



Mobile Game Development with Godot

A PRACTICAL GUIDE - 1ST EDITION

Daniel Buckley
Pablo Farias

academy.zenva.com

youtube.com/c/zenva

linkedin.com/company/zenva

©2026 Zenva Pty Ltd, all rights reserved

Table of Contents

Introduction.....	7
How to use this Book.....	7
Project Initialization and Design.....	9
Prerequisites.....	9
Project Files.....	10
Software requirements.....	12
Godot Version.....	12
Mobile Devices.....	12
Installing Godot.....	12
Game Design.....	14
The Core Game Concept.....	14
The Player.....	14
Obstacles and Spawning.....	15
Environment and Depth.....	16
User Interface (UI).....	16
The Main Menu.....	17
Project Setup.....	19
Creating the Main Game Scene.....	19
Importing Project Assets.....	21
Asset Overview.....	21
Core Mechanics: Environment and Player.....	24
The “Infinite” Environment Strategy.....	24
Setting up the Camera.....	24
Creating the Ground.....	26
Configuring the TileSet.....	27
Painting the Ground.....	28
Adding Collision.....	29
Fixing Blurry Pixels.....	29
Adding the Background.....	30
The Player Character.....	33
Player Node Setup.....	34
Adding Collision.....	36
Creating the Player Script.....	38

Handling Player Input.....	38
Implementing the Jump.....	40
Refining the Jump.....	41
World Logic: Movement and Obstacles.....	45
Moving the Ground.....	45
Creating the Game Manager.....	45
Scripting the Ground Movement.....	48
Creating an Infinite Loop.....	51
Seamless Tiling.....	52
Creating Obstacles.....	54
Creating the Obstacle Scene.....	54
Scripting the Obstacle's Movement.....	57
Creating the Obstacle Spawner.....	59
Scripting the Spawner Logic.....	61
Implementation and Troubleshooting.....	63
Randomizing Obstacles.....	65
Creating a New Tree Obstacle.....	65
Spawning Random Obstacles.....	70
Configuring the Spawner in the Editor.....	71
Visual Polish: Atmosphere and Animation.....	74
Creating Dynamic Clouds.....	74
Setting up the Cloud Spawner.....	78
Refactoring the Obstacle Script.....	81
Implementing a Parallax Effect.....	81
Randomizing Spawn Height.....	84
Adjusting the Draw Order.....	86
Player Animation.....	89
The AnimationPlayer Node.....	89
Creating the Run Animation.....	90
Creating the Jump Animation.....	93
Controlling Animations with Code.....	94
Game State Management.....	96
Implementing the Game Over Logic.....	96
Updating the Game Manager.....	96
Detecting Obstacles.....	97
Using Groups to Identify Obstacles.....	99

Connecting the Detector Signal.....	101
Implementing the Collision Logic.....	103
Finalizing the Game Over Effect.....	103
Building the Game Over UI.....	107
Setting up the UI Scene.....	107
Adding Game Over Text.....	110
Adding a Restart Button.....	111
Scripting the Restart Button.....	117
Managing Screen Visibility.....	119
Scoring and HUD.....	120
Tracking Player Distance.....	120
Displaying the Final Distance.....	121
Creating a Live Distance Counter.....	123
Updating the Distance in Real-time.....	126
User Interface and Menus.....	128
Creating the Menu Scene.....	128
The Background.....	129
The User Interface.....	132
Scripting the Play Button.....	141
In-game UI Buttons.....	144
Creating the Home Button.....	144
Implementing the Home Button Functionality.....	147
Adding a Pause Button.....	150
Implementing the Pause and Unpause Logic.....	151
Advanced Visuals: Particle Systems.....	156
Creating the Run Particle Effect.....	156
Amount and Time.....	157
Emission Shape.....	158
Movement.....	159
Initial Velocity.....	159
Visual Appearance.....	159
Controlling Particles with Code.....	161
Creating the Jump Particle Effect.....	163
Creating the Jump Particle Node.....	164
Configuring the Particle Emitter.....	164
Defining the Particle's Movement.....	166

Adjusting Visuals: Scale and Color.....	168
Triggering the Particles with Code.....	172
Game Feel: Audio and Input Buffering.....	175
Jump Buffering.....	175
The Problem with Precise Jumps.....	175
Adding the Buffer Timer.....	176
Scripting the Buffer Logic.....	177
Resetting the Input.....	178
Audio and Atmosphere.....	181
Implementing the Jump Sound Effect.....	181
Adding Background Music.....	183
Switching to Game Over Music.....	184
Looping Music Tracks.....	186
Finalization and Mobile Export.....	188
Screen Scaling.....	188
Configuring Stretch Settings.....	189
Understanding the Settings.....	190
Performance Optimization.....	191
Cleaning up Off-screen Nodes.....	191
Exporting to Android.....	193
Setting up the Android Export Preset.....	193
Installing Android Build Templates.....	194
Installing Required SDKs.....	195
Configuring SDK Paths in Godot.....	197
Final Project Configuration.....	198
Exporting the APK.....	200
Exporting to iOS.....	201
Export Templates.....	201
Project Configuration.....	201
Creating the Xcode Project.....	202
Closing Words.....	204
What you Accomplished.....	204
The Node System and Scene Tree.....	204
Scripting and Logic.....	204
Game Feel and Polish.....	205
User Interface and State Management.....	205

The Build Pipeline.....	205
Where to go from Here.....	205
Expand this Project.....	205
Participate in a Game Jam.....	205
Explore the Documentation.....	206
Join the Community.....	206
Continue Your Learning.....	206
Final Thoughts.....	206
Reference: Full Source Code.....	208
game_manager.gd.....	208
ground.gd.....	209
menu.gd.....	210
obstacle.gd.....	210
player.gd.....	211
spawner.gd.....	213

Introduction

Welcome to your journey with the Godot Engine. Game development is one of the most rewarding creative pursuits available today. It combines logic, art, sound, and design into an interactive experience that you can share with the world. Whether you're still just starting out with Godot or are an experienced developer curious about Godot's unique node-based workflow, this book is designed to get you building a real, playable game that can be played on the go.

We have structured this book to be hands-on. Theory is important, but nothing beats the feeling of seeing a character move across the screen because *you* told it to. Over the course of ten chapters, we will build a complete infinite runner game for mobile devices. We will start with the absolute basics of the Godot interface, then gradually layer on complexity: player controls, scrolling backgrounds, obstacles, particle effects, UI, audio, and finally a polished mobile export you can run on your phone.

How to use this Book

To get the most out of these pages, we recommend an active approach. Reading about code is different from writing it. Here are a few tips to help you succeed:

- **Follow along actively.** Open Godot and build the projects step-by-step with us. Muscle memory plays a huge role in learning a new interface.
- **Experiment often.** When we set a speed value to 200, try setting it to 500. See what happens when you change a color or a physics property. Breaking the game is often the best way to understand how it works.
- **Use the reference.** The appendix contains the full source code for our project. If you get stuck or your code behaves strangely, compare it against the reference to find the difference.

We have designed this path to be smooth, but game development is inherently a problem-solving discipline. Embrace the challenges. Every bug you fix makes you a better developer, and every small victory (your first moving character, your first particle effect) is worth celebrating.

The blank canvas awaits. Let's turn the page and start building your infinite runner.

Project Initialization and Design

Welcome to this guide on mobile game development using the Godot engine. Throughout this book, we will guide you through the entire process of creating a polished game from scratch and deploying it to mobile devices.

The core of this project is the development of a complete infinite runner game. In this game, the player’s character will continuously “move” to the right side of the screen, and the objective is to survive as long as possible by jumping over obstacles. Along the way, we will focus on key considerations for mobile development, such as implementing intuitive tap-based inputs and designing user interfaces with large, accessible buttons suitable for touchscreens.

We will cover three primary areas:

1. Creating a complete game in Godot.
2. Building and deploying your game for Android devices.
3. Building and deploying your game for iOS devices.

Prerequisites

To ensure a smooth learning experience, a basic understanding of Godot and GDScript is required. You should be familiar with the following fundamental concepts:

- Godot’s editor interface (Scene dock, Inspector, FileSystem dock)
- Nodes, scenes, and the parent-child hierarchy
- GDScript fundamentals (variables, types, functions, conditionals, loops)
- Using the 2D viewport to position and scale nodes
- Running a scene and reading console output
- What a tile map is (but not necessarily how to set one up in Godot)

If some of these concepts are new to you, we recommend reviewing the following resources first:

- [Learn Godot 4.6 From Scratch](https://academy.zenva.com/product/intro-to-godot-ebook/)
(<https://academy.zenva.com/product/intro-to-godot-ebook/>)
- [Mini-Projects with Godot 4.6](https://academy.zenva.com/product/godot-mini-projects-ebook/)
(<https://academy.zenva.com/product/godot-mini-projects-ebook/>)

Alternatively, you can complete the video course versions of these topics:

- [Intro to Godot 4 Game Development](https://academy.zenva.com/product/intro-to-godot-4-game-development/)
(<https://academy.zenva.com/product/intro-to-godot-4-game-development/>)
- [Godot 4 Mini-Projects](https://academy.zenva.com/product/godot-4-mini-projects/)
(<https://academy.zenva.com/product/godot-4-mini-projects/>)

That said, we will walk through each step carefully as we work in the editor and write code together.

Project Files

To follow along with the tutorial, you will need the project assets (sprites and audio) and can refer to the completed project source code if you get stuck.

- [Asset Files](https://academy.zenva.com/wp-content/uploads/2026/03/Assets-Godot-Mobile-Intro.zip)
(<https://academy.zenva.com/wp-content/uploads/2026/03/Assets-Godot-Mobile-Intro.zip>)
- [Completed Project](https://academy.zenva.com/wp-content/uploads/2026/03/Complete-Project-Godot-Mobile-Intro-v4.6.zip)
(<https://academy.zenva.com/wp-content/uploads/2026/03/Complete-Project-Godot-Mobile-Intro-v4.6.zip>)

Third Party Asset Licenses

Some of the assets provided in this course are subject to third-party licenses:

Assets by Pixel Frog

Environment and player sprites

URL: <https://pixelfrog-assets.itch.io/>

License: Creative Commons CC0
(<https://creativecommons.org/publicdomain/zero/1.0/>)

Assets by Kenney

UI elements

URL: <https://kenney.nl/>

License: Creative Commons CC0
(<https://creativecommons.org/publicdomain/zero/1.0/>)

Assets by SubspaceAudio

Background music

URL: <https://opengameart.org/users/subspaceaudio>

License: Creative Commons CC0
(<https://creativecommons.org/publicdomain/zero/1.0/>)

Assets by artisticdude

Jump SFX

URL: <https://opengameart.org/users/artisticdude>

License: Creative Commons CC0
(<https://creativecommons.org/publicdomain/zero/1.0/>)

Zenva-Created Assets

All other artwork and media assets included in the project files that are not listed above were created by **Zenva** and are released under:

Creative Commons CC0 (Public Domain)
<https://creativecommons.org/publicdomain/zero/1.0/>

Zenva-created source code included in the project files for this course is released under the MIT License (<https://opensource.org/license/MIT>). Third-party libraries and dependencies remain under their respective licenses.

Software requirements

Godot Version

Technology changes rapidly. To ensure an optimal learning experience and compatibility with the code provided, we recommend using **Godot 4.6** for this project. Using newer or older versions may result in compatibility issues.

Mobile Devices

To test the mobile-specific features, you should have an Android or iOS mobile device available.

Important Note for iOS Development: You can only build a Godot project for an iOS device if you are using a macOS computer. If you are using a Windows computer, you will not be able to build your project for an iOS device directly, as the necessary build tools (Xcode) are exclusive to macOS.

Installing Godot

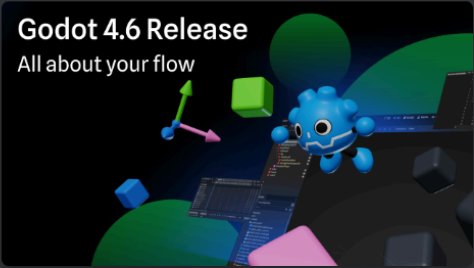
You can download the 4.6 version of the engine by visiting the official Godot download archive (<https://godotengine.org/download/archive/4.6-stable/>). On this page, select the Standard option that matches your operating system. Note that instructions provided in this ebook are intended for desktops only.

Godot 4.6-stable

26 January 2026

Godot 4.6 Release

All about your flow



With the stability gained over the past five releases, the engine has matured enough to enter a new phase. Godot 4.6 kicks off a period of polish, quality-of-life improvements, and doubled-down effort on performance optimization.

[Read more](#)[View changelog](#)

Supported platforms

 Android	Standard	
 Linux	Standard	.NET
 macOS	Standard	.NET
 Windows	Standard	.NET
 Web editor	Standard	
 Export templates	Standard	.NET

.NET builds offer support for C# as a scripting language.

[Show all downloads](#)

Once the file is downloaded, unzip the archive and simply click on the application launcher. No further installation steps are required.

With the engine installed and your project files ready, we can now begin our development journey.

Game Design

Before we begin developing in Godot, let's review the design of the game we will be creating. This project is an infinite runner where the primary objective is for the player to survive as long as possible by dodging obstacles while running to the right.

The Core Game Concept

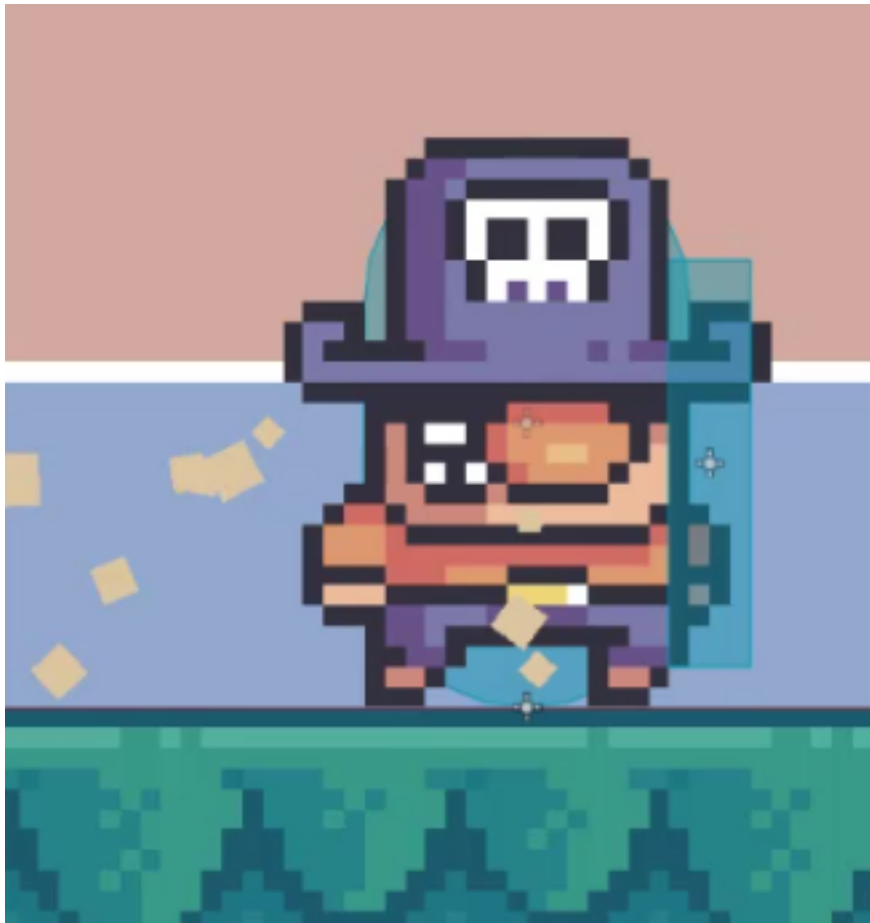
In this game, we will create an interesting illusion of movement. While it will appear that the player is running forward, the player character will actually remain stationary on the X-axis. Instead, the environment (including the ground and obstacles) will move from right to left past the player. This technique is common in infinite runner games and simplifies the logic for camera tracking and object spawning.



The Player

The player character will have simple controls - but will be enhanced with various effects to make the gameplay feel engaging.

- **Input:** The only player input will be tapping the screen (or clicking), which will cause the character to jump.
- **Collision:** The player will have two main collision shapes. A capsule collider will handle interactions with the ground, while a rectangular area in front of the player will detect collisions with obstacles, triggering a “game over” state.
- **Game Feel:** To enhance the player experience, we will add several features, often referred to as “juice.” This will include run and jump animations, particle effects, and a sound effect for jumping.

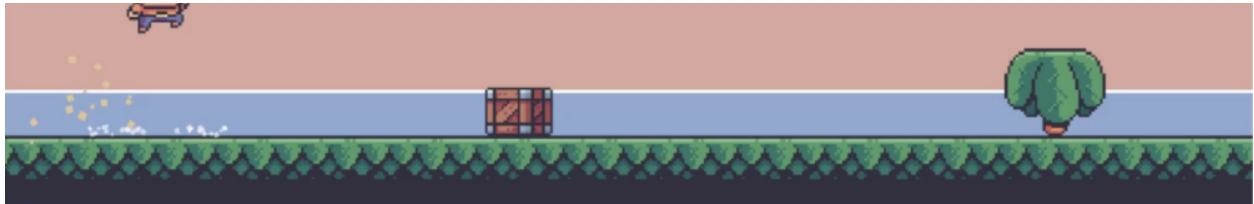


Obstacles and Spawning

The obstacles will be generated dynamically to create an endless and varied challenge. This will be managed by a dedicated Spawner node.

- Obstacles will spawn off-screen on the right-hand side and move towards the player on the left.

- The Spawner node will manage an array of different scenes (such as crates and trees) that it can spawn.
- We can configure the Spawner to randomly select which obstacle to create, adding variety to the gameplay.
- The distance between spawned obstacles can also be adjusted to control difficulty.



Environment and Depth

To create a more visually appealing and immersive environment, we will add background elements and an illusion of depth.

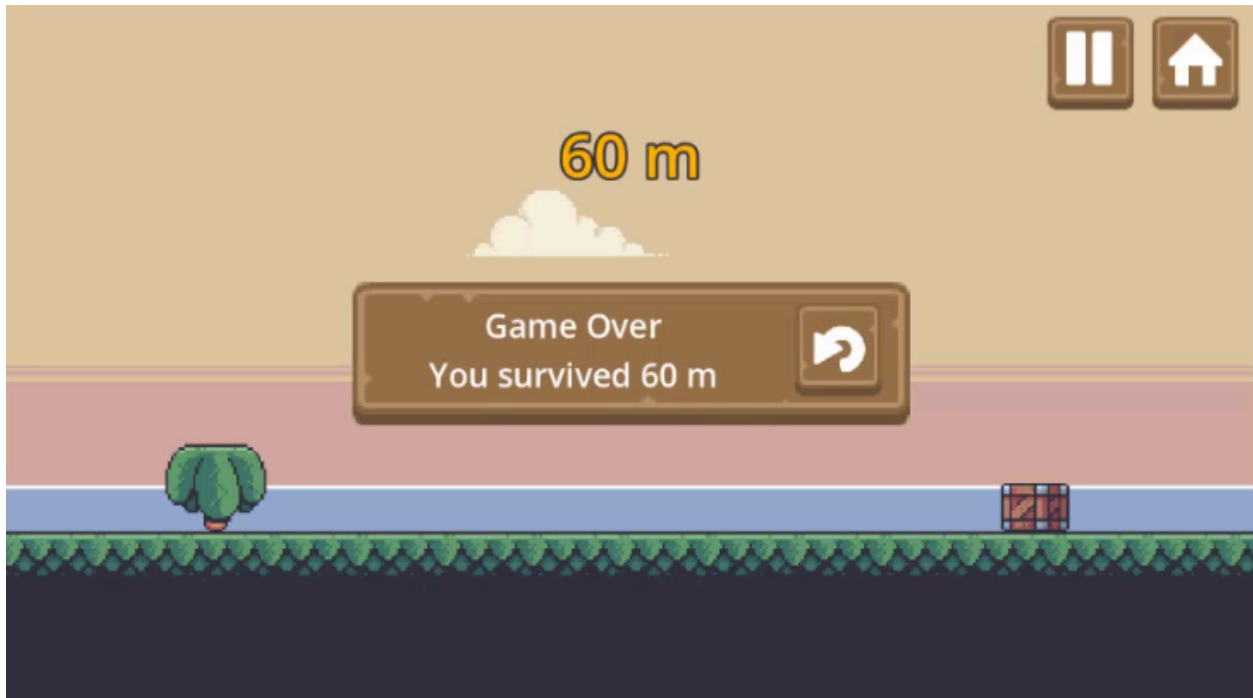
- The same spawner system used for obstacles will also be used to generate clouds in the sky.
- We will implement a parallax system. This will cause objects that are further in the background (like the clouds) to move slower than objects in the foreground (like the trees and the character). This technique creates a convincing illusion of depth in a 2D game.



User Interface (UI)

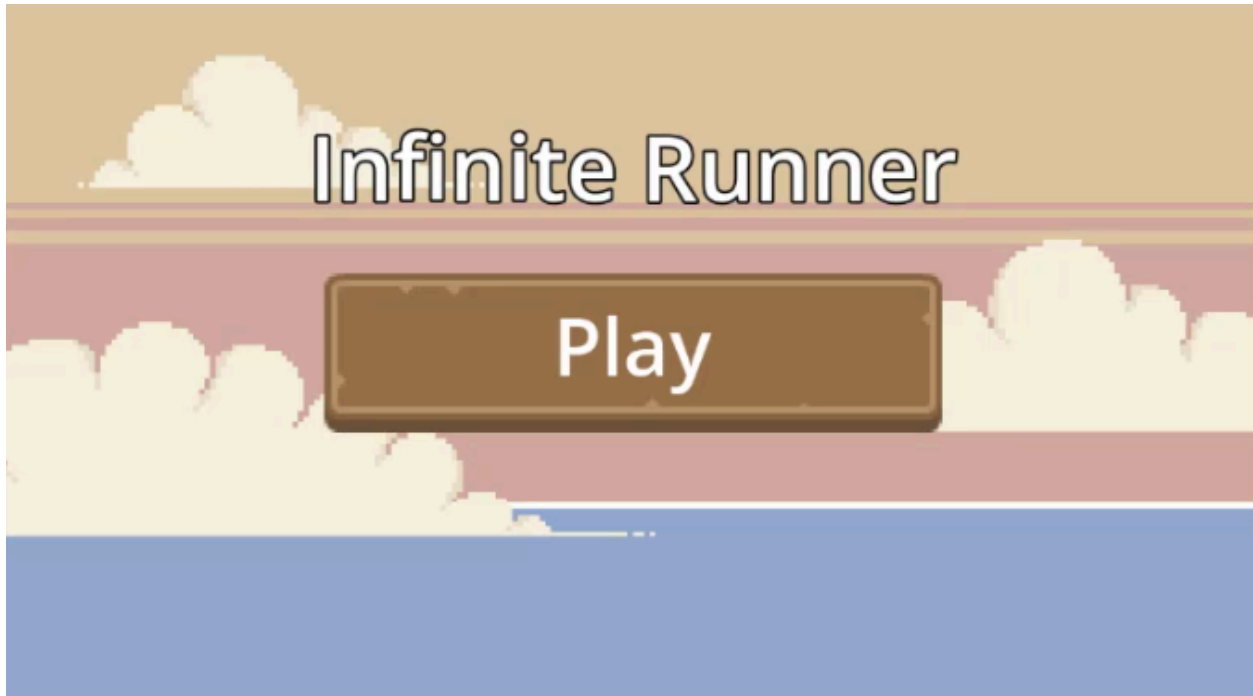
The game will feature a clean and functional user interface to provide players with necessary information and controls.

- **Distance Text:** A score display will show how far the player has traveled in meters.
- **Game Over Screen:** When the player hits an obstacle, a screen will appear displaying the final distance and a button to restart the game.
- **In-Game Buttons:** During gameplay, there will be buttons to pause the game and to return to the main menu.



The Main Menu

Finally, the game will start with a simple main menu screen, which will include the game's title and a "Play" button to begin the game.



Now that we have a clear overview of the project, we can proceed to Godot and begin setting it up.

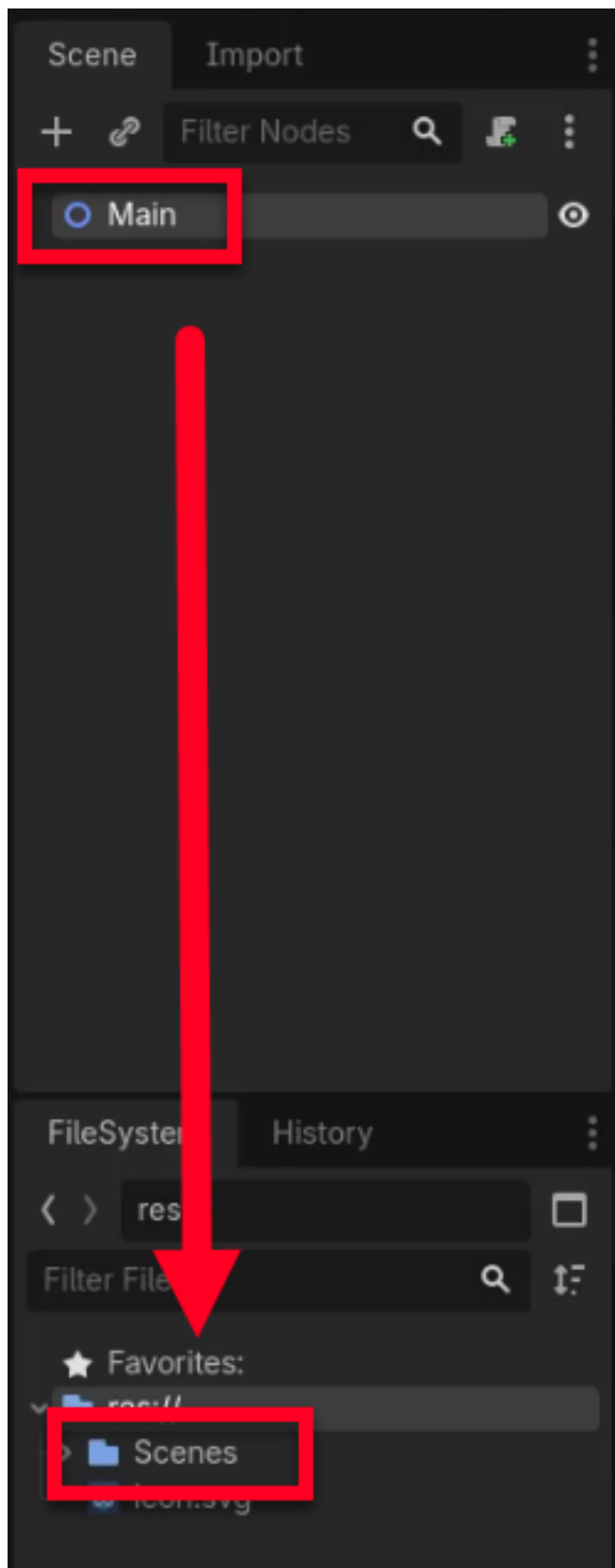
Project Setup

In this chapter, we will begin setting up our project. We will create the main scene for our infinite runner game and import all the necessary assets to get started.

Creating the Main Game Scene

The first step is to create the primary scene where our game will take place. This scene will contain all the elements of our game, such as the player, the environment, and the user interface.

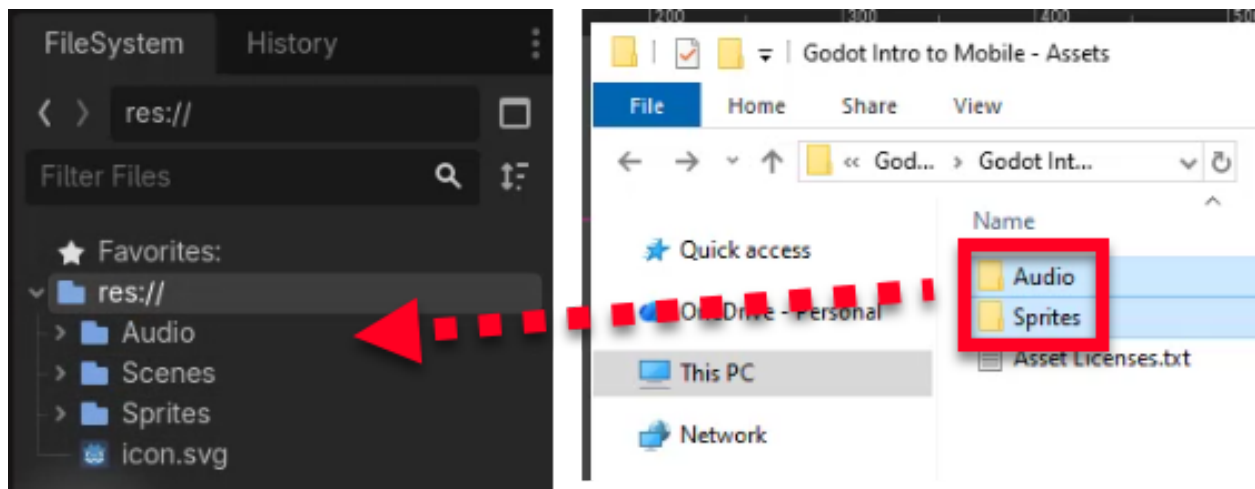
With Godot open, create a new 2D Scene and rename the root node to “Main”. Then, save the scene. It is good practice to create a new folder named “Scenes” to keep your project organized. Save your “Main” scene inside this folder.



Importing Project Assets

With our scene created, we now need to import the assets, such as sprites and audio clips. You can find all the required assets in the downloadable zip file listed in the **Project Files** section in the initialization chapter.

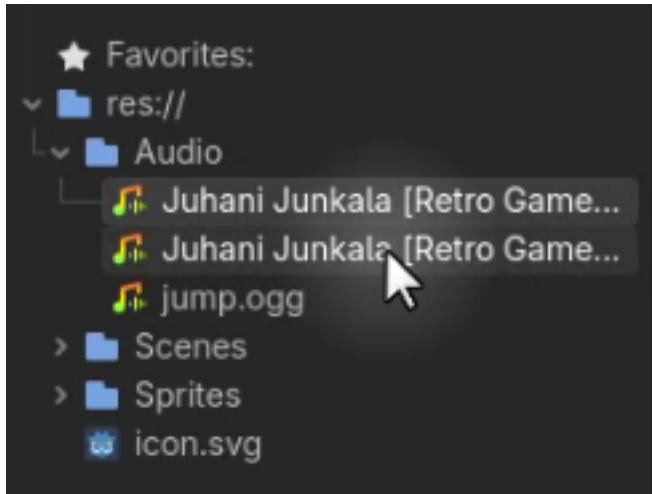
Once you have downloaded and extracted the zip file, you will find the “Audio” and “Sprites” folders. To import them into your Godot project, simply select both folders and drag them into the FileSystem dock within the Godot editor. The engine will automatically handle the import process.



Asset Overview

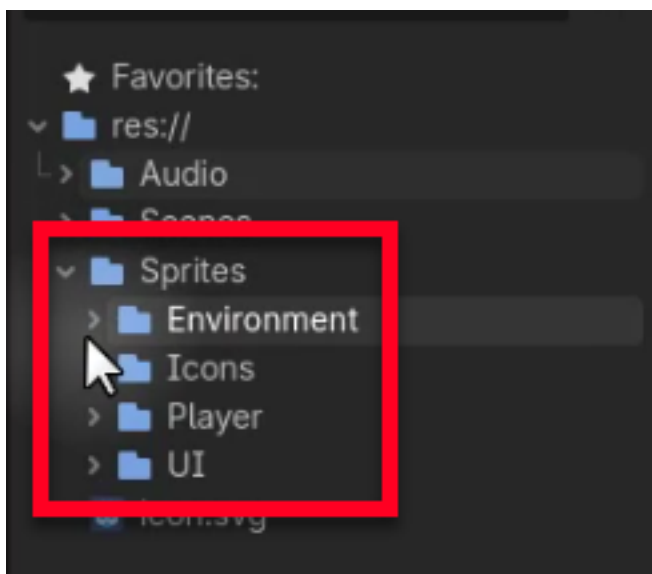
Let's take a moment to review the assets you have just imported. These files will provide the visual and auditory elements for our game.

Audio: This folder contains three audio clips. Two are music clips that we will switch between during gameplay, and one is a jump sound effect for our player.



Sprites: This folder is further divided into subfolders containing all our visual assets:

- **Environment:** Contains various tiles, trees, and clouds that we will use to build our game world.
- **Icons:** Includes different icons for the user interface (UI).
- **Player:** Holds the sprite animations for the player character, including idle, run, and jump states.
- **UI:** Contains assets for buttons and panels needed for the game's interface.



You are welcome to use your own assets, but using the provided files will make it easier to follow along with the instructions.

Now that our project is set up with the main scene and all necessary assets, we are ready to move on. In the next chapter, we will begin building the game environment, including the background and the looping ground for our player to run on.

Core Mechanics: Environment and Player

In this chapter, we will begin constructing the core elements of our game. A well-designed environment is crucial for setting the stage, and a responsive player character is the heart of the gameplay experience. We will focus on setting up a scrolling background, a foreground platform for the player to run on, and the player character itself, complete with physics-based jumping mechanics.

The “Infinite” Environment Strategy

For an infinite runner game, one might assume that the camera and player move continuously through an ever-expanding world. While this is a valid approach, it can lead to technical issues over long gameplay sessions, specifically *floating-point errors*. In Godot, as objects move further from the world’s origin point (0,0), the precision of their position coordinates can decrease. This often results in visual glitches and jittery movement.

To avoid this, we will employ a different strategy:

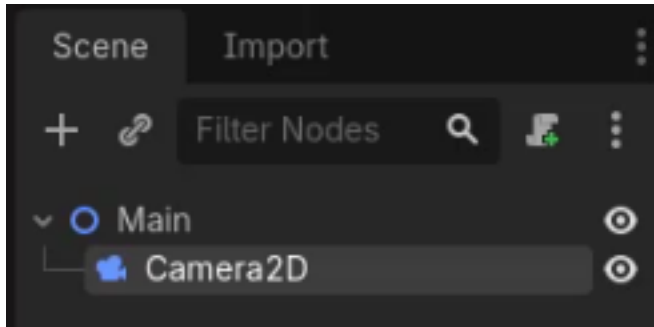
- The player and the camera will remain stationary in the center of the scene.
- The environment (the ground and background) will move towards the player to create the illusion of forward motion.

This method keeps all game objects close to the origin, ensuring positional accuracy and simplifying the logic for spawning new elements.

Setting up the Camera

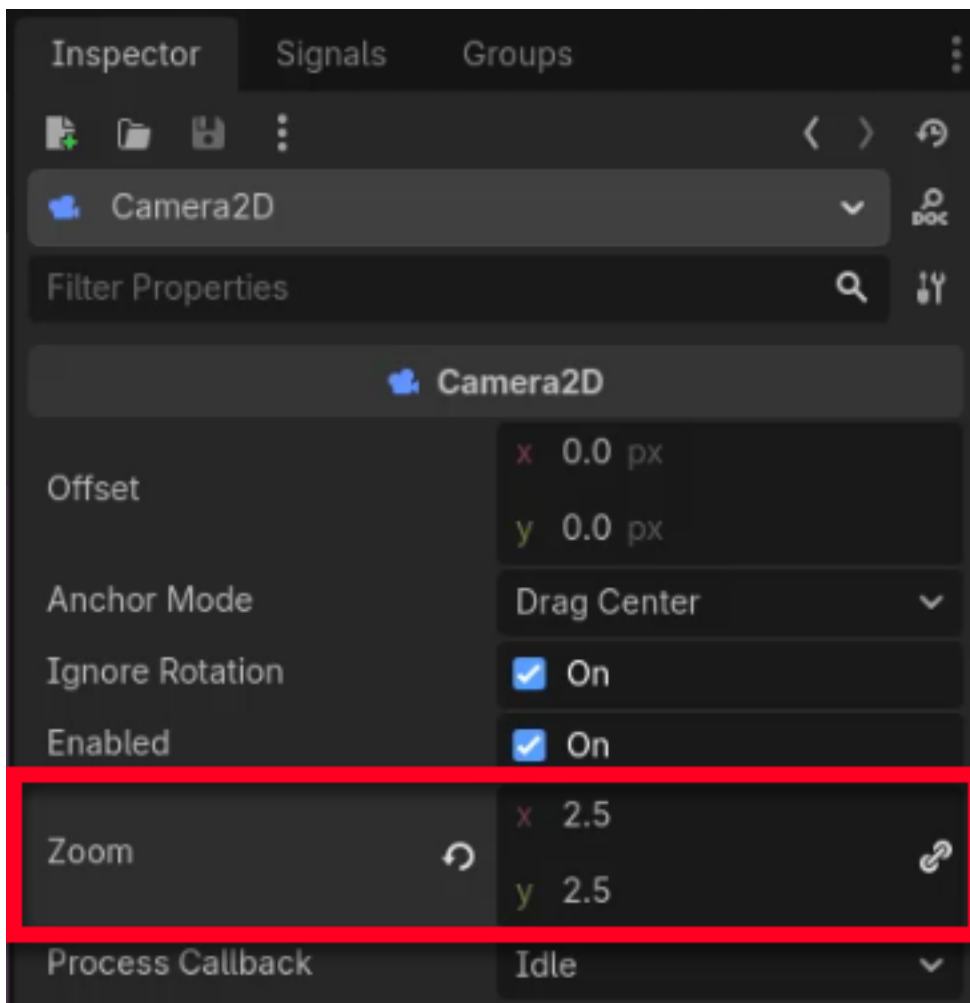
First, we need a camera to define the player’s view. We will also adjust its zoom to suit our pixel-art style.

Start by adding a Camera2D node to your scene.



To get a sense of scale, temporarily drag a player sprite from the FileSystem dock into the scene. You will likely notice that the default camera view is too zoomed out for our assets.

With the Camera2D node selected, find the Zoom property in the Inspector and set both its X and Y values to 2.5. This will make each pixel of your art take up 2.5 pixels on the screen, giving it a chunkier, clearer look.



Once you are satisfied with the scale, you can delete the temporary player sprite from the scene.

Creating the Ground

The ground is the surface the player will run on. It needs to have a visual representation and a physical collider.

Create a `StaticBody2D` node and rename it to “Ground”. We use a `StaticBody2D` because it is a physics object that can collide with other objects but will not be moved by external forces.

Godot Insight: Physics body types in 2D

Godot provides three main types of physics bodies, each suited to a different role in your game:

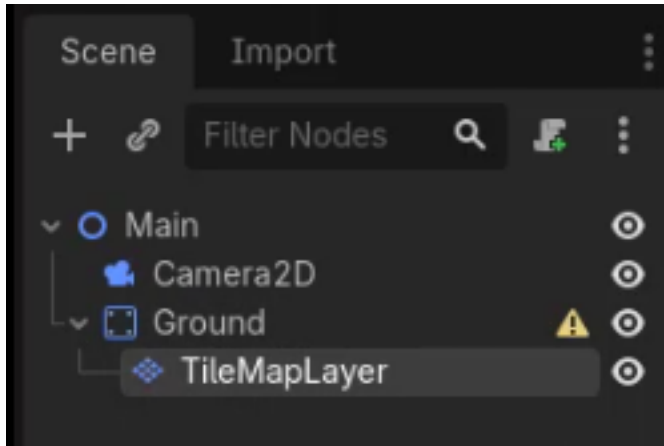
StaticBody2D never moves on its own. Use it for floors, walls, and platforms that the player stands on. You can still move it through code (as we will do later to scroll the ground), but the physics engine will not push it around.

CharacterBody2D is moved entirely by your code. Use it for player characters and NPCs where you need precise control over movement, along with built-in helpers like `move_and_slide()` for collision response.

RigidBody2D is driven by the physics engine. Use it for objects that should respond to gravity, collisions, and forces automatically, such as bouncing crates or ragdolls.

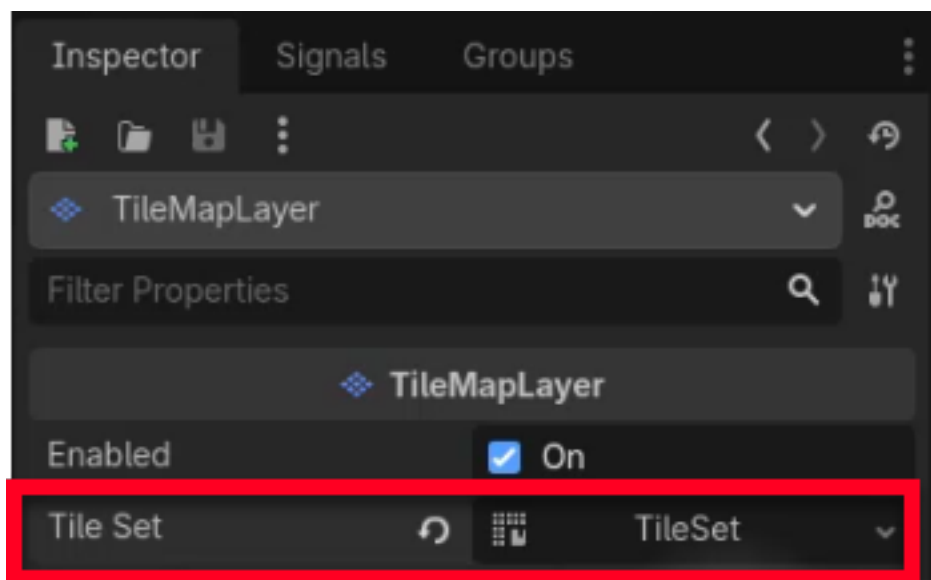
Learn more: [StaticBody2D](https://docs.godotengine.org/en/4.6/classes/class_staticbody2d.html)
(https://docs.godotengine.org/en/4.6/classes/class_staticbody2d.html)

To create the visuals, we will use a `TileMap`. Right-click on the “Ground” node and add a `TileMapLayer` node as a child.



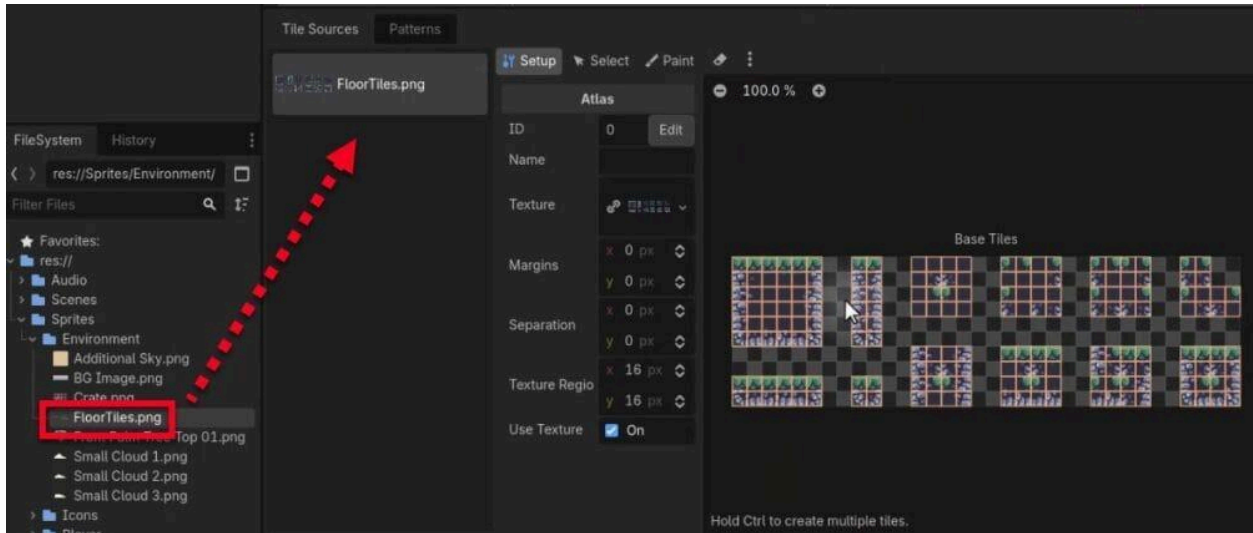
Configuring the TileSet

In order to use our TileMap, we need to give it a TileSet resource. To do so, first select the TileMapLayer node. In the Inspector, find the Tile Set property and select “[empty]” > “New TileSet”.



At the bottom of the editor, a new “TileSet” panel will appear. Click on it to open the TileSet editor.

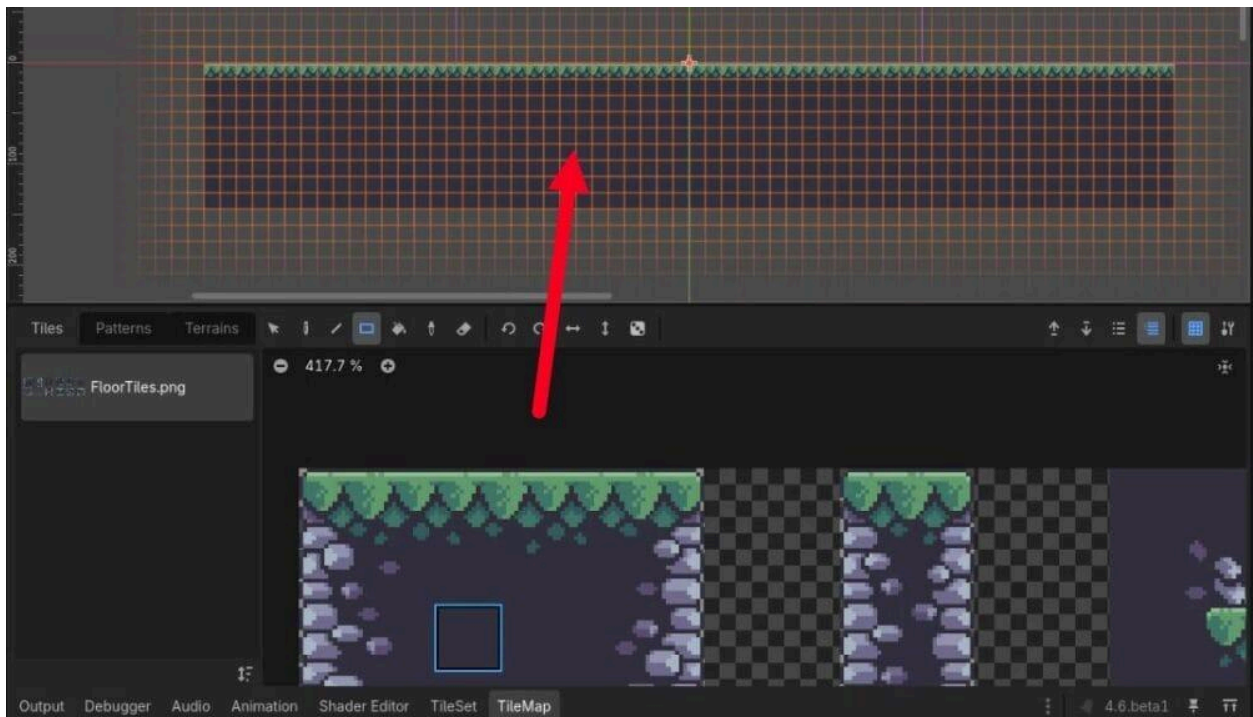
From the FileSystem dock, drag your ground tile texture (e.g., FloorTiles.png) into the left-hand panel of the TileSet editor. Godot will ask if you want to automatically create tiles; select “Yes”. Godot will then slice the image into a grid of tiles, which are 16×16 by default.



Painting the Ground

With the TileSet configured, we can now actually set up our map. Once again, select the TileMapLayer node again in the Scene dock. At the bottom of the editor, switch to the “TileMap” panel. You can now select tiles and paint them directly into your scene to build the ground platform.

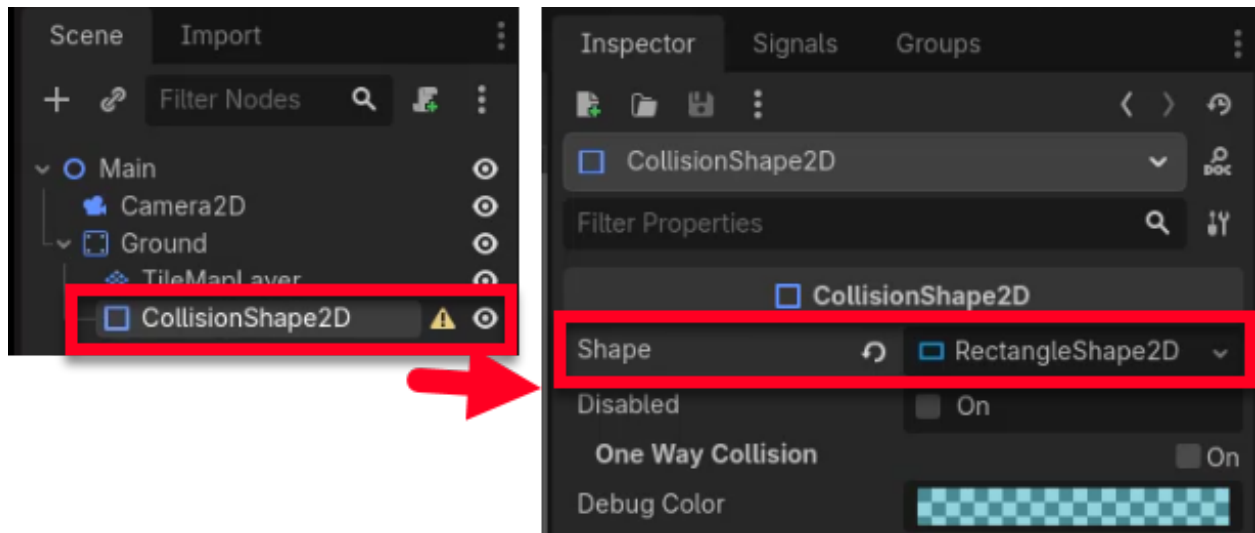
While painting, ensure the platform is wider than the camera’s view to prevent the edges from being visible on wider screens.



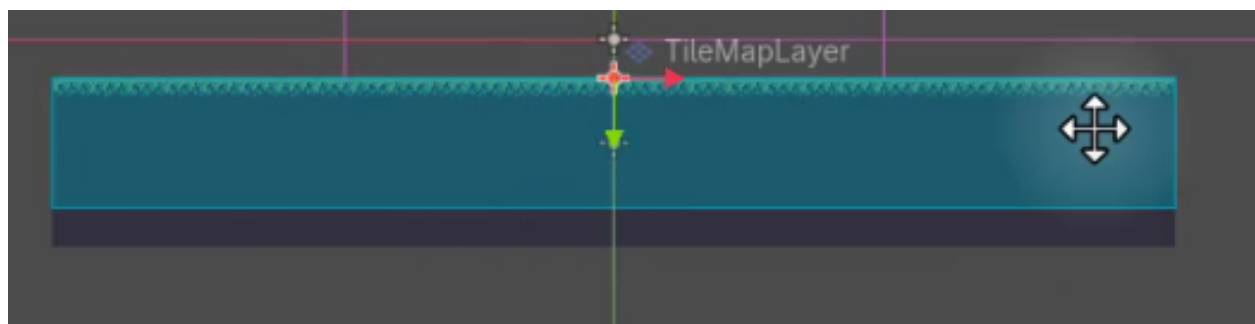
Adding Collision

Finally, the ground needs a collider so the player does not fall through it. Select the “Ground” node and add a CollisionShape2D as a child.

In the Inspector for the CollisionShape2D, set its Shape property to “New RectangleShape2D”.



Now we can use the handles in the 2D viewport to resize the rectangle so that it covers the entire visible area of your tilemap ground.

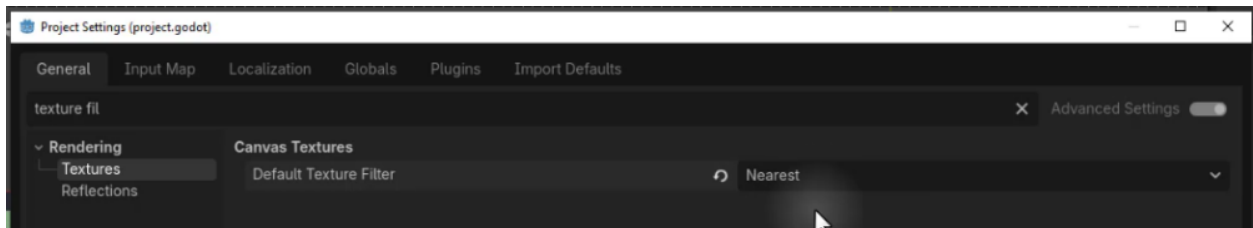


Fixing Blurry Pixels

You may notice that the pixel art appears blurry. This is due to Godot’s default texture filtering, which smooths pixels when they are scaled up. For pixel art, we want the opposite: each source pixel should map to a sharp, hard-edged block on screen. Switching the filter to “Nearest” (also called nearest-neighbor interpolation) tells the renderer to pick the closest pixel value instead of blending between neighbors, preserving those crisp edges.

We can adjust the texture filtering for the whole project by following these steps:

1. Go to the main menu and select **Project > Project Settings**.
2. In the Project Settings window, navigate to the **Rendering > Textures** section on the left.
3. Find the **Default Texture Filter** setting and change its value from “Linear” to “Nearest”.
4. Close the settings window.



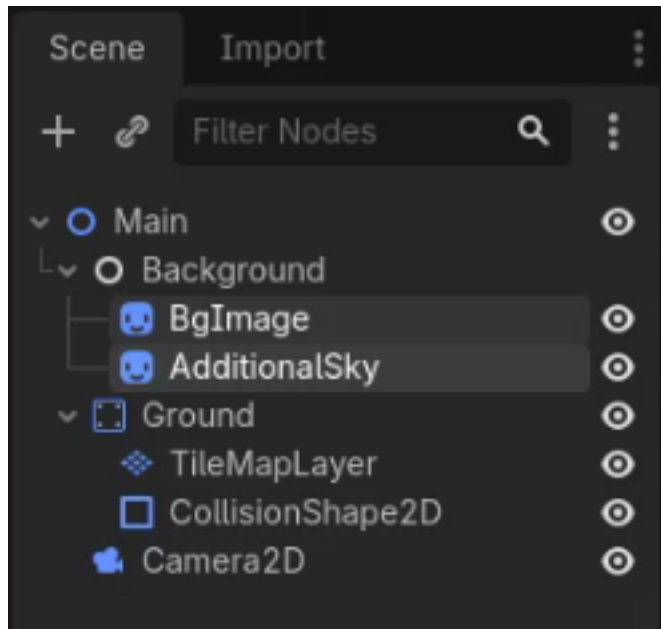
Your game’s graphics should now appear sharp and crisp!

Adding the Background

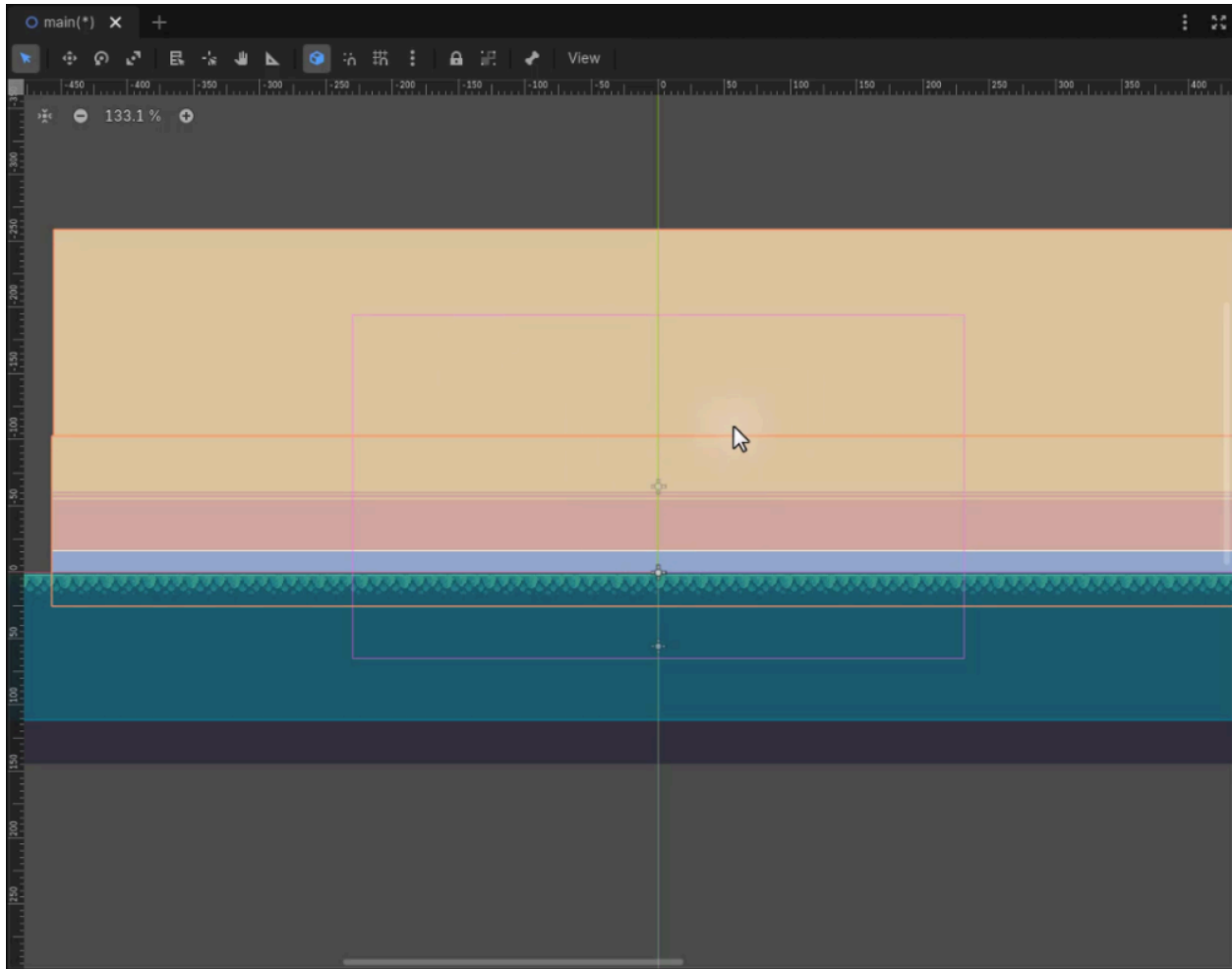
Next, let’s add a background to our project. The background adds depth and atmosphere to our scene even if it's not important to the core mechanics. We will construct it from multiple images and ensure it always renders behind everything else.

To start, create a new, empty Node and rename it “Background” to serve as a container for our background elements.

Next, drag your background image (e.g., BG Image.png) and any additional sky images you want into the scene as children of the “Background” node.



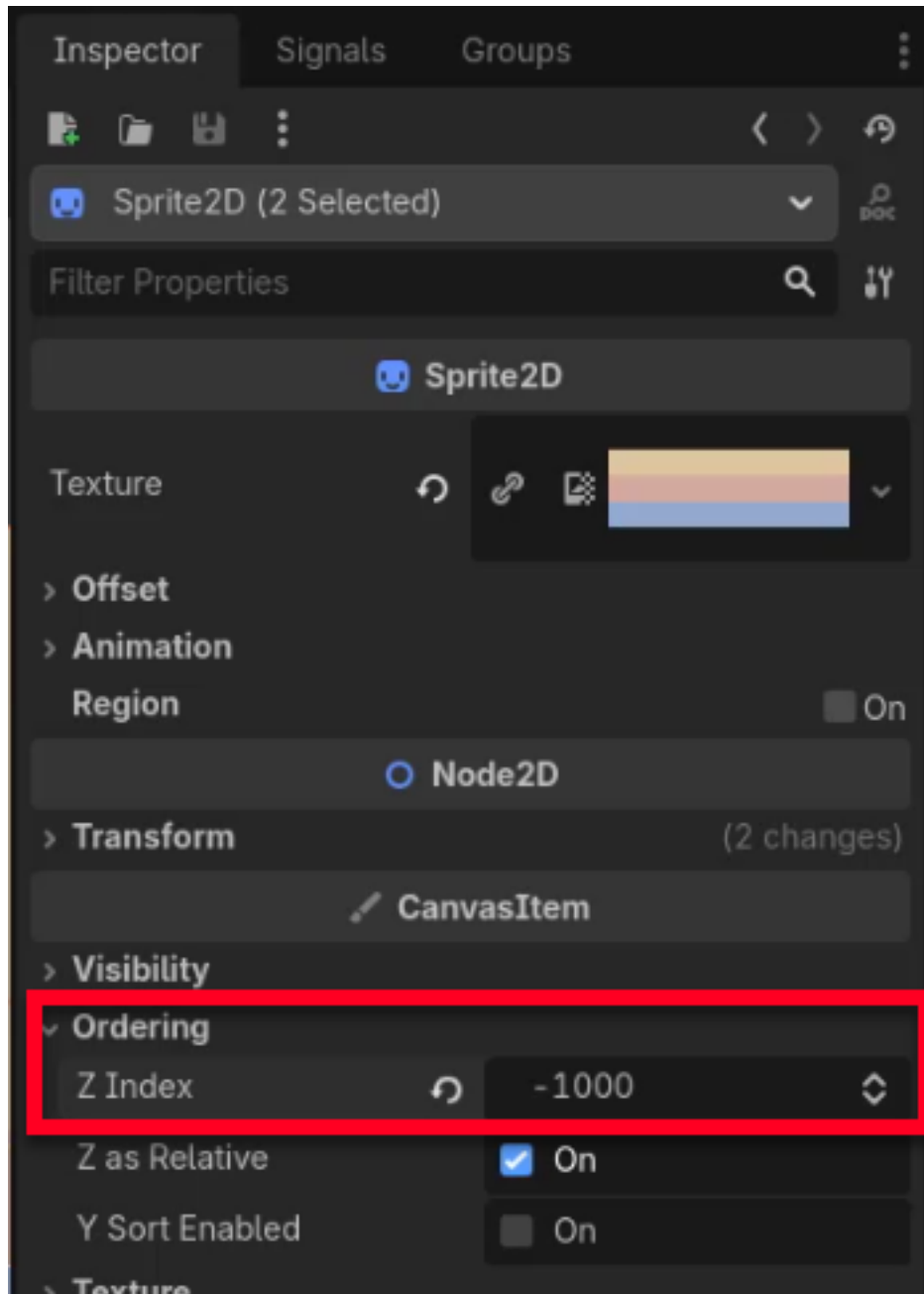
Position and scale these sprites in the 2D viewport to create a complete background scene that fills the camera's view.



By default, Godot draws nodes in the order they appear in the scene tree. If you later rearrange nodes, the background could accidentally render on top of the player. Setting a Z-index gives us explicit control over draw order, so the background stays behind everything no matter how the scene tree changes.

Let's begin by selecting both background sprites.

In the Inspector, find the **Ordering** section and set the Z Index property to a large negative number, such as -1000. Objects with a lower Z-index are drawn before objects with a higher Z-index, effectively placing them in the background.



With the camera, ground, and background in place, our basic environment is now set up. Next, we will focus on implementing the player character.

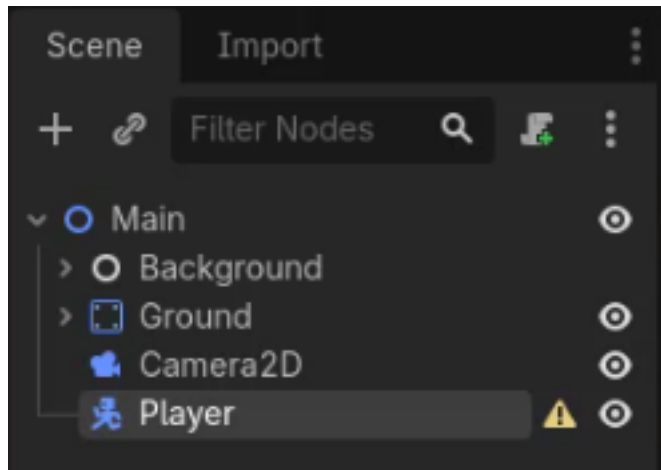
The Player Character

In this section, we will set up our player character. We will add the necessary nodes to the scene to represent the player and then implement the code required to make the player jump.

Player Node Setup

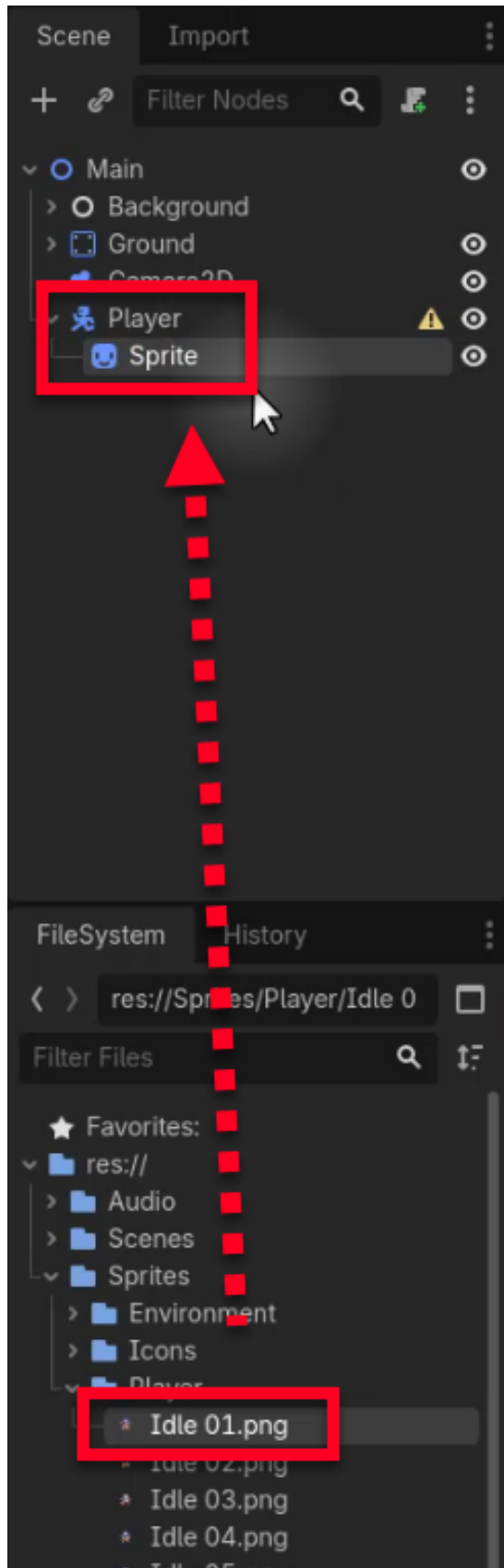
First, we need to create the main node for our player. In Godot, a `CharacterBody2D` is the ideal node for a character that will be controlled by player input and affected by physics like gravity.

In the Scene dock, click the plus icon and add a `CharacterBody2D` node. For organizational purposes, rename this node to “Player”.

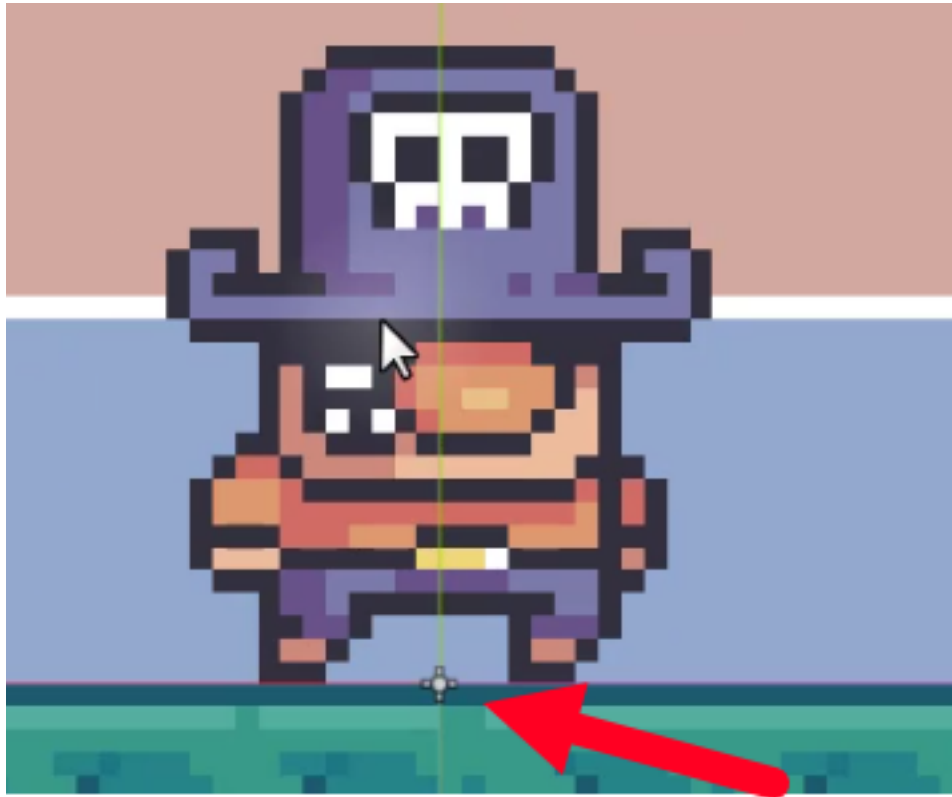


To make our player visible, we need to give it a visual representation. We will do this by adding a `Sprite2D` node as a child of the Player.

Select the “Player” node and, in the FileSystem dock, navigate to `Sprites/Player/`. Drag one of the idle sprite images onto the “Player” node in the Scene dock; this will automatically create a `Sprite2D` node. Rename this new node to “Sprite” so we can easily identify it in the hierarchy.



With the “Sprite” node selected, use the move tool in the 2D viewport to position it so that the Player node’s origin (the center of the crosshairs) is at the character’s feet. This will make it easier in the future for ground detection.

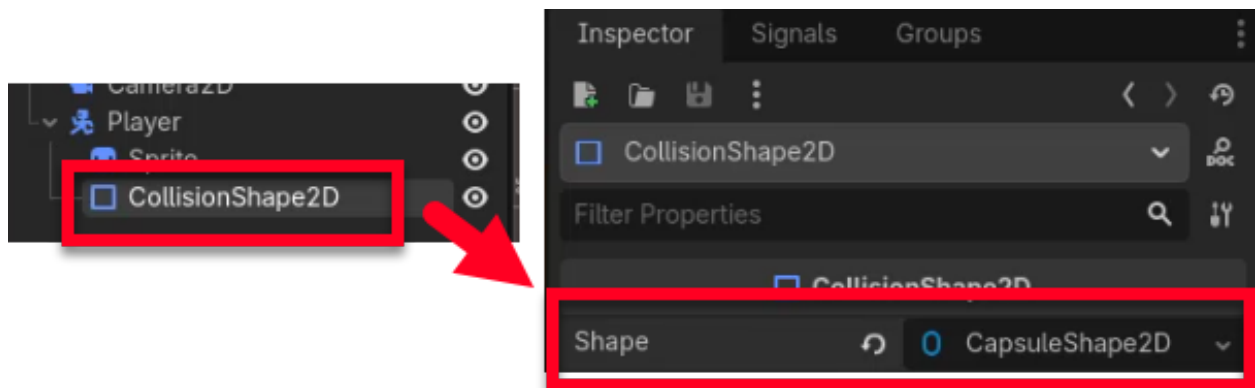


Adding Collision

A `CharacterBody2D` requires a collision shape to interact with the physics world. Without one, it cannot detect collisions or the ground.

Select the “Player” node and add a `CollisionShape2D` node as a child.

In the Inspector for the `CollisionShape2D`, click the empty field next to the “Shape” property and select “New `CapsuleShape2D`”. A capsule shape is often preferred for characters as its rounded edges can help prevent the player (or any entity for that matter) from getting stuck on corners.



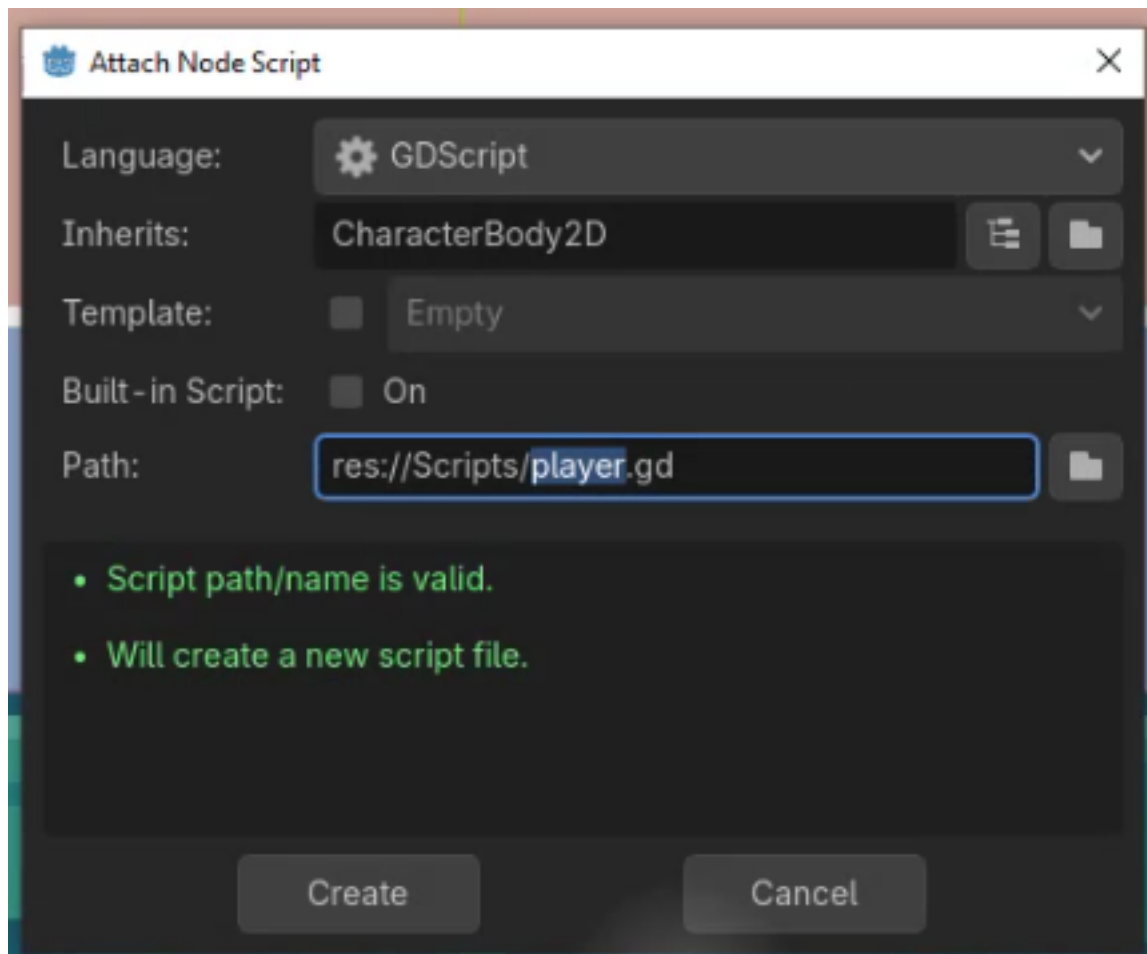
In the 2D viewport, adjust the size and position of the capsule shape to roughly match the player's sprite. It is often good practice to make the collider slightly smaller than the sprite. This provides a small margin of error, improving the game feel, as the player will not feel like they were unfairly hit by an obstacle.



Creating the Player Script

With the player's scene structure complete, we can now create a script to control its behavior.

1. With the “Player” node selected, find the “Script” property in the Inspector and select “[empty]” > “New Script...”.
2. Before saving, create a new folder for our scripts. Click the folder icon, create a new folder named “Scripts”, and open it.
3. Name the script `player.gd` and click “Create”.



Handling Player Input

We need a way to detect when the player wants to jump. This can be done via a mouse click or a screen tap. Godot's built-in `_input(event)` function is called every time any input is detected.

First, let's lay out the functions we will need in our script.

```
extends CharacterBody2D

func _physics_process(delta):
    pass

func _try_jump():
    pass

func _input(event):
    pass
```

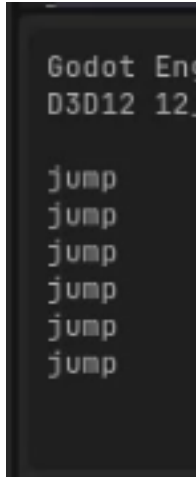
Inside the `_input` function, we can check the type of event that occurred. We will check for a left mouse button press and a screen touch, and when either occurs, we will call our `_try_jump` function. We can also temporarily add a print statement to our `_try_jump` function so we can test it out.

```
func _input(event):
    if event is InputEventMouseButton:
        if event.button_index == MOUSE_BUTTON_LEFT:
            if event.pressed:
                _try_jump()

    if event is InputEventScreenTouch:
        if event.pressed:
            _try_jump()

func _try_jump():
    print("jump")
```

If you run the scene now and click the game window, you should see the word “jump” printed in the Output panel each time you click.



Implementing the Jump

Now, let's add the physics for the jump. We will define variables for jump force and gravity, and then use them within the `_physics_process` function. This function is called at a fixed interval, making it ideal for physics calculations.

First, add these variables to the top of your script.

```
extends CharacterBody2D
```

```
@export var jump_force : float = 500
```

```
@export var gravity : float = 2200
```

```
var jump_input : bool = false
```

- `jump_force`: The upward velocity applied when the player jumps. We use `@export` to make it editable in the Inspector.
- `gravity`: The downward force applied every frame the player is in the air (which we also make editable in the Inspector).
- `jump_input`: A boolean flag we will use to signal that a jump has been requested.

Next, modify `_try_jump` to set this flag to true.

```
func _try_jump():  
    jump_input = true
```

Finally, we can update the `_physics_process` function to handle gravity and jumping. We must also call the `move_and_slide()` function, which is essential for a `CharacterBody2D` as it applies the velocity and handles collisions.

```
func _physics_process(delta):  
    # Apply gravity only while airborne  
    if not is_on_floor():  
        velocity.y += gravity * delta  
  
    # Handle jump  
    if jump_input:  
        velocity.y = -jump_force  
        jump_input = false  
  
    move_and_slide()
```

Note that we set `velocity.y` to a negative value for the jump. This is because in Godot's 2D coordinate system, the positive Y-axis points downwards.

Godot Insight: `_physics_process` vs `_process`

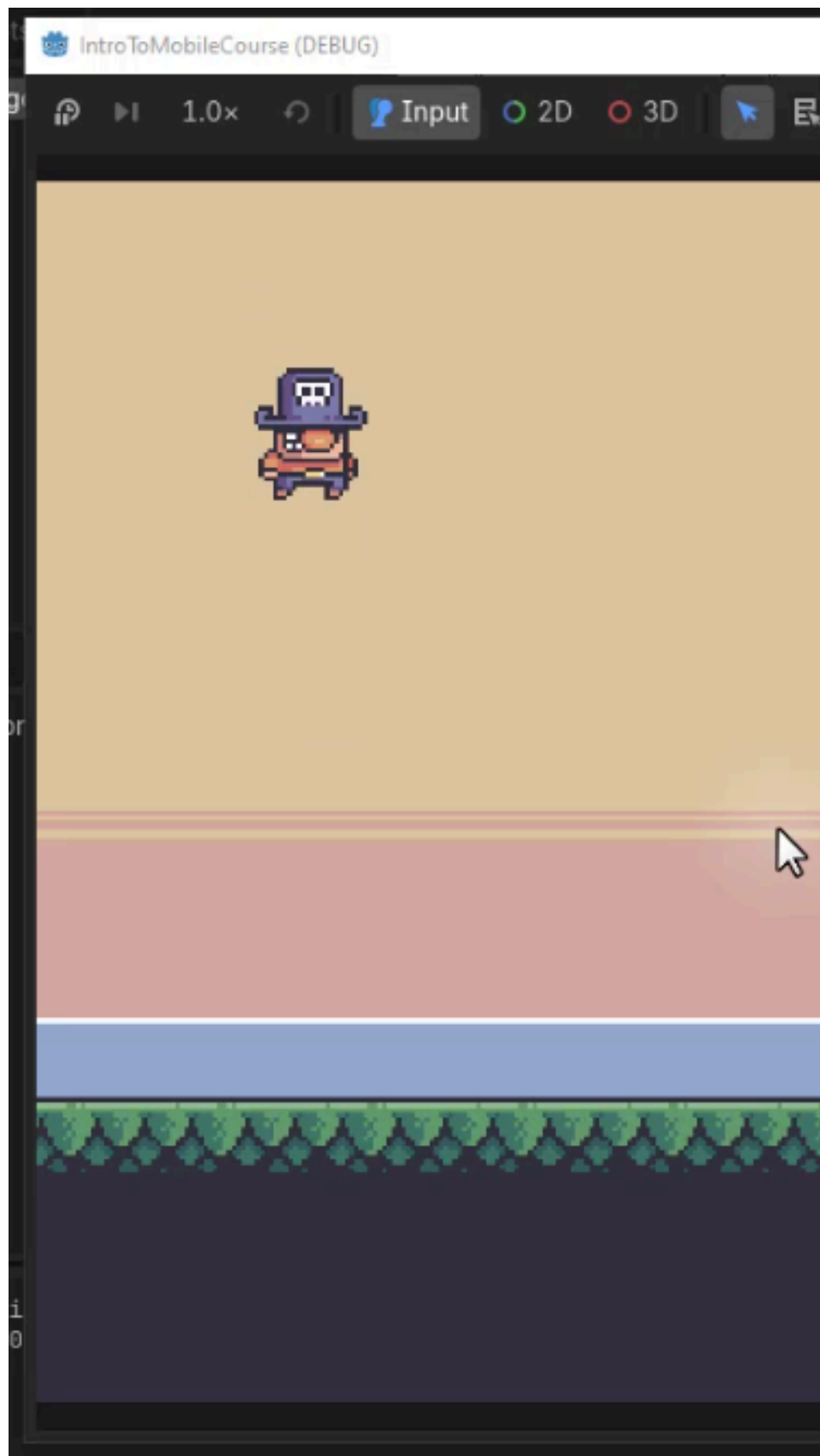
Godot offers two main per-frame callbacks. `_process(delta)` runs once per rendered frame, so its rate varies with the display's refresh rate. `_physics_process(delta)` runs at a fixed interval (60 times per second by default), which makes it ideal for anything involving physics, movement, or collision detection because the results stay consistent regardless of frame rate.

As a rule of thumb, put physics-related logic (gravity, velocity, `move_and_slide()`) in `_physics_process` and put visual-only updates (UI text, camera effects) in `_process`.

Learn more: [CharacterBody2D](https://docs.godotengine.org/en/4.6/classes/class_characterbody2d.html)
(https://docs.godotengine.org/en/4.6/classes/class_characterbody2d.html)

Refining the Jump

If you run the game now, you will notice you can jump repeatedly in mid-air.



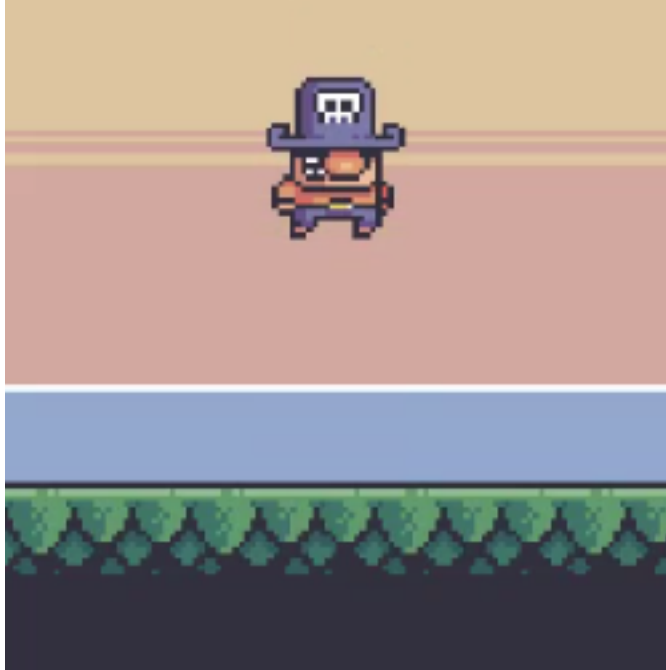
We want to restrict jumping to only when the player is on the ground. We can do this by adding a check for `is_on_floor()` to our jump condition.

```
func _physics_process(delta):  
    # Apply gravity only while airborne  
    if not is_on_floor():  
        velocity.y += gravity * delta  
  
    # Execute a jump if the player is on the ground  
    if jump_input and is_on_floor():  
        velocity.y = -jump_force  
        jump_input = false  
  
    move_and_slide()
```

This works, but introduces a new issue. If you click while in the air, the `jump_input` flag is set to true, and the player will jump immediately upon landing. To fix this, we will prevent the jump input from being registered if the player is not on the ground. Modify the `_try_jump` function as follows:

```
func _try_jump():  
    if not is_on_floor():  
        return  
  
    jump_input = true
```

Now, the player can only jump when on the ground, and any clicks while airborne are ignored.



With our player's basic movement set up, we can proceed to the next chapter where we will make the environment move.

World Logic: Movement and Obstacles

In this chapter, we will create the illusion that our player is running by making the environment move. Instead of moving the player or the camera, we will move the ground and other environmental elements backward. This approach keeps the player centered on the screen, simplifying many aspects of the game's logic, such as camera tracking and object spawning.

Once the environment is moving, we will implement the core challenge of the game: obstacles. We will create a system to spawn crates and trees at random intervals, requiring the player to jump to survive.

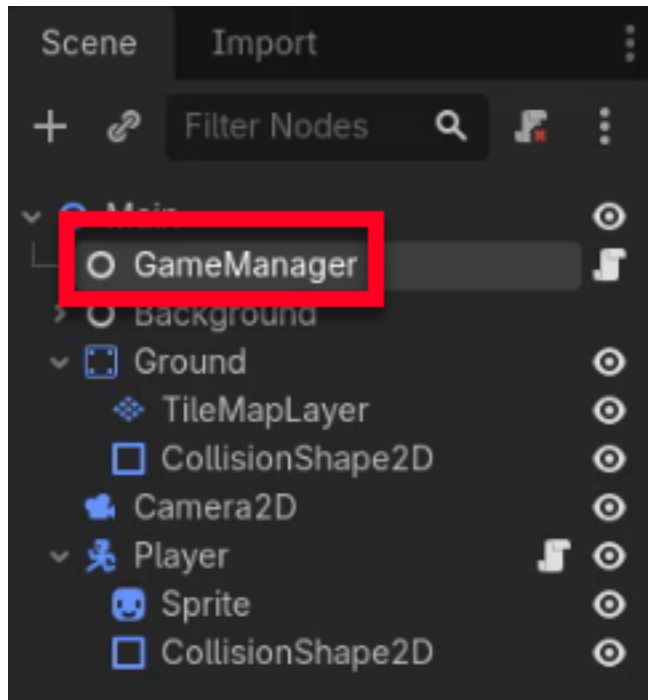
Moving the Ground

To simulate movement, we need to move the ground to the left. However, we also need to ensure that the ground, obstacles, and background all move at the same speed. To achieve this, we will create a central manager to handle the game's speed.

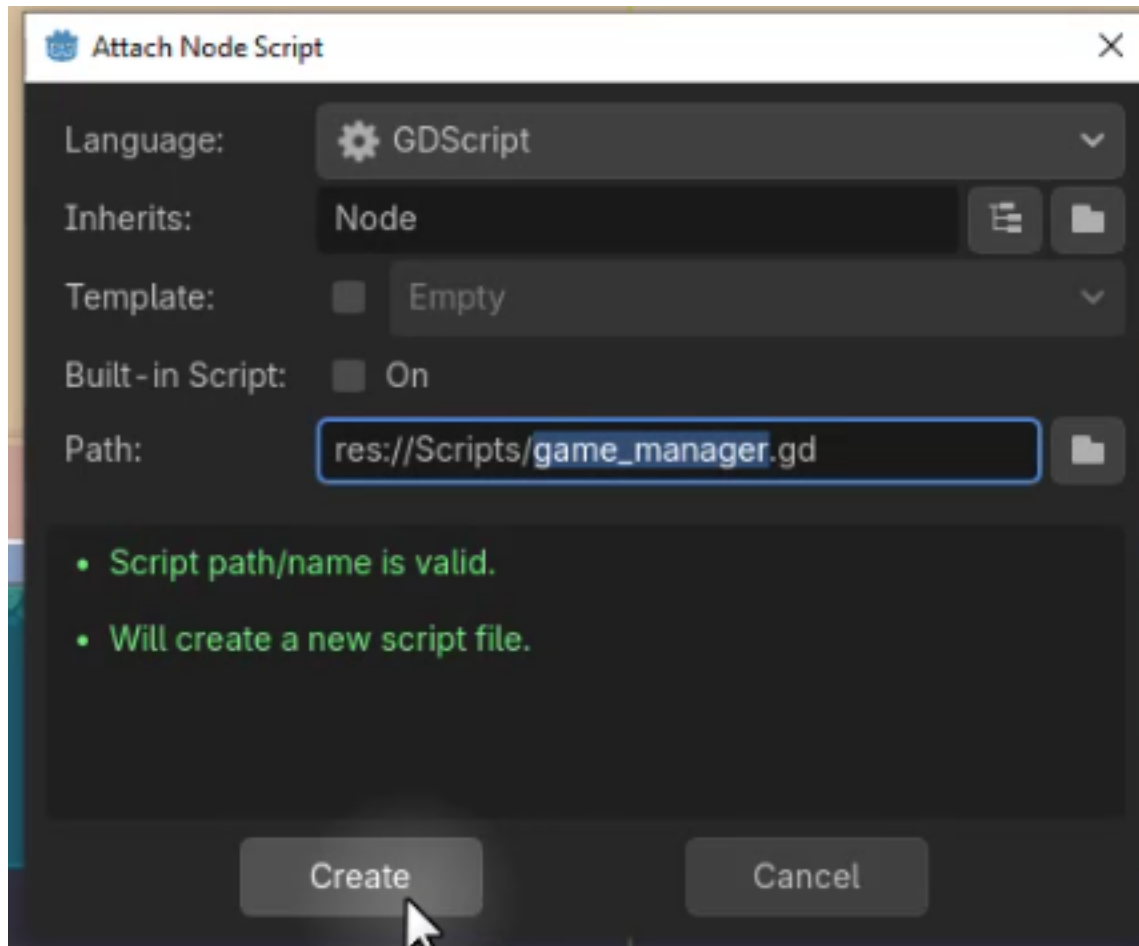
Creating the Game Manager

To ensure all moving objects in our scene move at the same, consistent speed, we need a centralized place to manage this value. As the game progresses, this speed will increase, making the game more challenging. We will create a `GameManager` node to handle this responsibility.

Let's start by creating a new Node in the Scene dock and renaming it to **GameManager**.



With the GameManager node selected, go to the Inspector and attach a new script named `game_manager.gd`.



Once created, we can open the script. We will start by giving it a `class_name`. This allows us to easily reference this script and its properties from other scripts without needing a direct node reference.

Godot Insight: `class_name` and global references

When you add `class_name GameManager` at the top of a script, Godot registers that name globally. Other scripts can then use it as a type hint (e.g., `var gm : GameManager`), which enables autocompletion and type checking in the editor. Without `class_name`, you would need to use `preload()` to reference the script file directly.

Learn more: [Node](https://docs.godotengine.org/en/4.6/classes/class_node.html)
(https://docs.godotengine.org/en/4.6/classes/class_node.html)

Next, we will define three variables: `current_speed` to hold the game's speed, `max_speed` to cap how fast the game can get, and `speed_increase_rate` to control how quickly the speed increases.

Finally, in the `_process` function, we will gradually increase `current_speed` over time, as long as it hasn't reached `max_speed`.

Your `game_manager.gd` script should look like this:

```
class_name GameManager
extends Node

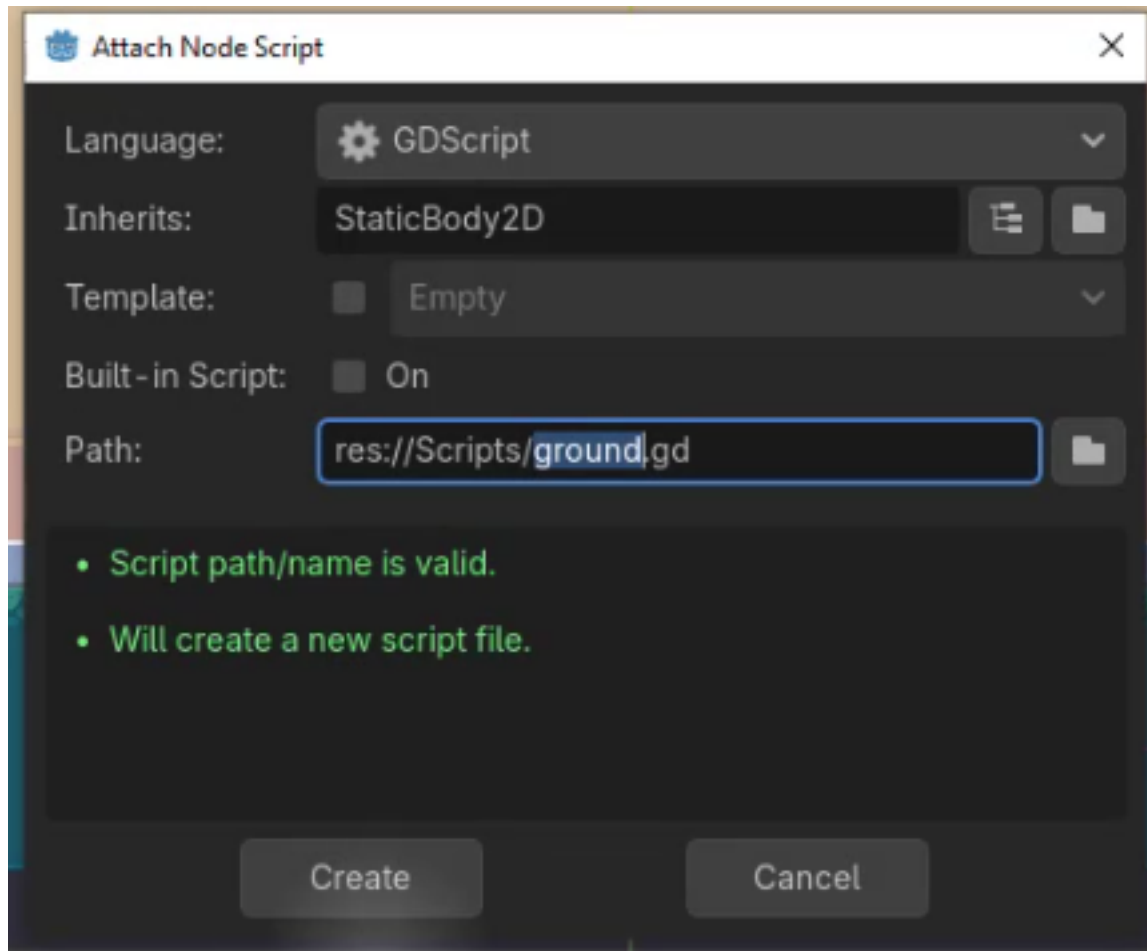
var current_speed : float = 200
var max_speed : float = 1500
var speed_increase_rate : float = 5

func _process(delta):
    if current_speed < max_speed:
        # Gradually ramp up difficulty over time
        current_speed += speed_increase_rate * delta
```

Scripting the Ground Movement

Now that we have a central manager for our game's speed, we can make the ground move. We will create a script for our ground object that reads the speed from the `GameManager` and updates its position accordingly.

To start, select the **Ground** node in the Scene dock and attach a new script to it. We will name this script `ground.gd`.



Inside this script, we first need a reference to our GameManager node. We can use an @onready variable for this. By specifying the type as GameManager, Godot provides helpful autocompletion thanks to the class_name we defined earlier.

We also need two @export variables, start and end. These will define the positions for our looping ground effect, which we will implement shortly in the Inspector.

In the _process function, we'll get the current_speed from the GameManager, multiply it by delta to ensure frame-rate independent movement, and then update the ground's global_position. We use Vector2.LEFT to move it to the left.

Add the following code to your ground.gd script:

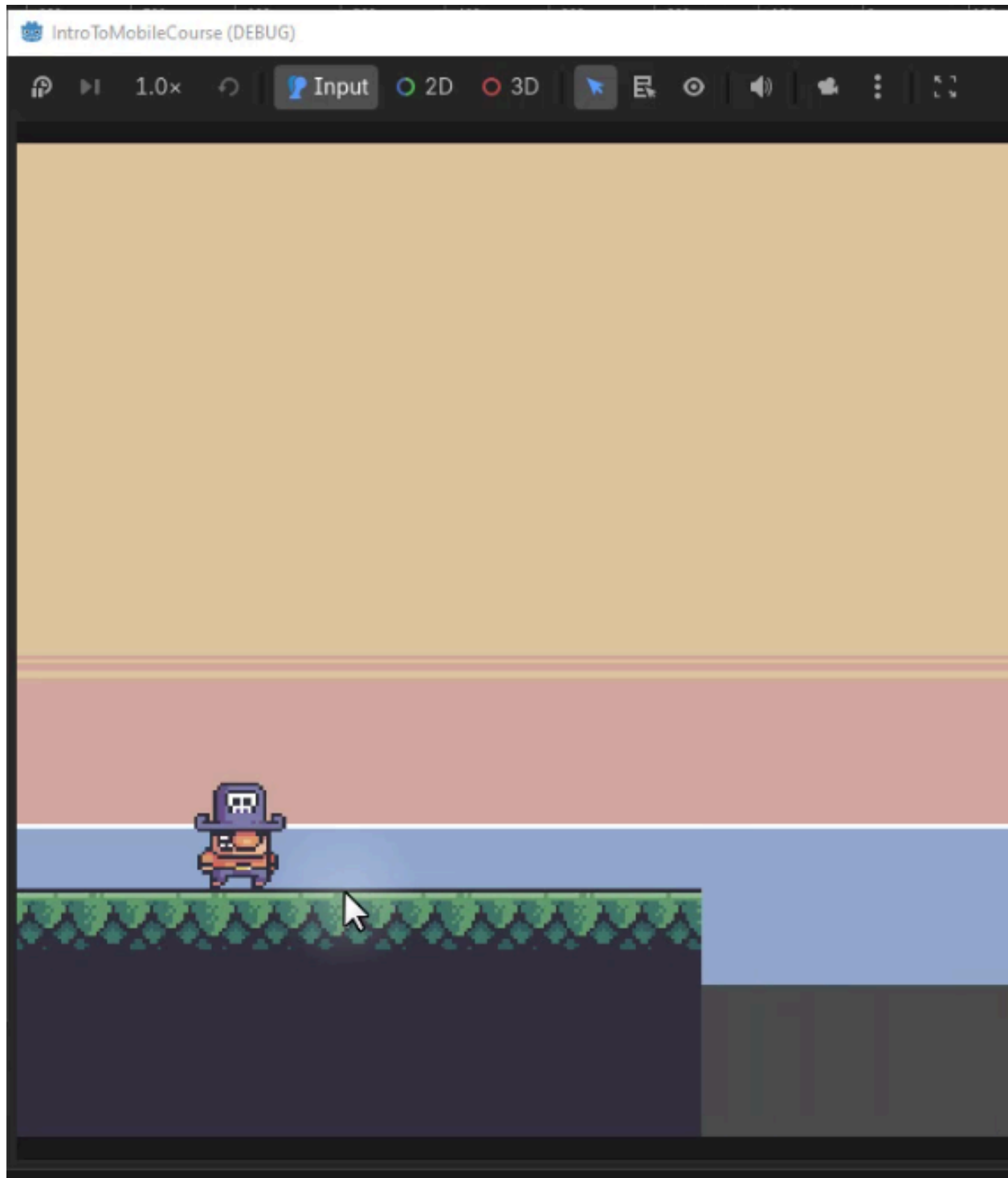
```
extends StaticBody2D

@onready var game_manager : GameManager = $"./GameManager"

# Loop boundaries set in the Inspector to match the tile pattern
@export var start : float
@export var end : float

func _process(delta):
    # Move the ground leftward at the current game speed
    var speed : float = game_manager.current_speed * delta
    global_position += Vector2.LEFT * speed
```

If you run the game now, you will see the ground start to move to the left, and its speed will slowly increase. However, the player will eventually fall off the edge.



Creating an Infinite Loop

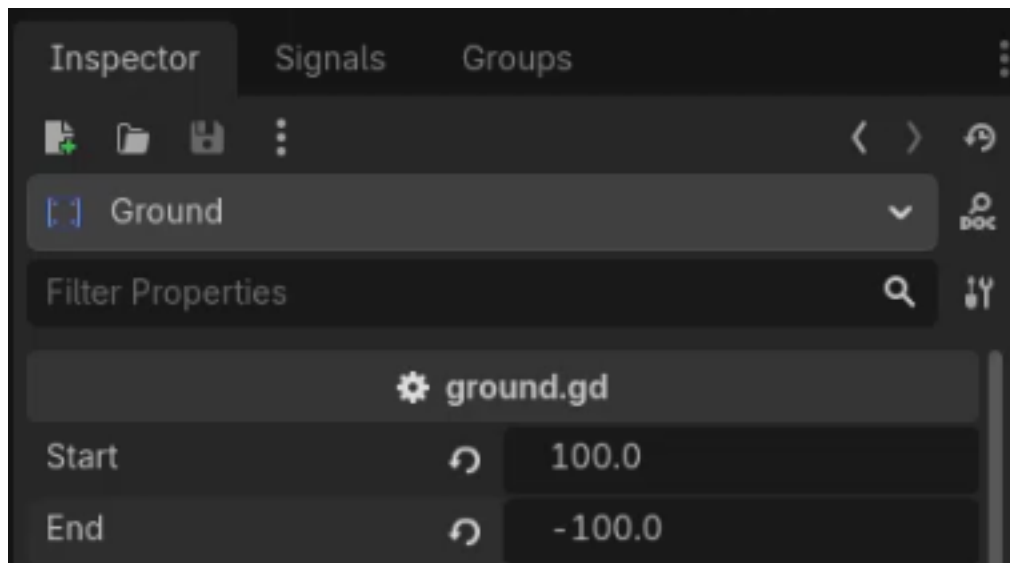
To make the ground scroll infinitely, we will teleport it forward once it moves past a certain point. This is where our start and end variables come

into play. When the ground's x-position is less than the end value, we will reset its x-position to the start value.

Let's update the `_process` function in `ground.gd`:

```
func _process(delta):  
    # Move the ground leftward at the current game speed  
    var speed : float = game_manager.current_speed * delta  
    global_position += Vector2.LEFT * speed  
    # Teleport back to create a seamless infinite loop  
    if global_position.x < end:  
        global_position.x = start
```

Now, select the **Ground** node and look at the Inspector. You will see the “Start” and “End” properties we exported. Let's try setting **Start** to 100 and **End** to -100.



Seamless Tiling

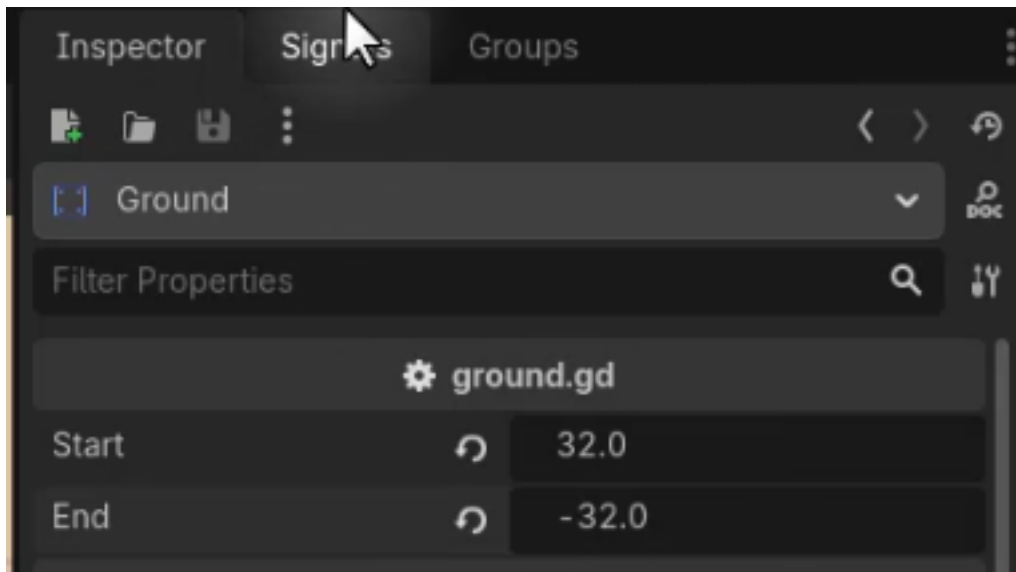
If you run the game with the values above, you might notice a slight visual “jump” or “jitter” when the ground teleports. This happens because the tile pattern at the end position does not perfectly match the pattern at the start position.

To fix this, we need to choose start and end values that align with our tile pattern. Our ground tiles are built from four 16×16 sprites, but for a seamless

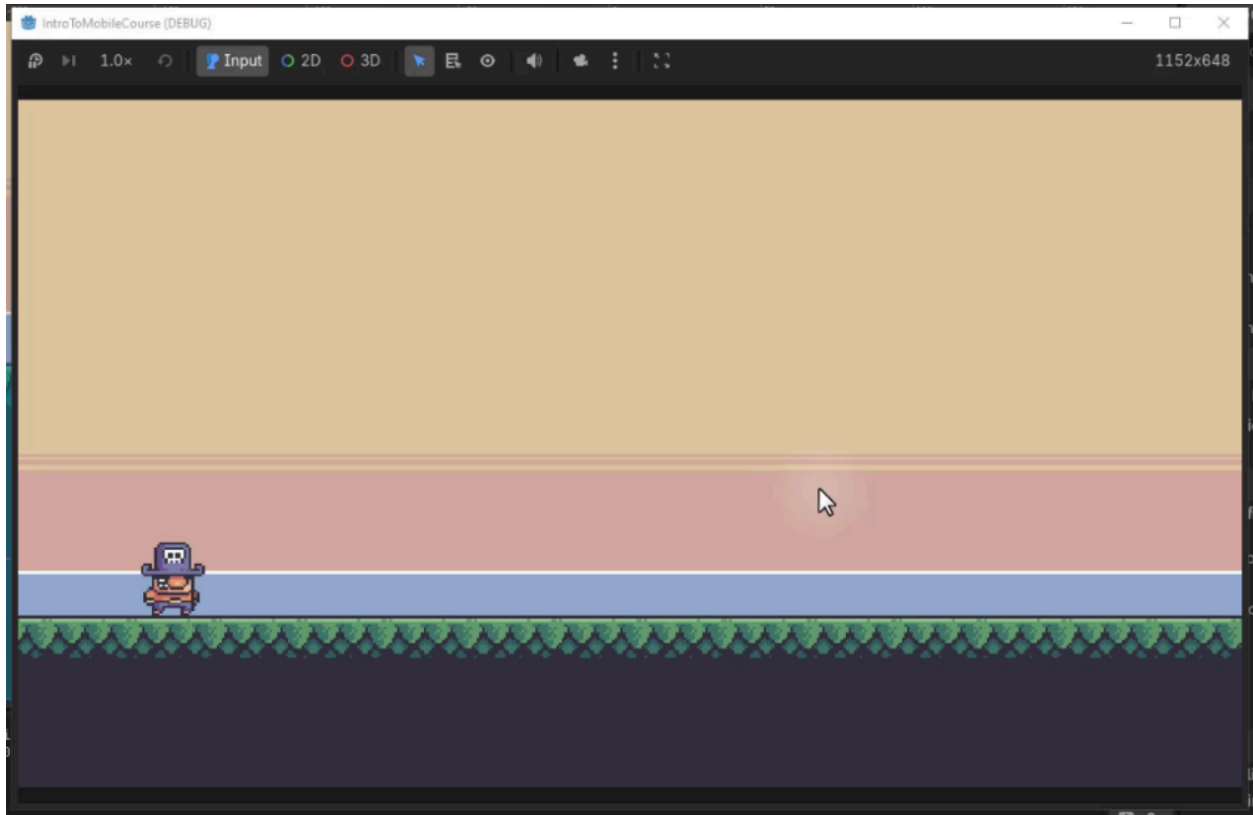
loop, we can use a distance that is a multiple of our tile size. A value of 32 works well in this case.

Select the **Ground** node again and update the values in the Inspector:

- Set **Start** to 32.
- Set **End** to -32.



Now, when you run the game, the ground will scroll backward smoothly and infinitely, with no noticeable seam when it teleports. The player appears to be running, but is in fact standing still while the world moves beneath them.



Creating Obstacles

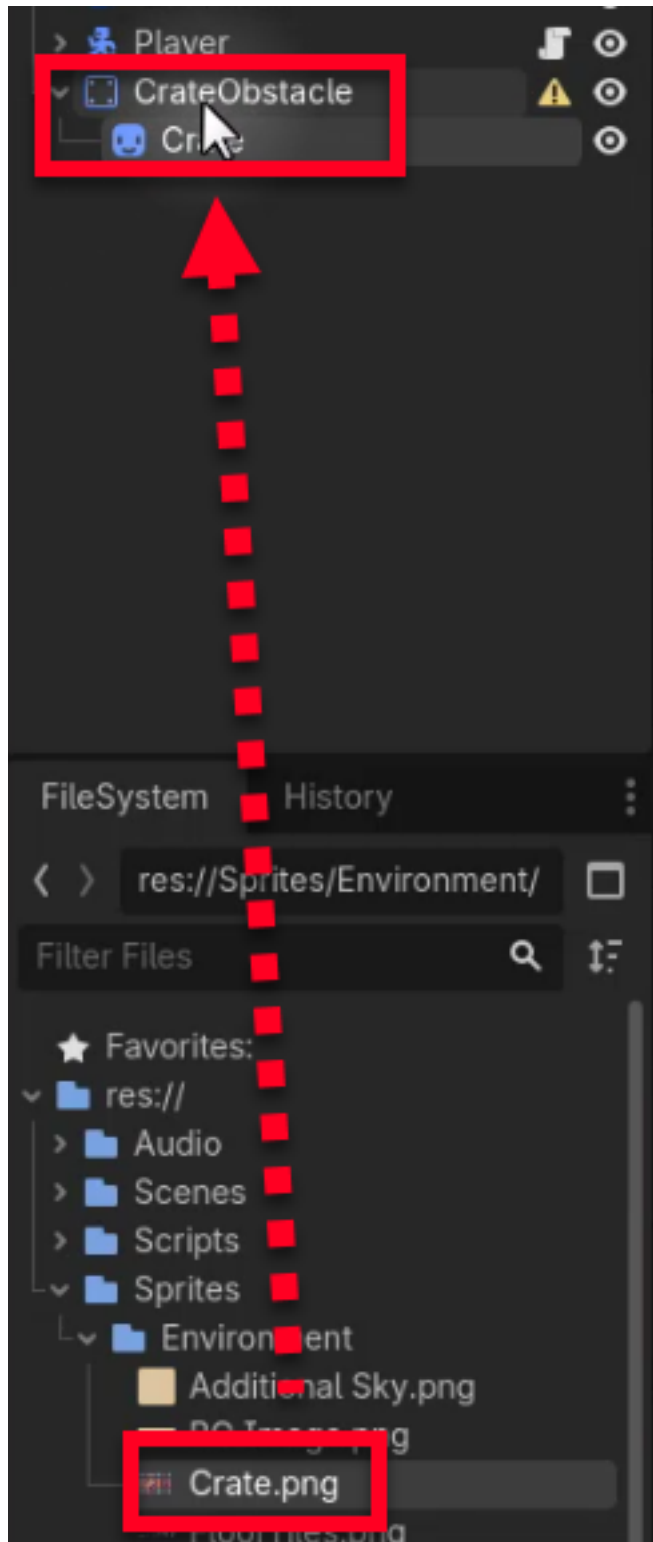
With the ground moving, we can now add obstacles for the player to avoid. We will begin by creating the obstacle object itself, then build a spawner to automatically generate these obstacles at random intervals, creating an endless challenge for the player.

Creating the Obstacle Scene

Before we can spawn obstacles, we need to create one. Our obstacles will be `StaticBody2D` nodes that move across the screen since they don't need to be affected by physics. Let's create our first obstacle: a crate.

In the Scene dock, add a `StaticBody2D` child node to your main scene and name it `CrateObstacle`.

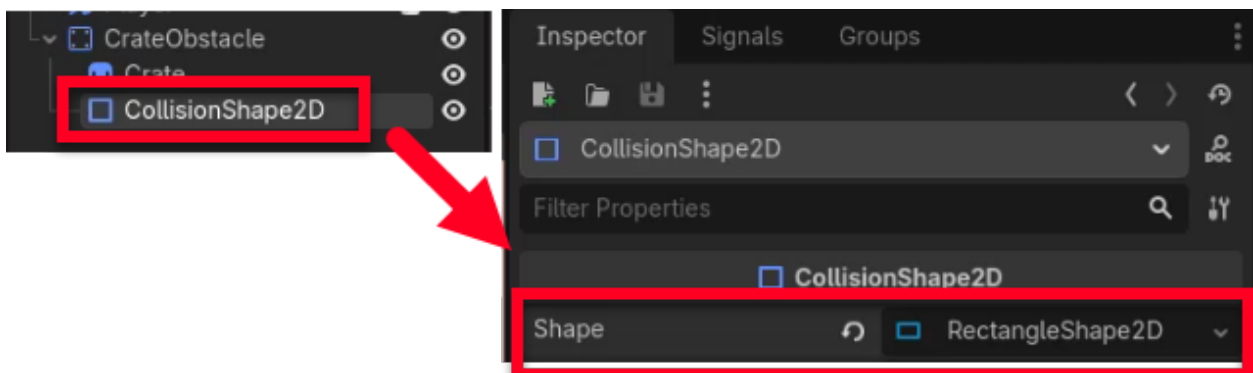
Find the crate sprite in your file system (under `Sprites/Environment`) and drag it onto the `CrateObstacle` node to add it as a child `Sprite2D`. Rename it to "Crate" in this case.



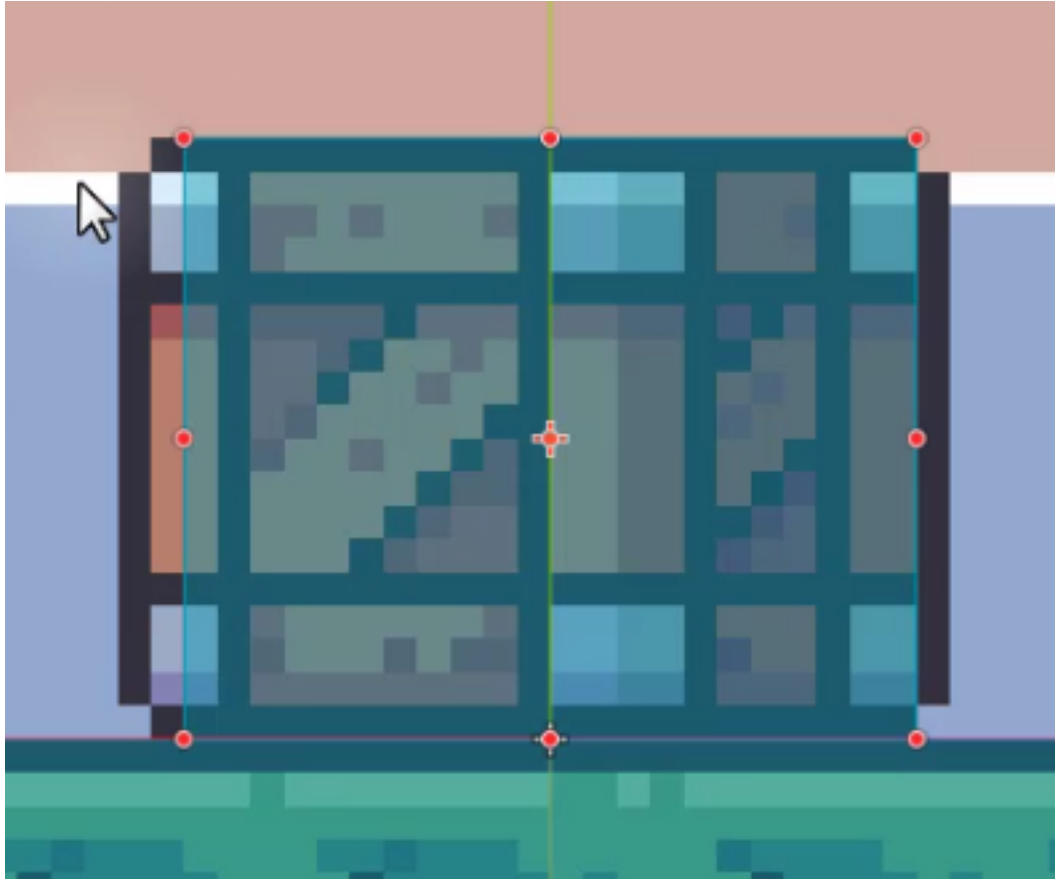
Position the sprite so its bottom edge rests on the node's origin point. This alignment is important because the spawner will place the obstacle at ground level.



Next, add a `CollisionShape2D` node as a child of `CrateObstacle`. In the Inspector for the `CollisionShape2D`, assign a new `RectangleShape2D` to its *Shape* property.



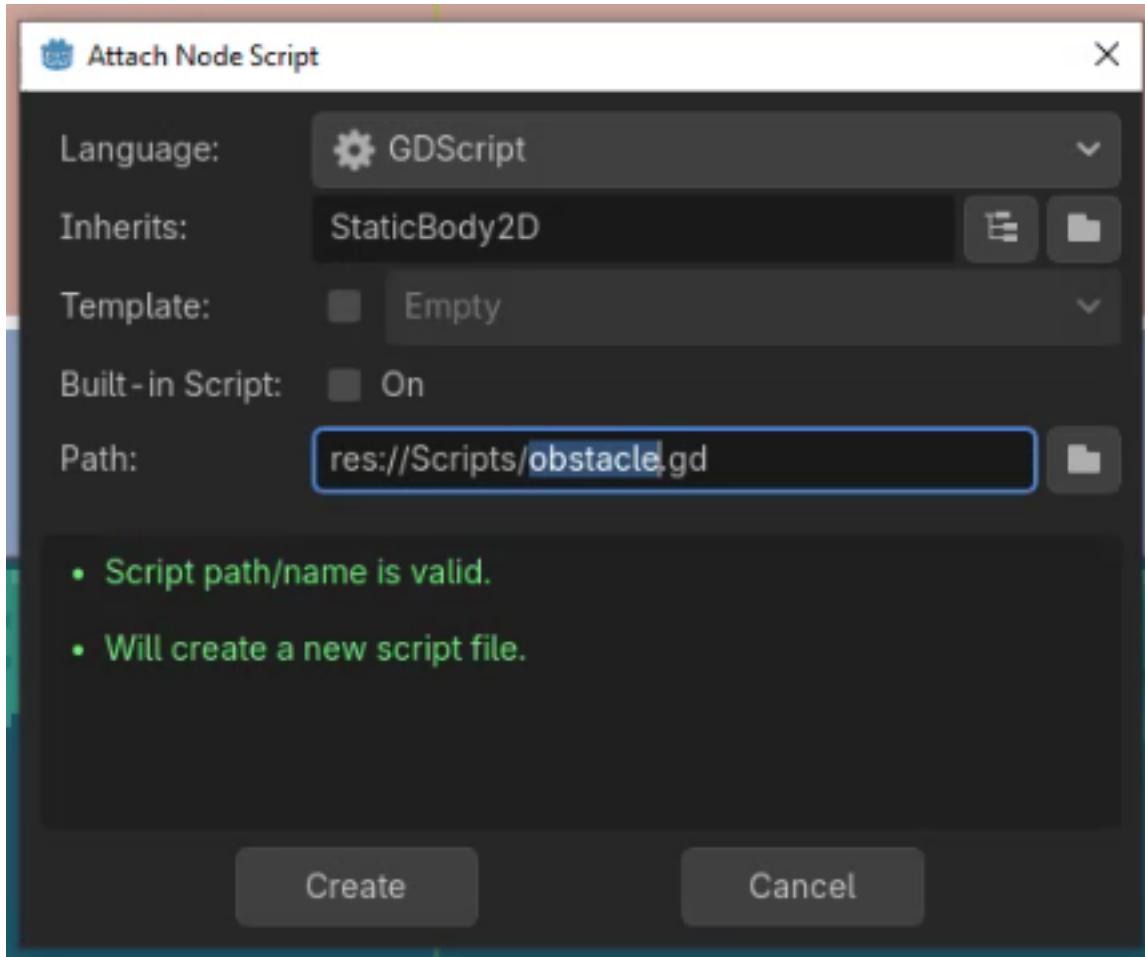
Then, resize the rectangle to roughly match the dimensions of the crate sprite. Remember it doesn't have to be exact and, depending on your preference, the shape can be smaller to give the player more room for error.



Scripting the Obstacle's Movement

Now, we need to make the obstacle move. We will attach a script that moves it to the left based on the game's current speed, which is managed by our `GameManager`.

To begin, select the `CrateObstacle` node and attach a new script. Save it as `obstacle.gd` in your `Scripts` folder. We will call it just `obstacle` in this case because we will be reusing it again later when we create a second kind of obstacle.



Inside the script, we first need a reference to the GameManager. Then, in the `_process` function, we'll update the obstacle's position every frame.

Add the following code to your `obstacle.gd` script:

```
extends StaticBody2D

@onready var game_manager : GameManager = $"../GameManager"

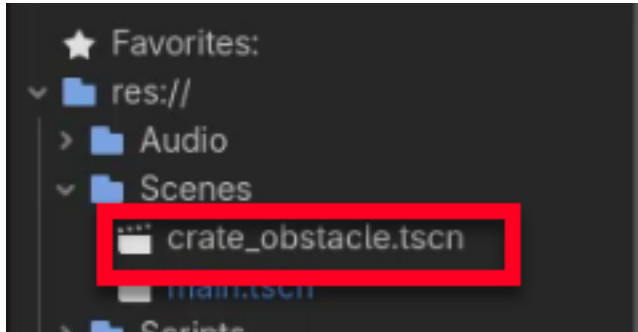
func _process(delta):
    var speed : float = game_manager.current_speed * delta
    global_position.x -= speed
```

This code is very similar to our ground script. It fetches the current speed from the GameManager, scales it by delta time, and subtracts it from the obstacle's horizontal position, causing it to move left.

Creating the Obstacle Spawner

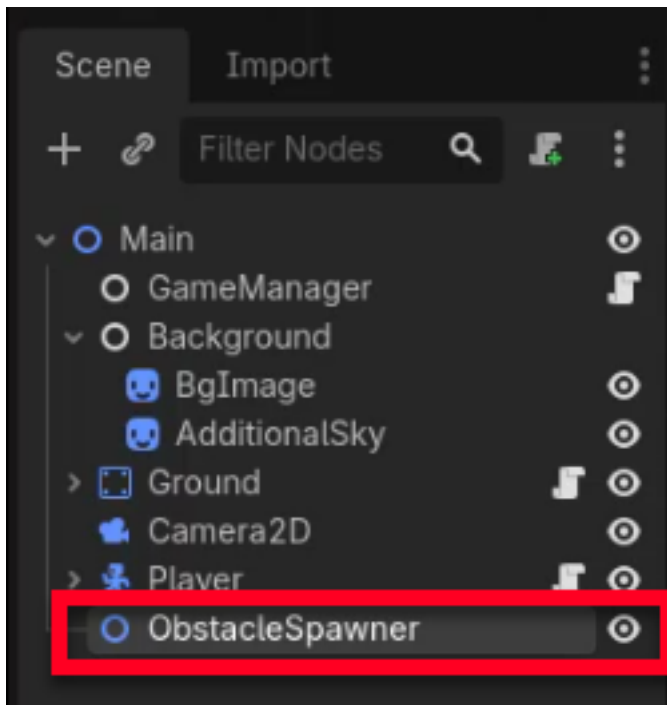
Manually placing obstacles is not practical for an endless runner. We need a spawner to create them automatically. First, let's save our configured obstacle as a reusable scene.

In the Scene dock, right-click on the `CrateObstacle` node and select "Save Branch as Scene". Save it as `crate_obstacle.tscn` in your Scenes folder.

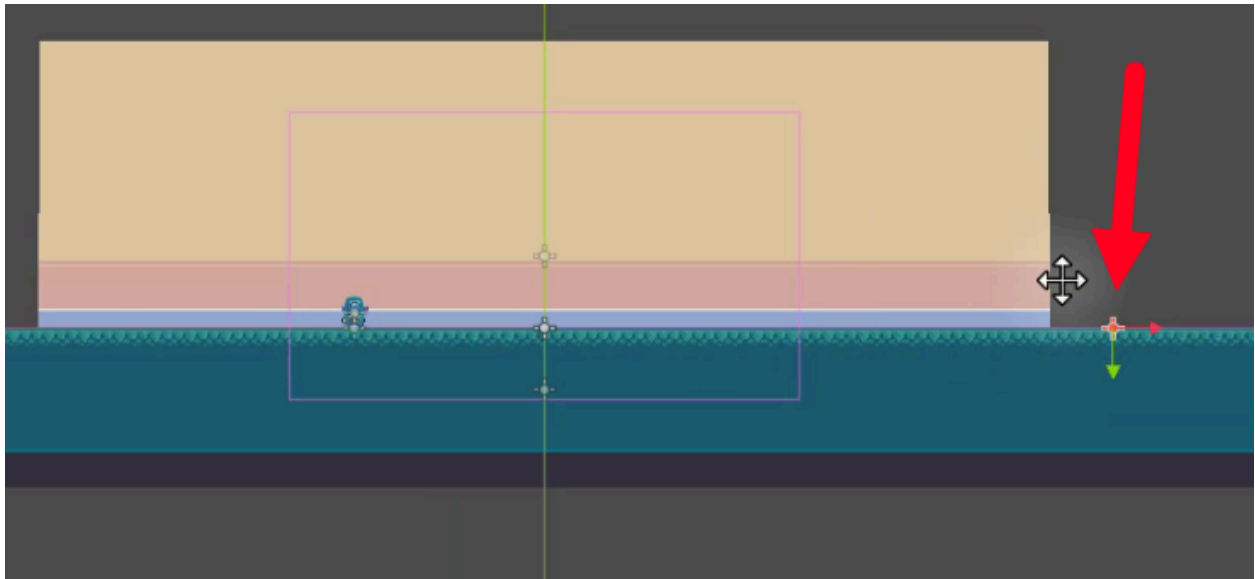


Once saved, you can delete the `CrateObstacle` instance from your main scene since we'll control the instantiation via code now.

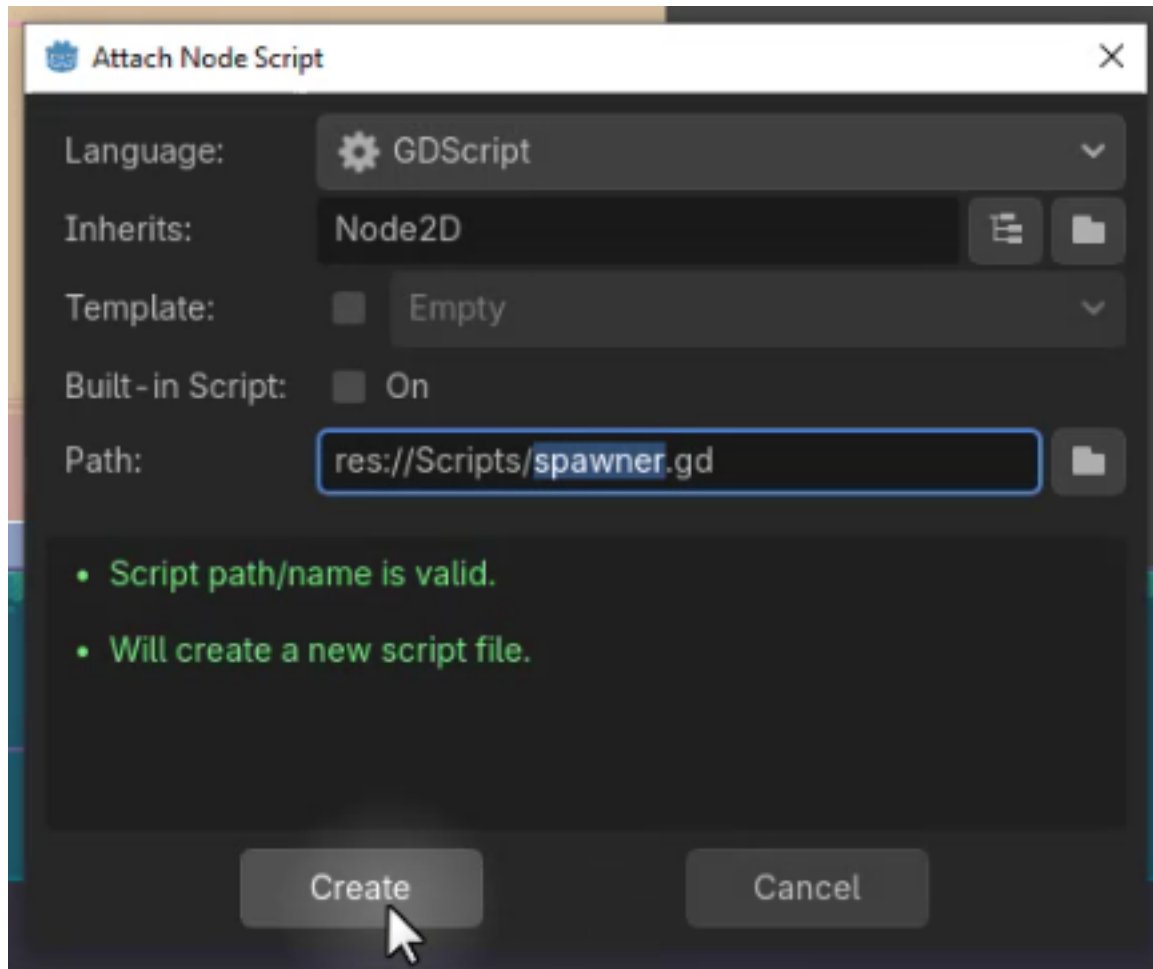
Now, create a new Node2D in your main scene and name it `ObstacleSpawner`. This node will serve as the spawn point.



Position the `ObstacleSpawner` just off-screen to the right. This is where new obstacles will appear.



Attach a new script to `ObstacleSpawner` and name it `spawner.gd`. Like the obstacle, we are giving it a generic name because we will later reuse this spawner for other objects, like clouds.



Scripting the Spawner Logic

The spawner script will instantiate our obstacle scene at random intervals. It needs to know what to spawn and the distance range between spawns.

Here is the initial code (which we will explain below) for `spawner.gd`:

```
extends Node2D

@export var scene_to_spawn : PackedScene

@export var min_spawn_range : int
@export var max_spawn_range : int

var next_spawn_range : float
var previous_object : Node2D

func _process(delta):
```

```

# Spawn the first object immediately, then wait for distance
if not previous_object:
    _spawn()
    return

# Spawn again once the last object has moved far enough away
if global_position.x - previous_object.global_position.x >
next_spawn_range:
    _spawn()

func _spawn ():
    var obj : Node2D = scene_to_spawn.instantiate()
    add_child(obj)

    obj.global_position = global_position

# Randomize the gap before the next spawn
next_spawn_range = randi_range(min_spawn_range, max_spawn_range)

previous_object = obj

```

Let's break down this script:

- `scene_to_spawn`: An exported `PackedScene` variable. We will drag our `crate_obstacle.tscn` file here in the Inspector.
- `min_spawn_range` and `max_spawn_range`: These exported integers define the minimum and maximum distance (in pixels) between spawned obstacles, adding variation.
- `next_spawn_range`: Stores the randomly chosen distance for the next spawn.
- `previous_object`: A reference to the last object that was spawned.
- `_process()`: This function checks if an object has been spawned yet. If not, it spawns one immediately. Otherwise, it continuously checks the distance between the spawner and the last spawned object. If that distance exceeds `next_spawn_range`, it calls `_spawn()` again.
- `_spawn()`: This function instantiates the scene, adds it as a child to the spawner, sets its initial position to the spawner's position, calculates the next random spawn distance, and updates the `previous_object` reference.

Godot Insight: PackedScene and scene instantiation

A `PackedScene` is a resource that holds a saved scene tree. Think of it as a blueprint: calling `instantiate()` creates a fresh copy of that scene as live nodes you can add to the game. This pattern is central to Godot's design and is how you spawn enemies, projectiles, pickups, and any other runtime objects.

The typical workflow is: export a `PackedScene` variable, assign a `.tscn` file in the Inspector, then call `instantiate()` followed by `add_child()` to bring it into the scene tree.

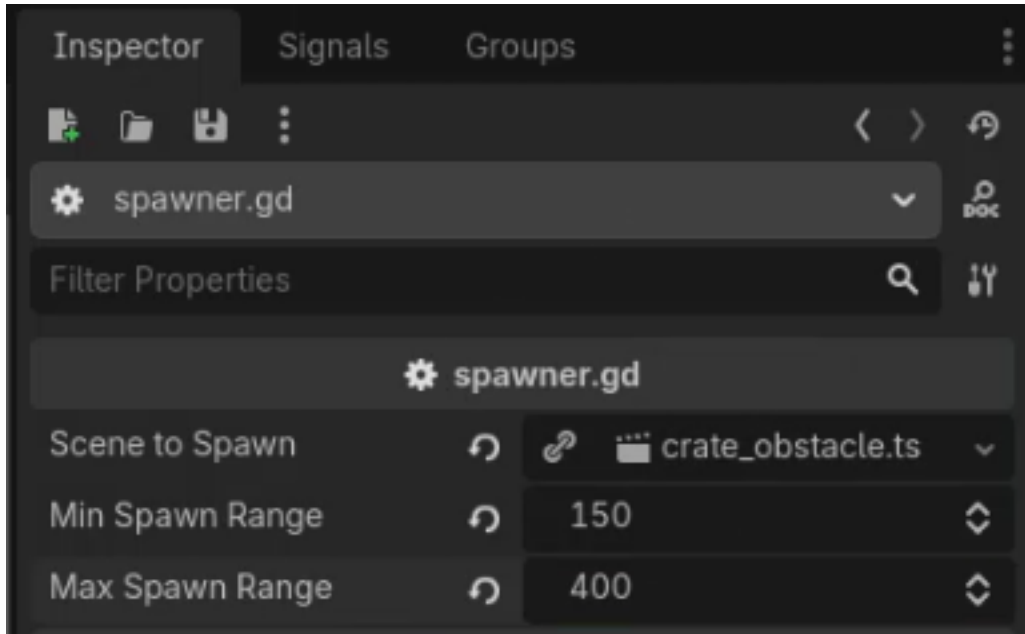
Learn more: [PackedScene](https://docs.godotengine.org/en/4.6/classes/class_packedscene.html)

(https://docs.godotengine.org/en/4.6/classes/class_packedscene.html)

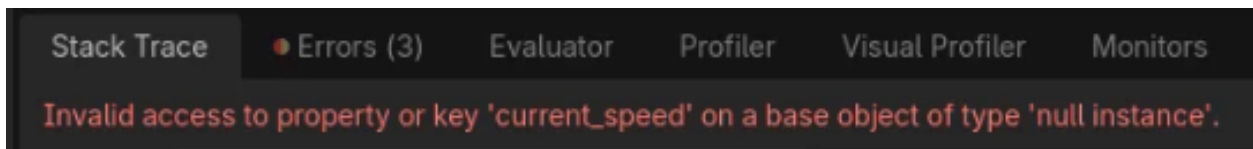
Implementation and Troubleshooting

Now we need to wire up the spawner in the editor so it knows which obstacle scene to create and how far apart to space them. These exported properties keep the spawner flexible: you can tweak timing and swap obstacle types without touching the script.

First, select the `ObstacleSpawner` node and, in the Inspector, drag `crate_obstacle.tscn` from the `FileSystem` dock to the *Scene To Spawn* property. Then, set the *Min Spawn Range* to 150 and the *Max Spawn Range* to 400.



If you run the game now, you will encounter an error. The game crashes because the spawned obstacle cannot find the GameManager.

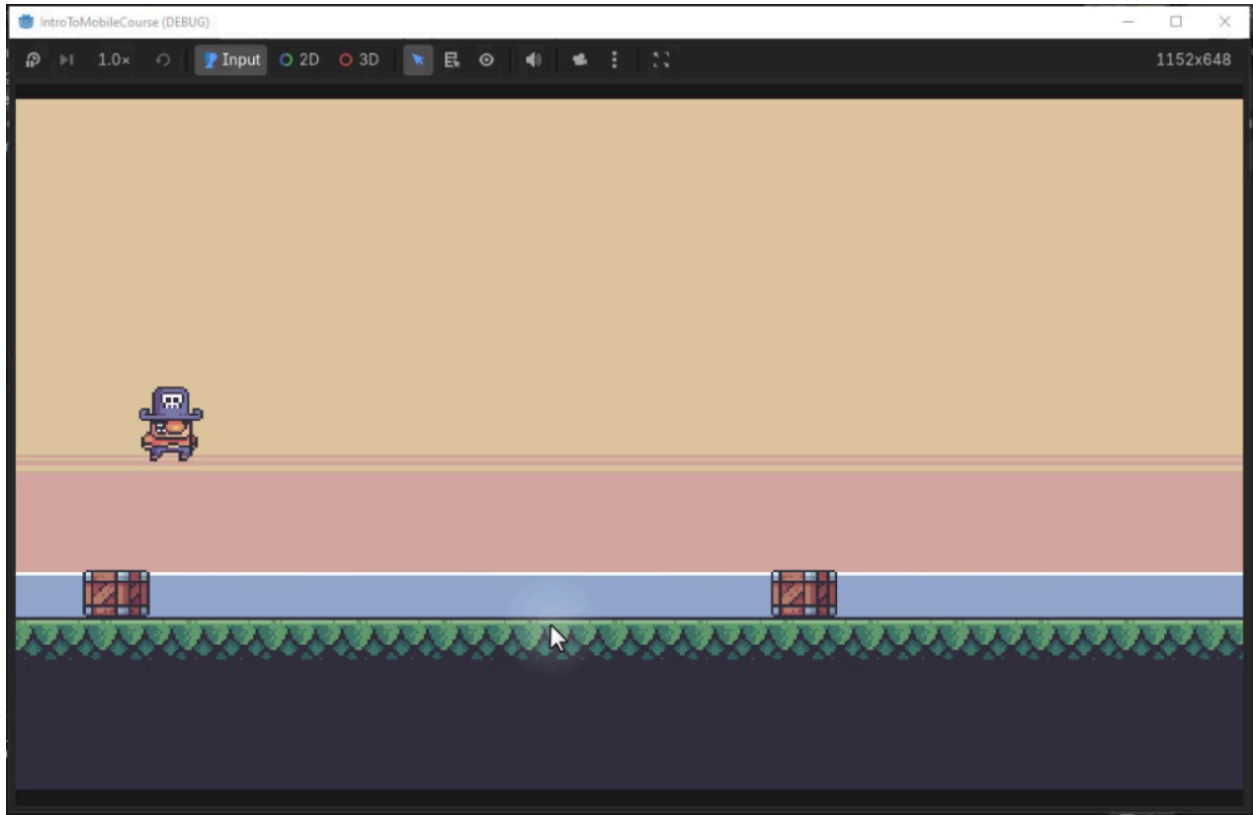


The path `$" ./GameManager"` is a relative path. When the obstacle is instantiated as a child of the spawner, this path no longer points to the correct node. To fix this, we must use an absolute path from the root of the scene tree. The root of our main running scene is named “Main”.

Modify the `game_manager` variable line in `obstacle.gd`:

```
# Absolute path because spawned objects are children of the spawner, not Main
@onready var game_manager : GameManager =
    get_node("/root/Main/GameManager")
```

Now, when you run the game, the spawner will correctly create crate obstacles that move towards the player, with randomized spacing between them. You can jump over them, and they will continue to spawn indefinitely.



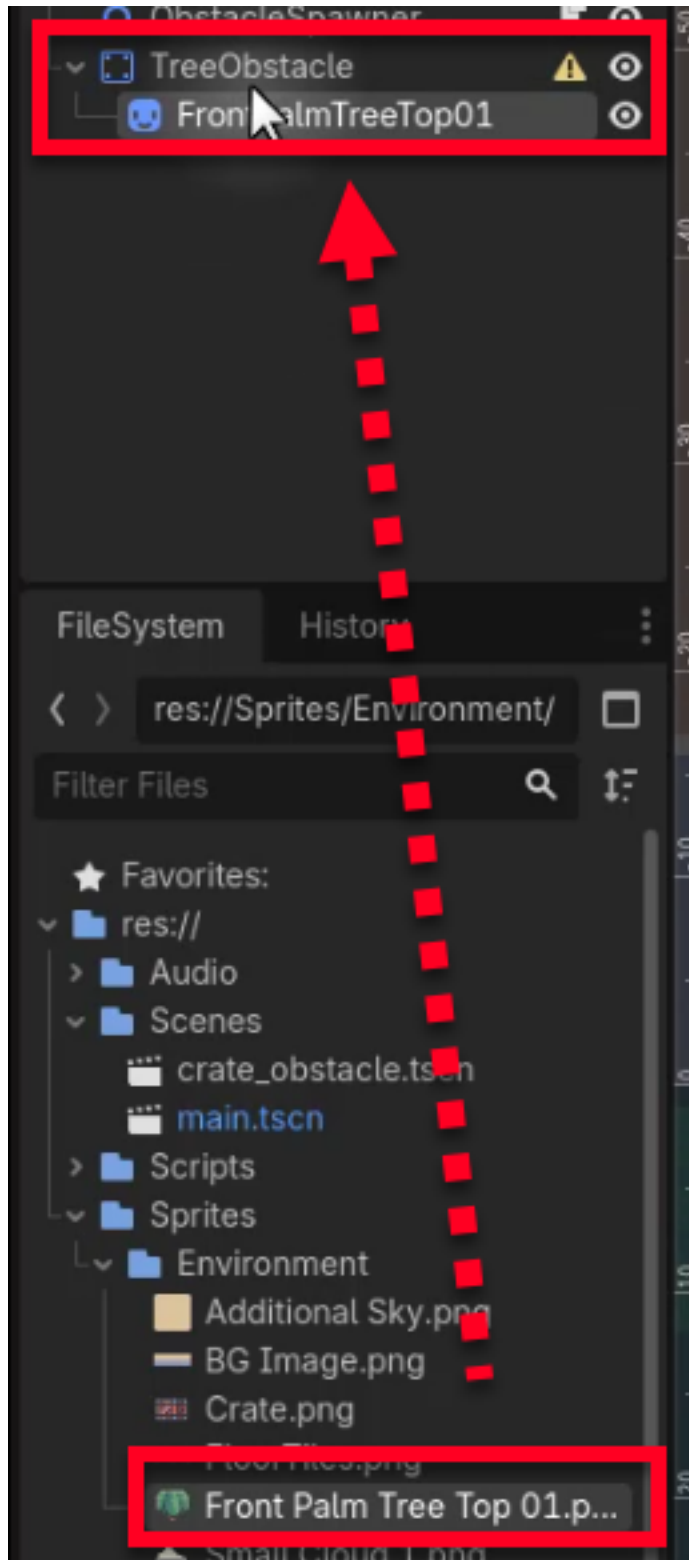
Randomizing Obstacles

To make the gameplay more engaging and less predictable, we will introduce a new type of obstacle and modify our spawner to randomly select which obstacle to create.

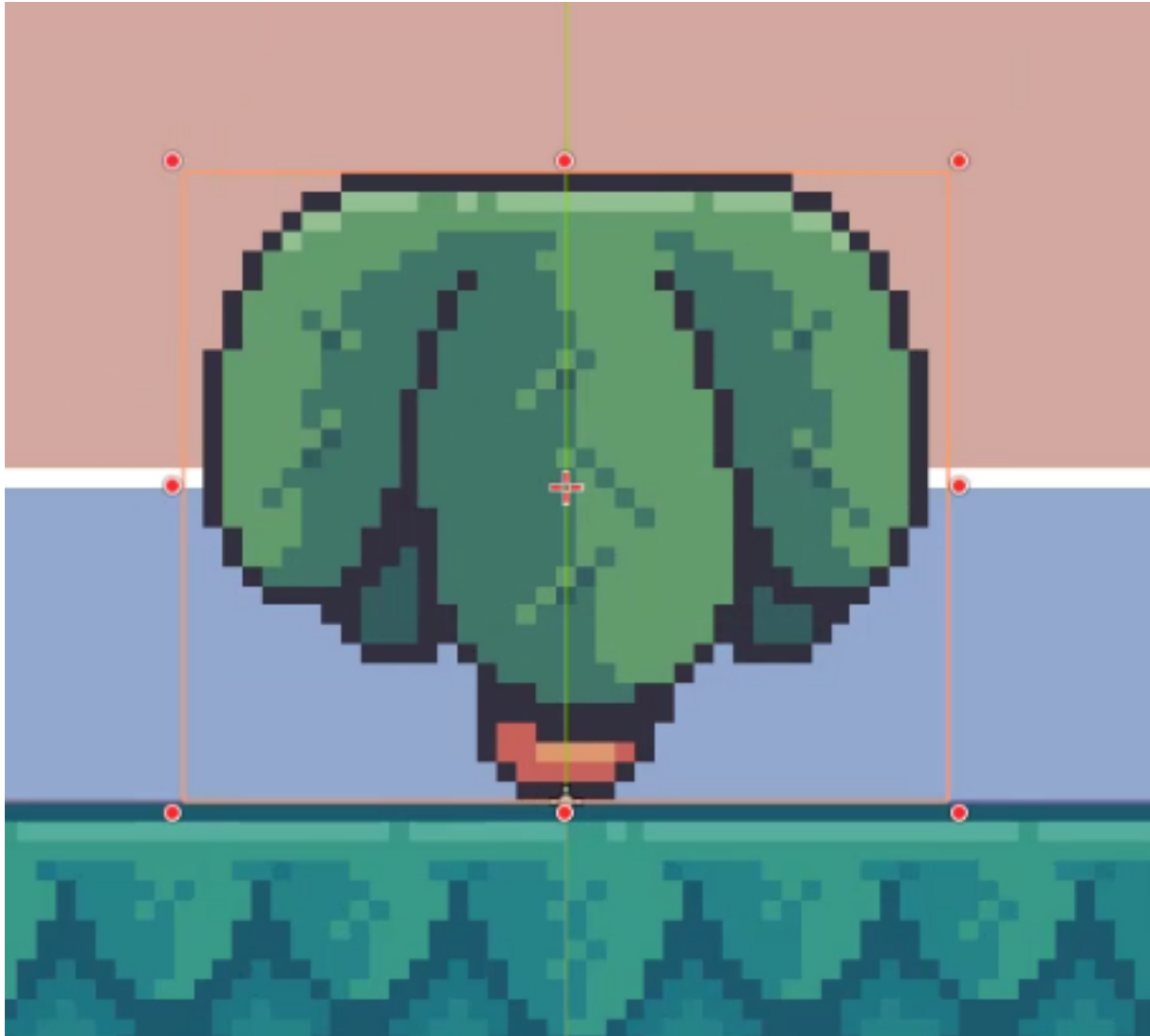
Creating a New Tree Obstacle

To start, we will create a new tree obstacle. This process is similar to how we created the crate obstacle, but with a different sprite and collision shape.

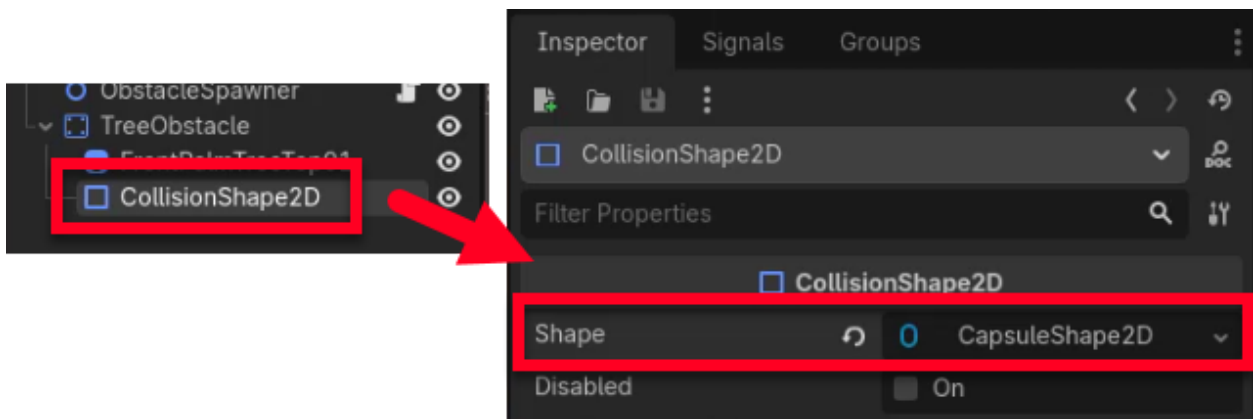
Create a new scene with a `StaticBody2D` root node and rename it to `TreeObstacle`. After, add a `Sprite2D` node as a child of `TreeObstacle` and assign the palm tree texture to it.



With the visual in place, position the sprite appropriately so it is on the ground.



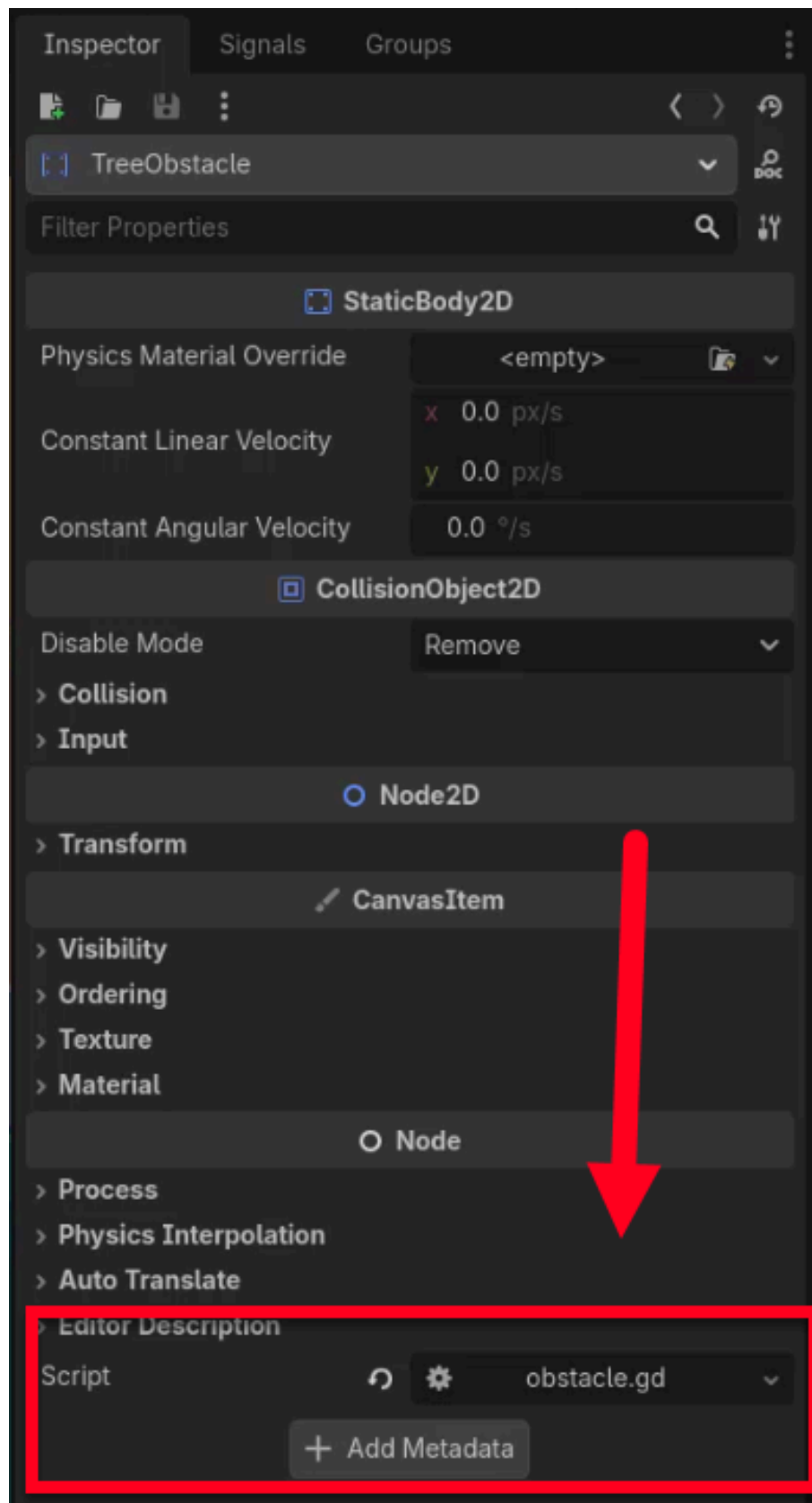
Per usual, for collisions to work, add a `CollisionShape2D` node as another child. In the Inspector, set its shape to a new `CapsuleShape2D`.



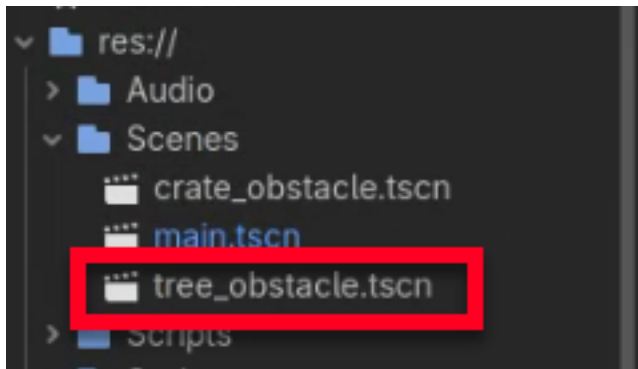
Adjust the capsule's size and position to roughly match the tree trunk. With trees, you usually want to focus more on the trunk than the leaves for collision detection to make the gameplay satisfying.



Select the root `TreeObstacle` node. In the Inspector, attach the existing `obstacle.gd` script using the “Quick Load” option.



Finally, save this new obstacle as `tree_obstacle.tscn`. You can now delete any temporary instances from your main scene.



Spawning Random Obstacles

Currently, our spawner is configured to spawn only one type of obstacle. To handle multiple types, we will modify the spawner script to use an array of scenes instead of a single scene variable.

Let's open the spawner .gd script. First, we will change the exported variable from a single `PackedScene` to an array of them. Replace the old variable with the following:

```
@export var scenes_to_spawn : Array[PackedScene]
```

Next, we need to update the `_spawn` function. Instead of instantiating the same scene every time, we will now randomly select one from our new `scenes_to_spawn` array. We can achieve this by getting a random index within the bounds of the array's size.

Inside the `_spawn` function, add the following line to pick a random scene:

```
# Instantiate a random scene from the array and place it at the spawner
func _spawn ():
    var scene_to_spawn : PackedScene = scenes_to_spawn[randi() %
len(scenes_to_spawn)]
    var obj : Node2D = scene_to_spawn.instantiate()
    add_child(obj)

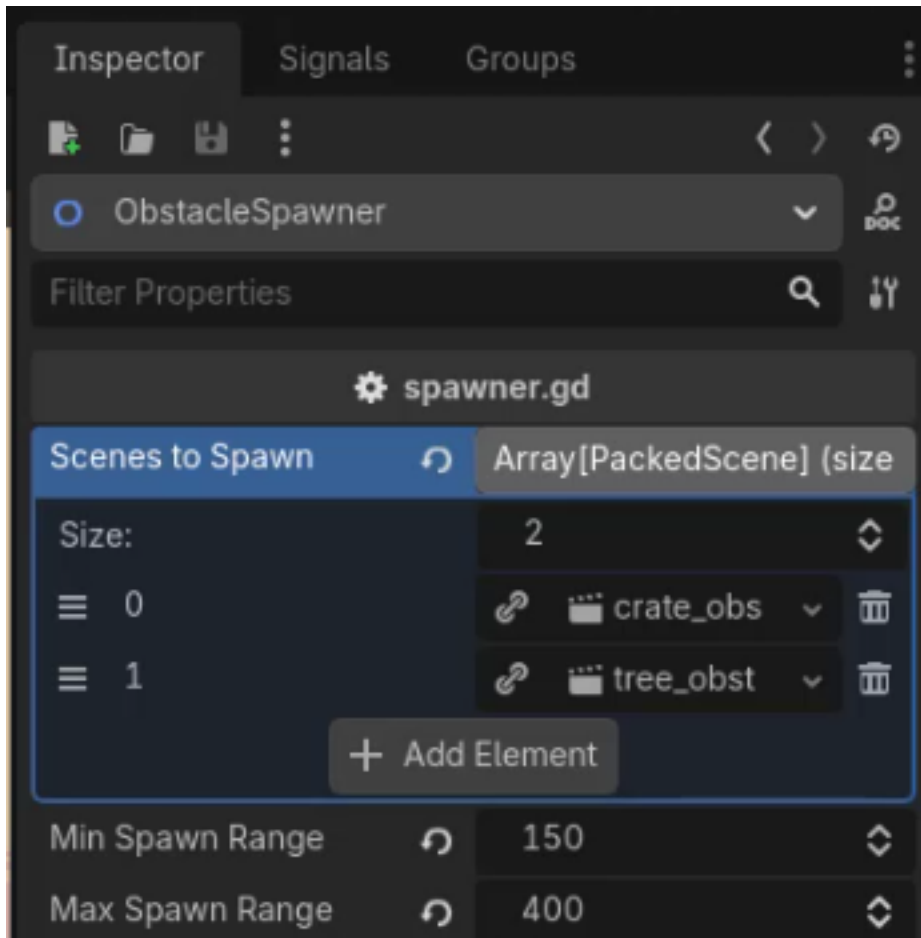
# ... rest of the function remains the same
```

The expression `randi() % len(scenes_to_spawn)` generates a random integer between 0 and the number of elements in the array minus one. This gives us a valid random index to select an element from the `scenes_to_spawn` array.

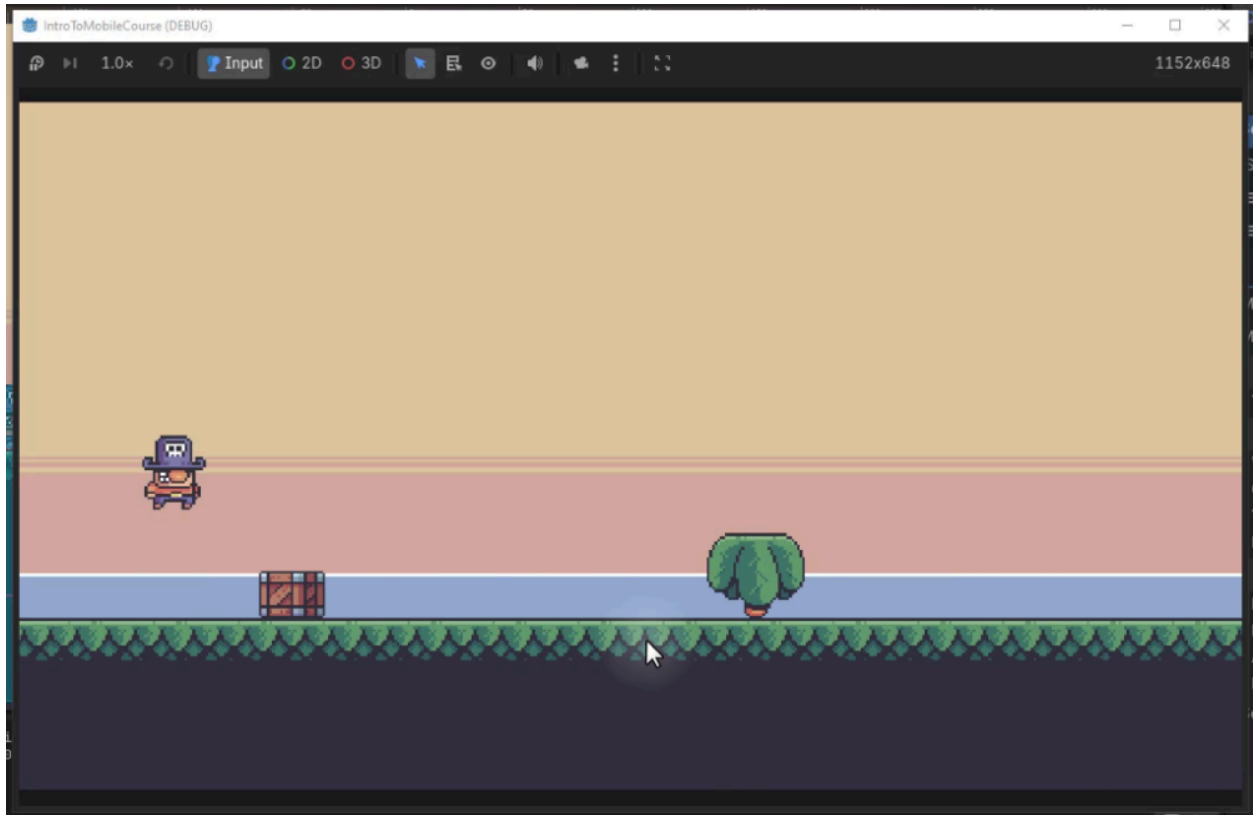
Configuring the Spawner in the Editor

With the script updated, the final step is to configure the spawner in the Godot editor.

1. Return to your main scene and select the `ObstacleSpawner` node.
2. In the Inspector, you will now see the “Scenes To Spawn” property, which is an array.
3. Set the size of the array to 2.
4. Drag `crate_obstacle.tscn` into the first element slot (index 0) and `tree_obstacle.tscn` into the second slot (index 1).



Now, if you run the game, you will see that either a crate or a tree is spawned randomly each time, making the gameplay more dynamic. You are encouraged to create and add even more obstacle variations to this array to further enhance the player's experience.



In the next chapter, we will turn away from our gameplay for a moment to give our game some visual polish. We will improve our background by adding clouds with a parallax effect. Then, we will set our player up with animations so the player isn't just gliding.

Visual Polish: Atmosphere and Animation

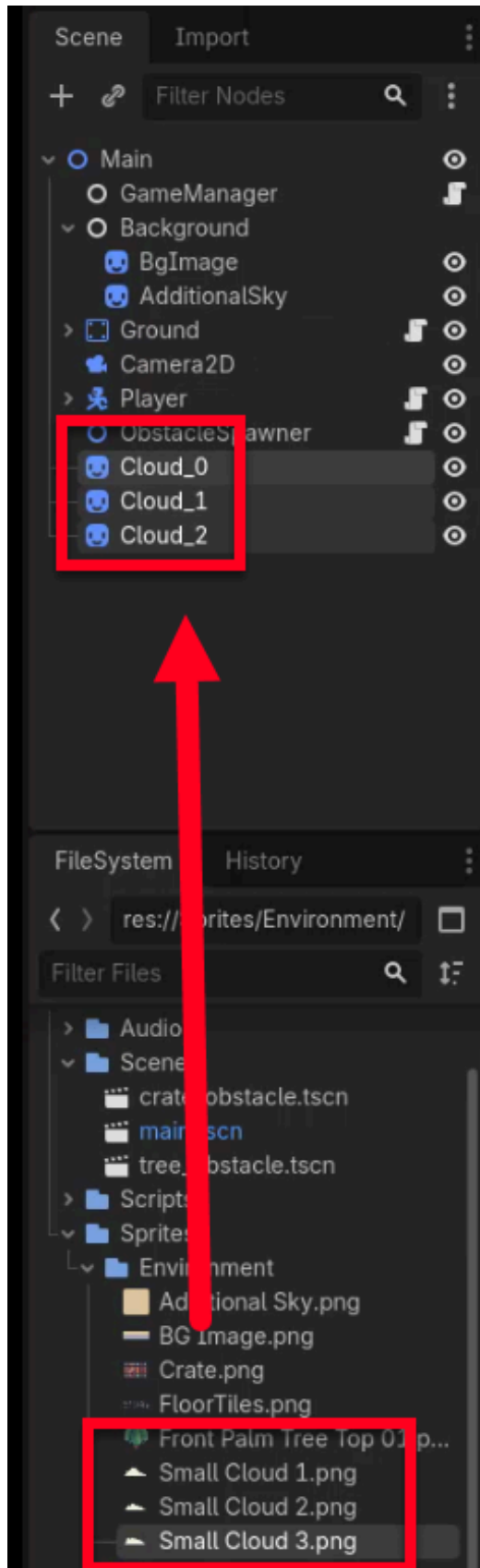
In this chapter, we will enhance the visual appeal of our game by adding clouds to the sky and animations to our player character. Currently, our player can run and jump over obstacles, but the background is static, and the character slides across the screen without moving their legs. By introducing clouds, we can create a more dynamic and visually interesting environment. We will achieve this by reusing our existing spawner and obstacle systems, and then introducing new features like parallax scrolling and randomized spawn heights to create a sense of depth.

Following that, we will bring our character to life by implementing run and jump animations using Godot's powerful animation tools.

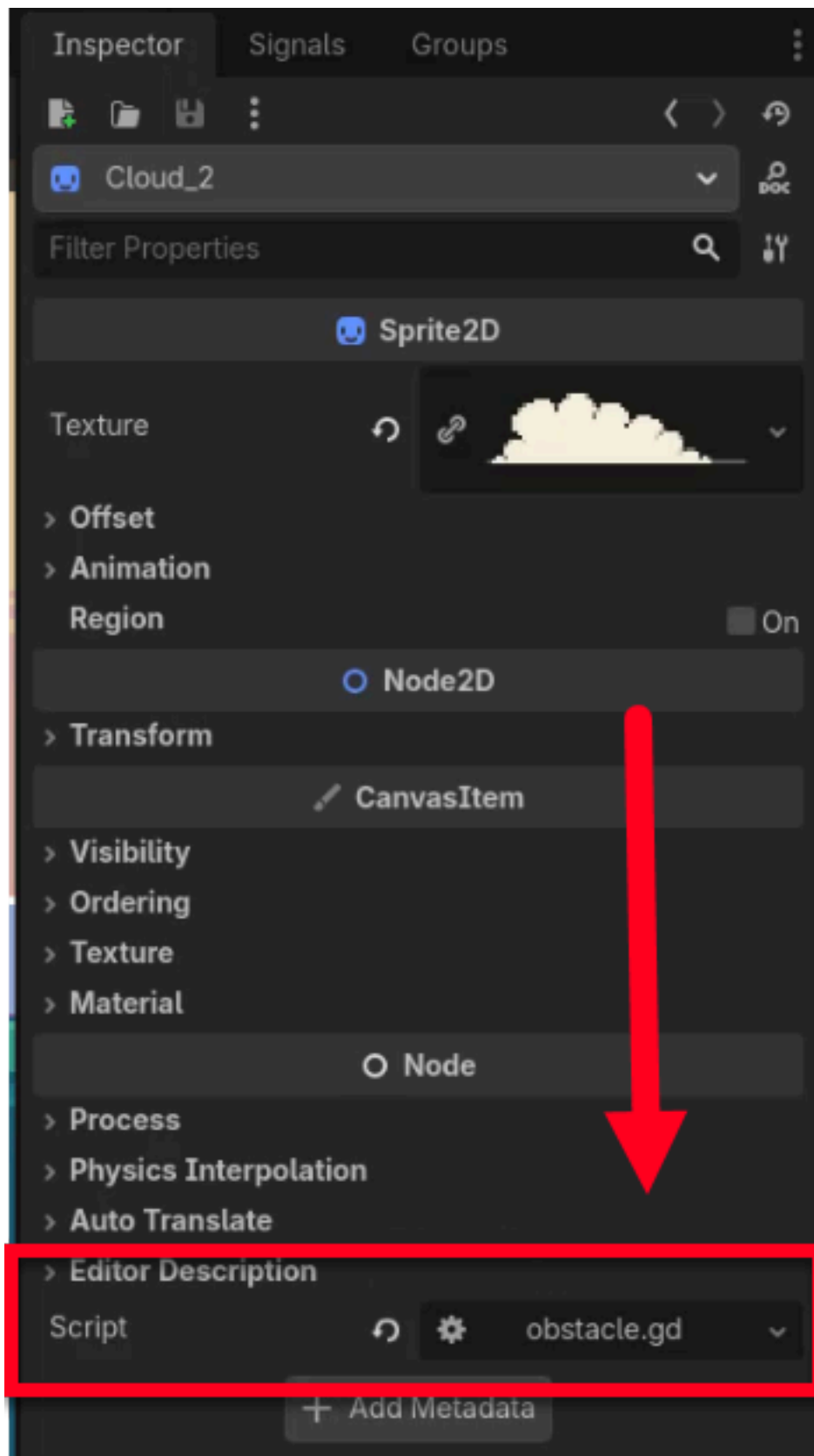
Creating Dynamic Clouds

To start, we need to create the scenes for our clouds. We will use three different cloud sprites to add variety to the sky.

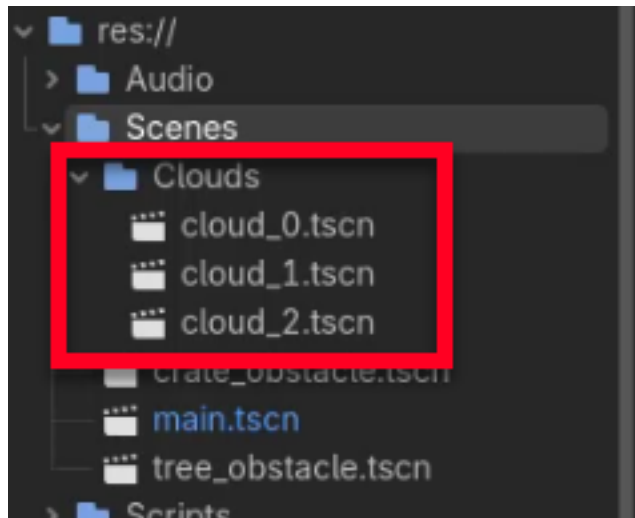
First, verify that your cloud sprite images are imported from the Sprites folder in the FileSystem dock. Drag each one into the main scene to create a `Sprite2D` node. Then, rename them to `Cloud_0`, `Cloud_1`, and `Cloud_2` to keep things organized.



Next, we need to apply movement logic to these clouds. Attach the existing `obstacle.gd` script to each of the three cloud nodes. This script currently handles the movement of our crates and trees, but we will adapt it to handle clouds as well.



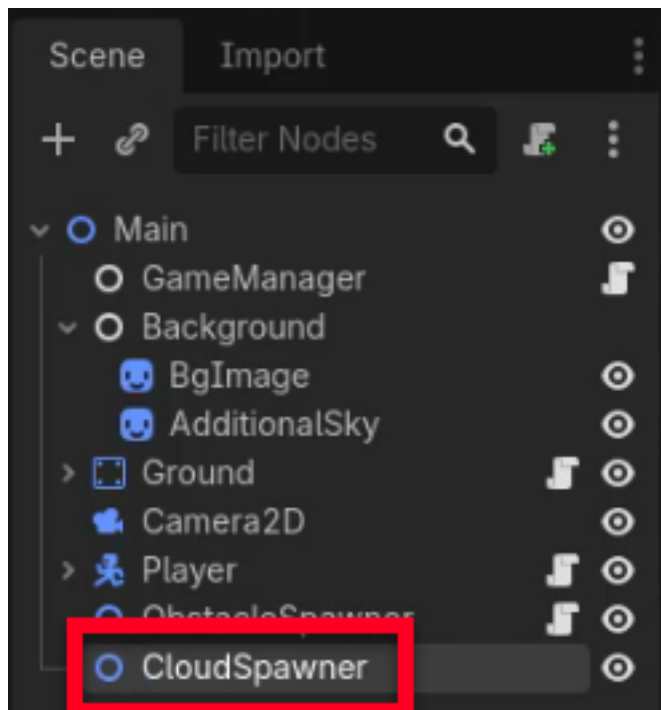
Now, save each cloud node as a separate scene. It is good practice to create a new subfolder, such as Scenes/Clouds, to store them. Once they are saved as scenes, you can delete the cloud nodes from your main scene.



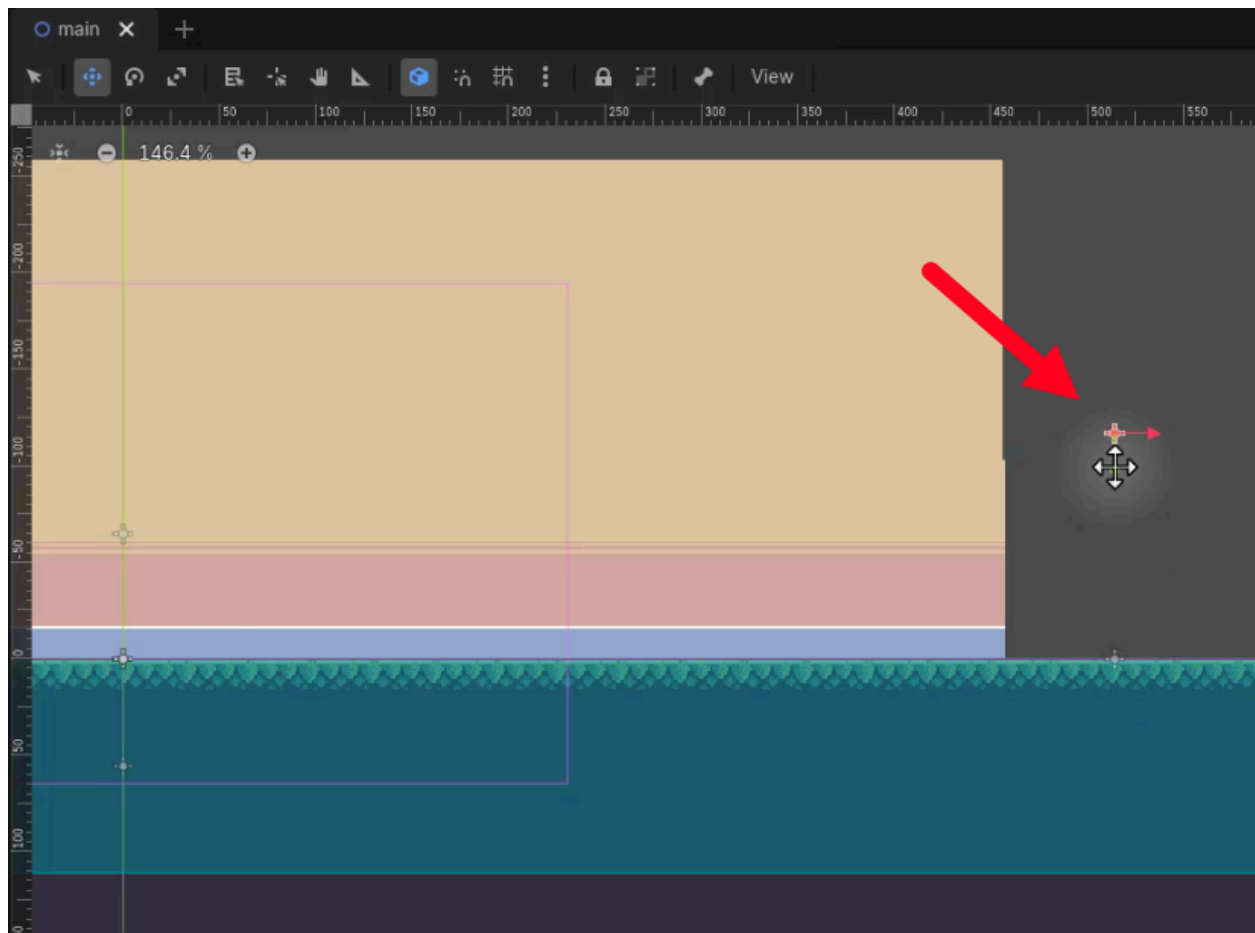
Setting up the Cloud Spawner

With our cloud scenes ready, we need a spawner to create them in the game world. We can reuse our existing spawner .gd script for this purpose.

In your main scene, create a new Node2D and name it CloudSpawner.

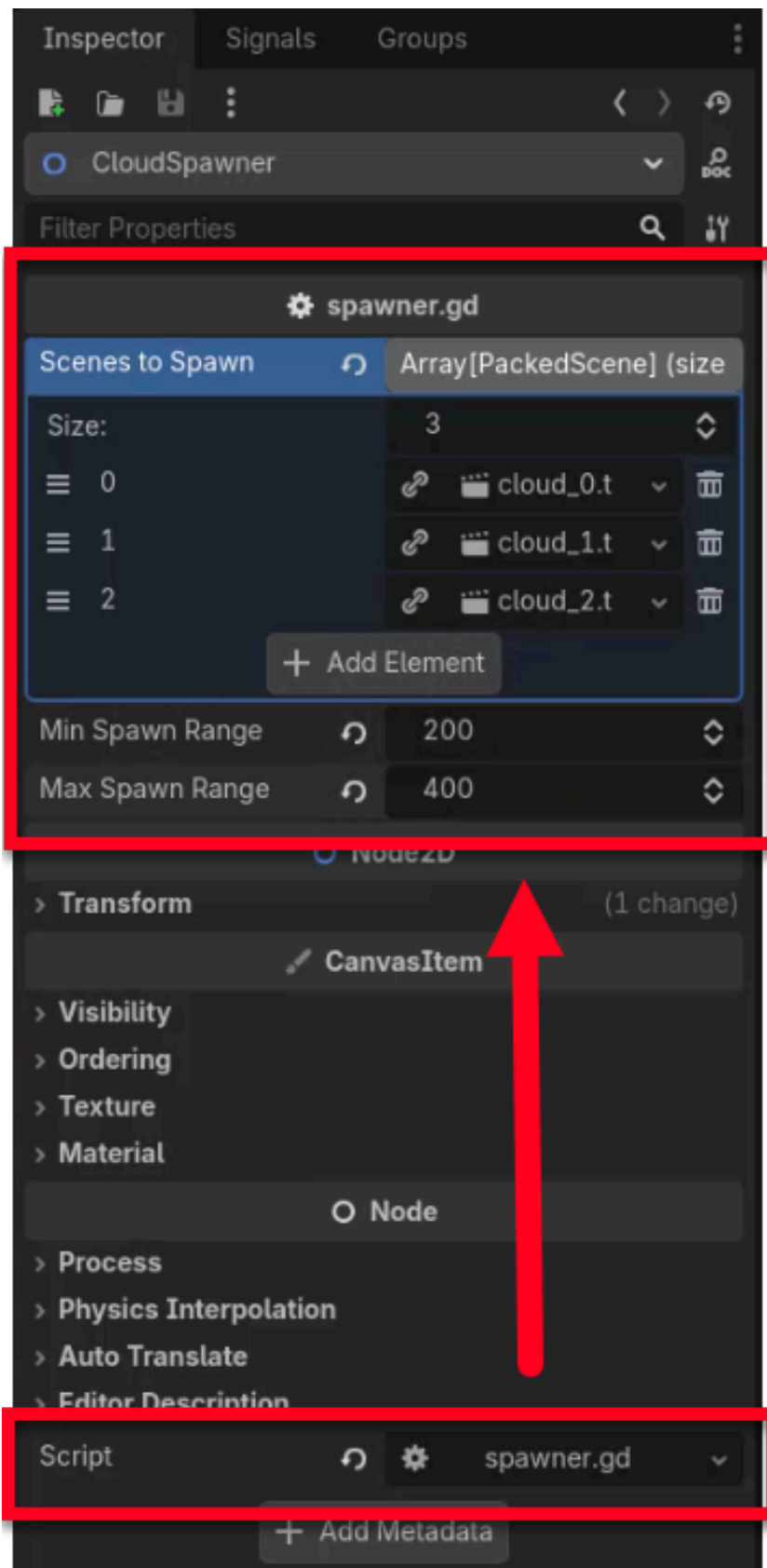


Position the CloudSpawner in the sky area where you want the clouds to appear, typically off-screen to the right so they can drift into view.



To make our spawning work, attach the spawner .gd script to the CloudSpawner node. We'll then configure its properties in the Inspector:

1. Set the size of the Scenes To Spawn array to 3.
2. Drag your three cloud scenes (cloud_0.tscn, cloud_1.tscn, cloud_2.tscn) into the array slots.
3. Set the Min Spawn Range and Max Spawn Range to appropriate values, for example, 200 and 400.



Refactoring the Obstacle Script

If you try to run the game now, you will encounter an error. This is because our `obstacle.gd` script currently extends `StaticBody2D`, but our cloud scenes are of type `Sprite2D`. A script that inherits from one node type cannot be attached to a different, incompatible node type.

To fix this, we will change the script to extend from a more general node type that both objects share. Both `StaticBody2D` and `Sprite2D` inherit from `Node2D`.

As such, let's open the `obstacle.gd` script and change the first line:

```
extends Node2D
```

By changing the inheritance from `StaticBody2D` to `Node2D`, the script becomes compatible with our ground obstacles (which are `StaticBody2D`) and our new clouds (which are `Sprite2D`). Since we are only modifying the `global_position` in our script, `Node2D` provides all the functionality we need for both.

Implementing a Parallax Effect

To create a sense of depth, we will implement a parallax effect. In the real world, objects further in the distance appear to move slower than objects in the foreground. We can simulate this by adding a speed multiplier to our movement logic.

First, let's add an export variable to control this effect from the Inspector.

```
extends Node2D
```

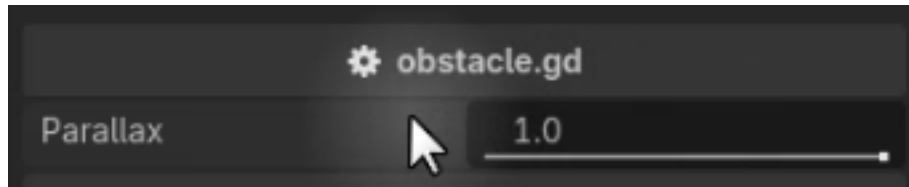
```
# Absolute path because spawned objects are children of the spawner, not Main
```

```
@onready var game_manager : GameManager =  
get_node("/root/Main/GameManager")
```

```
# Controls depth-based speed: 1.0 = foreground, lower = further back
```

```
@export_range(0.0, 1.0) var parallax : float = 1.0
```

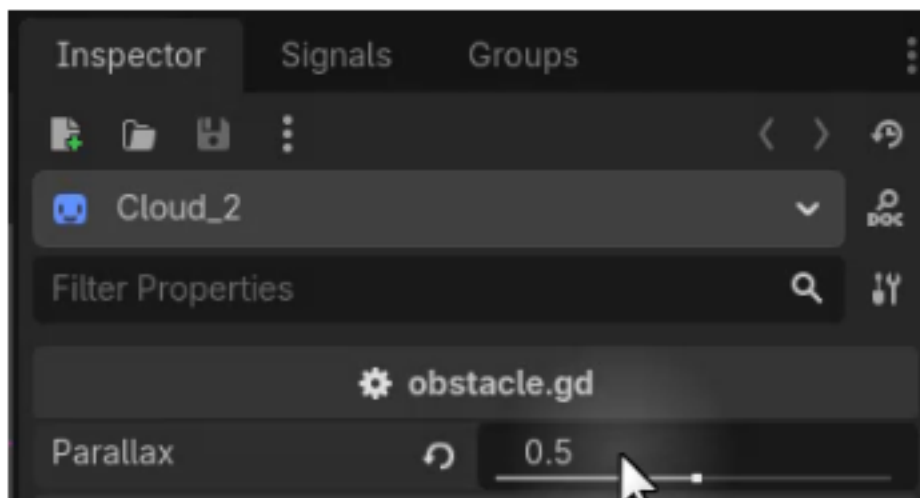
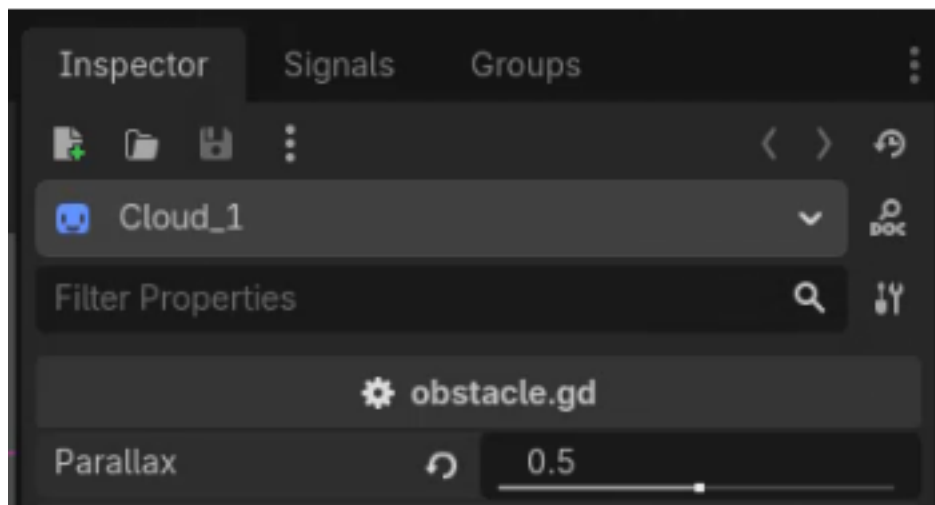
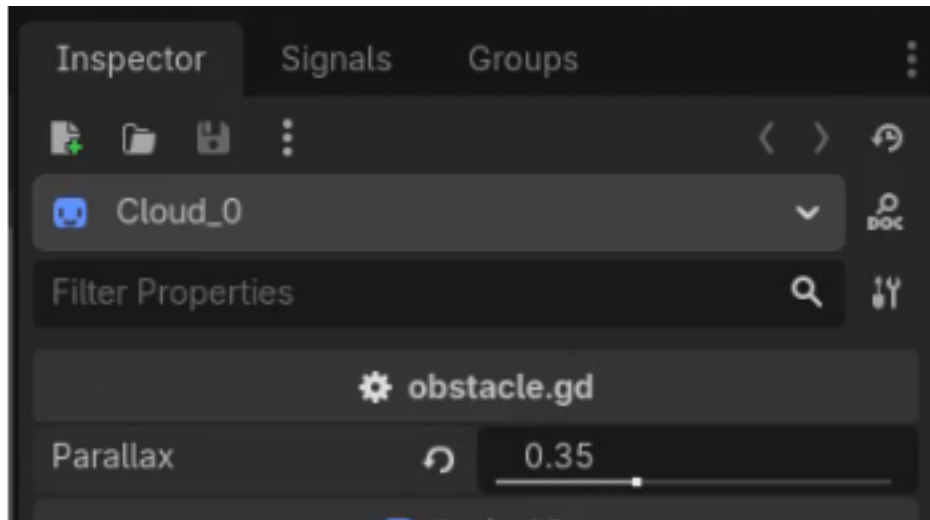
The `@export_range(0.0, 1.0)` annotation creates a convenient slider in the Inspector, limited to values between 0 and 1. A value of 1.0 means the object moves at full speed (foreground), while a value closer to 0 means it moves slower (background).



Next, we'll modify the `_process` function to use this new variable:

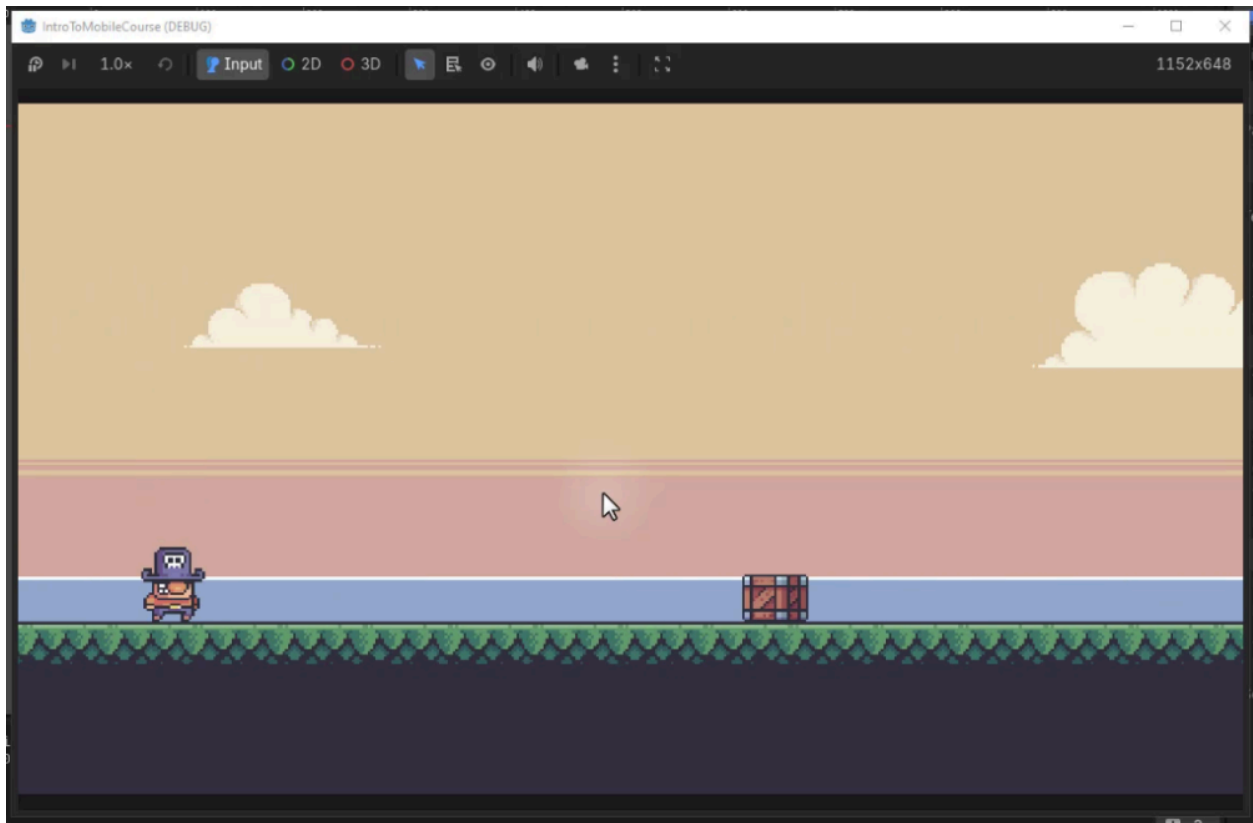
```
func _process(delta):  
    # Multiply by parallax so background elements drift slower  
    var speed : float = game_manager.current_speed * delta * parallax  
    global_position.x -= speed
```

Now, you can select each of your cloud scenes and set their Parallax value in the Inspector. For example, you could set larger clouds to 0.5 and smaller, more distant-looking clouds to 0.35.



Your ground obstacles should keep the default value of 1.0. This way the speed of our foreground isn't affected.

By now, you should have clouds spawning and moving slower than the obstacles, creating a true sense of depth.



Randomizing Spawn Height

To make the sky look more natural, the clouds should spawn at slightly different heights rather than in a single straight line. We can add this functionality to our `spawner.gd` script.

First, add a new export variable to define the maximum random height offset.

```
# Maximum vertical offset for spawn variety (clouds at different heights)  
@export var max_height : int
```

Next, in the `_spawn` function, we will calculate a random Y position for each new object.

```

# Instantiate a random scene from the array and place it at the spawner
func _spawn ():
    var scene_to_spawn : PackedScene = scenes_to_spawn[randi() %
len(scenes_to_spawn)]
    var obj : Node2D = scene_to_spawn.instantiate()
    add_child(obj)

    obj.global_position = global_position

    # Add a random vertical offset for natural placement
    var y_pos : float = global_position.y + randi_range(0, max_height)
    obj.global_position.y = y_pos

    # Randomize the gap before the next spawn
    next_spawn_range = randi_range(min_spawn_range, max_spawn_range)

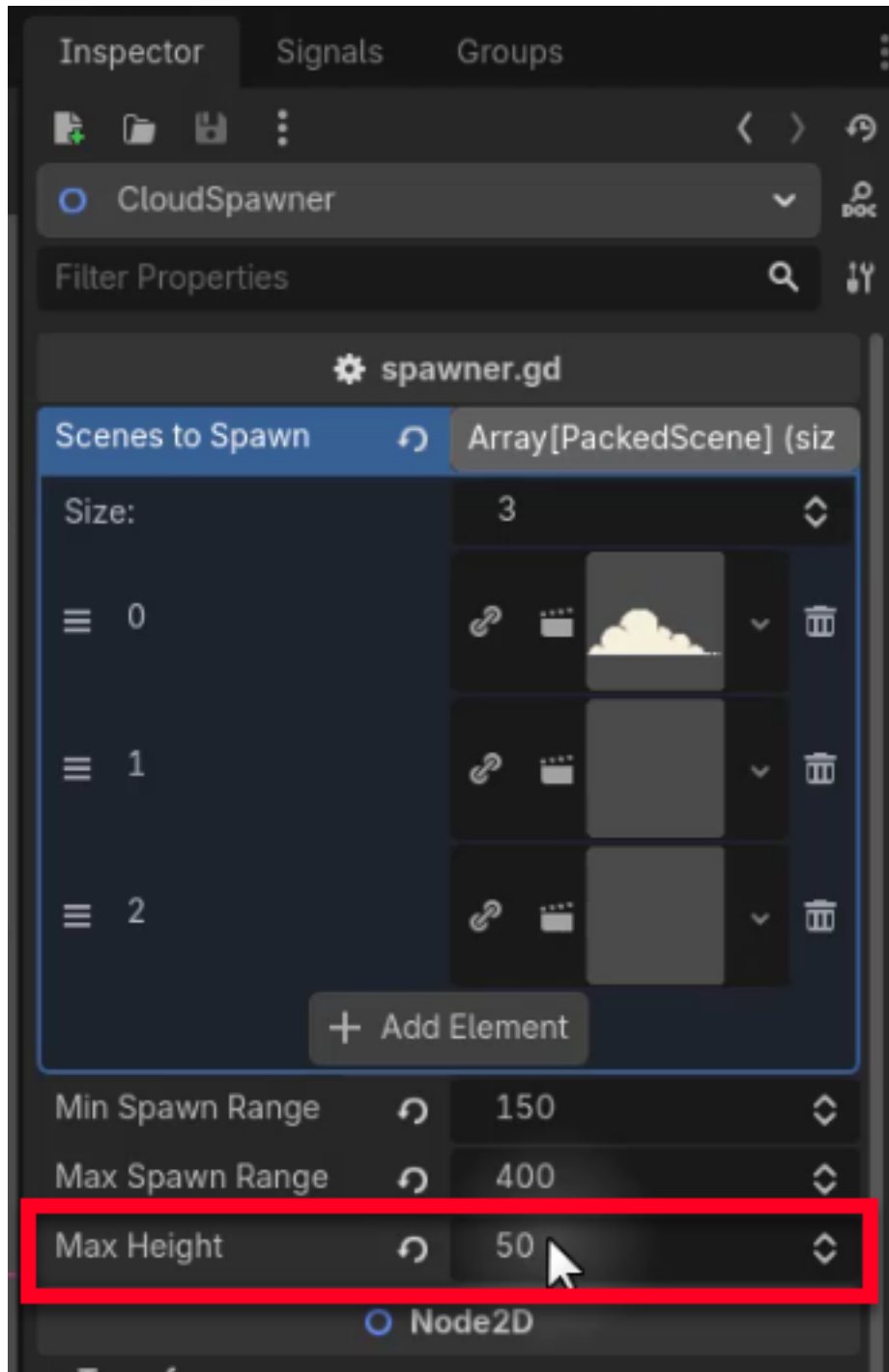
    previous_object = obj

```

This code takes the spawner's Y position and adds a random integer between 0 and max_height to it.

Select your CloudSpawner in the main scene and set the Max Height property to a value like 50. This will cause clouds to spawn within a 50-pixel vertical range from the spawner's position.

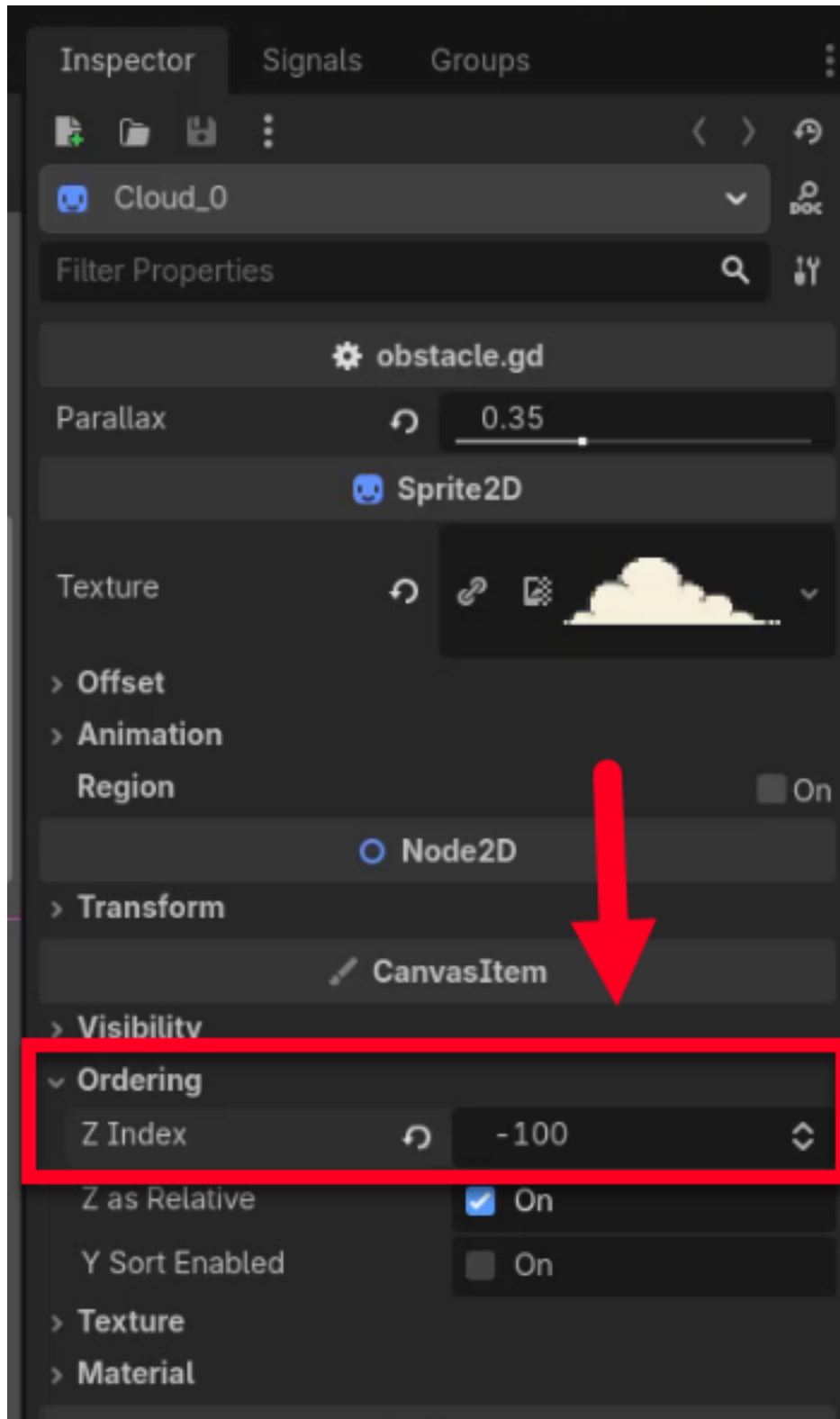
Note that we will leave this as its default for the ObstacleSpawner. In this way, our obstacles will remain unaffected by the height offsets and spawn on the ground as intended.



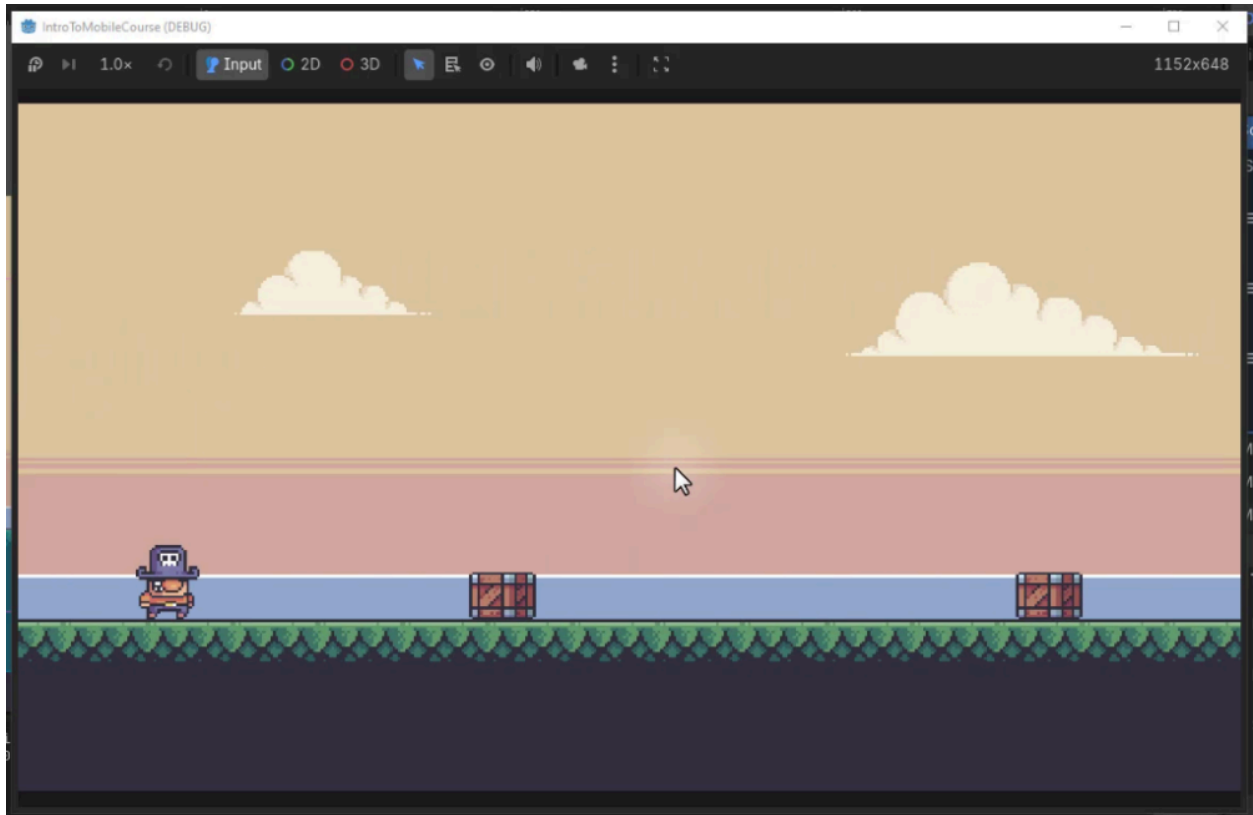
Adjusting the Draw Order

A final issue you might notice is that when the player jumps, they appear behind the clouds. This is a draw order problem. As a refresher, in Godot, nodes are drawn in tree order by default, but we can override this using the Z Index.

In each cloud scene, select the root node and set its Z Index (under **Ordering** in the Inspector) to a negative value, such as -100. This ensures the clouds are drawn behind the player and other game elements, which have a default Z Index of 0. Of course, make sure that this value is larger than your background's Z Index, since we still want the clouds to appear on top of the background.



With these changes, you now have a dynamic sky with clouds that add depth and visual interest to your game.



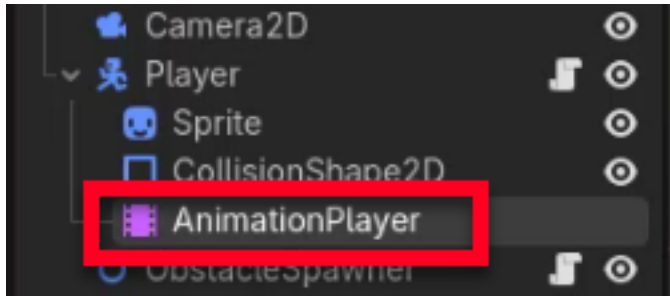
Player Animation

Now that our environment looks lively, let's focus on the player. We will create two distinct animations: a "run" animation for when the player is on the ground and a "jump" animation for when the player is in the air. This will provide important visual feedback to the player.

The AnimationPlayer Node

To manage our animations, we will use Godot's `AnimationPlayer` node. This powerful node allows you to animate almost any property of any other node over time. For our 2D sprite, we will animate the `texture` property of the `Sprite2D` node, effectively changing the displayed image at set intervals to create the illusion of movement.

To begin, select your `Player` node in the Scene dock. Then, add a new child node of type `AnimationPlayer`.



Godot Insight: AnimationPlayer and property tracks

The AnimationPlayer node can animate virtually any property on any node in the scene tree, not just sprites. You can use it to animate positions, colors, shader parameters, audio volume, and even custom script variables. Each animation is stored as a resource inside an AnimationLibrary, making it easy to organize and reuse.

For frame-by-frame sprite animation (like our run cycle), you create a property track targeting the Sprite2D's texture property and set a different image at each keyframe. For smoother motion, you can use value interpolation between keyframes.

Learn more: [AnimationPlayer](https://docs.godotengine.org/en/4.6/classes/class_animationplayer.html)

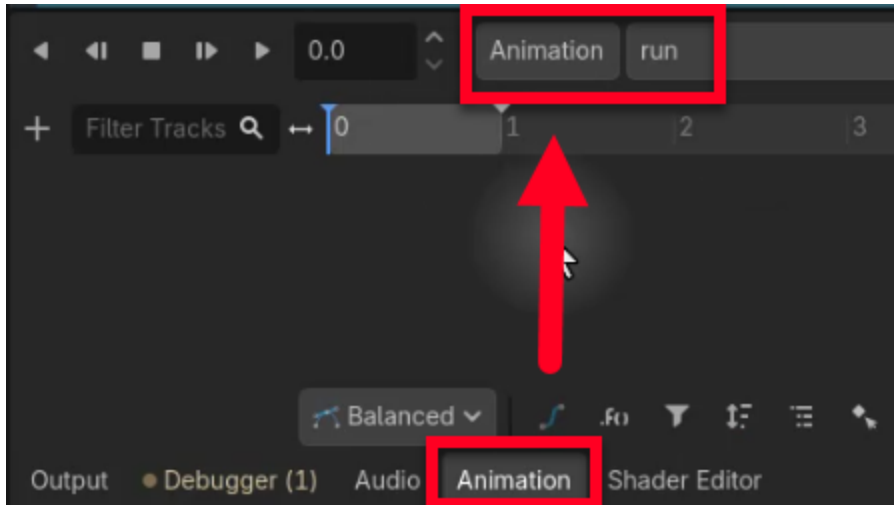
(https://docs.godotengine.org/en/4.6/classes/class_animationplayer.html)

Creating the Run Animation

The overall workflow for building a frame-by-frame animation in Godot is: create a named animation, add a property track that targets the sprite's texture, and then insert a keyframe for each frame of the cycle. Once you understand this pattern, you can use it to animate any sprite sequence in your projects!

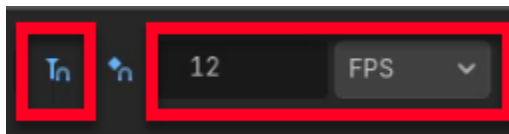
First, we will create the running animation. Our game's art assets, located in the Sprites/Player/ folder, include several frames for a run cycle.

With the AnimationPlayer node selected, open the **Animation** panel at the bottom of the editor. Click **Animation > New**, name the animation "run", and click OK.

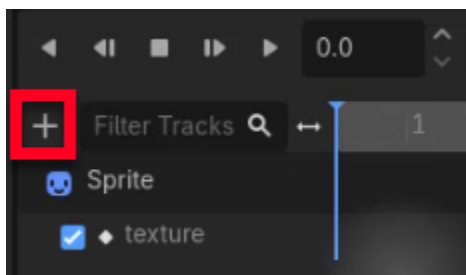


The animation timeline defaults to showing time in seconds, which is useful for smooth motion tweens. For frame-by-frame animation, however, it is better to work with Frames Per Second (FPS) so you can place each sprite on an exact frame number rather than calculating fractional seconds. Change the timeline mode from “Seconds” to “FPS” and set the FPS value to 12. This is a common frame rate for 2D sprite animations.

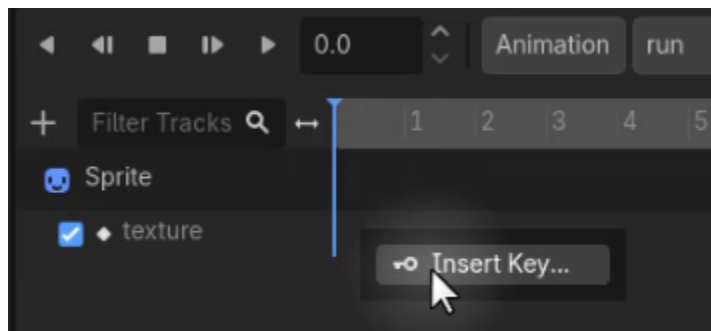
To make placing keyframes easier, also enable **Apply snapping to timeline cursor** in the timeline options.



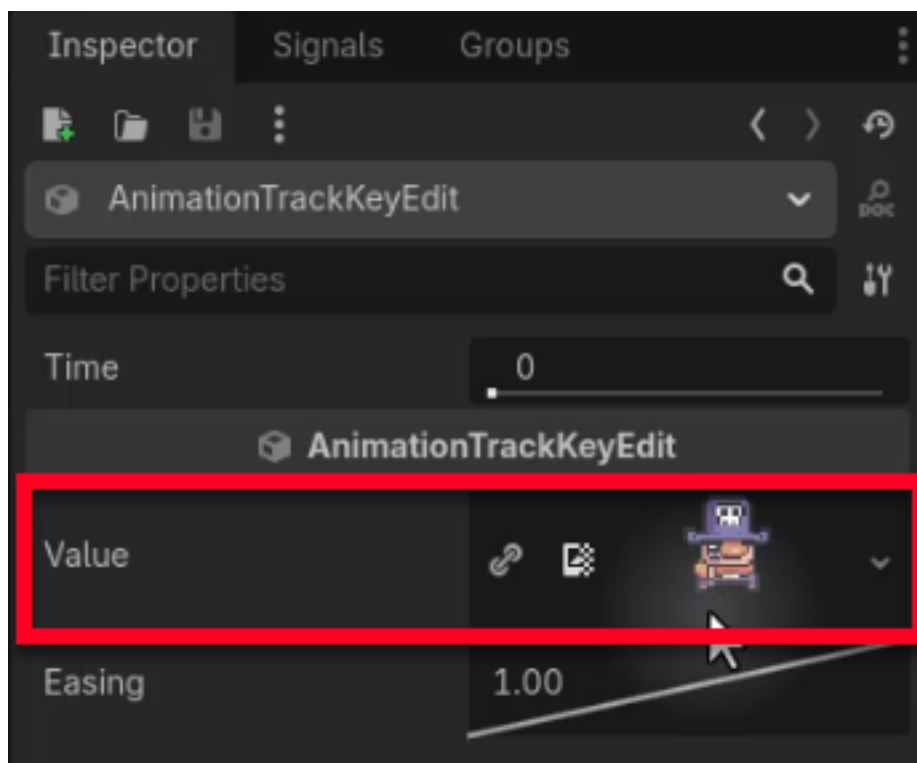
Now, we need to tell the AnimationPlayer which property to animate. Click the **Add Track** button (+ icon) and select **Property Track**. In the “Select a Node” window, choose your player’s Sprite node and click OK. From the list of properties, select texture and click “Open”. After this, a new track for the sprite’s texture will appear.



With the track added, move the timeline cursor to frame 0, then right-click on the track and select **Insert Key**.

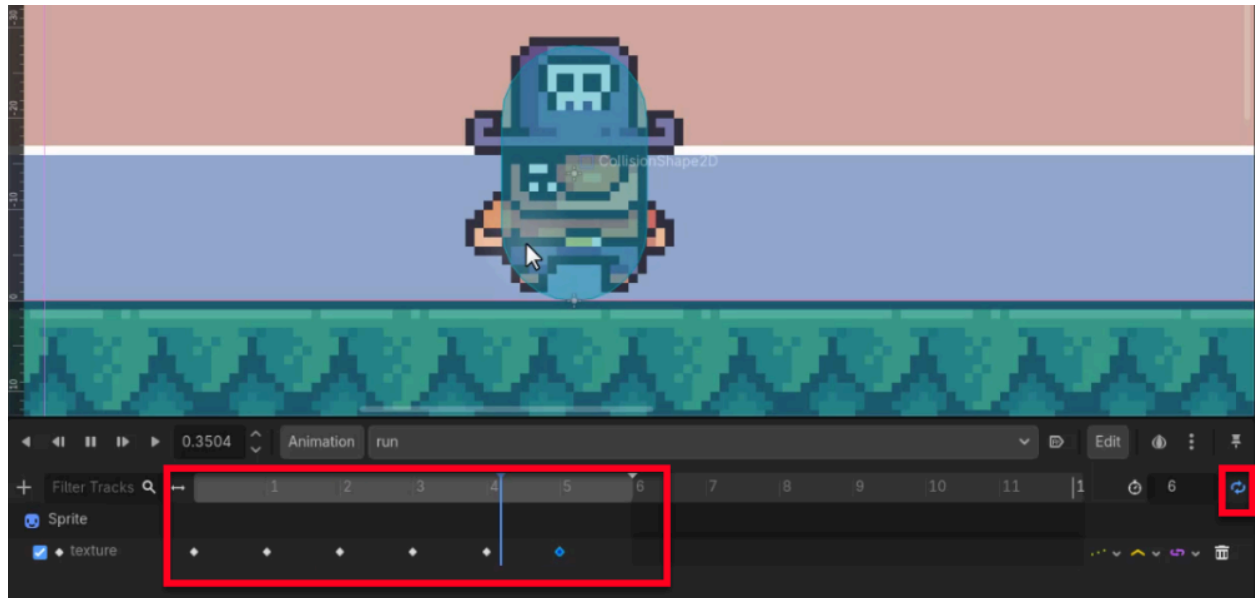


After inserting the keyframe, you should see a dot appear indicating there is a keyframe. Click on the newly created keyframe and, in the Inspector, you will see a **Value** field. Drag the run_1.png sprite from the FileSystem dock into this field.



Move the timeline cursor to the next frame (frame 1). Then, insert another key and set its value to the run_2.png sprite. Continue this process for all the run animation frames, placing each subsequent frame on the next keyframe in the timeline.

Once all frames are added, enable **Animation Looping** (the loop icon in the animation toolbar) so the animation repeats. Finally, adjust the total length of the animation to match your number of frames. If you have 6 frames, drag the end marker of the timeline to the 6-frame mark.



You can now press the **Play** button in the animation panel to preview your running animation.

Creating the Jump Animation

The jump animation will be much simpler, as it is just a single static frame. This frame will be displayed for the entire duration the player is in the air.

In the Animation panel, click on the animation name (“run”) and select **New**. Name this new animation “jump” and click OK.

Next, add a Property Track for the Sprite node’s texture property, just as you did for the run animation. In this case, we just need to insert a single keyframe at the beginning of the timeline and set its value to the `jump.png` sprite.



Controlling Animations with Code

With our animations created visually, we need to write code to switch between them based on the player's state. We will play the "run" animation when on the ground and the "jump" animation when in the air.

Let's open the `player.gd` script. First, we need a reference to our `AnimationPlayer` node so we can actually manipulate our animations. We can create an `@onready` variable for this.

```
@onready var anim : AnimationPlayer = $AnimationPlayer
```

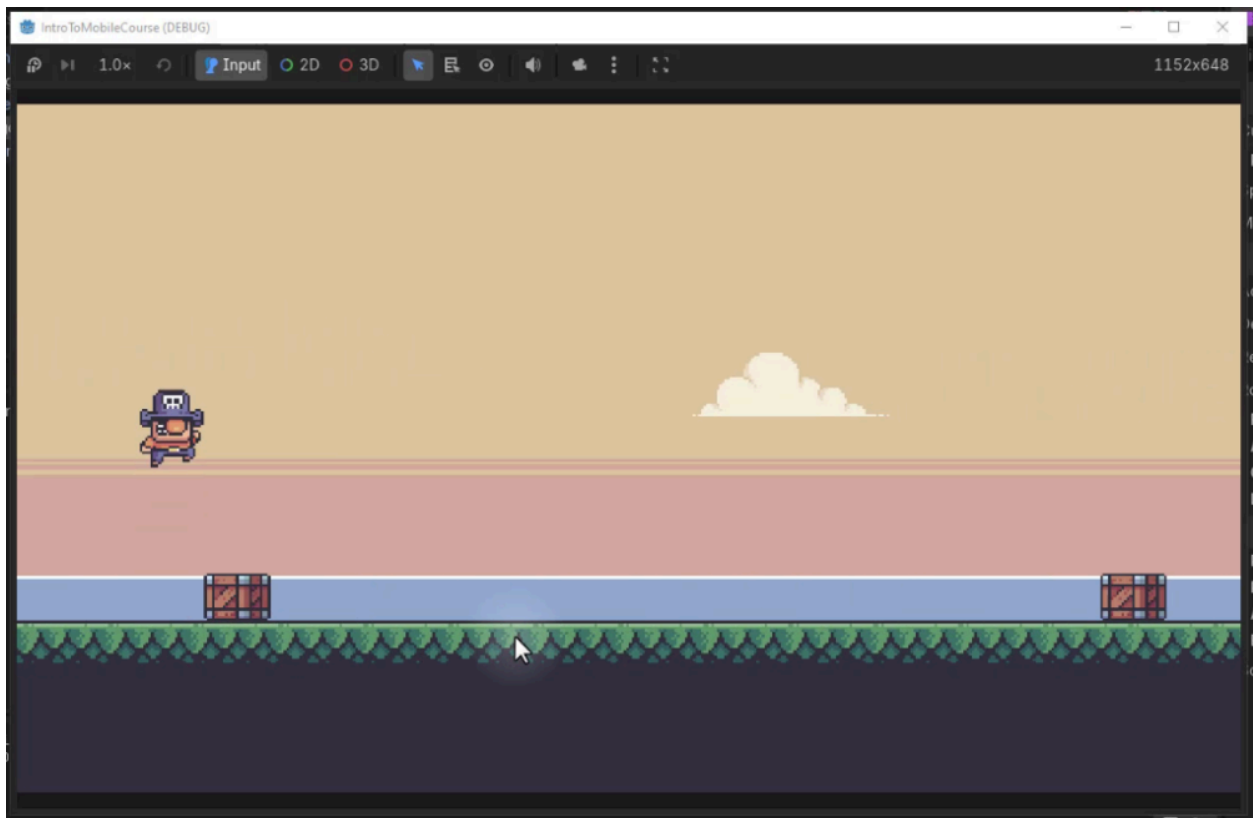
Next, at the end of the `_physics_process` function (after the `move_and_slide()` call), we will add the logic to control the animation.

```
func _physics_process (delta):  
    # ... existing movement code ...  
  
    move_and_slide()  
  
    # Switch animation based on ground contact  
    if is_on_floor():  
        anim.current_animation = "run"
```

```
else:  
    anim.current_animation = "jump"
```

This code checks if the player is on the floor each physics frame. If they are, it sets the `current_animation` property of our `AnimationPlayer` to “run”. Otherwise, it sets it to “jump”. Make sure that the names in the code match what you named the animations exactly; otherwise, the system will not work.

With this done, save your script and run the game. Your player should now display the running animation while on the ground and switch to the jump frame as soon as you jump. This adds a significant layer of polish and makes the game feel much more dynamic.



In the next chapter, we will move back to our gameplay mechanics themselves and work on adding a game over state to our game.

Game State Management

In this chapter, we will implement the “Game Over” state for our project. Currently, the game runs indefinitely - even if the player collides with an obstacle. We will modify this so that hitting an obstacle stops the world movement, provides a visual feedback effect where the player is thrown backward, and displays a user interface allowing the player to restart.

We will also implement a scoring system based on the distance traveled, displaying both a live counter during gameplay and a final score on the game over screen.

Implementing the Game Over Logic

First, we need a way to track the game’s state. We will handle this within our central `game_manager.gd` script like we did with our speed. We need a boolean variable to track if the game has ended and a function to trigger this state change.

Updating the Game Manager

Let’s begin by opening the `game_manager.gd` script. First, we’ll declare a new variable called `is_game_over` and set its default value to `false`.

```
var is_game_over : bool = false
```

Next, create a new function named `game_over`. This function will be called when the player hits an obstacle. Inside this function, we will set the `current_speed` to zero to stop the world from moving and set `is_game_over` to `true`.

```
func game_over ():  
    current_speed = 0.0  
    is_game_over = true
```

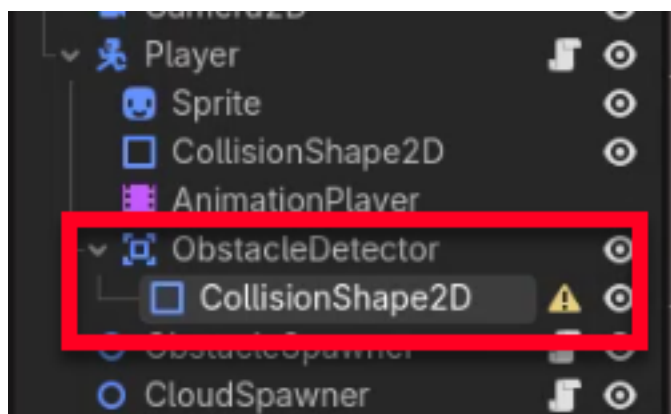
To prevent the game from speeding up again after a game over, add a check at the very beginning of the `_process` function. If `is_game_over` is true, we will exit the function early using `return`.

```
func _process(delta):  
    # Stop updating once the game ends  
    if is_game_over:  
        return  
  
    if current_speed < max_speed:  
        # ... rest of the function
```

Detecting Obstacles

In order for our game over to trigger, we need a way for the player to detect a head-on collision with an obstacle. The player already has a capsule collider for physics interactions with the ground, but we want to allow the player to stand on top of obstacles without triggering a game over. Therefore, we will create a separate detector specifically for frontal collisions.

With the Player node selected in the Scene dock, add an Area2D child node and rename it to `ObstacleDetector`. Then, add a `CollisionShape2D` as a child of the new `ObstacleDetector`.



Godot Insight: Area2D vs physics bodies

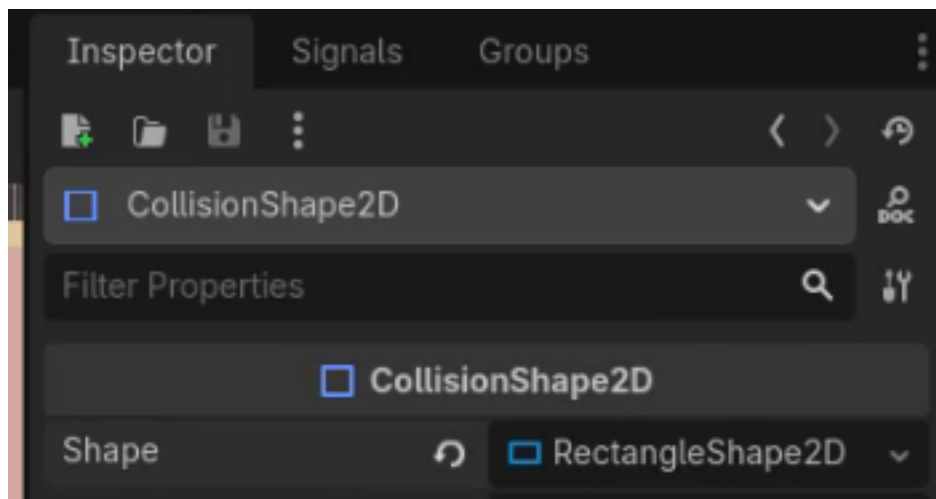
An Area2D detects when other physics objects enter or leave its region, but it does not physically block them. This makes it ideal for triggers, hit zones, and detection areas. In contrast, StaticBody2D and CharacterBody2D provide solid collision that prevents objects from passing through.

In our case, we use an Area2D as the obstacle detector because we want to know when the player touches an obstacle without adding a second physics body that could interfere with the player's ground collisions.

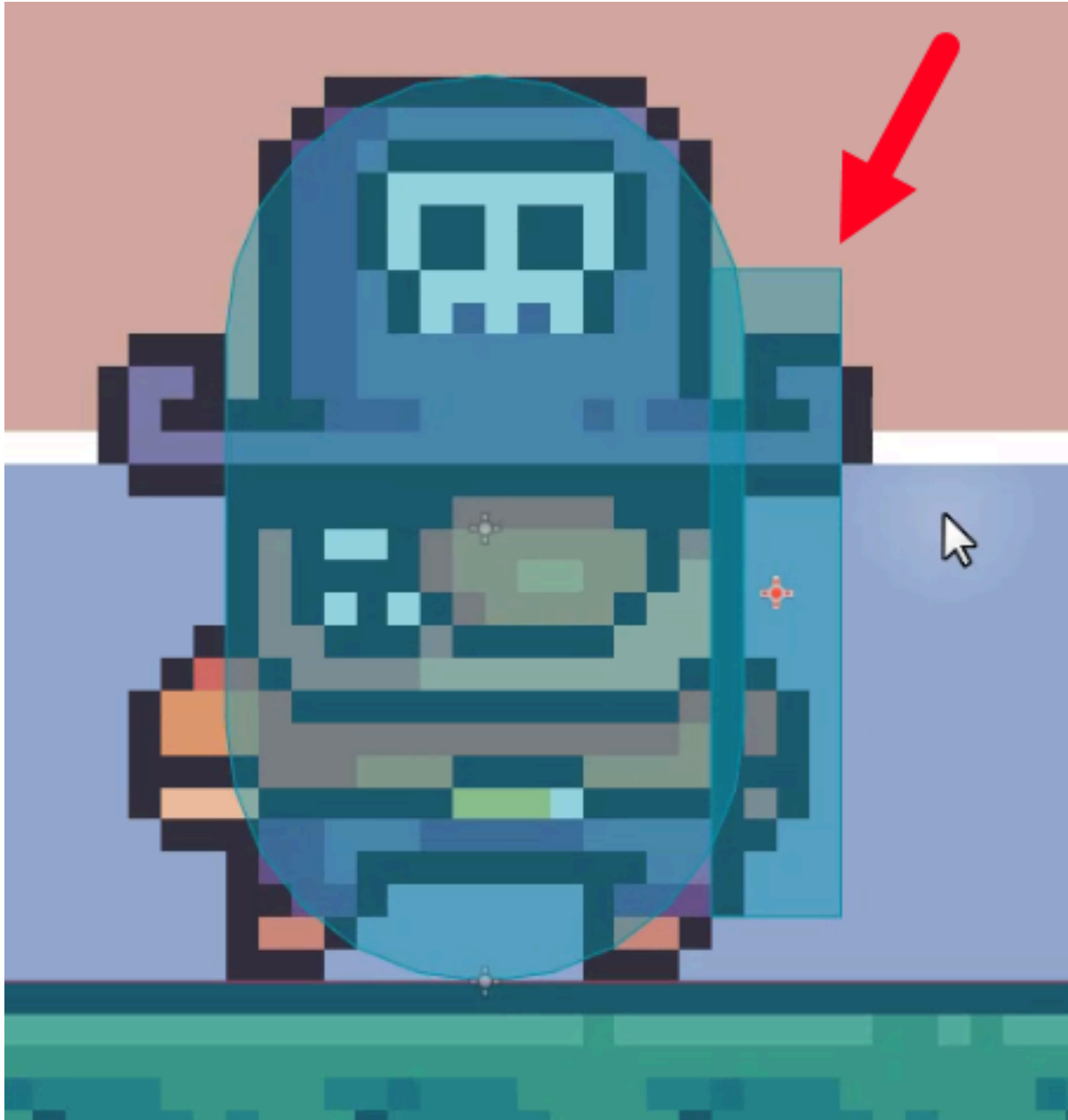
Learn more: [Area2D](https://docs.godotengine.org/en/4.6/classes/class_area2d.html)

(https://docs.godotengine.org/en/4.6/classes/class_area2d.html)

In the Inspector, set the shape of the CollisionShape2D to a RectangleShape2D. Note that as our detector is not involved in movement detection, we don't need a capsule shape to stop us from getting stuck in corners. This is why a rectangle works fine for our purposes.



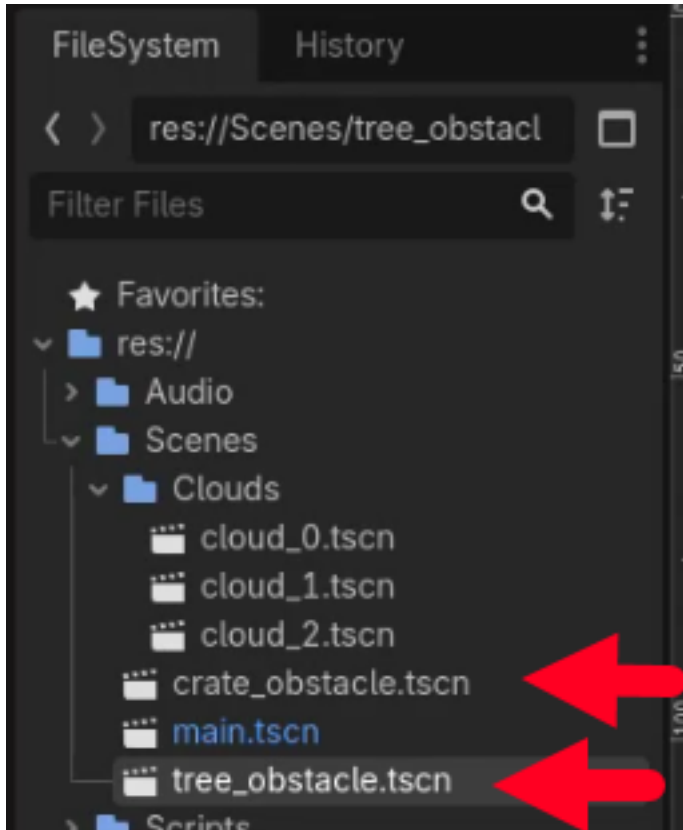
Resize and position this shape in front of the player, covering the area where a head-on collision would occur.



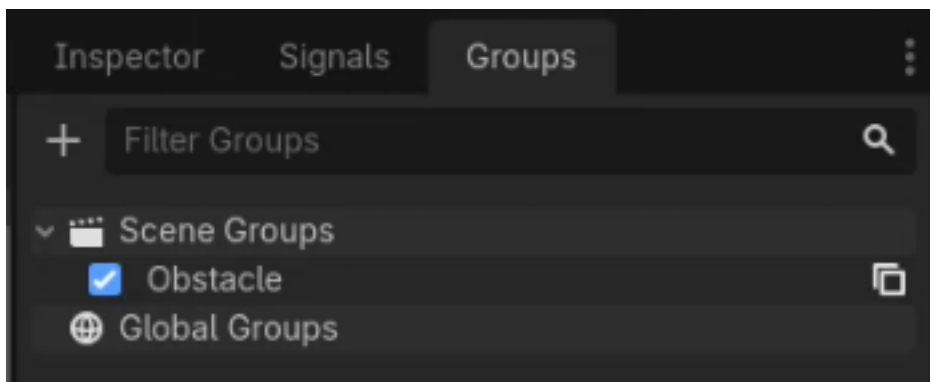
Using Groups to Identify Obstacles

To ensure our detector only triggers a game over when hitting an actual obstacle (and not the ground or other elements), we will assign our obstacle scenes to a specific group.

First, open both your obstacle scenes (e.g., `tree_obstacle.tscn` and `crate_obstacle.tscn`).



For each obstacle, select the root node (e.g., TreeObstacle), then navigate to the **Node** tab in the Inspector and select **Groups**. Type “Obstacle” and click **Add**. Make sure to repeat this for all of your obstacle scenes.



Godot Insight: Groups as a lightweight tagging system

Groups let you tag nodes with string labels and then query them at runtime. Any node can belong to multiple groups, and you can check membership with `is_in_group("name")` or retrieve all members with

`get_tree().get_nodes_in_group("name")`. Groups are defined per-scene in the editor or added dynamically via code.

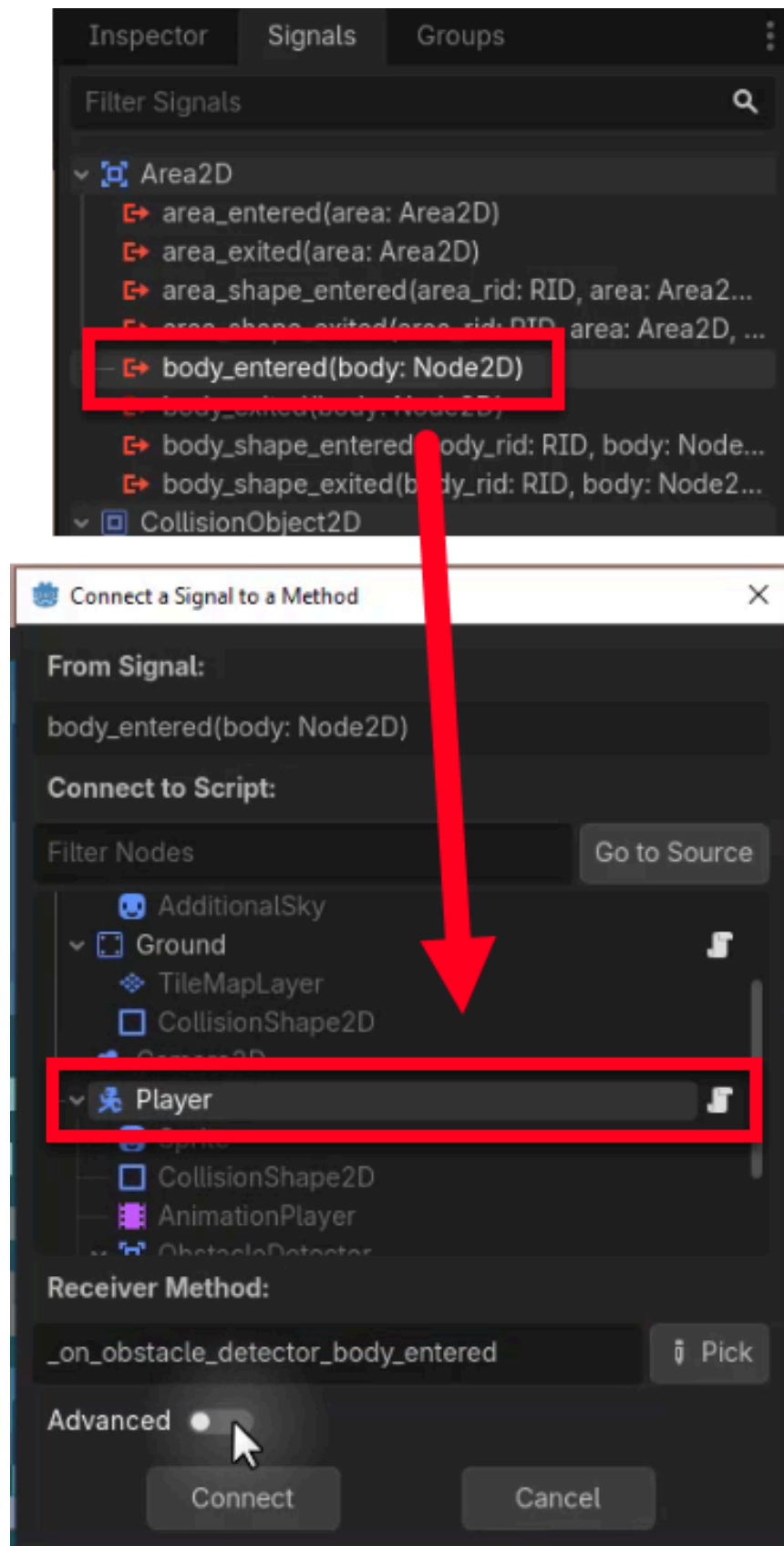
This is much lighter than creating a class hierarchy just for identification. In our project, tagging obstacles with the “Obstacle” group lets the player’s detector distinguish obstacles from the ground without checking node types.

Learn more: [Node](https://docs.godotengine.org/en/4.6/classes/class_node.html)
(https://docs.godotengine.org/en/4.6/classes/class_node.html)

Connecting the Detector Signal

Now, we will connect the `ObstacleDetector`’s signal to the player script to handle the collision logic.

1. Select the `ObstacleDetector` node in the Player scene.
2. In the **Node** tab of the Inspector, click **Signals** and double-click the `body_entered(body: Node2D)` signal.
3. In the connection window that appears, ensure the `Player` node is selected as the target, then click **Connect**.



This will create a new function, `_on_obstacle_detector_body_entered`, in your `player.gd` script.

Implementing the Collision Logic

Inside the `player.gd` script, we will now write the code to handle the game over logic.

First, we need a reference to the `GameManager`. Add the following line at the top of your script with your other variable declarations.

```
@onready var game_manager : GameManager = $"./GameManager"
```

Now, locate the newly created `_on_obstacle_detector_body_entered` function. Inside this function, we will first check if the game is already over to prevent the code from running multiple times. Then, we'll check if the colliding body is part of the "Obstacle" group. If both conditions are met, we call the `game_over` function from our game manager.

```
# Triggered by the Area2D detector in front of the player
func _on_obstacle_detector_body_entered(body):
    if game_manager.is_game_over:
        return

    if body.is_in_group("Obstacle"):
        game_manager.game_over()
```

Finalizing the Game Over Effect

If you run the game now, you will notice the screen stops moving, but the player's animation continues, and they can still jump. Let's fix this and add a visual effect.



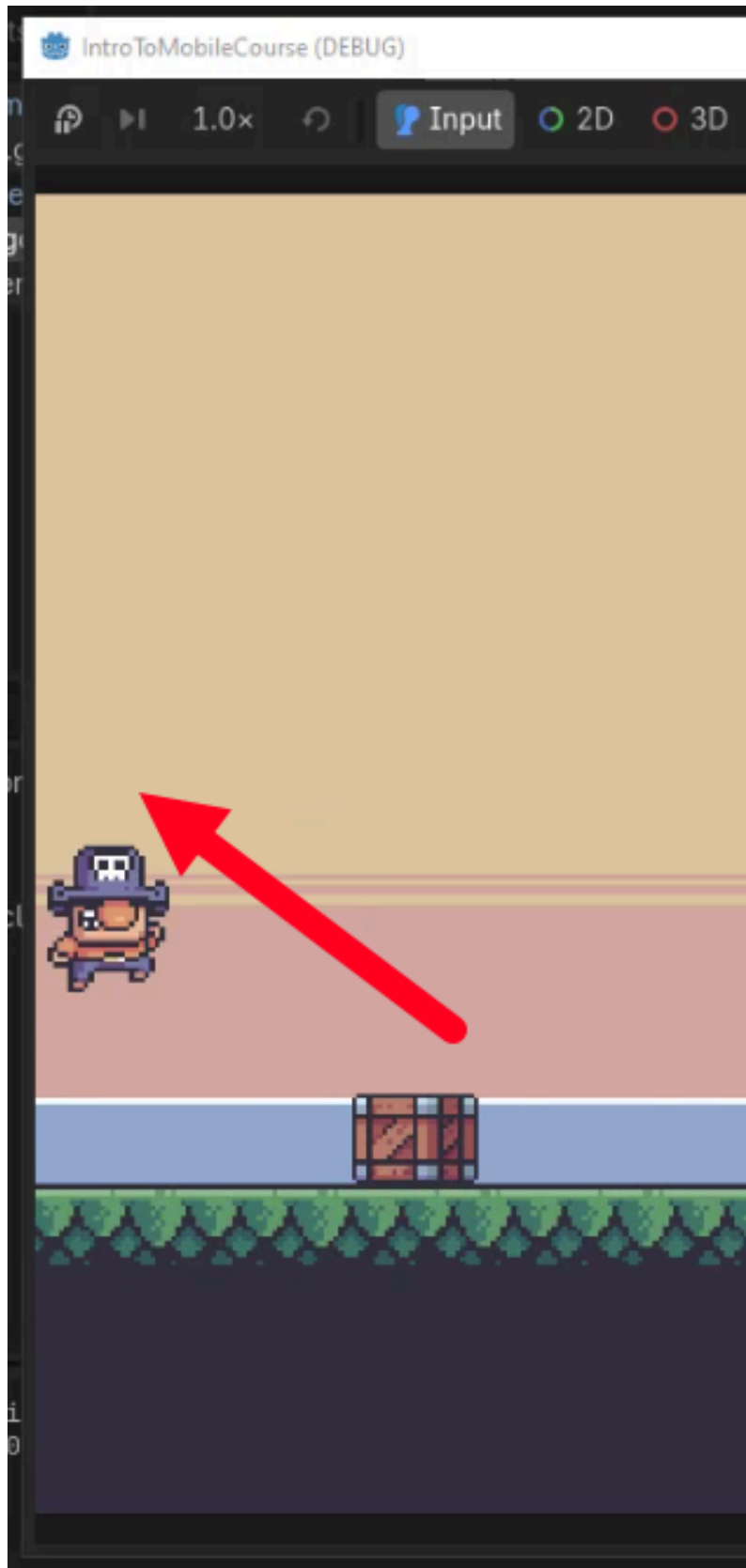
First, disable jumping after the game is over. In the `_try_jump` function, add a check at the beginning.

```
func _try_jump():  
    if game_manager.is_game_over:  
        return  
  
    jump_input = true
```

Next, let's make the player get "thrown" backward upon collision. In the `_on_obstacle_detector_body_entered` function, after calling `game_manager.game_over()`, we will directly manipulate the player's velocity to launch them up and to the left.

```
# Triggered by the Area2D detector in front of the player  
func _on_obstacle_detector_body_entered(body):  
    if game_manager.is_game_over:  
        return  
  
    if body.is_in_group("Obstacle"):  
        game_manager.game_over()  
  
    # Fling the player backward for visual feedback  
    velocity.x = -500  
    velocity.y = -jump_force
```

With these changes, when the player hits an obstacle, the game will stop, jumping will be disabled, and the player character will be flung backward off the screen. This provides clear feedback to the player that the game is over.



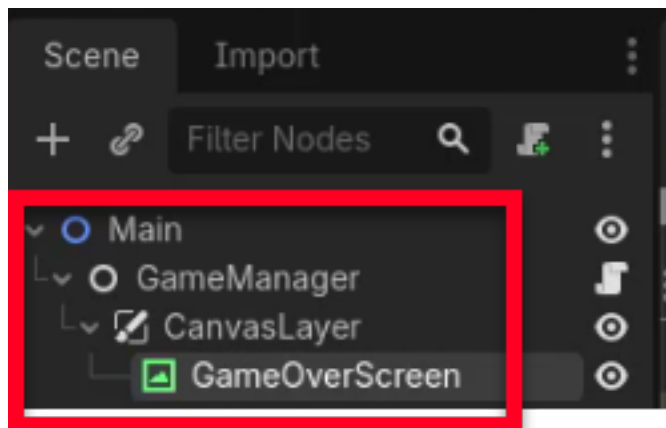
Building the Game Over UI

Now that we have the logic in place, we need a user interface to display the score and offer an option to restart the level. The UI will consist of three elements, each with a specific role: a `TextureRect` for the background panel, a `Label` to show the “Game Over” message and final distance, and a `TextureButton` that lets the player restart with a single tap.

Setting up the UI Scene

Game UI needs to stay fixed on screen - even if the camera were to move with the player. A `CanvasLayer` solves this by rendering its children on a separate drawing layer that is independent of the game world’s camera, so buttons and text always appear in the same screen position. All UI elements will be children of this node.

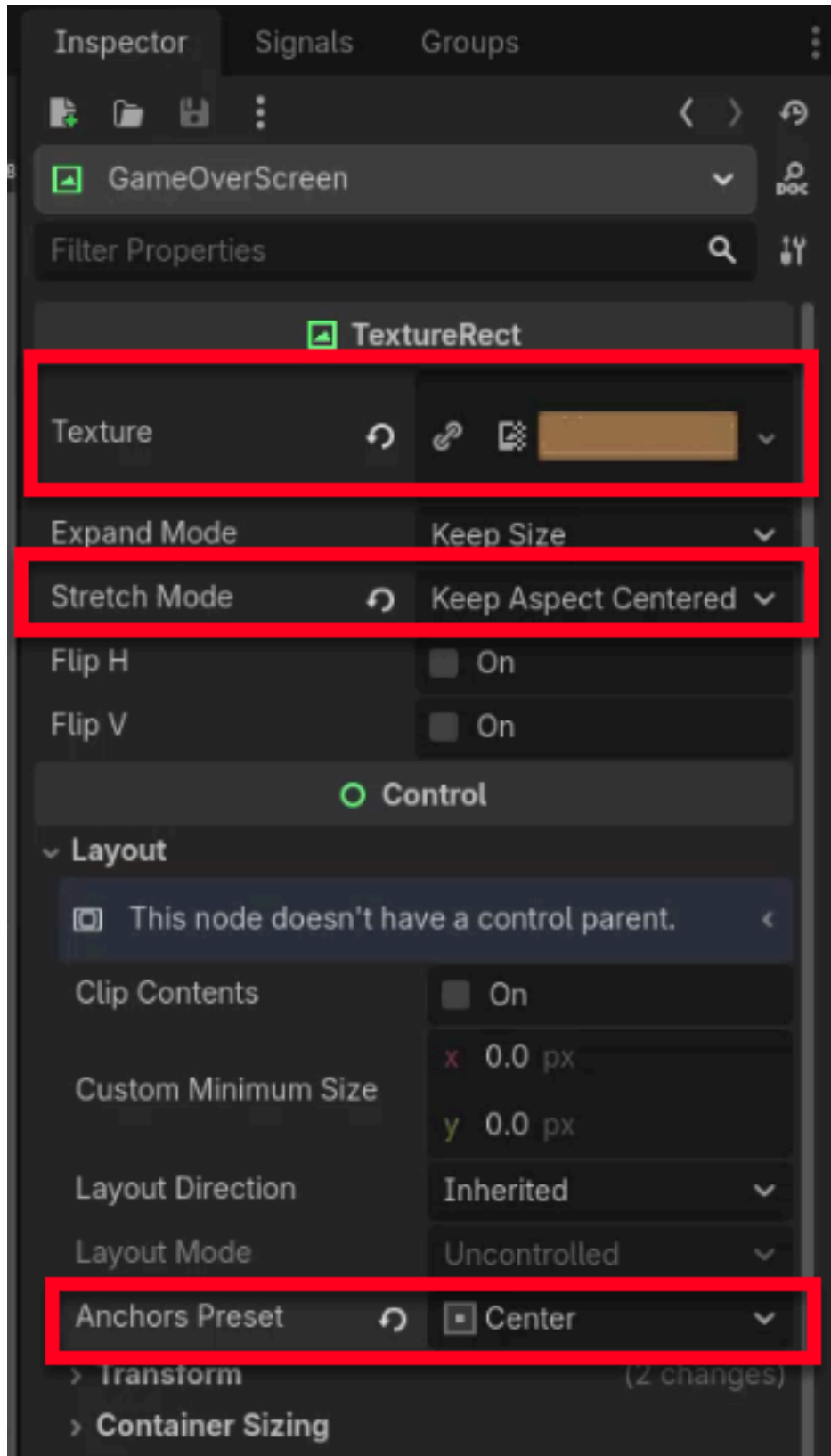
Under the `GameManager` node, create a new `CanvasLayer` child, then add a `TextureRect` node inside it and rename it to `GameOverScreen`. This will serve as the background panel for our game over menu.



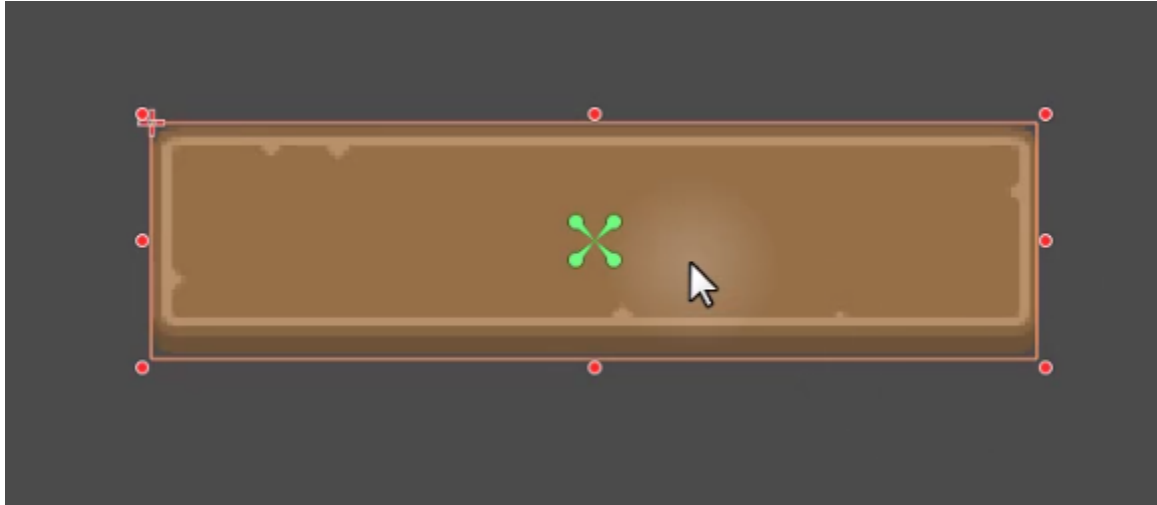
With the `GameOverScreen` node selected, navigate to the Inspector. Find the `Texture` property and drag the `button_long_brown.png` sprite from the `res://Sprites/UI/` folder into it.

If you try resizing the panel now, you will notice that it distorts the image. To fix this, change the **Stretch Mode** property from **Scale** to **Keep Aspect Centered**.

To position the panel perfectly in the center of the screen, go to the **Layout** menu in the toolbar above the viewport and select **Anchors > Center**.

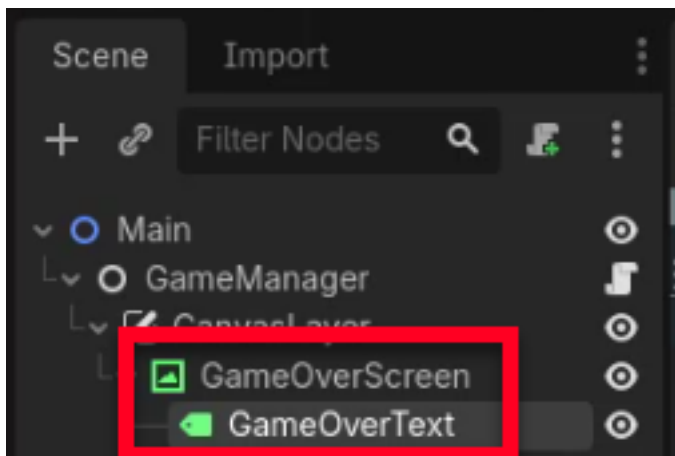


After making these adjustments, you can size the box as needed. The exact size isn't important now, as we'll get a clearer idea of the final size once we add the other elements.



Adding Game Over Text

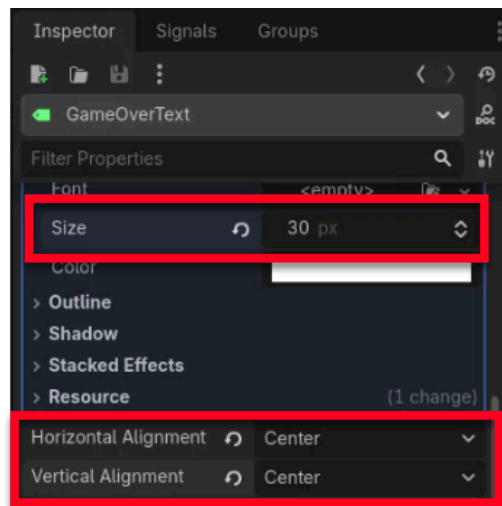
With our screen itself ready to go, let's add a `Label` node as a child of `GameOverScreen` and name it `GameOverText`.



In the Inspector, set its **Text** property to a placeholder like "Game Over! You survived 300 m". The exact text doesn't matter, as we will update this via code later.

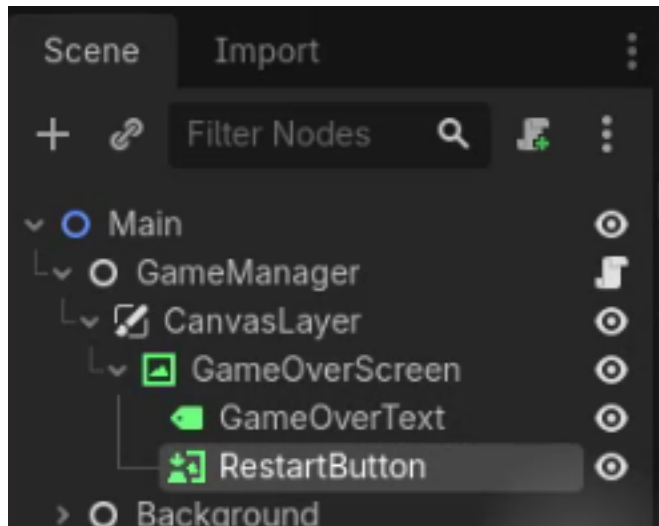
To adjust the font size, create a new `Label Settings` resource and increase the **Font Size** to something like 30px. Set both **Horizontal Alignment** and

Vertical Alignment to Center to properly position the text within its bounding box. Then, adjust the size and position of the text as needed.

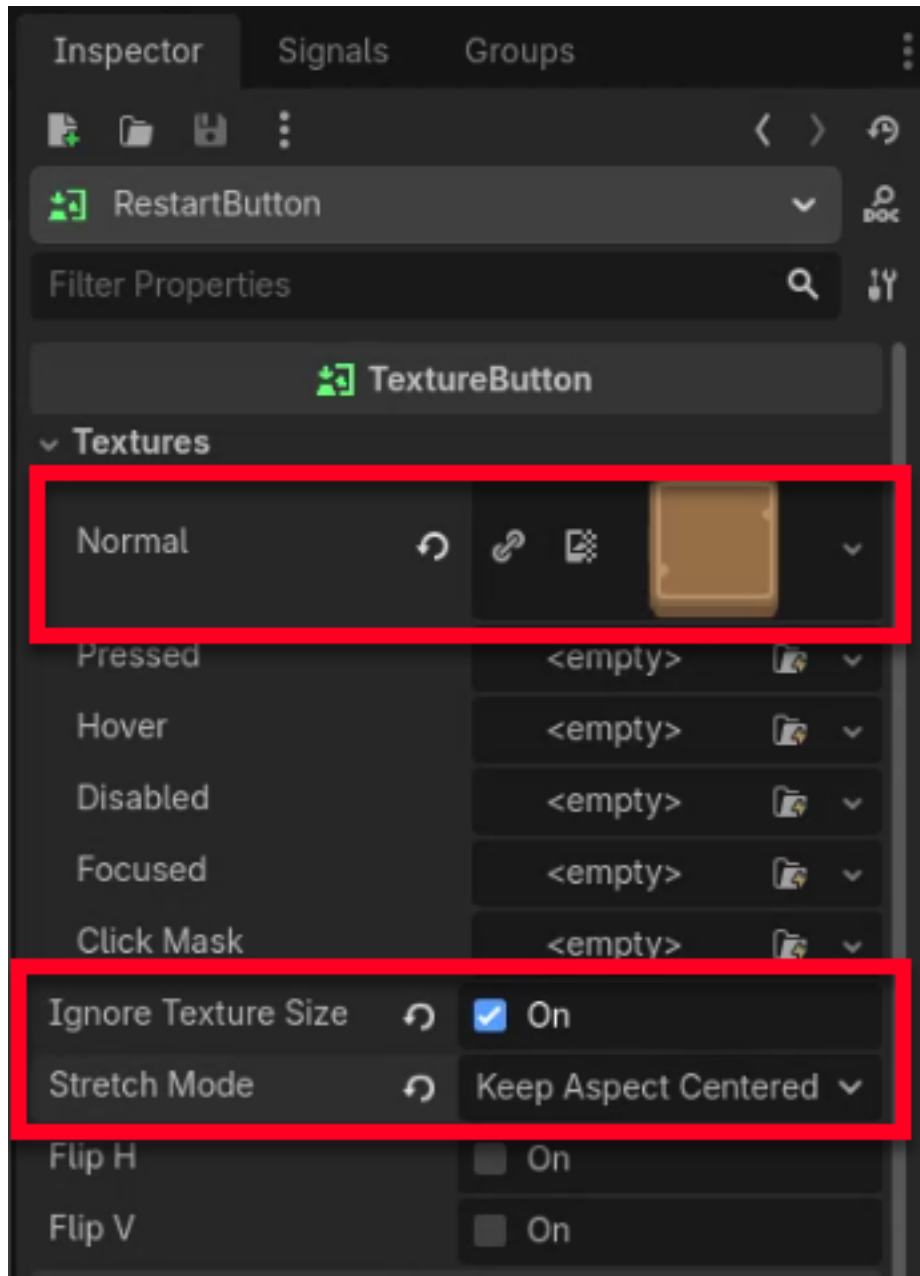


Adding a Restart Button

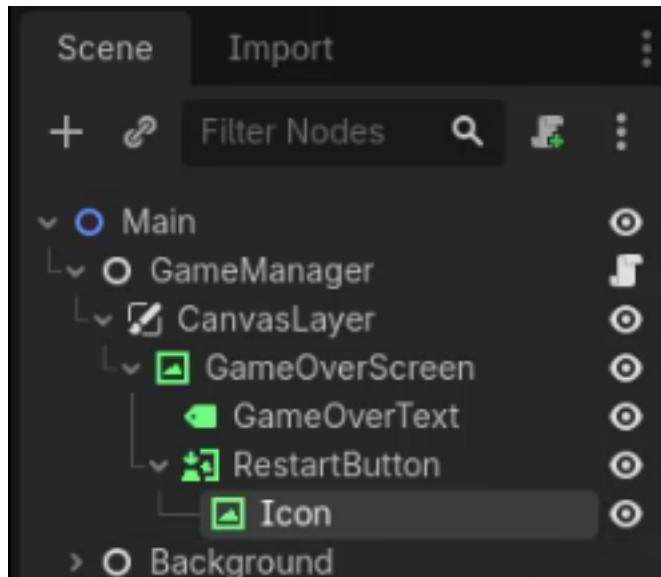
Next, we need our button to restart the game. Add a TextureButton node as a child of GameOverScreen and name it RestartButton.



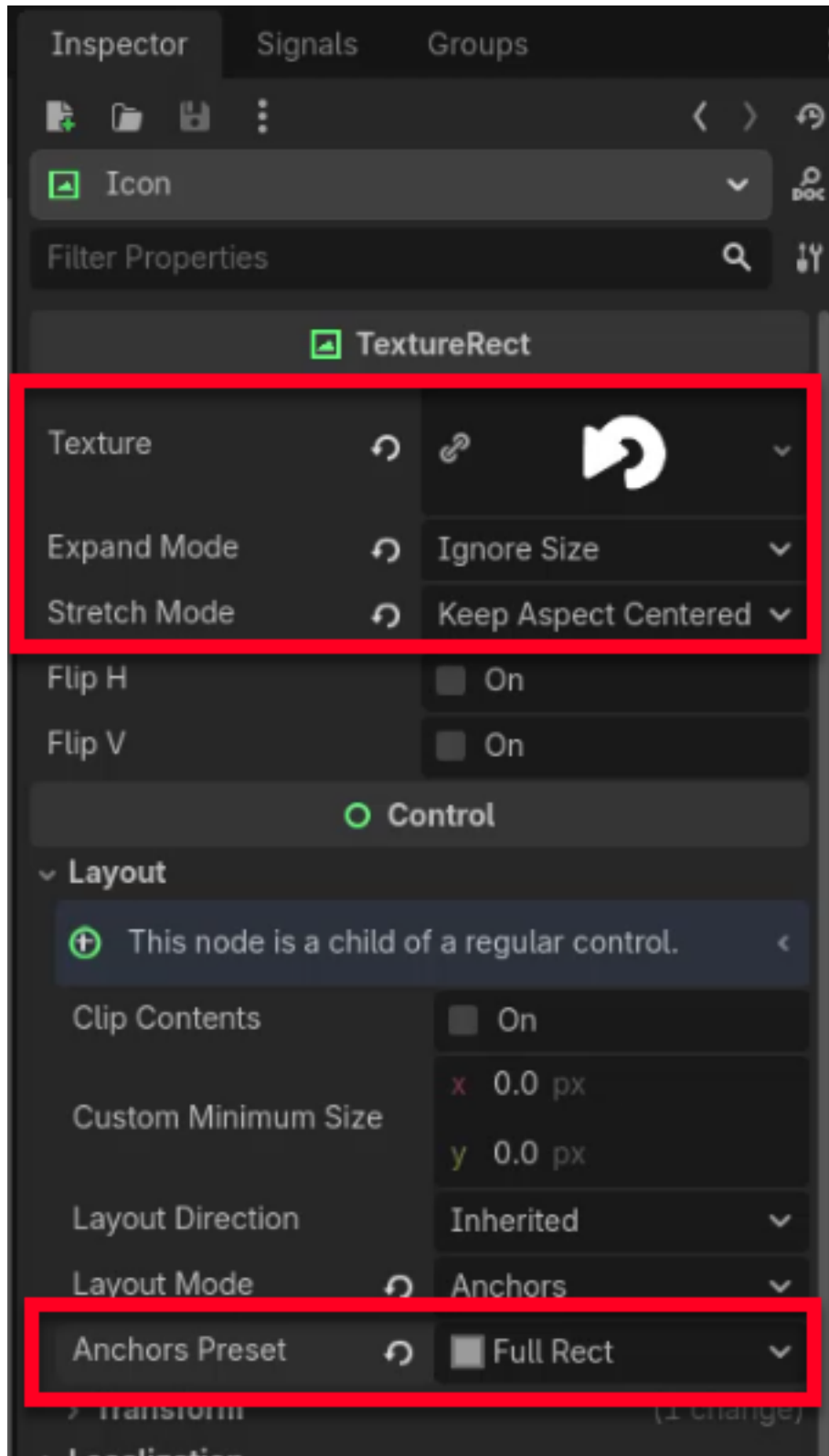
This type of button uses textures instead of text. In the Inspector, under **Textures**, drag the `button_square_brown.png` sprite into the **Normal** texture slot. To allow the button to be resized without distortion, enable **Ignore Texture Size** and set its **Stretch Mode** to **Keep Aspect Centered**.



To add a visual cue, add a TextureRect as a child of the RestartButton and name it Icon.

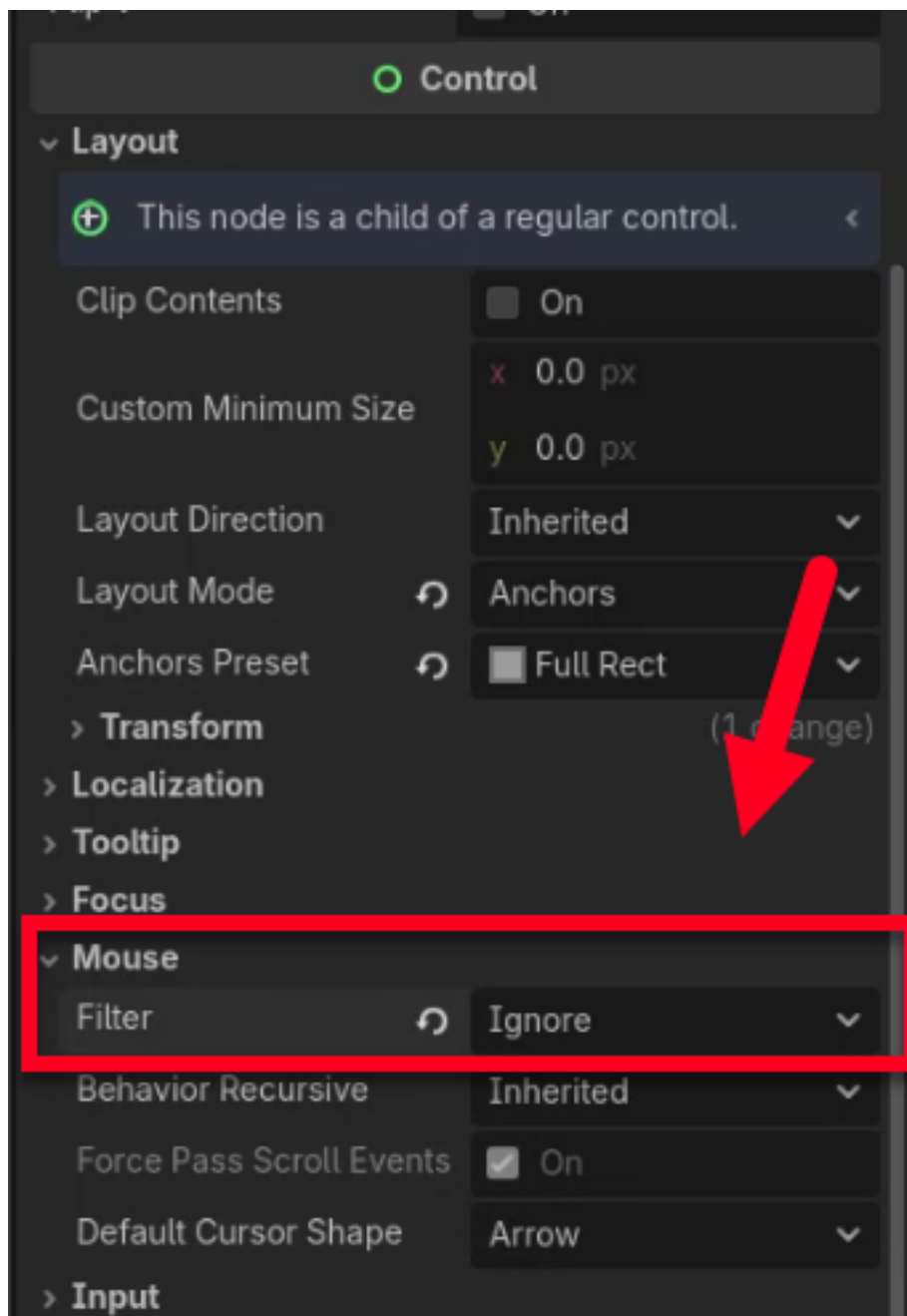


Then, assign the `restart.png` sprite from `res://Sprites/Icons/` to its **Texture** property. To make the icon fill the button, set its **Expand Mode** to **Ignore Size**, **Stretch Mode** to **Keep Aspect Centered**, and under **Layout** > **Layout Mode**, change it to **Anchors** and set the preset to **Full Rect**.

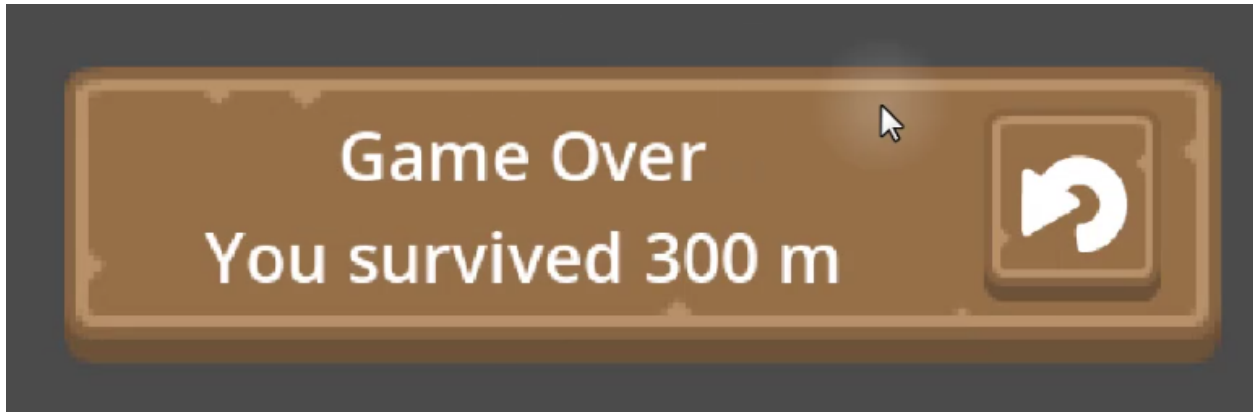


To ensure that clicking the icon still triggers the button, select the Icon node and in the Inspector, find the **Mouse** section and set its **Filter** property to

Ignore. This allows mouse input to pass through the icon to the button beneath it.



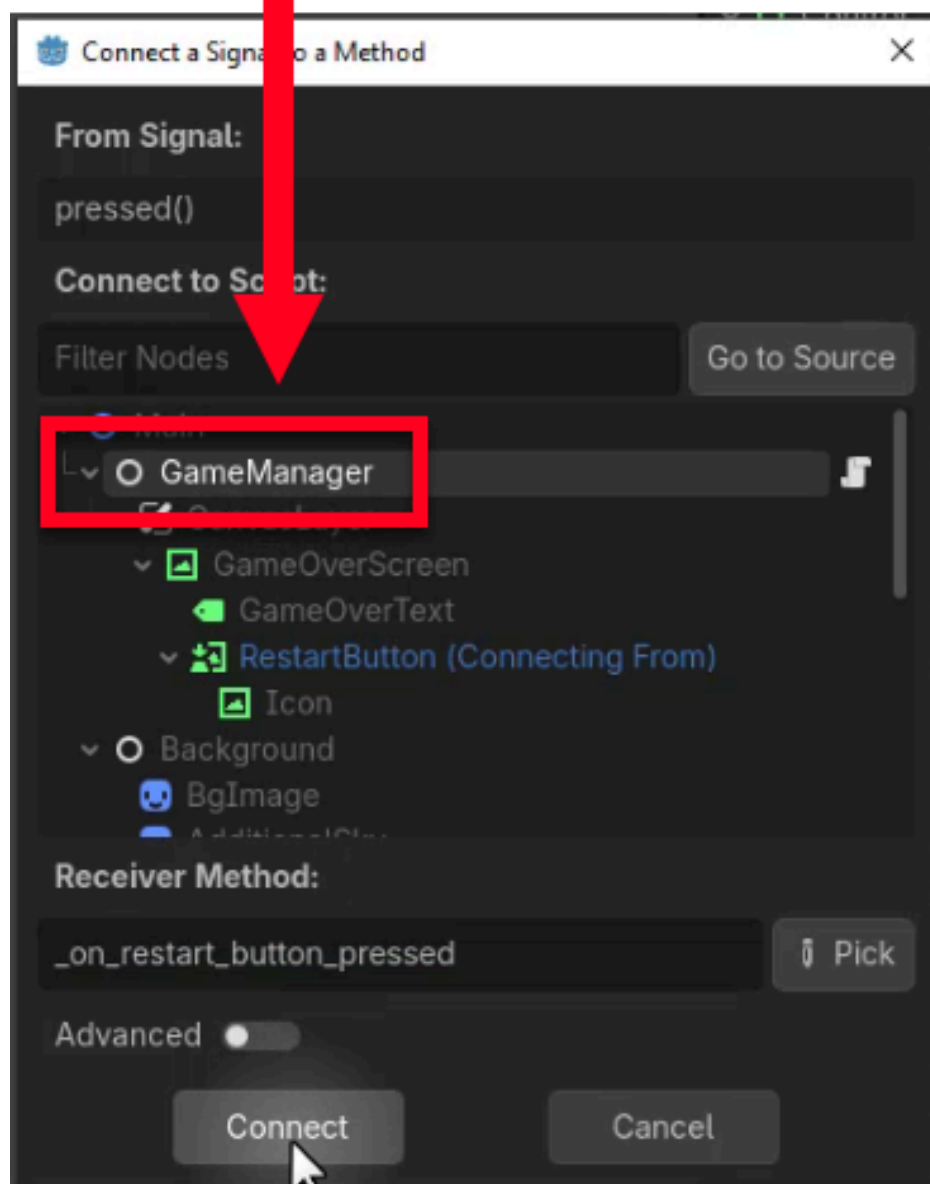
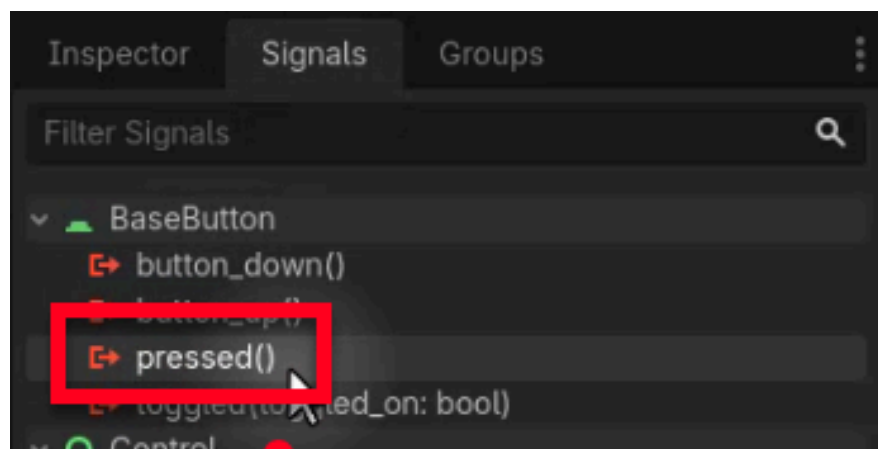
Finally, position the button elements (and text if needed) as desired until you get a clean-looking UI as shown below. Don't be afraid to resize the main game over screen box as well to get a look at the fit you like.



Scripting the Restart Button

Now, let's make the restart button actually functional. We will connect its pressed signal to a function in our GameManager script.

1. Select the RestartButton node.
2. Go to the **Node** tab next to the Inspector and select the **Signals** tab.
3. Double-click the pressed() signal.
4. In the connection window, select the GameManager node as the target and click **Connect**.



This will create a new function in `game_manager.gd`. To restart the game, we simply need to reload the current scene.

```
func _on_restart_button_pressed():  
    # Reload the entire scene to reset all state  
    get_tree().reload_current_scene()
```

Managing Screen Visibility

While we have our screen partially functional, the game over screen should only be visible when the game has ended. We will control its visibility from the `GameManager` script.

First, we must get a reference to the `GameOverScreen` node.

```
@onready var game_over_screen : TextureRect =  
$CanvasLayer/GameOverScreen
```

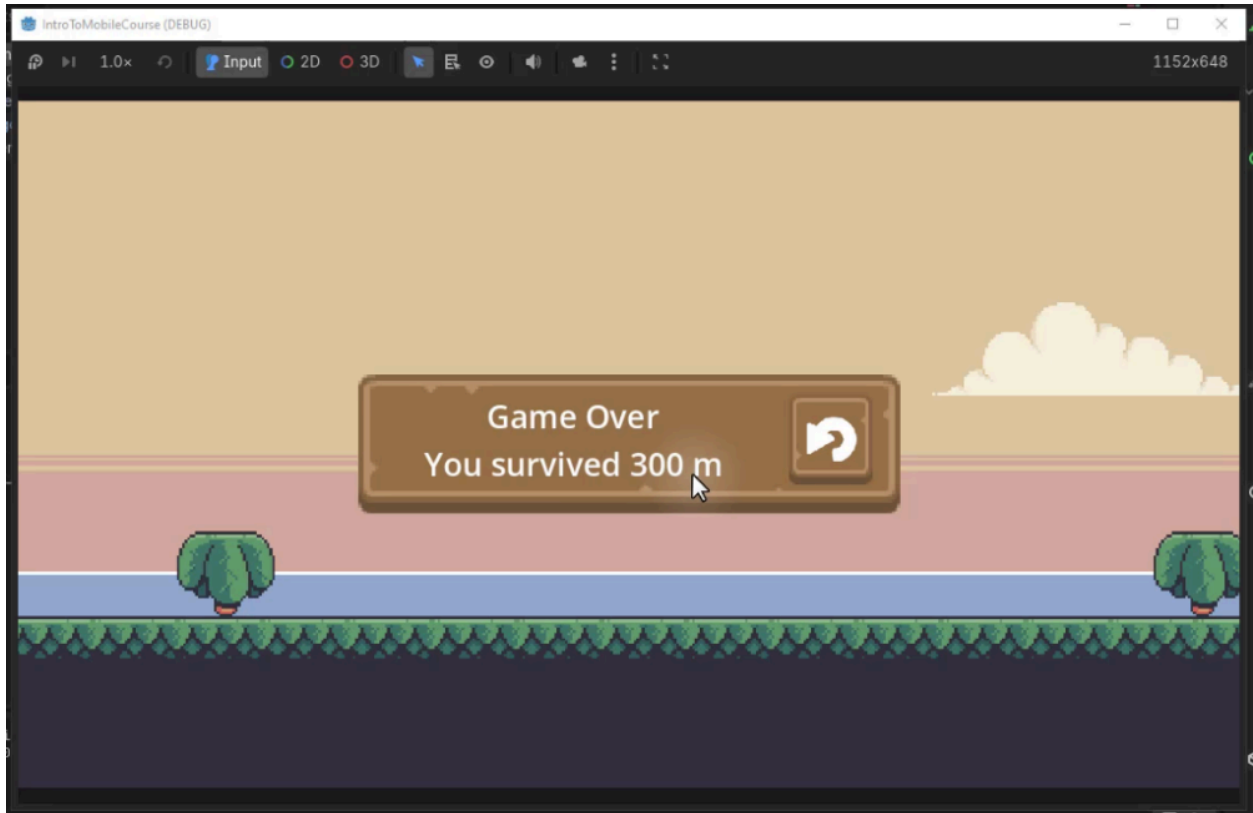
In the `_ready()` function, we'll hide the screen by default.

```
func _ready():  
    # Hide the game over panel until the player actually loses  
    game_over_screen.visible = false
```

Then, in the `game_over()` function, make it visible again.

```
func game_over():  
    # ... existing code ...  
    game_over_screen.visible = true
```

Now the screen will dynamically appear after the player actually loses the game.



Scoring and HUD

To give the player a score, we will track the distance they travel. We can calculate this using our `current_speed` variable. We will display this distance on the game over screen and also create a live counter (HUD) during gameplay.

Tracking Player Distance

First, let's declare a new variable in `game_manager.gd` to store the distance.

```
var distance : float = 0
```

Next, in the `_process(delta)` function, we will increment the distance based on the current speed. To make the number more readable and represent “meters,” we will scale it down. Dividing by 32 provides a good-feeling progression.

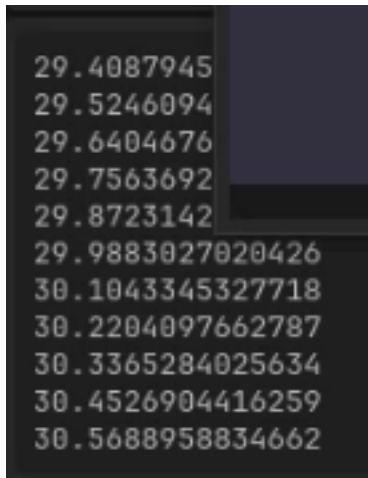
```

func _process(delta):
    # Stop updating once the game ends
    if is_game_over:
        return

    if current_speed < max_speed:
        # Gradually ramp up difficulty over time
        current_speed += speed_increase_rate * delta
        # Convert speed into a human-readable "meters" value
        distance += current_speed / 32 * delta

```

You can test this by temporarily adding `print(distance)` inside the `_process` function to see the value increase in the output log as you play. Note that unlike the output log, our HUD will only display whole numbers.



A screenshot of a game's output log showing a sequence of floating-point numbers representing distance over time. The numbers are displayed in a dark, monospaced font on a black background. The sequence starts with 29.4087945 and increases to 30.5688958834662.

```

29.4087945
29.5246094
29.6404676
29.7563692
29.8723142
29.9883027020426
30.1043345327718
30.2204097662787
30.3365284025634
30.4526904416259
30.5688958834662

```

Displaying the Final Distance

Currently, our game over screen appears when the player collides with an obstacle, but it doesn't provide any feedback on their performance. As you'll remember, our text is merely static right now. Let's update the text on the game over screen to show the final distance traveled.

First, we need to modify the `game_over` function in our `game_manager.gd` script. We will access the `GameOverText` label and set its `text` property to a custom message that includes the distance.

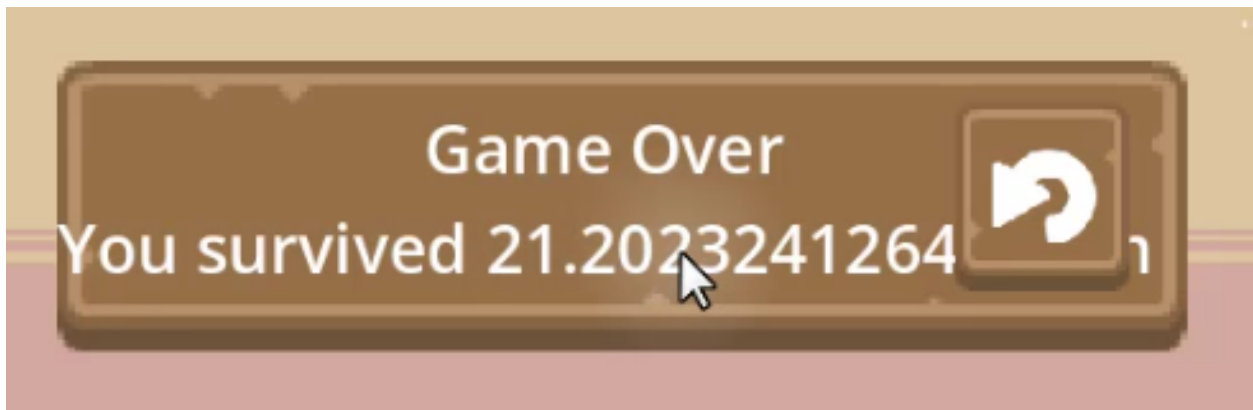
Locate the `game_over` function and add the following line to update the text. Note that `\n` creates a new line in the text.

```
# Called by the player script when an obstacle is hit
func game_over ():
    current_speed = 0.0
    is_game_over = true

    game_over_screen.visible = true

    $CanvasLayer/GameOverScreen/GameOverText.text = "Game Over\nYou
survived " + str(distance) + " m"
```

If you run the game now and collide with an obstacle, you will see the game over message. However, the distance is displayed with many decimal places (e.g., “21.123456 m”).



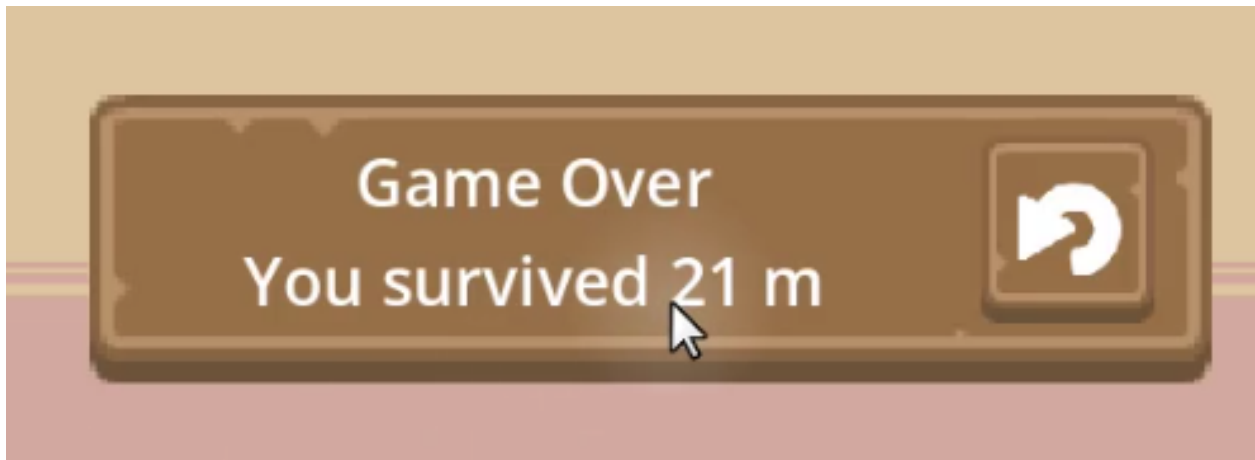
To make it cleaner, we can convert the distance from a float to an integer before converting it to a string. This will remove the numbers after the decimal point, which is fine for this purpose since we don’t need fine-tuned precision.

```
# Called by the player script when an obstacle is hit
func game_over ():
    current_speed = 0.0
    is_game_over = true

    game_over_screen.visible = true

    $CanvasLayer/GameOverScreen/GameOverText.text = "Game Over\nYou
survived " + str(int(distance)) + " m"
```

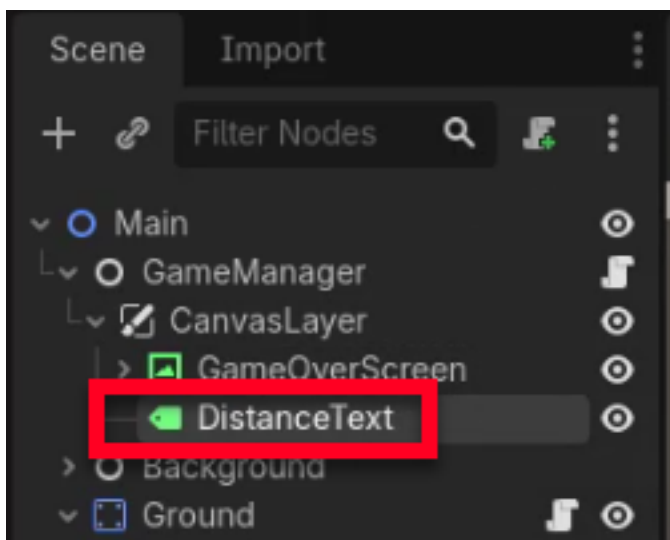
Now, the distance will be displayed as a clean, whole number, providing much clearer feedback to the player.



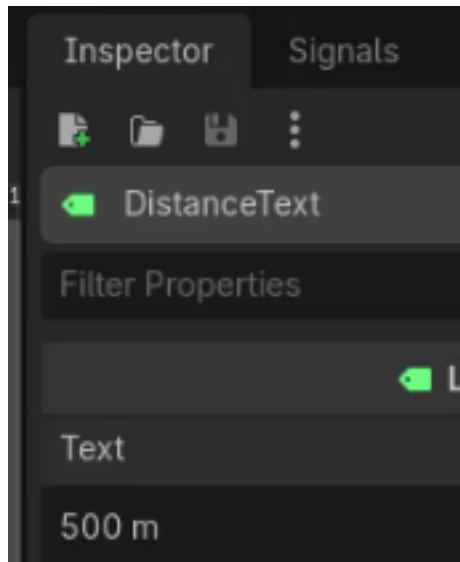
Creating a Live Distance Counter

Next, let's add a text label to the main game screen that updates in real-time to show the player's current distance. This gives the player a constant sense of progress.

First, we will create the UI element in the scene editor. Open your main game scene and, in the Scene dock, right-click on the CanvasLayer node and add a Label child node. We will rename the new label to DistanceText.



With DistanceText selected, go to the Inspector. In the Text property, enter a placeholder like “500 m” to help with positioning and styling.

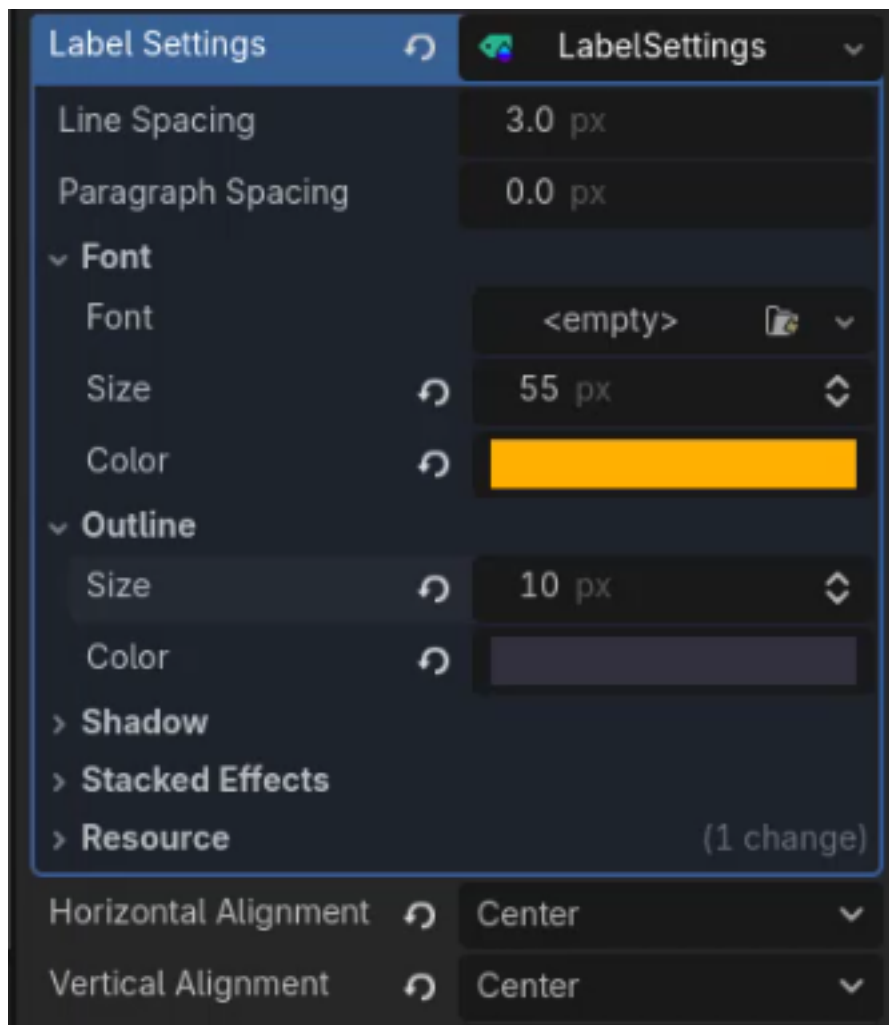


To position it, use the Layout menu at the top of the 2D viewport or the Inspector to select the **Top Wide** anchor preset. You can then drag it down slightly if needed so it's not right against the top edge.

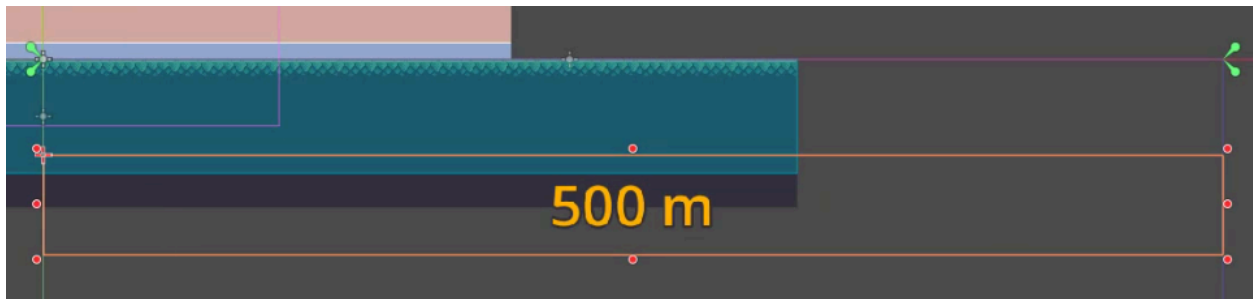


To style the text, navigate to the Inspector and adjust the following properties:

- Under the **Label** section, set Horizontal Alignment and Vertical Alignment to **Center**.
- Create a new LabelSettings resource and expand them..
- Inside the new resource, increase the **Font Size** to a suitable value (55px).
- Change the **Font Color** to something visible, like a yellowish hue (#ffb200).
- Under **Outline**, you can add a small outline (e.g., **Outline Size** of 10) and set its **Outline Color** to a contrasting color like dark purple (#33323d).



Preview in the editor and make additional tweaks as you would like. You can use whatever values look better to you for this visual element!



Updating the Distance in Real-time

With the UI element in place, we need to connect it to our script so we can update its text dynamically.

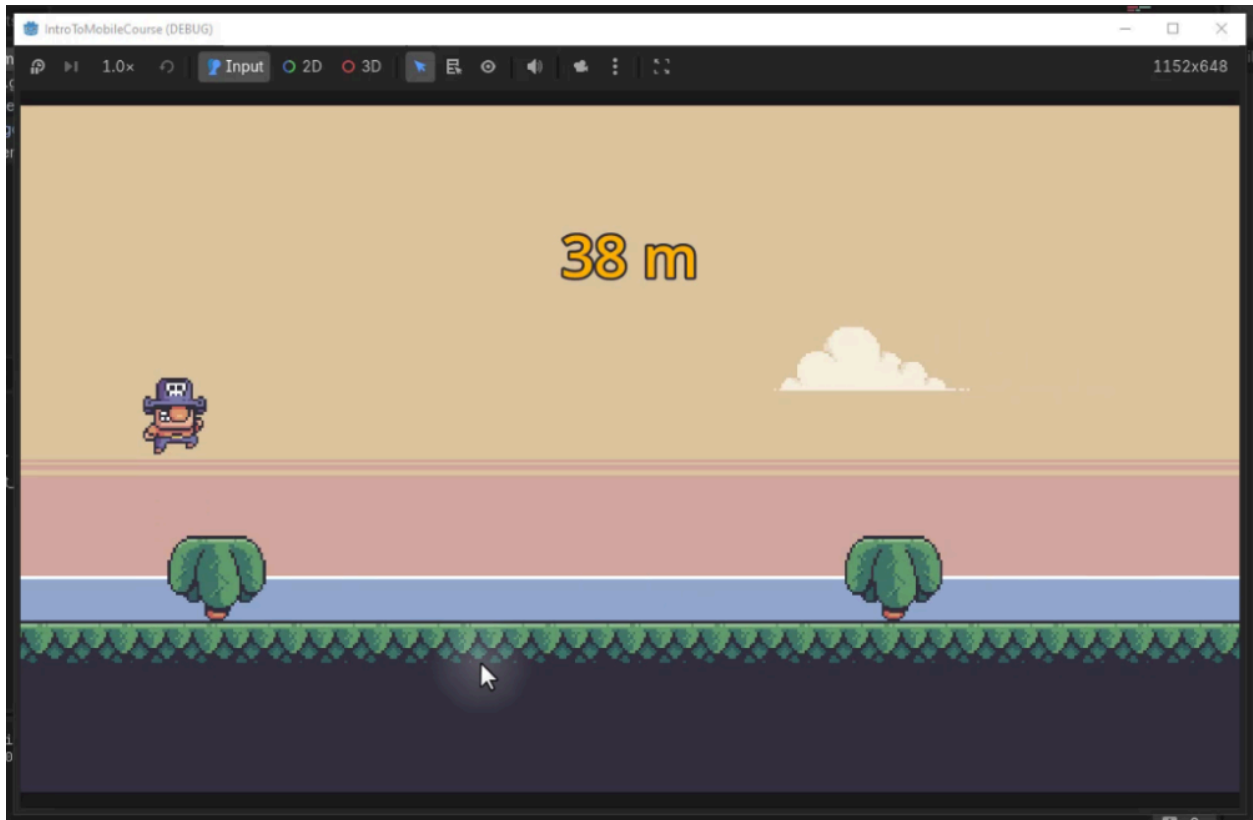
First, let's get a reference to our new DistanceText label in the `game_manager.gd` script. We can do this by creating a new `@onready` variable at the top of the script.

```
@onready var game_over_screen : TextureRect =  
$CanvasLayer/GameOverScreen  
@onready var distance_text : Label = $CanvasLayer/DistanceText
```

Now that we have a reference, we can update its text every frame within the `_process` function. This is where we also calculate the distance, making it the perfect place for this logic.

```
func _process(delta):  
    # Stop updating once the game ends  
    if is_game_over:  
        return  
  
    if current_speed < max_speed:  
        # Gradually ramp up difficulty over time  
        current_speed += speed_increase_rate * delta  
        # Convert speed into a human-readable "meters" value  
        distance += current_speed / 32 * delta  
        distance_text.text = str(int(distance)) + " m"
```

Now, when you run the game, you will see the distance counter at the top of the screen, updating in real-time as you play. When you hit an obstacle, the game over screen will appear with your final score. This completes the implementation of our game's scoring and feedback system.



In the next chapter, we will set up our main menu. We will also add some other UI elements to give our game a bit more quality of life for our players.

User Interface and Menus

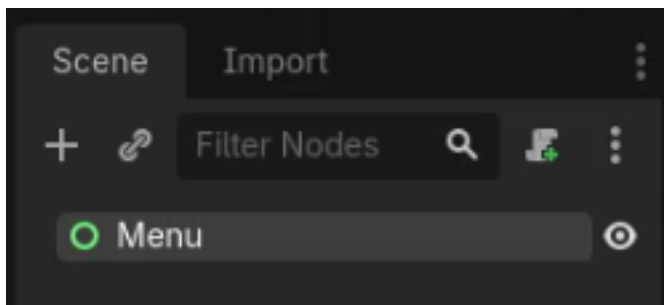
In this chapter, we will focus on creating a dedicated menu scene for our game. This scene will serve as the entry point for the player, featuring a title and a button to start the game. We will construct the visual elements and then add the necessary script to transition to the main game scene upon clicking the play button.

Following that, we will enhance our game's user interface by adding two essential buttons to the main game loop: a “Home” button to return to the main menu and a “Pause” button to temporarily halt the gameplay. These features are standard in many games and will significantly improve the overall player experience.

Creating the Menu Scene

First, we need a new scene to work with. This scene will be a “User Interface” type, which provides a Control node as the root. This is ideal for UI-focused scenes like menus.

1. Navigate to the top menu and select **Scene > New Scene**.
2. In the “Create New Scene” dialog, select **User Interface**.
3. Click **Create** to generate a new scene with a **Control** root node, then rename it to **Menu**.
4. Save the scene (Ctrl+S) inside your Scenes folder, naming it `menu.tscn`.



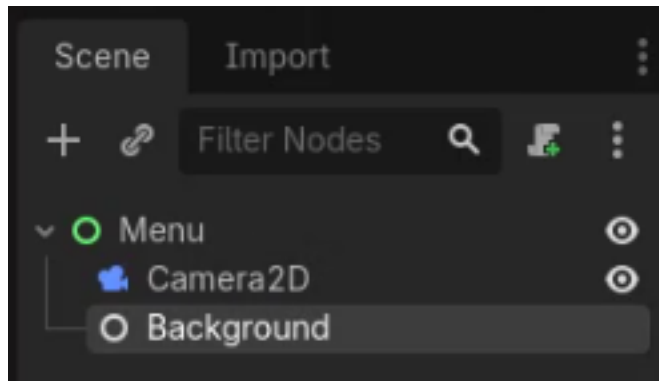
Our menu will have two main components: a static background with some decorative elements and the UI elements that the player interacts with. Let's get started.

The Background

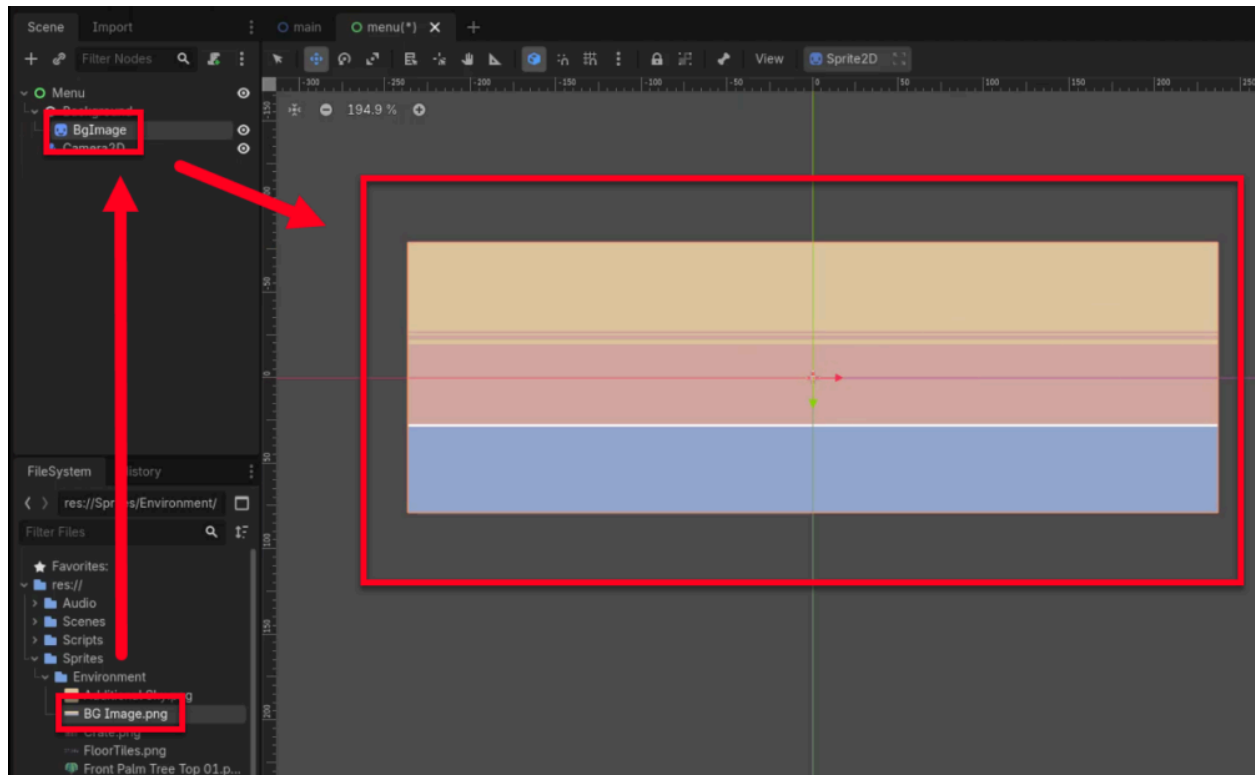
For our background, we will re-use some of the visual sprite elements we used in the main game.

However, because our menu scene uses a Control node as its root (a UI-type scene), Godot does not automatically set up a 2D viewport camera. Without a Camera2D, any Sprite2D nodes we add would have no camera to render them, so they would not appear on screen. Adding a Camera2D gives us a working 2D viewport alongside our UI layer.

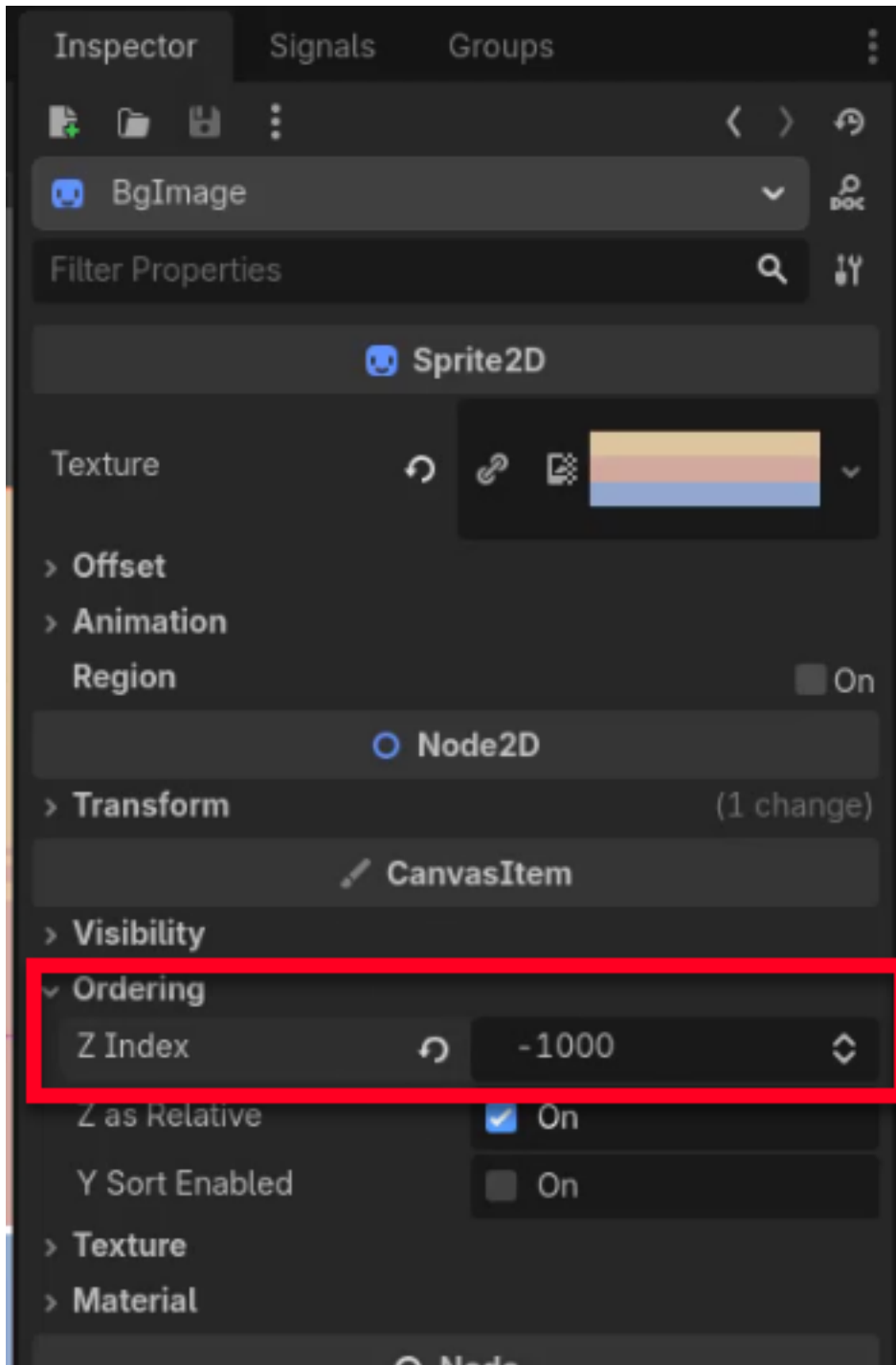
As such, add a **Camera2D** node as a child of the **Menu** node. After which, we can add a **Node2D** and rename it to **Background**. This will hold our background sprites.



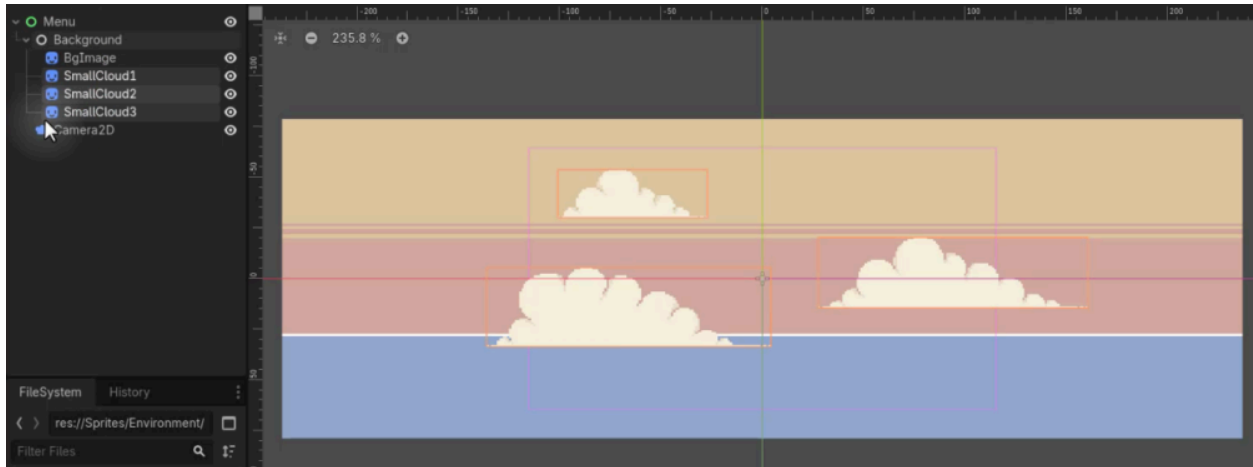
Now we can officially populate the background. Drag your background sky sprite from the FileSystem dock into the scene as a child of the **Background** node. You can scale it to fit the view.



To ensure the sky is always rendered behind other elements, select it, and in the Inspector, find the **Ordering** section. Set the **Z Index** to a large negative value, such as -1000.



Feel free to add other decorative elements, like clouds, as children of the **Background** node to create a more visually appealing environment. The exact look is up to you, so let your creativity shine here!

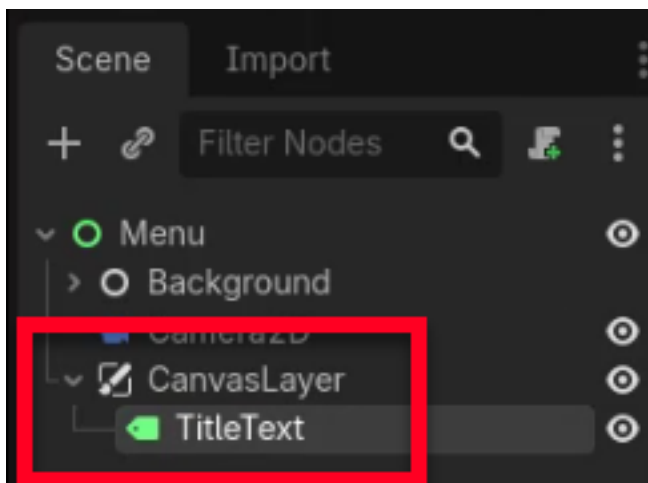


The User Interface

For the UI, we will use a **CanvasLayer**. As before, this node ensures that its children are drawn on a separate layer, on top of the rest of the 2D scene, and that they remain fixed in place regardless of camera movement.

Title Text

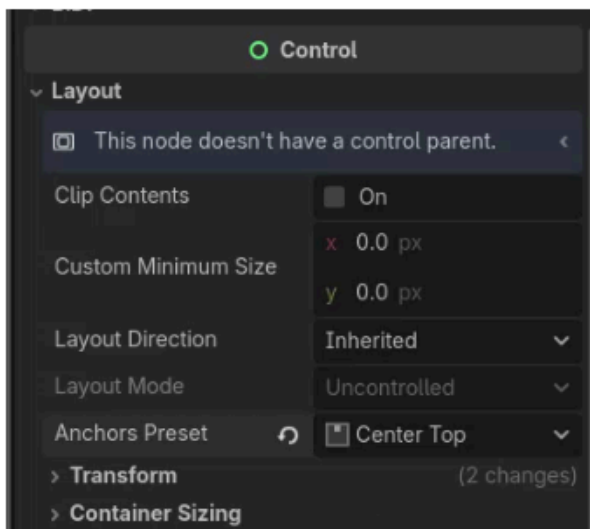
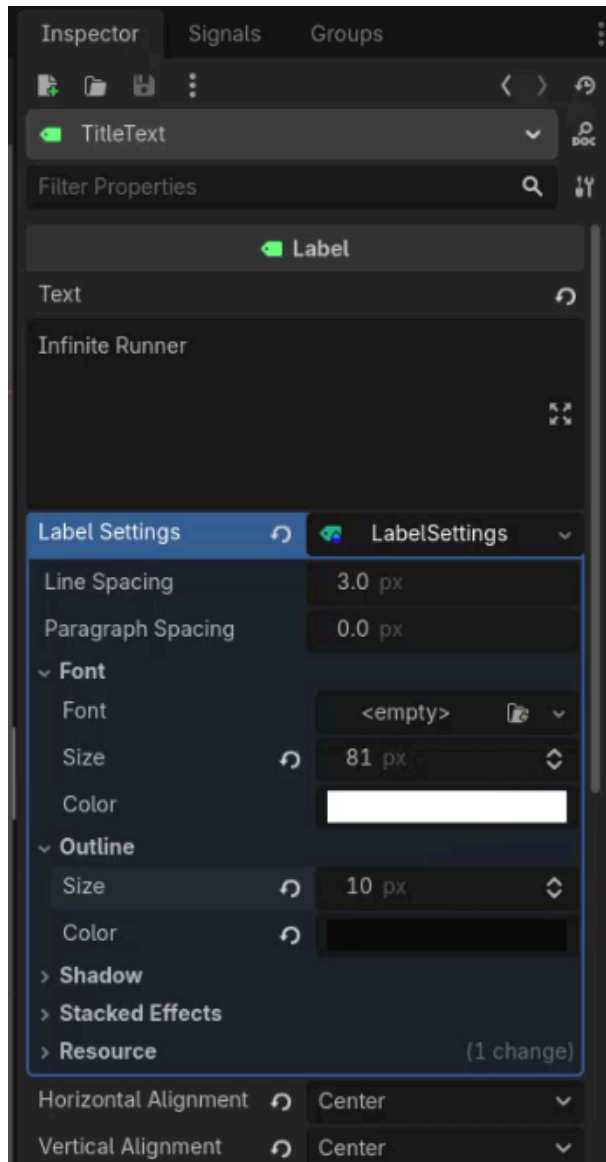
Let's add the title text first. Add a **CanvasLayer** node as a child of **Menu**, then add a **Label** child inside it and rename it to **TitleText**.



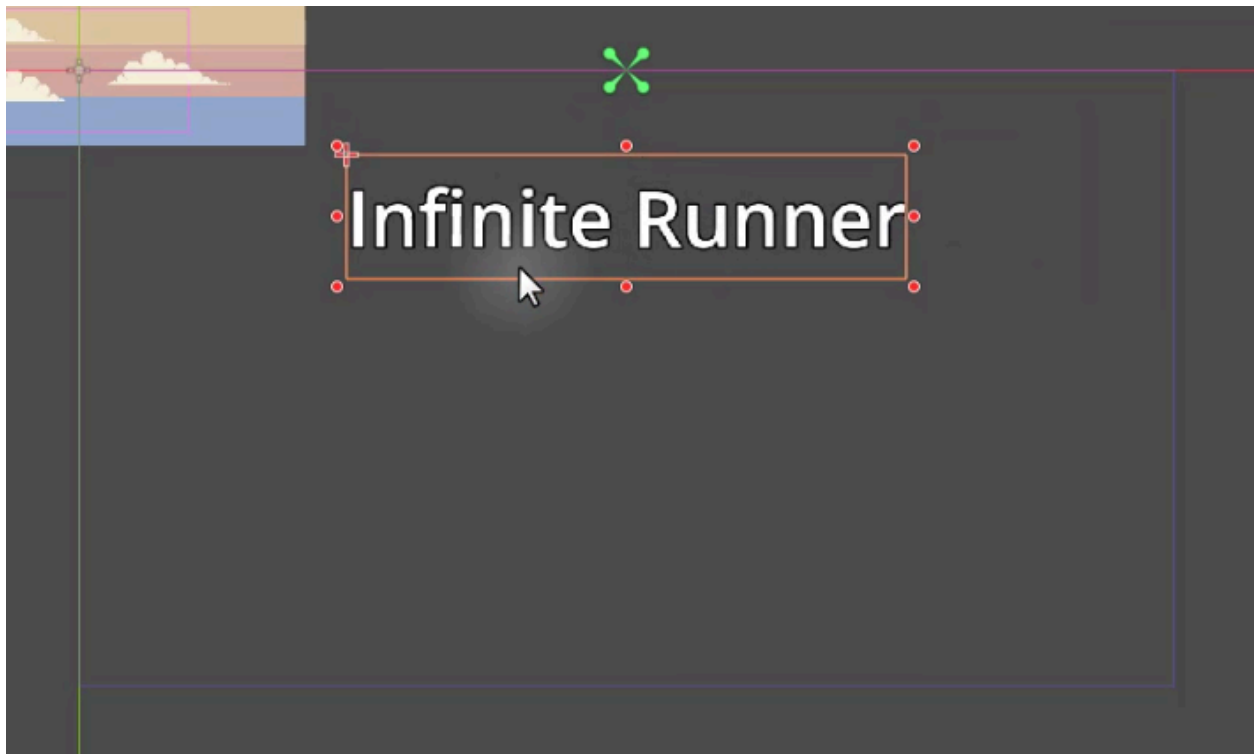
In the Inspector, set its **Text** property to "Infinite Runner". To help with positioning, find the **Anchor Presets** in the Inspector and select **Center Top**.

You can customize the font's appearance by creating new **LabelSettings** and adjusting them as you see fit. For ours, we increased the font size to 81px

and added a 10px outline. Set the Vertical and Horizontal alignment to be centered.



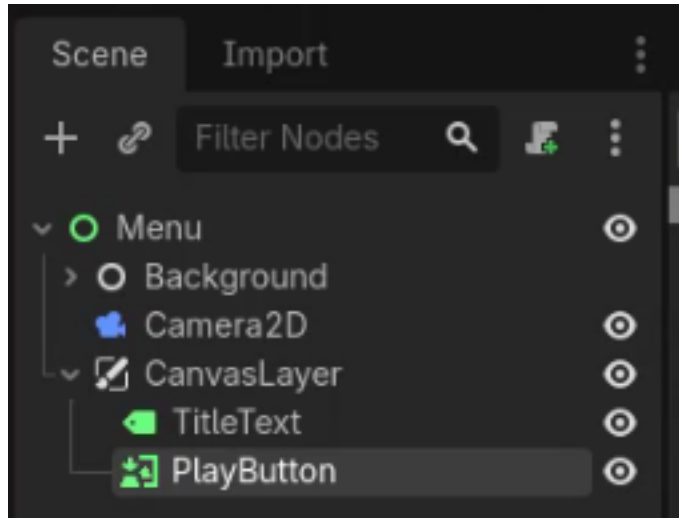
Make any other positional adjustments you want to the title text (such as moving it down). There is no wrong answer here!



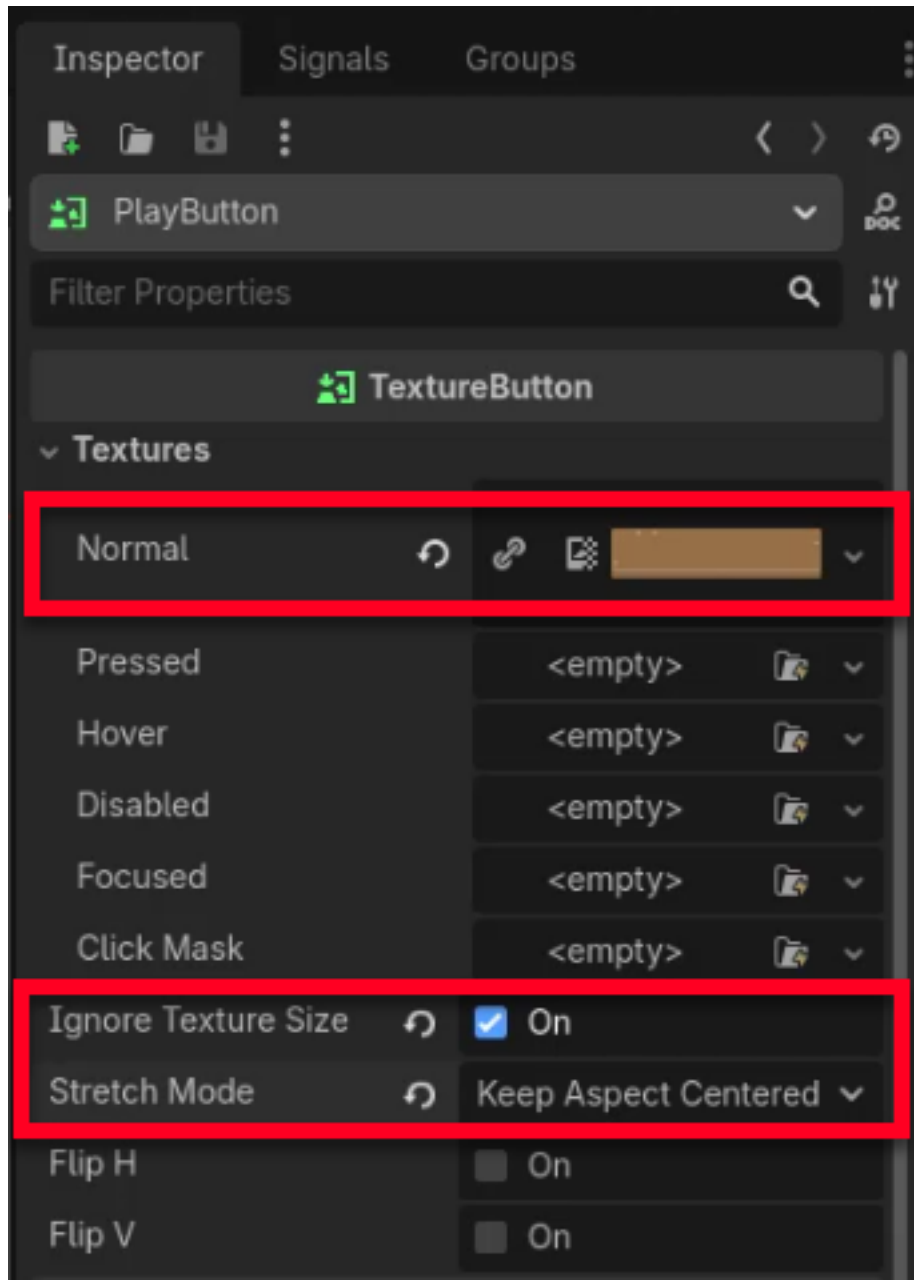
Play Button

Next, we need a button that lets us play the game. We will use a `TextureButton` instead of a regular `Button` because it lets us display a custom sprite as the button background, keeping the visual style consistent with our pixel-art theme.

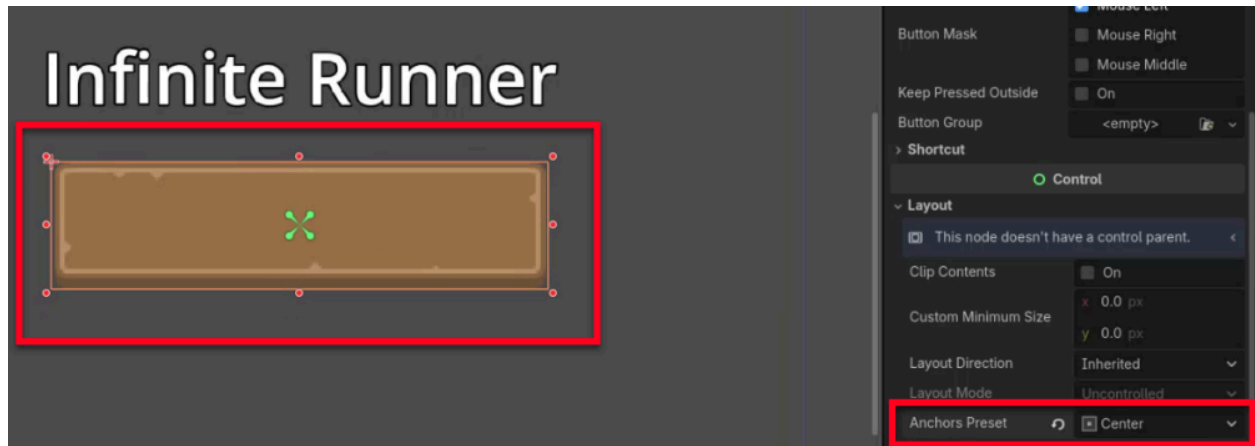
To begin, add a **`TextureButton`** as a child of the **`CanvasLayer`**. Rename it to **`PlayButton`**.



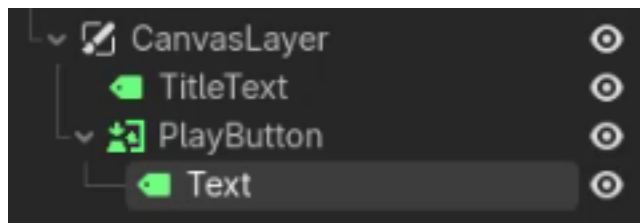
In the Inspector, under the **Textures** section, drag your button image (e.g., `button_long_brown.png`) to the **Normal** slot. By default, the button locks to the image's native resolution. To allow free resizing, enable **Ignore Texture Size** and set **Stretch Mode** to **Keep Aspect Centered** so the image scales without distortion.



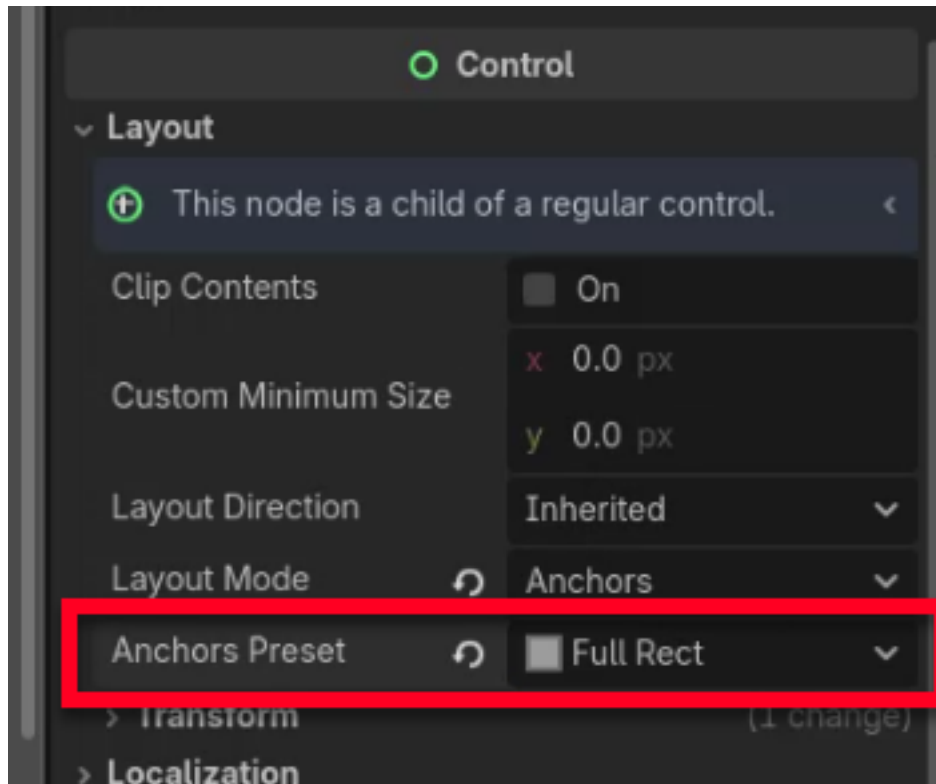
We can now position and resize the button in the center of the screen. Use the **Anchor Presets** and select **Center** to help you.



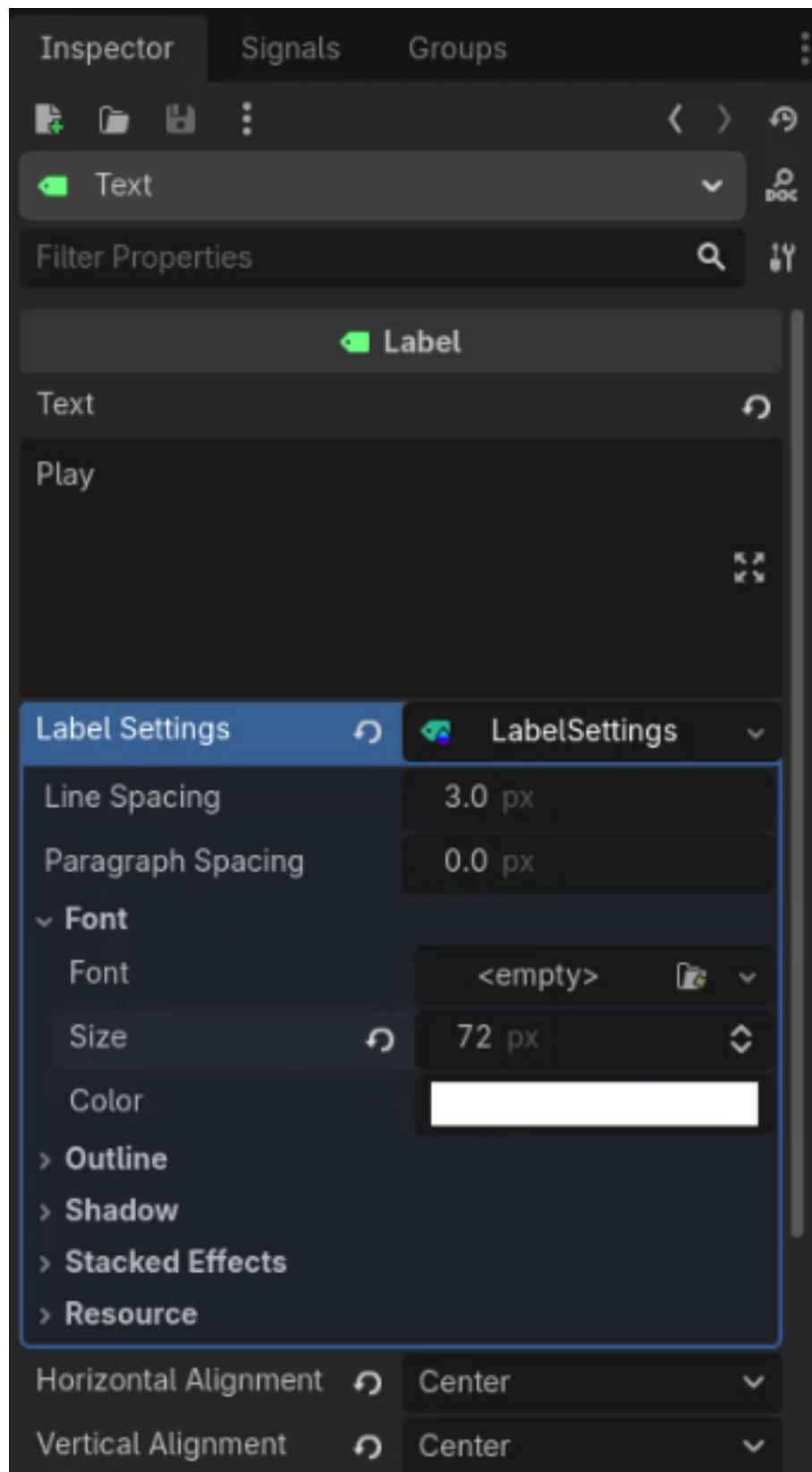
To add text to our button, add a **Label** as a child of the **PlayButton**. Name it “Text” for clear identification.



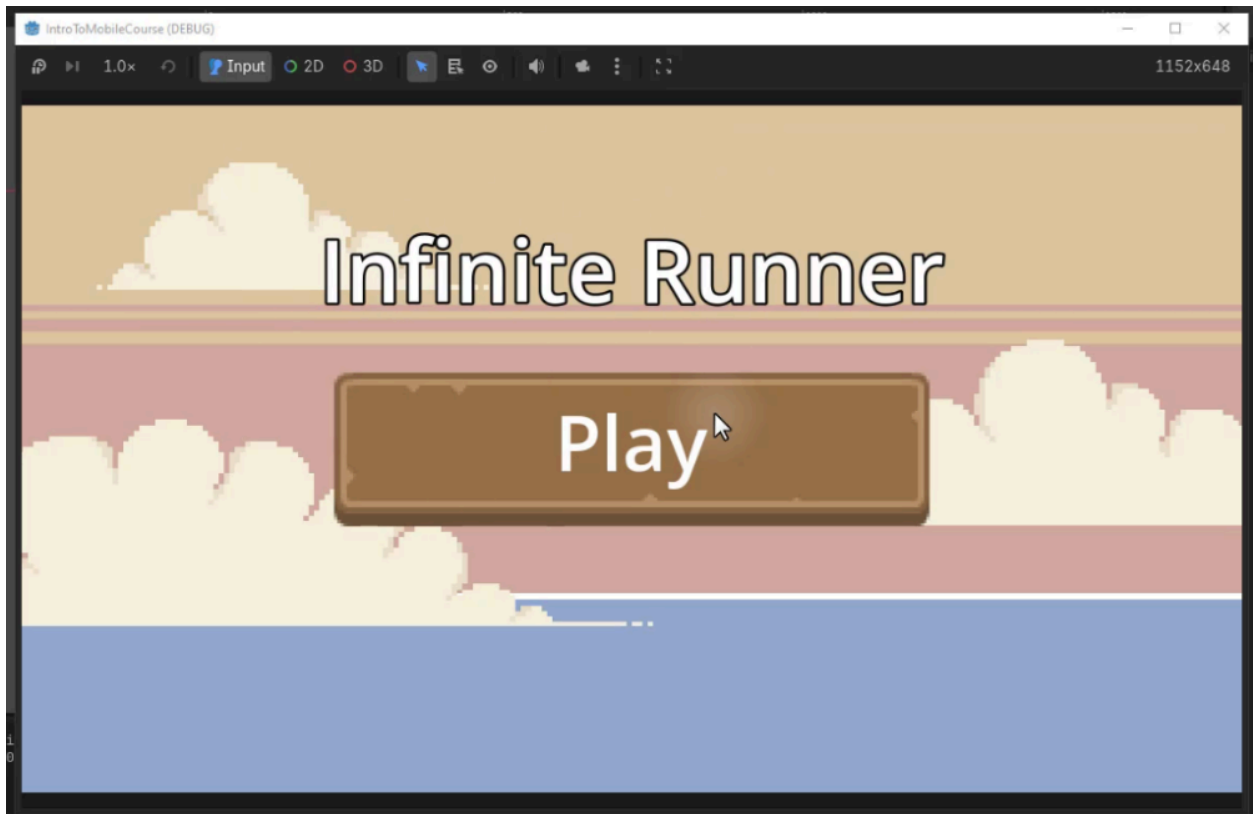
Set its **Text** property to “Play”. To make the label fill the entire button area, select it and use the **Anchor Presets** to choose **Full Rect**.



Center the text horizontally and vertically using the **Horizontal Alignment** and **Vertical Alignment** properties in the Inspector. We can then customize the font as you did for the title. We simply set our font size to 72px.



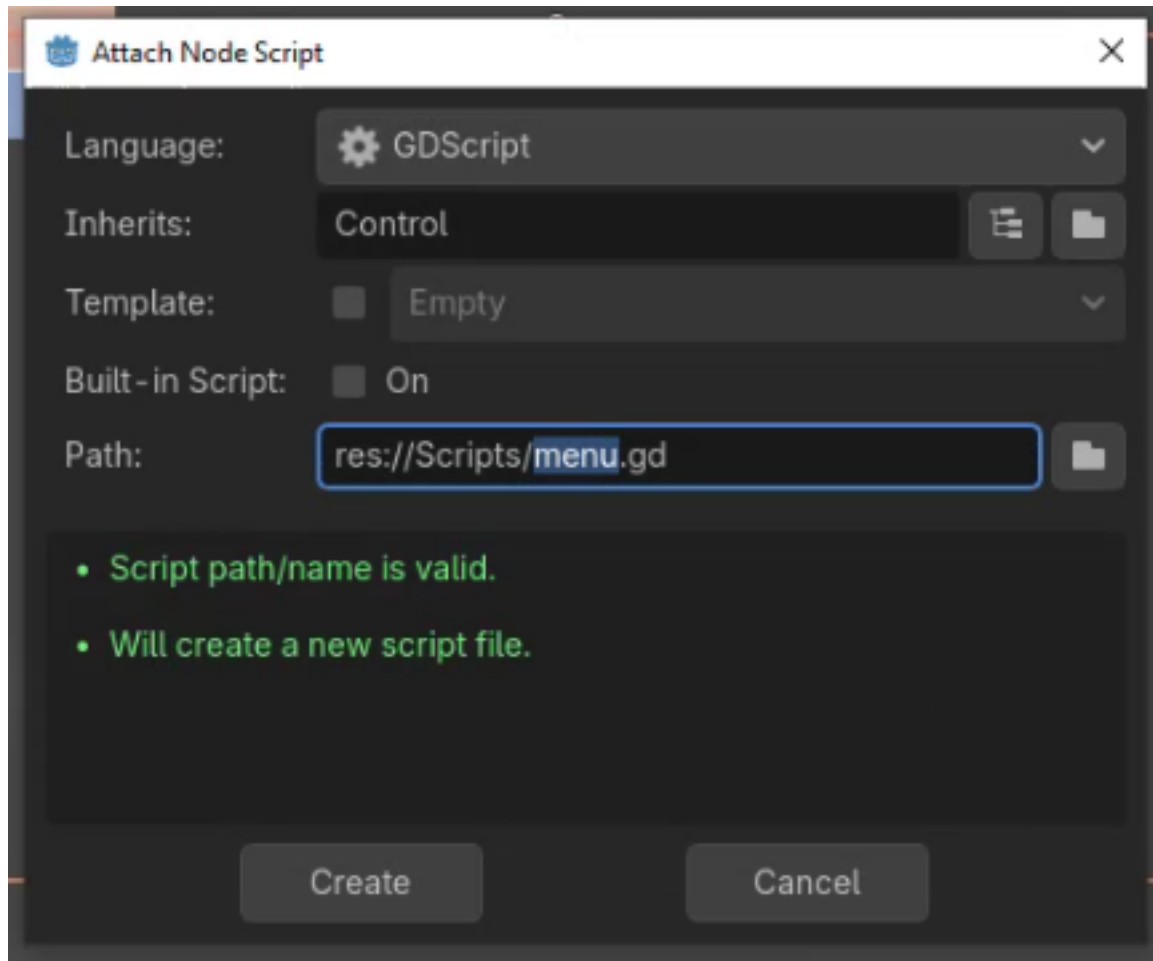
Make sure to adjust the position and size of the text as needed. You can preview your scene by pressing F6. It should display the background, title, and the play button.



Scripting the Play Button

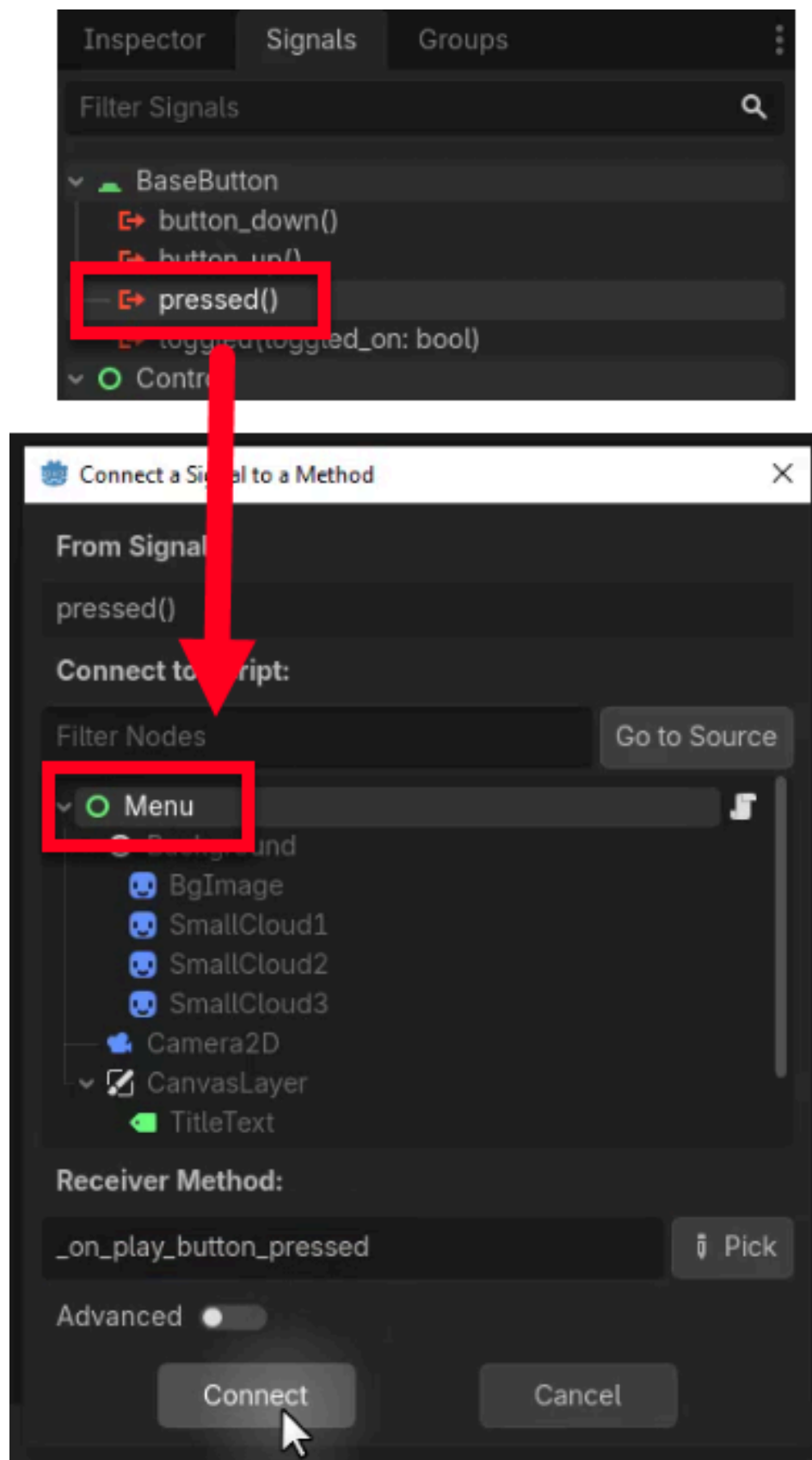
The final step is to make the play button functional. When clicked, it should load our main game scene.

First, select the root **Menu** node and attach a new script to it. Name it `menu.gd` and save it in the Scripts folder.



To connect the button's action:

1. Select the **PlayButton** node.
2. Go to the **Node** tab next to the Inspector and select the **Signals** tab.
3. Double-click the **pressed()** signal. A dialog will appear to connect the signal.
4. Ensure the **Menu** node is selected in the list and click **Connect**. This will create a new function named `_on_play_button_pressed` in your `menu.gd` script.



Inside this new function, add the code to change the scene.

```
extends Control
```

```
func _on_play_button_pressed():  
    get_tree().change_scene_to_file("res://Scenes/main.tscn")
```

This code gets the current scene tree and calls the `change_scene_to_file` method, passing the path to our main game scene. Now, when you run the menu scene and click the “Play” button, it will load and start the game.

That’s it! We now have a functional main menu to give our game a more professional feel.

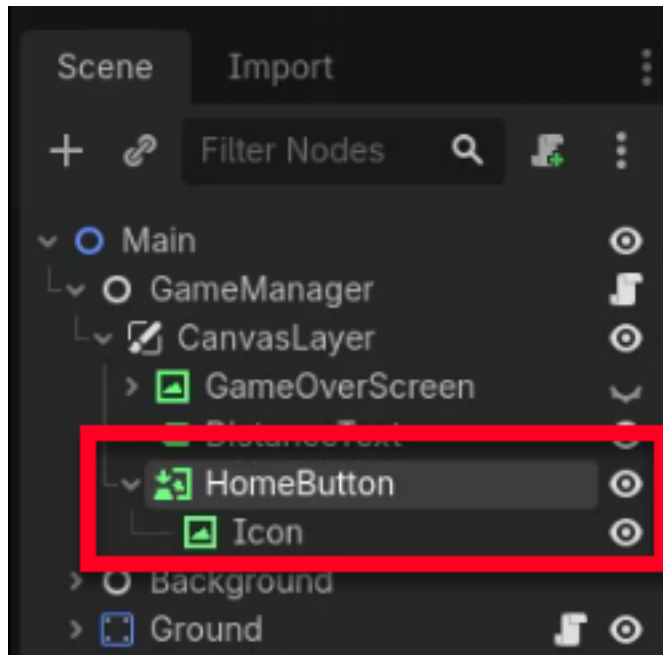
In-game UI Buttons

With our main menu complete, we need to ensure the player can navigate back to it from the game. We will also add a pause button to allow players to take a break. The last one is super important since not allowing players to pause in an “infinite” runner can cause frustration.

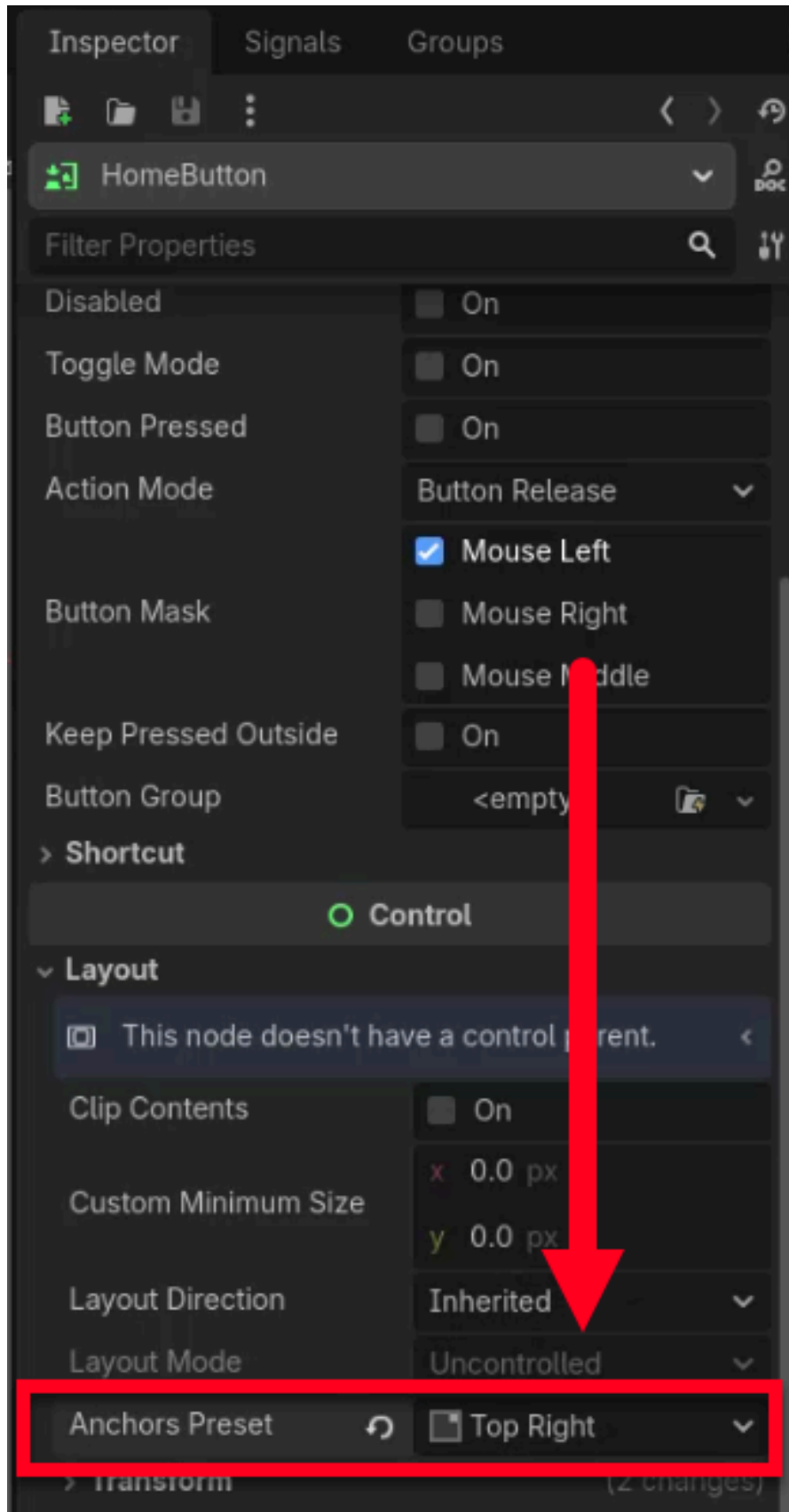
Creating the Home Button

Players need a way to exit the current run and return to the main menu without closing the app. Rather than building a new button from scratch, we can duplicate the restart button we already configured and repurpose it, saving time while keeping the visual style consistent.

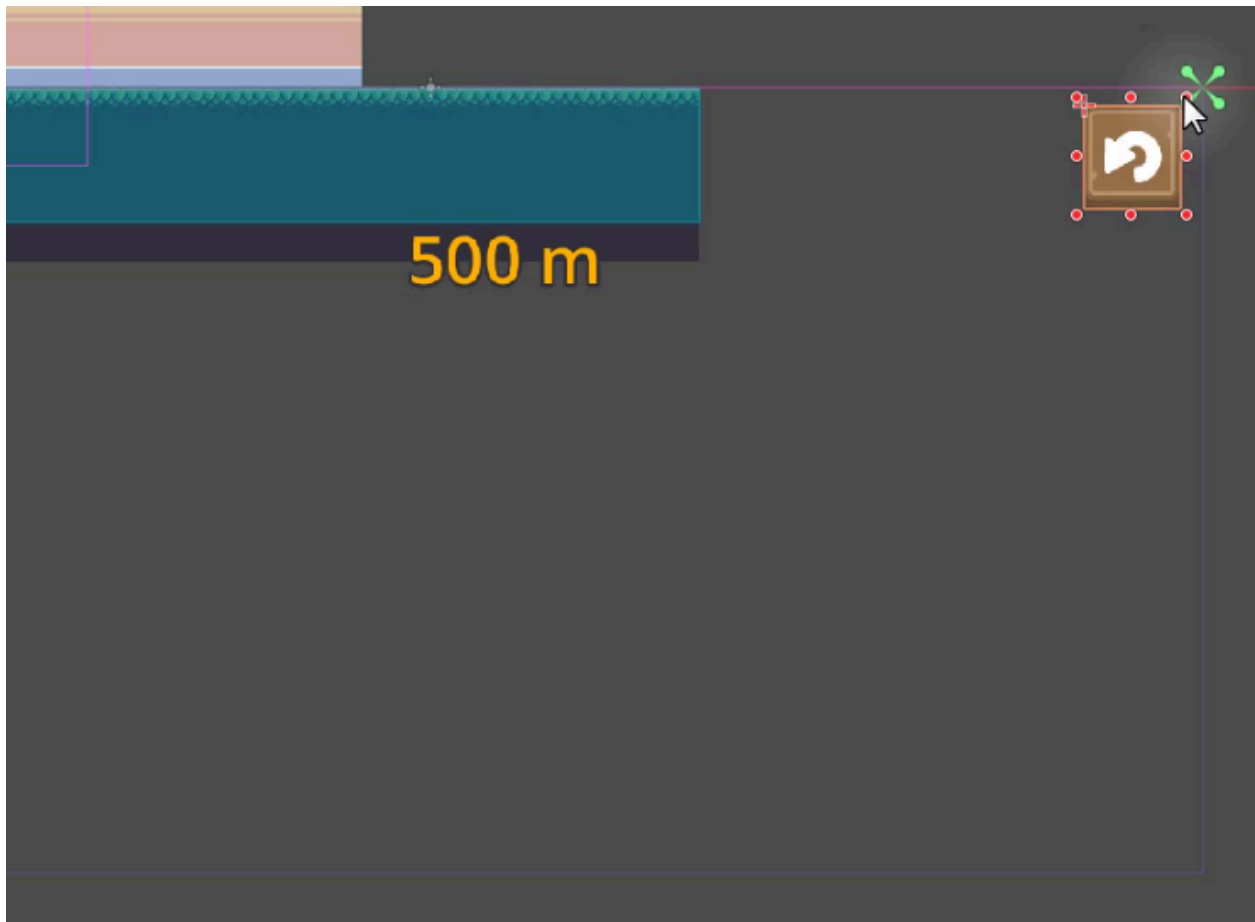
1. Navigate to your Main scene and expand the CanvasLayer node in the Scene dock.
2. Select the RestartButton (which we created in the previous chapter inside the Game Over screen), right-click it, and choose **Duplicate** (or press Ctrl+D).
3. Rename the duplicated button to HomeButton.
4. Drag the HomeButton out of the GameOverScreen node and drop it directly onto the CanvasLayer node. This ensures the button is always visible during gameplay, not just when the game is over.



With the HomeButton selected, go to the Inspector. Under the **Layout** section, set the **Anchors Preset** to **Top Right**.



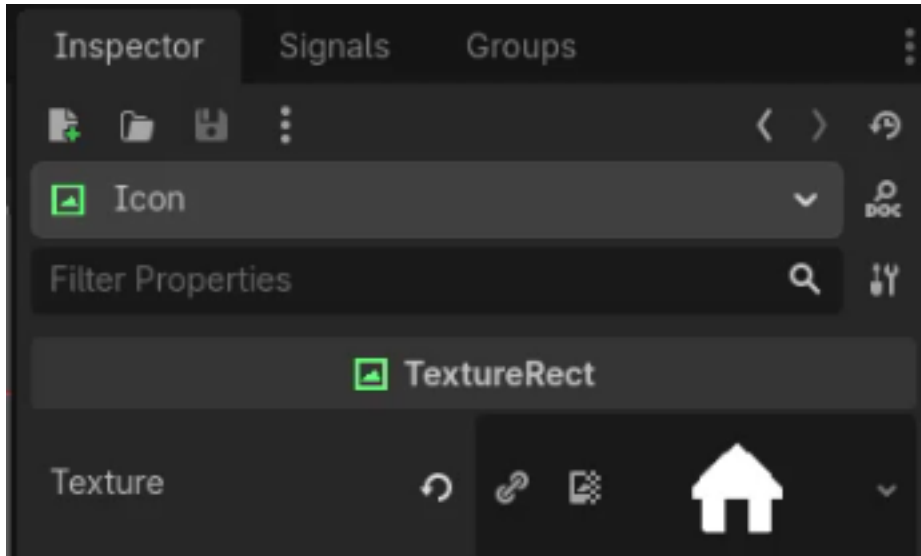
Next, we want to adjust the position of the HomeButton so it sits neatly in the top-right corner of the game view.



Implementing the Home Button Functionality

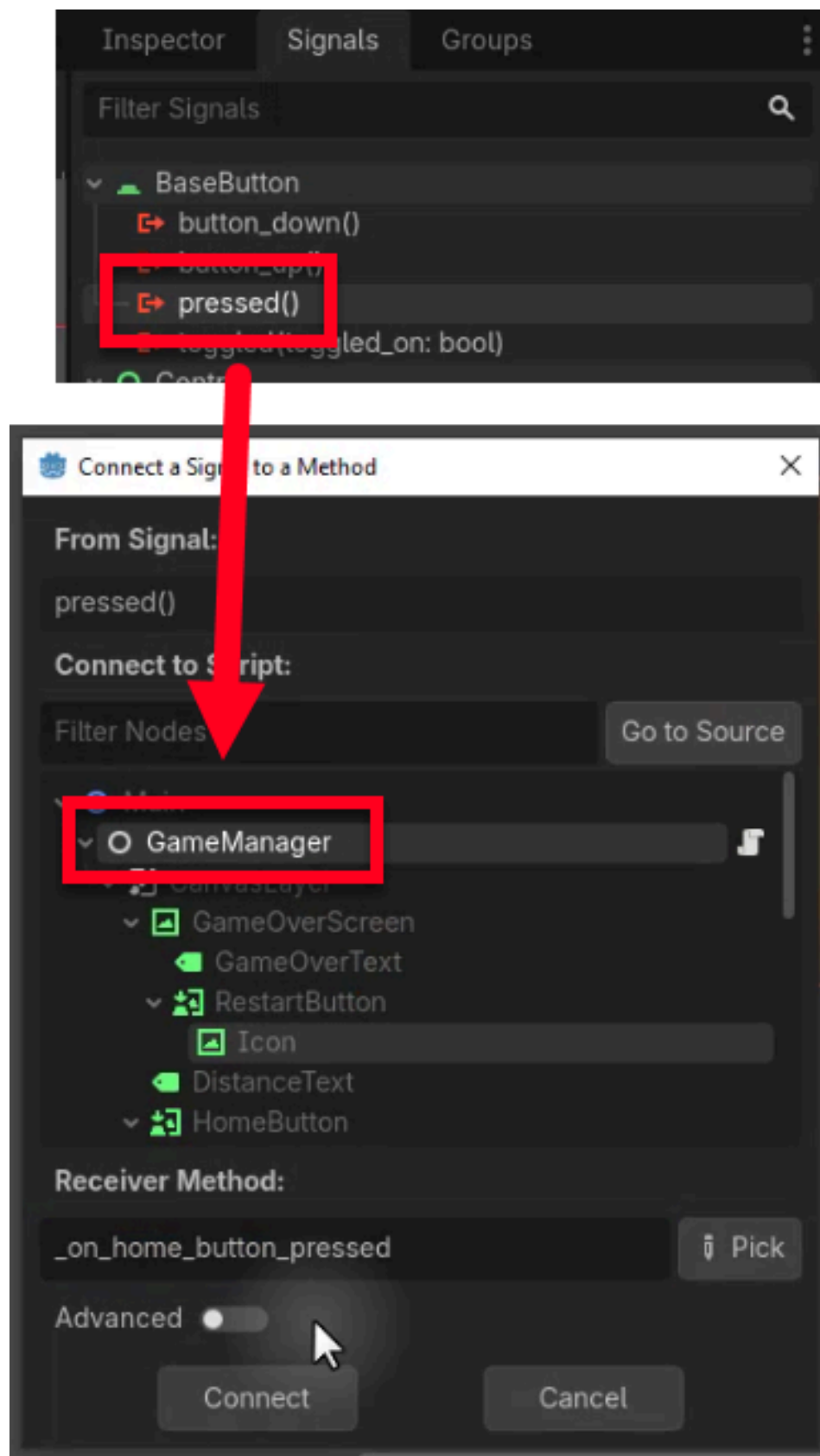
Now that the button is in place, we need to make it functional. This involves changing its appearance and connecting its signal to a script.

To begin, select the HomeButton's child Icon node. In the Inspector, find the **Texture** property and replace the current icon with the "home" icon from your project's assets (e.g., `res://Sprites/Icons/home.png`).



Next, we need to connect the button's signal.

1. Select the HomeButton node and switch to the **Node** tab to view its signals.
2. Since we duplicated this button, it might still be connected to the restart function. If so, right-click the existing connection and disconnect it.
3. Select the `pressed()` signal and click **Connect...**
4. In the "Connect a Signal" window, select the GameManager node and click **Connect**. This will create a new function in your `game_manager.gd` script.



Let's open the `game_manager.gd` script. Simply add the following code to the newly created `_on_home_button_pressed` function to load the menu scene when the button is clicked.

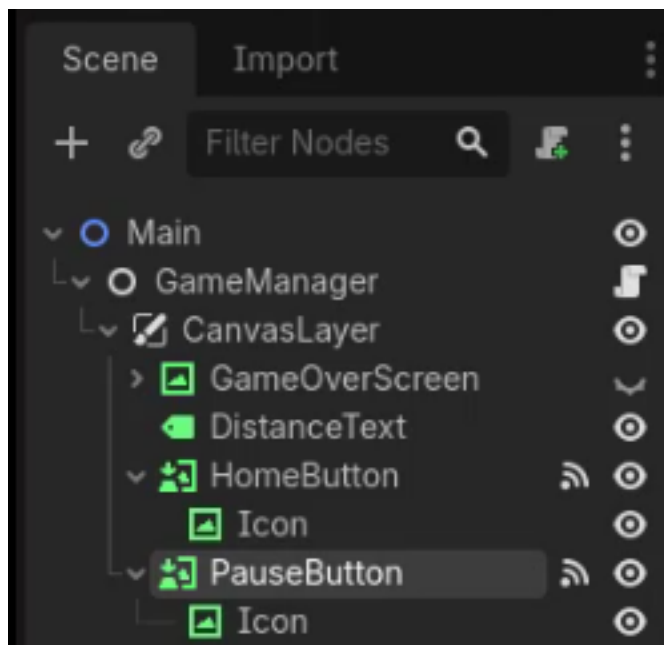
```
func _on_home_button_pressed():  
    get_tree().change_scene_to_file("res://Scenes/menu.tscn")
```

Running the game now will show the home button, and clicking it will take you back to the main menu, creating a complete loop between your game and menu scenes.

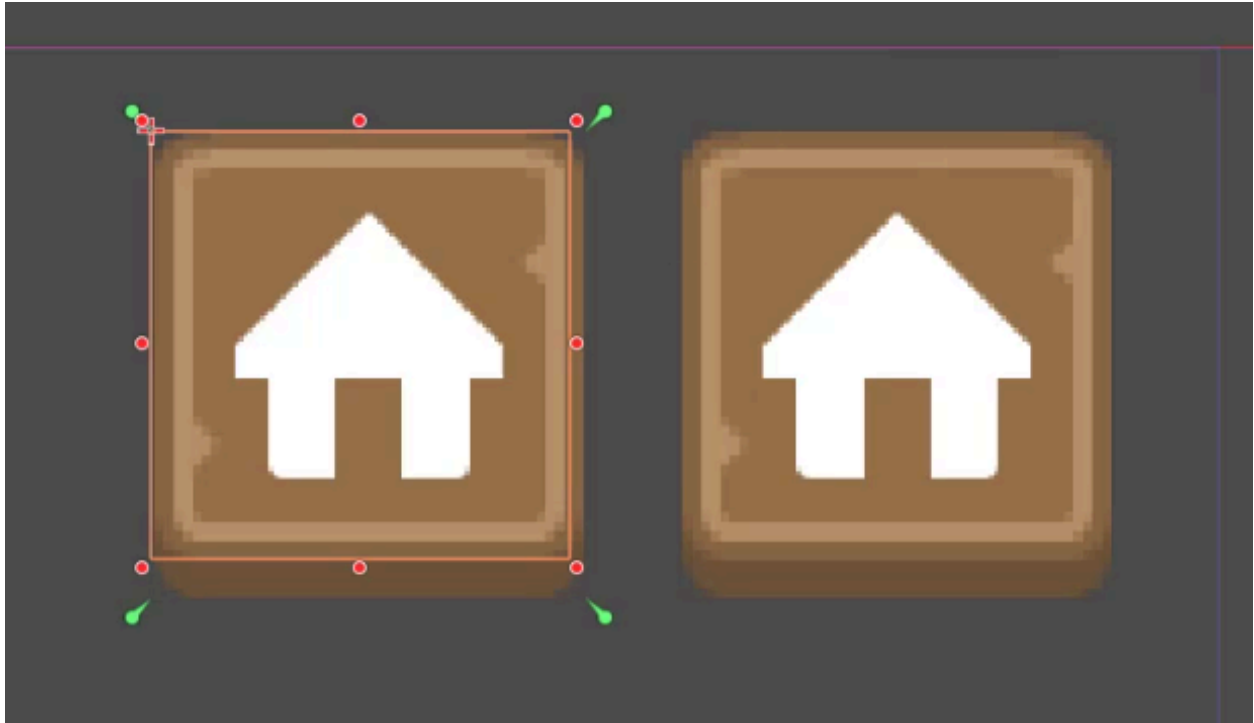
Adding a Pause Button

A pause button is a crucial feature for any game, especially an infinite runner where players might need a break. We will add this next to our home button.

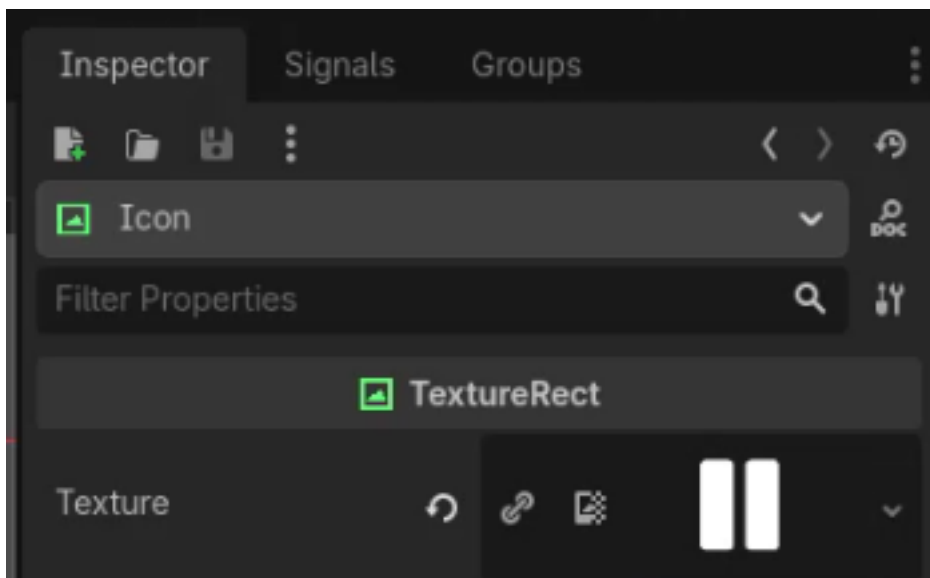
In the Main scene, duplicate the HomeButton and rename the copy to PauseButton.



Then, position the PauseButton next to the HomeButton in the top-right corner.



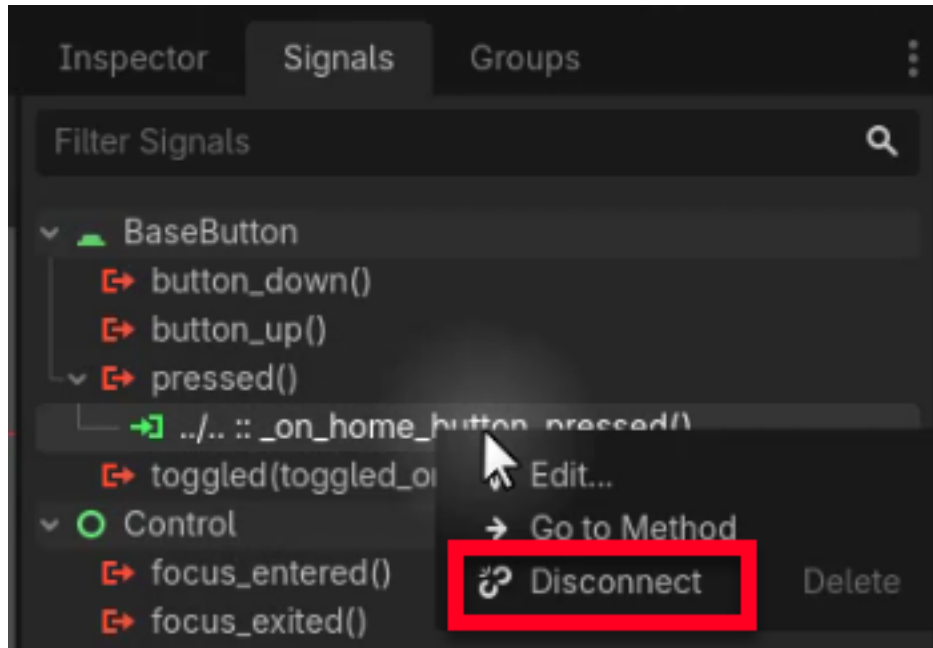
In the Inspector, select the child `Icon` node of the `PauseButton` and change its **Texture** to the “pause” icon.



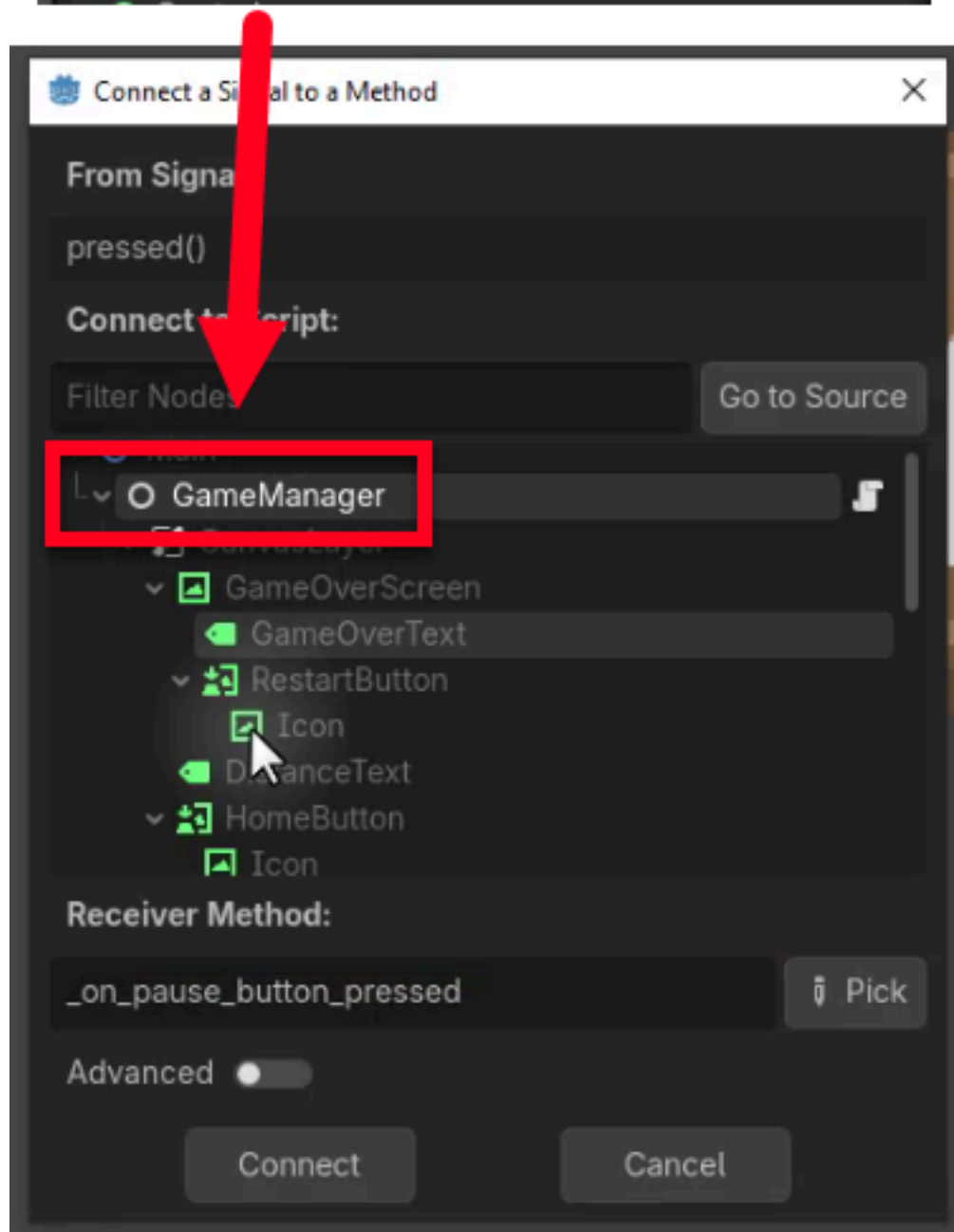
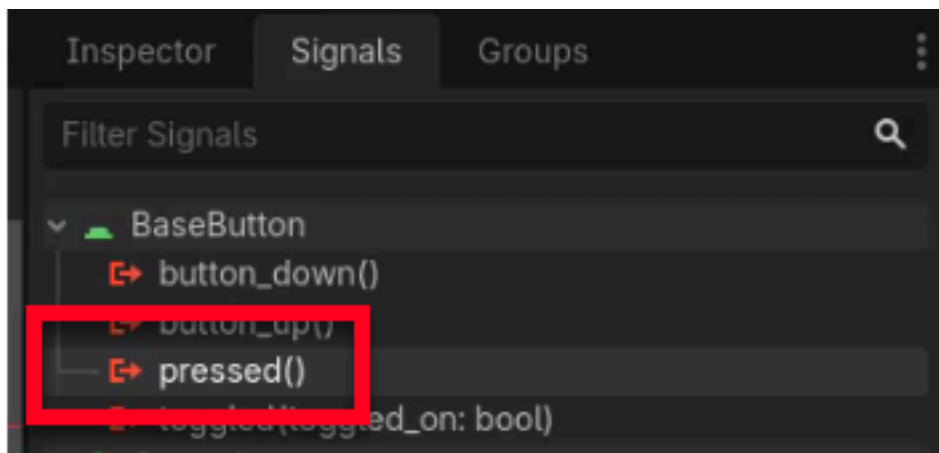
Implementing the Pause and Unpause Logic

To implement the pause functionality, we will use a built-in Godot property that controls the flow of time in the game. Since we duplicated the home button, we first need to disconnect its old signal before creating a new one.

1. Select the PauseButton and go to the **Node** tab to view its signals.
2. You will see the pressed() signal is already connected to the _on_home_button_pressed function. Right-click this connection and select **Disconnect**.



3. Now, with the signal disconnected, double-click the pressed() signal again.
4. Connect it to the GameManager node to create a new function, _on_pause_button_pressed.



In the `game_manager.gd` script, we will add logic to toggle the game's paused state. We can achieve this by manipulating `Engine.time_scale`. A value of `1.0` means the game runs at normal speed, while `0.0` freezes the game.

Add the following code to your new function:

```
# Toggle pause by freezing/unfreezing the engine clock
func _on_pause_button_pressed():
    if Engine.time_scale > 0:
        Engine.time_scale = 0.0
    else:
        Engine.time_scale = 1.0
```

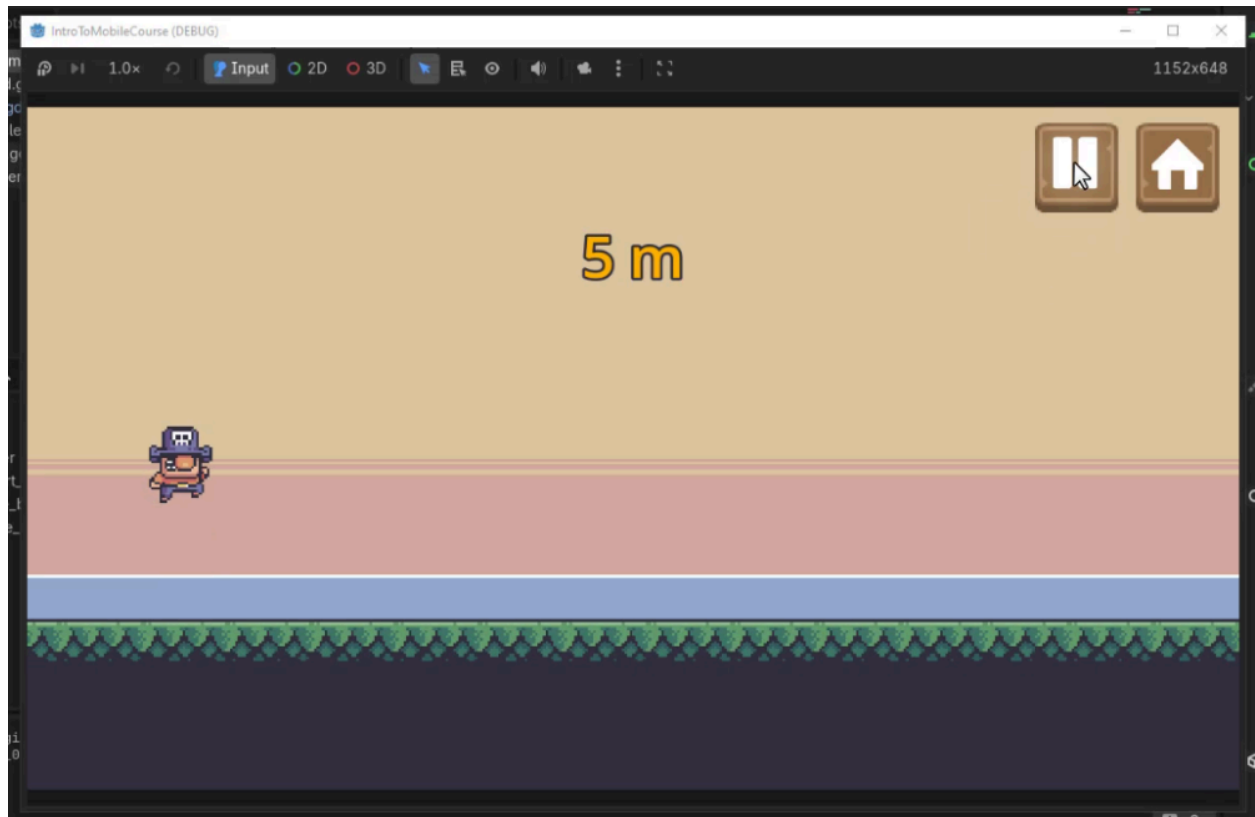
This code checks if the game is currently running (`time_scale > 0`). If it is, it sets the `time_scale` to `0.0`, pausing the game. If the game is already paused, it sets the `time_scale` back to `1.0`, resuming it. Now you can pause and unpause the game at will.

Godot Insight: `Engine.time_scale` vs `SceneTree.paused`

Godot offers two main ways to pause a game. `Engine.time_scale` controls how fast time passes: setting it to `0.0` freezes all `_process` and `_physics_process` callbacks globally, while values above `1.0` speed the game up. This approach is simple but affects everything, including UI animations.

The alternative is `get_tree().paused = true`, which pauses the scene tree. Nodes can opt out by setting their `process_mode` to `PROCESS_MODE_ALWAYS`, letting menus and pause screens continue to function while gameplay freezes. For more complex games, this approach gives finer control.

Learn more: [Engine](https://docs.godotengine.org/en/4.6/classes/class_engine.html)
(https://docs.godotengine.org/en/4.6/classes/class_engine.html)



That's it for our UI. We now have a more satisfying experience, both with a menu and quality-of-life buttons in our actual game.

In the next chapter, we will make another visual improvement to our player movement to create a better user experience for players. While not necessary to the core gameplay, it does add “game juice”, which is important for the overall feel of our project.

Advanced Visuals: Particle Systems

In this chapter, we will begin adding more visual flair to our game. Specifically, we will implement particle effects that trigger when the player runs and jumps. These effects help make the player feel more connected to the game environment, adding a layer of “juice” that makes movement feel satisfying and responsive.

Creating the Run Particle Effect

First, we will create a dust effect that appears whenever the player is running on the ground. We need to add a particle emitter node to our player, which will be responsible for generating and managing the particles.

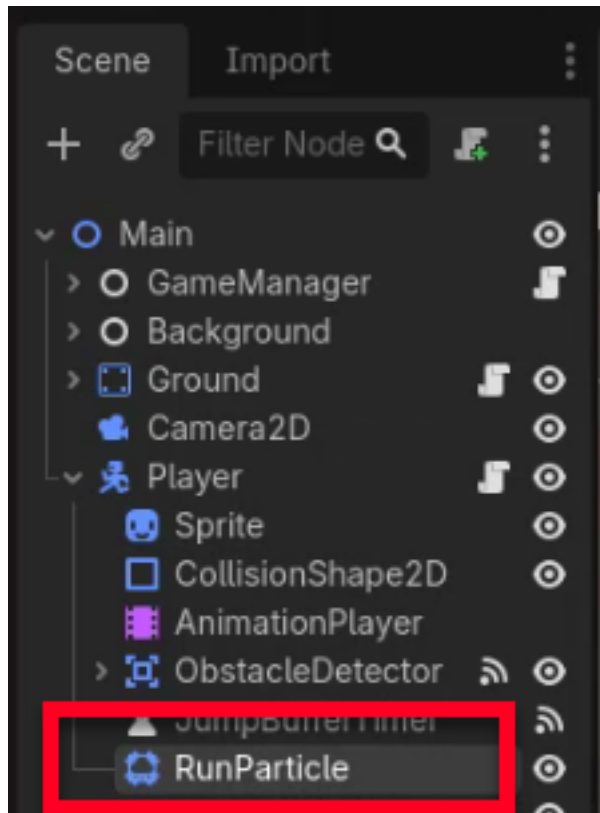
Godot Insight: CPUParticles2D vs GPUParticles2D

Godot provides two particle nodes for 2D effects. `CPUParticles2D` processes particles on the CPU. It works on all hardware (including older mobile devices and web exports) and exposes every setting directly in the Inspector, making it beginner-friendly.

`GPUParticles2D` offloads the work to the GPU via a `ParticleProcessMaterial` or custom shader. It handles larger particle counts more efficiently, but requires hardware that supports compute shaders. For a mobile project like ours, `CPUParticles2D` is the safer choice.

Learn more: [CPUParticles2D](https://docs.godotengine.org/en/4.6/classes/class_cpuparticles2d.html)
(https://docs.godotengine.org/en/4.6/classes/class_cpuparticles2d.html)

To begin, let’s select the Player node in the Scene dock, right-click it, and choose “Add Child Node”. Search for `CPUParticles2D`, add it, and rename the new node to `RunParticle`.

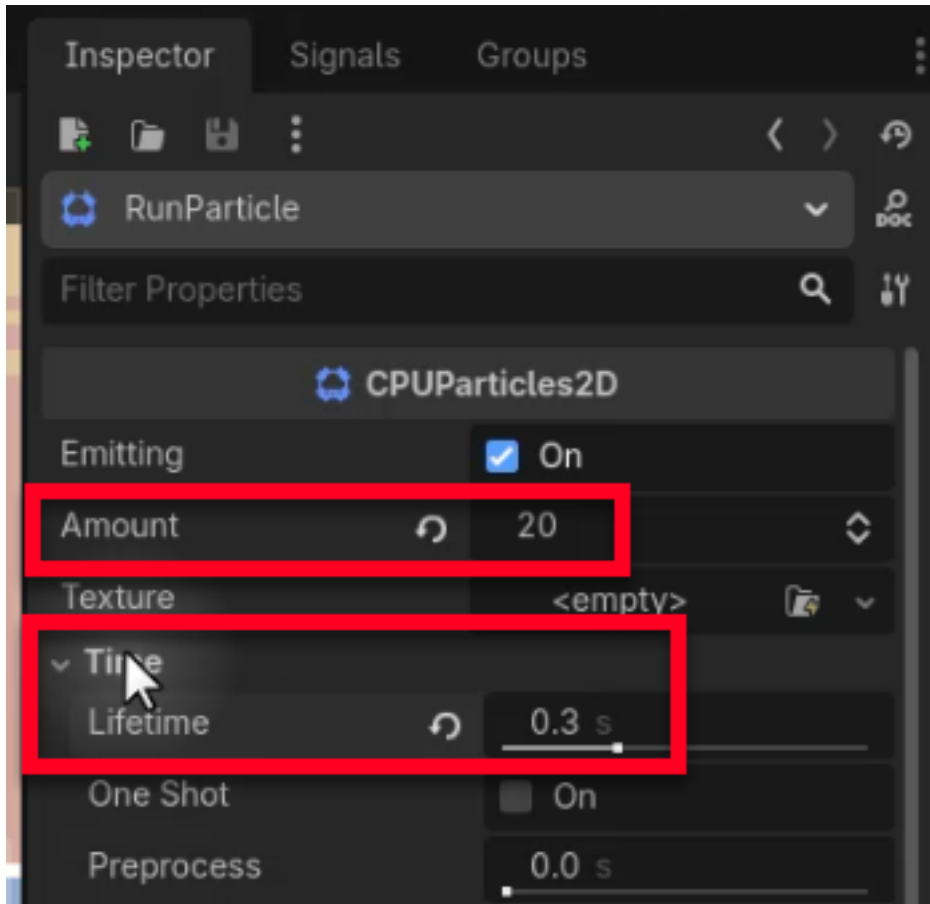


With the `RunParticle` node selected, we can now adjust its properties in the Inspector to create a “kicking up dust” effect. The default particle effect is not what we need, so we will make several changes.

Amount and Time

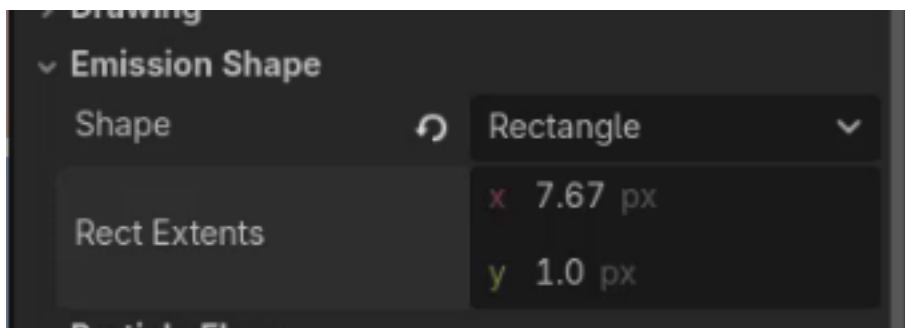
The **Amount** property controls how many particles are alive at any given moment. In the Inspector, increase it from 8 to around 20 for a fuller, more convincing dust cloud.

Next, change the **Lifetime** property under the **Time** section from 1.0 to 0.3. Lifetime determines how long each particle stays visible before it disappears. A shorter value keeps the effect tight and fast-paced, matching the speed of a running character.



Emission Shape

To make particles spawn from under the player's feet instead of a single point, change the **Emission Shape** from "Point" to "Rectangle". Adjust the **Emission Rect Extents** so the rectangle is wider along the X-axis (e.g., X: 7.67, Y: 1). The Rect Extents define the half-width and half-height of the spawning area, so a wider X value spreads particles across the player's full footprint.

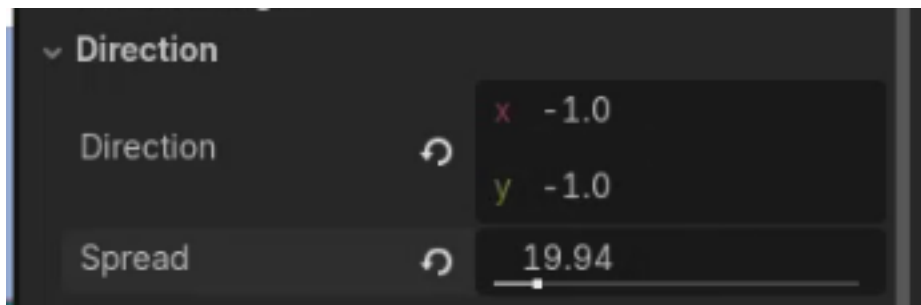


Movement

Next, we will define how the particles move.

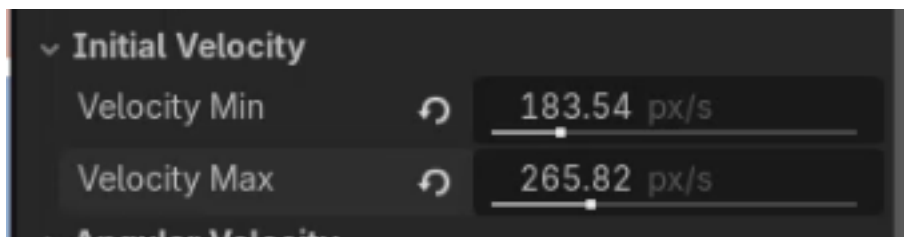
Set the **Direction** vector's X and Y values to -1. This will direct the particles up and to the left, as if being kicked up behind the player.

To add some randomness to the particle direction, set the **Spread** to about 20 degrees. A spread of 0 would make all particles travel in the exact same direction, which looks unnatural.



Initial Velocity

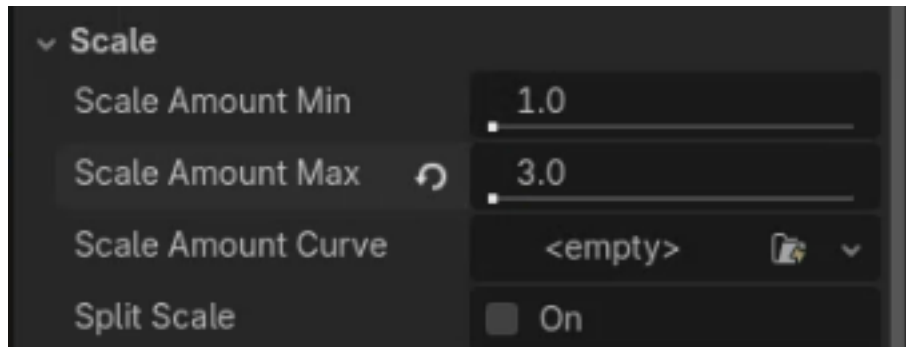
Initial Velocity controls how fast particles travel when they first spawn. Set **Initial Velocity Min** to around 180 and **Initial Velocity Max** to around 270. Using a range instead of a single value adds natural variation, so the dust cloud does not look uniform or mechanical.



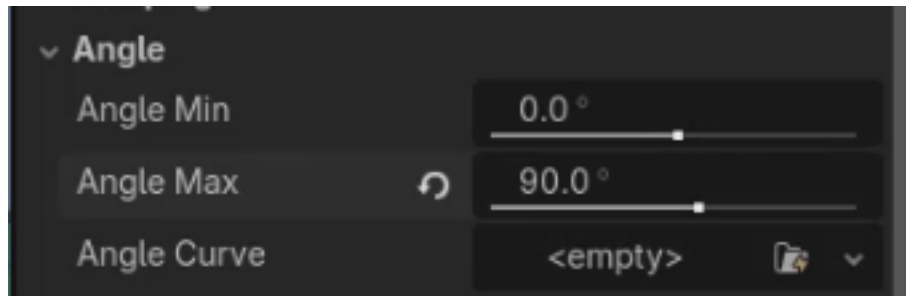
Visual Appearance

To finalize the visual appearance, we will adjust the scale, angle, and color. These properties add the variety that makes particle effects look natural rather than artificial.

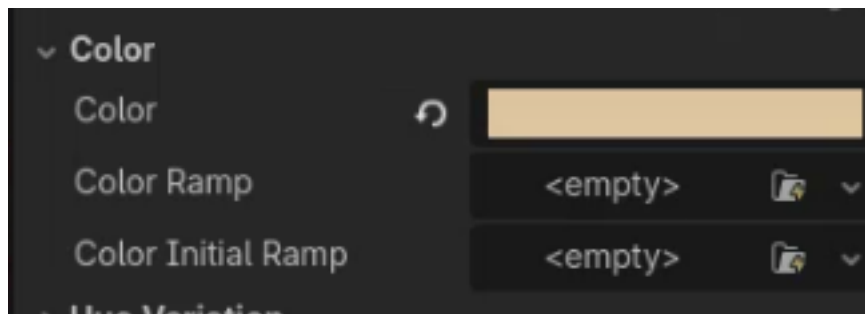
First, expand the **Scale** section. We'll set **Scale Min** to 1 and **Scale Max** to 3. Randomizing size prevents every dust speck from looking identical, which would break the illusion.



Next, let's expand the **Angle** section and set **Angle Max** to 90. This rotates each particle by a random amount up to 90 degrees, adding further visual variety so the dust does not look like a grid of identical squares.



Finally, expand the **Color** section and click on the **Color** property. Choose a color that fits the environment, such as a brownish-orange to represent dust (#ddc6a1).

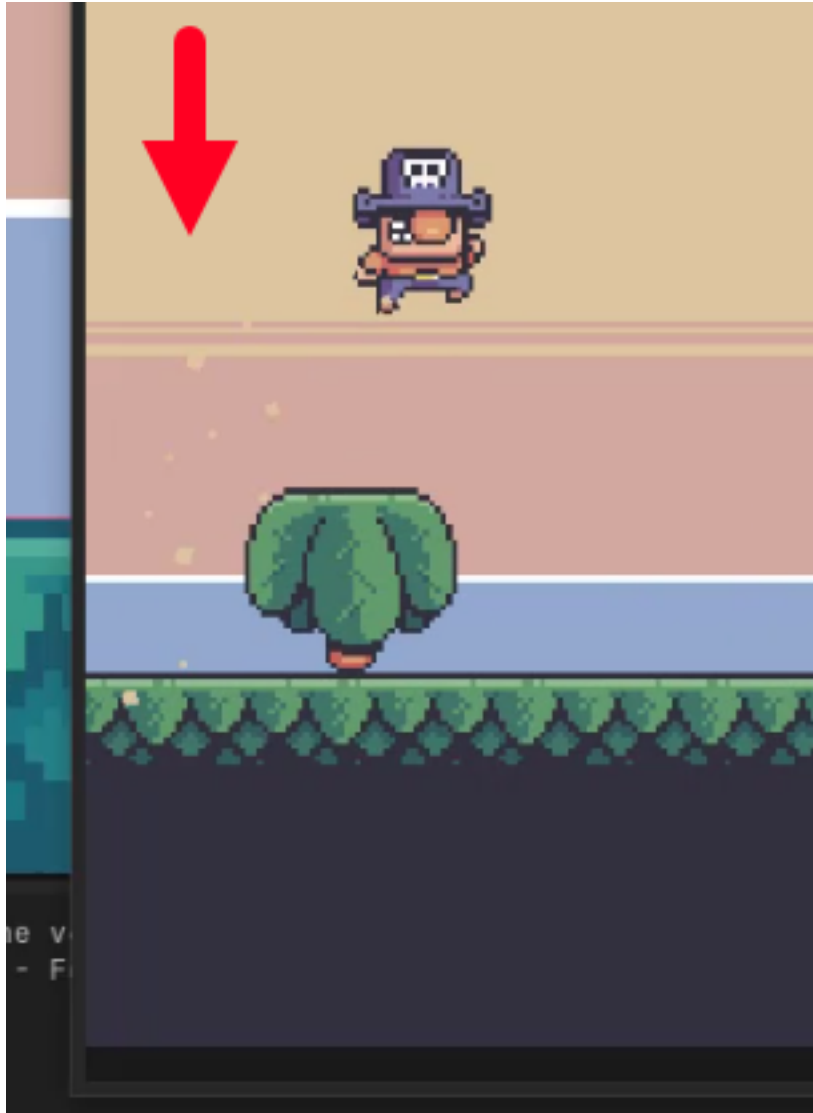


Feel free to play around with the settings until you get an effect that looks like dust being kicked up.



Controlling Particles with Code

If you run the game now, you will notice the particle effect plays continuously, even when the player is jumping. We only want the particles to emit when the player is on the ground. We can control this with a script.



First, open the `player.gd` script. We'll need to create a reference to our new `RunParticle` node.

```
@onready var run_particle : CPUParticles2D = $RunParticle
```

Now, we can use this reference to enable or disable the particle emission based on the player's state. We will modify the existing logic inside the `_physics_process` function that also handles animations.

```
func _physics_process (delta):  
    # ... existing gravity and jump code ...
```

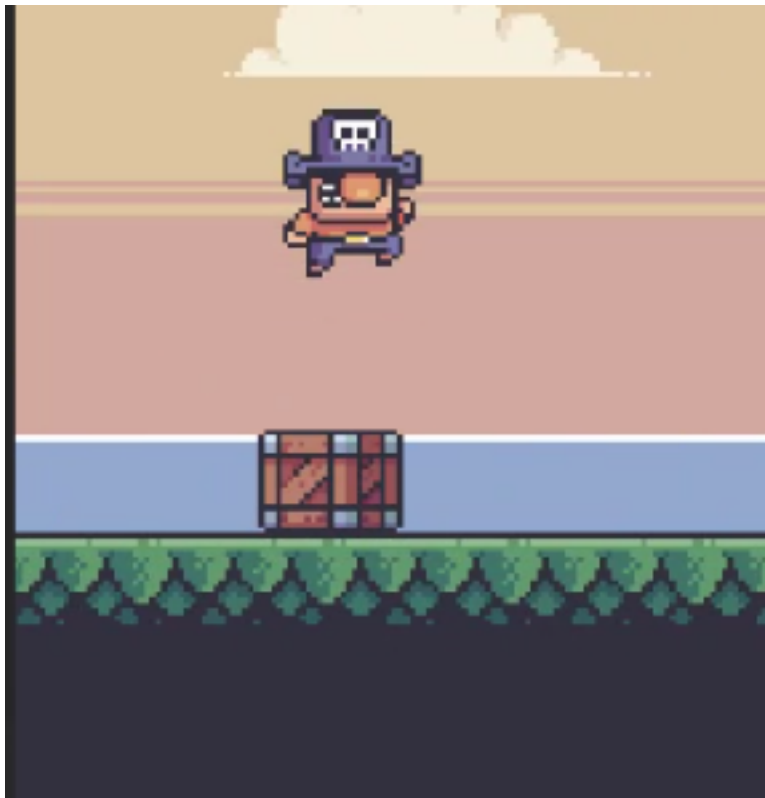
```

move_and_slide()

# Switch animation and particles based on ground contact
if is_on_floor():
    anim.current_animation = "run"
    run_particle.emitting = true
else:
    anim.current_animation = "jump"
    run_particle.emitting = false

```

In this code, we set the emitting property of our particle node to true when the player is on the floor and to false when they are in the air. Now, if you run the game, the dust particles will only appear when the player is running on the ground.



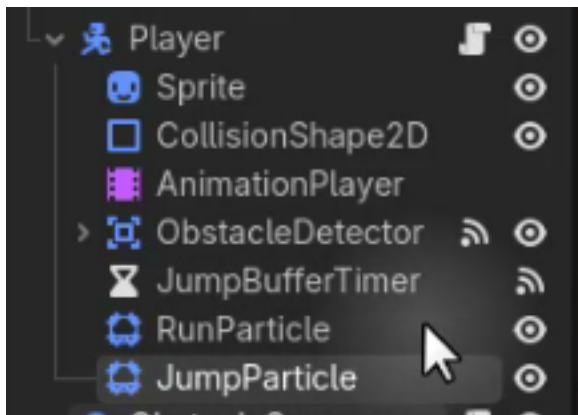
Creating the Jump Particle Effect

In addition to the running dust, we will enhance our game's visual feedback by implementing a particle effect for the player's jump. This effect will

provide a satisfying burst of particles each time the player leaps into the air, making the action feel more dynamic and impactful.

Creating the Jump Particle Node

First, we need to create the particle node itself. This will be a child of our Player scene, ensuring it moves with the player. Similar to what we did before, select the Player node in the Scene dock. Then, add a new CPUParticles2D child node and rename it to JumpParticle.



Unlike the continuous run particle, the jump particle will be a “one-shot” effect. This means it will play once and then stop, perfectly suited for a singular action like a jump.

Configuring the Particle Emitter

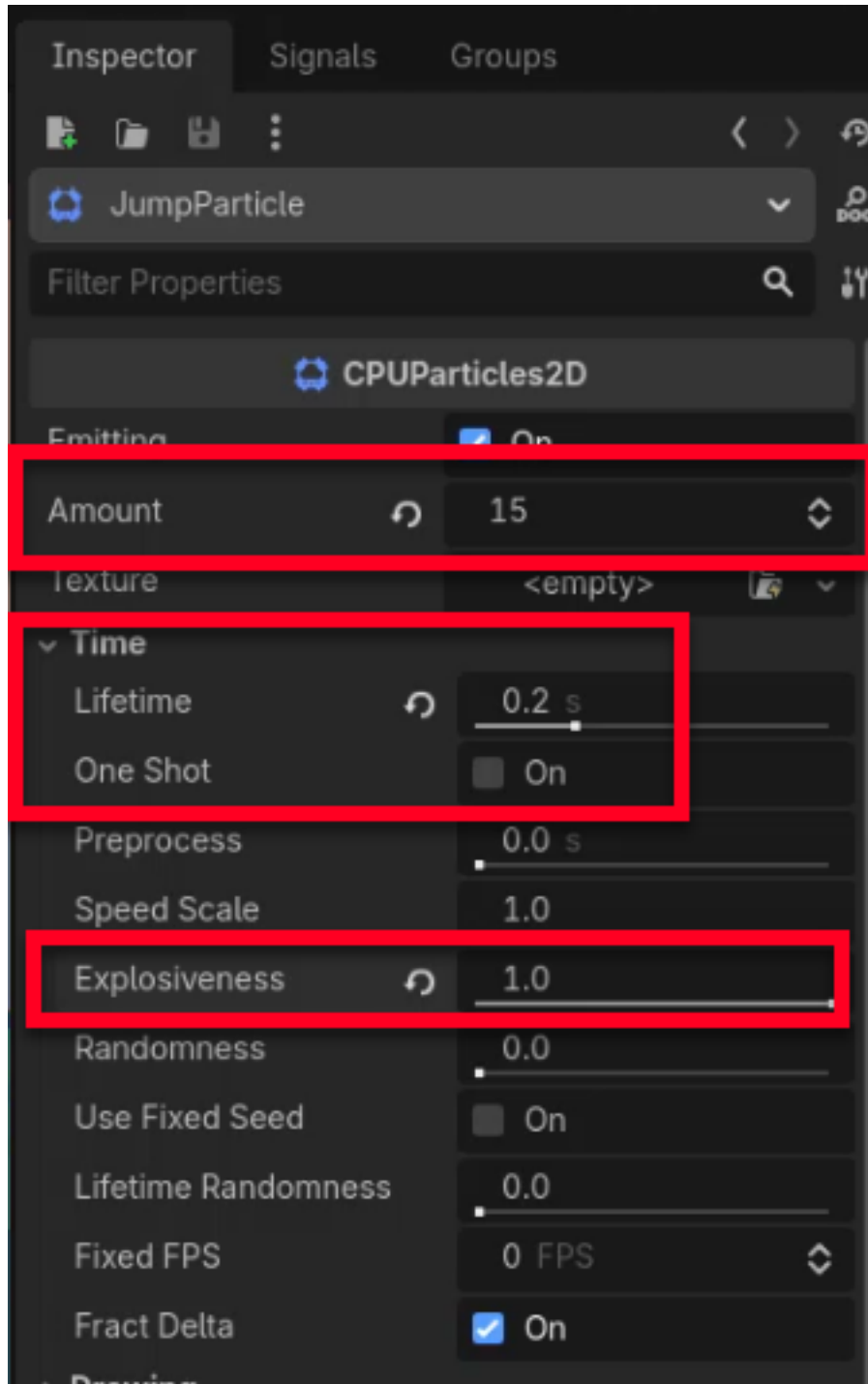
With the JumpParticle node selected, navigate to the Inspector to configure its properties.

We'll first set **Amount** to 15. This is the total number of particles that will be emitted.

Under the **Time** section:

- Set **Lifetime** to 0.2. This is how long each particle will exist.
- Enable **One Shot**. Unlike the run particle (which emits continuously while the player moves), a jump is a single discrete action, so the particle system should fire once and stop. Note that while testing, you may find it helpful to selectively toggle the Emitting property and One Shot property to see the effect of the changes you make.

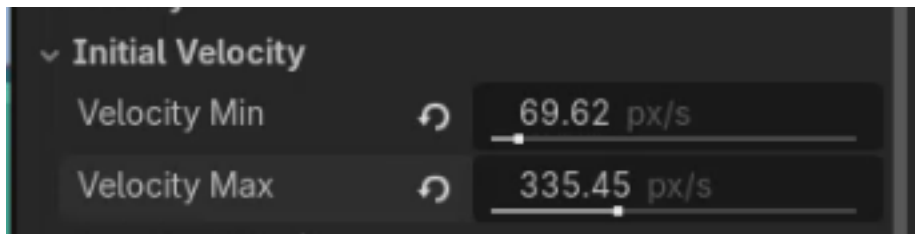
- Set **Explosiveness** to 1. By default, particles spawn gradually over the lifetime of the effect. An Explosiveness value of 1 forces all particles to emit at the exact same instant, creating the snappy burst that matches the sudden force of a jump.



Defining the Particle's Movement

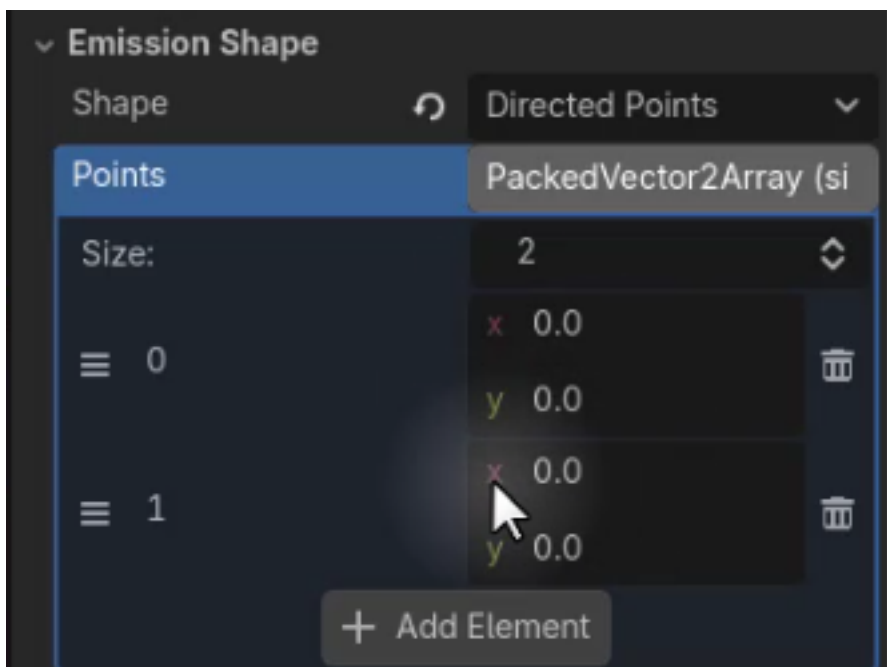
Next, we will define how the particles move. We want them to shoot out to the left and right, simulating air being pushed away by the jump.

Under **Initial Velocity**, set **Velocity Min** to around 70 and **Velocity Max** to around 330.

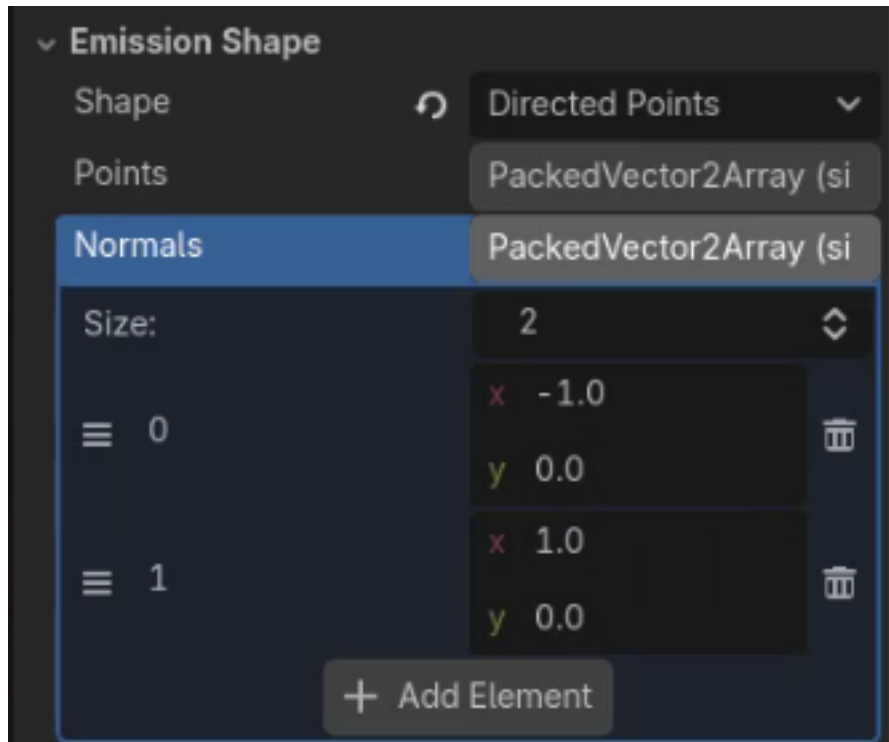


For **Emission Shape**, change the **Shape** from Point to Directed Points. This allows us to specify emission points and their corresponding directions.

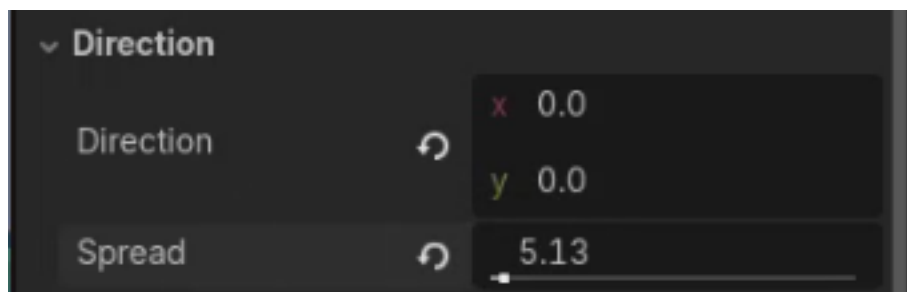
We'll first set the size of the **Points** array to 2, but leave both points at (0, 0).



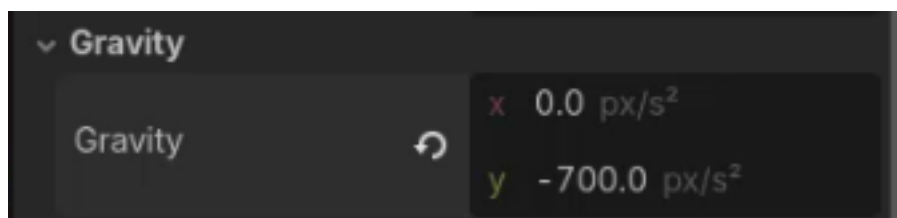
We want to set the **Normals** array size to 2 as well, but with the first normal's X at -1 (left) and the second at 1 (right).



Next, under **Direction**, set the **Spread** to around 5. This tightens the particle stream.



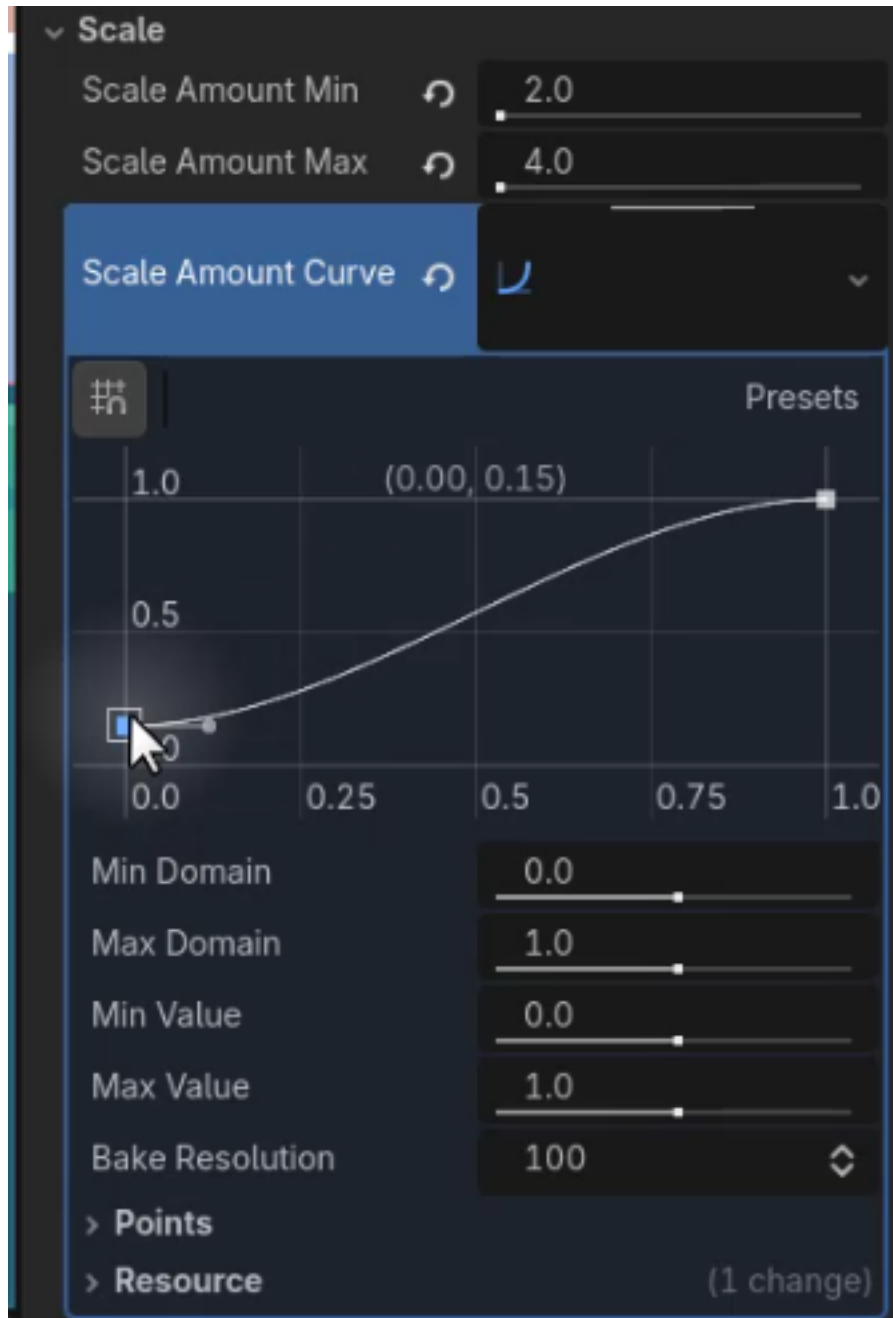
Then, under **Gravity**, set the Y value to -700. This negative gravity will cause the particles to rise after their initial burst, creating a nice “evaporating” effect.



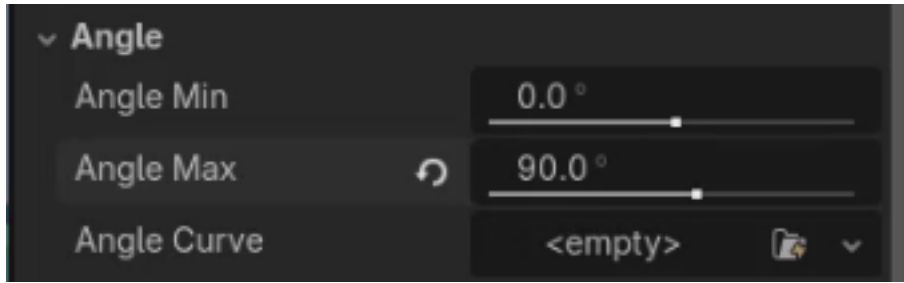
Adjusting Visuals: Scale and Color

To make the particles look more interesting, we will adjust their scale and color over their lifetime.

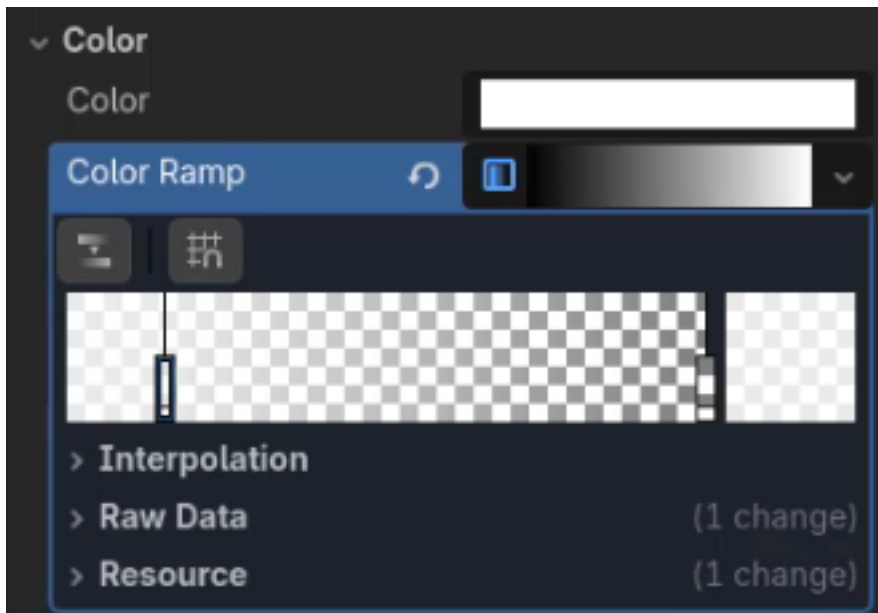
In the **Scale** section, first set **Scale Min** to 2 and **Scale Max** to 4. For **Scale Amount Curve**, create a new Curve resource and click it to open the curve editor. Drag the first point down to the bottom-left corner so the particles start small and grow larger over time.



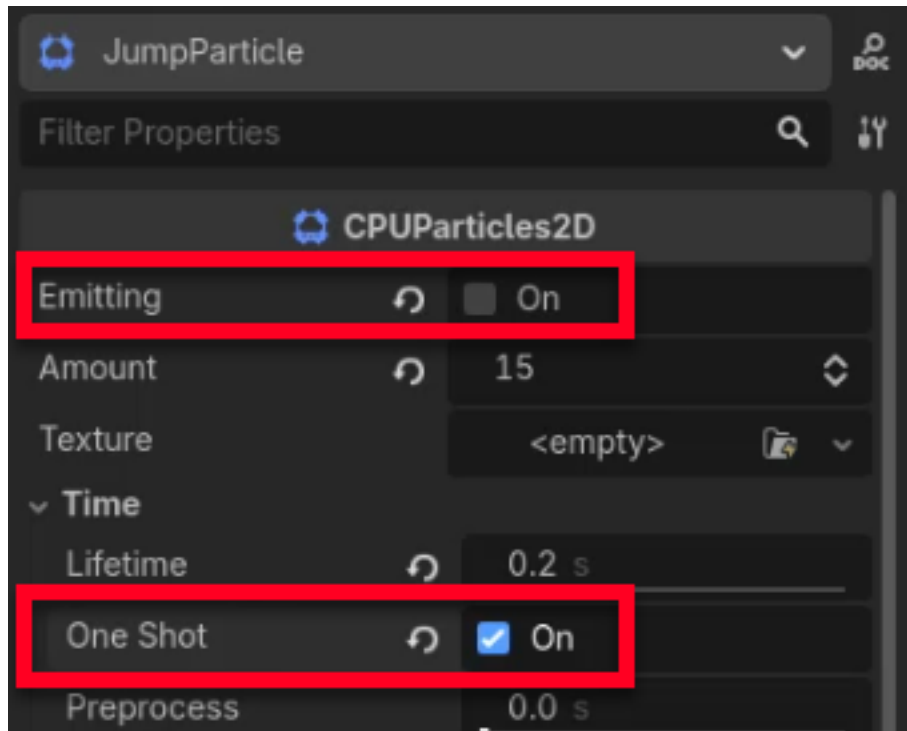
Under **Angle**, set **Angle Max** to 90. This adds some random rotation to each particle.



Next we'll work with **Color**. Locate the **Color Ramp** and create a new Gradient resource. Click it to edit. We'll set the starting color to white with full alpha (opacity), and the ending color to white with an alpha of 0. This creates a smooth fade-out before the particles disappear.



After configuring all the properties, go back to the **Time** section and make sure that **One Shot** is enabled and that **Emitting** is off if you changed them at all while testing.



Lastly, position the JumpParticle node at the player's feet.



Triggering the Particles with Code

Now we need to write a small amount of code to trigger this particle effect whenever the player jumps. Let's add this to the `player.gd` script.

First, create a reference to our new particle node at the top of the script with your other variables.

```
@onready var jump_particle : CPUParticles2D = $JumpParticle
```

Next, find the `_physics_process` function. Inside the condition that handles the jump (if `jump_input` and `is_on_floor()`), add a line to enable emitting on the jump particle.

```

func _physics_process (delta):
    # Apply gravity only while airborne
    if not is_on_floor():
        velocity.y += gravity * delta

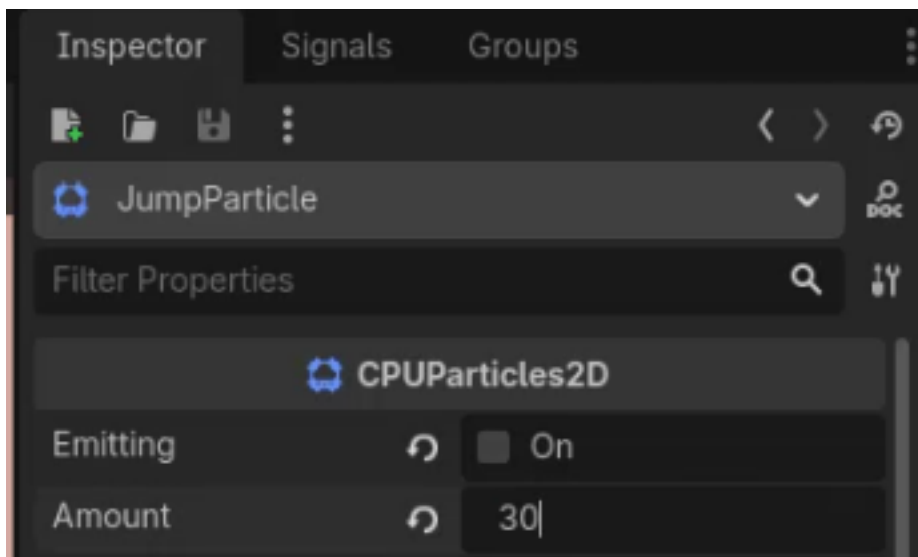
    # Execute a buffered jump if the player is on the ground
    if jump_input and is_on_floor():
        jump_input = false
        velocity.y = -jump_force
        jump_particle.emitting = true

    move_and_slide()
    # ...

```

Because the particle system is set to **One Shot**, setting emitting to true will cause it to play its full animation once and then automatically set emitting back to false. Thus, we don't need to manually do it like we did with running.

Run the game and test it out. Every time you jump, you should see a burst of particles. For a more pronounced effect, you can return to the JumpParticle node's properties and increase the **Amount** to 30.



With both running and jumping particles implemented, our player's movements now have much more visual flair.



In the next chapter, we will continue to polish our game by adding sound effects and music. This will add another element of game juice to our project that rounds out how satisfying our game feels. We will also tweak our jump slightly to give more leeway to our player.

Game Feel: Audio and Input Buffering

In this chapter, we will once again focus on “game feel” - the intangible quality that makes a game satisfying to play. We will first address a common issue in platformer and runner games related to jump timing by implementing a technique known as “jump buffering.” This will make the controls feel more responsive and forgiving.

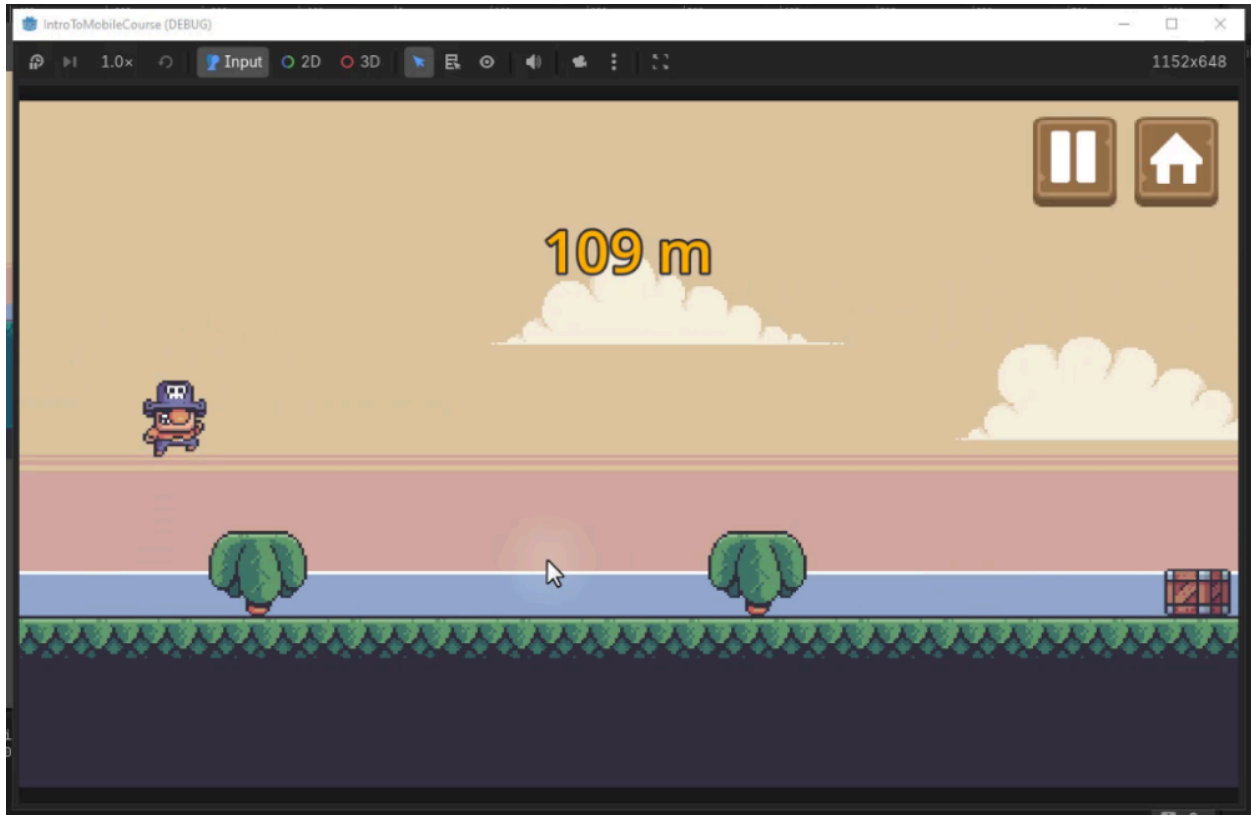
Following that, we will enhance the game’s atmosphere by implementing sound effects and music. We will add a sound effect for the player’s jump, include background music for gameplay, and switch to a different track when the game is over.

Jump Buffering

As you play the game, you will frequently need to jump over obstacles, often immediately after landing from a previous jump. Currently, the jump input is only registered if the player is strictly on the ground.

The Problem with Precise Jumps

If you press the jump button a fraction of a second too early, while the player is still in the air, the input is ignored. This requires perfect timing from the player and can lead to frustrating moments where a jump fails to execute as intended.



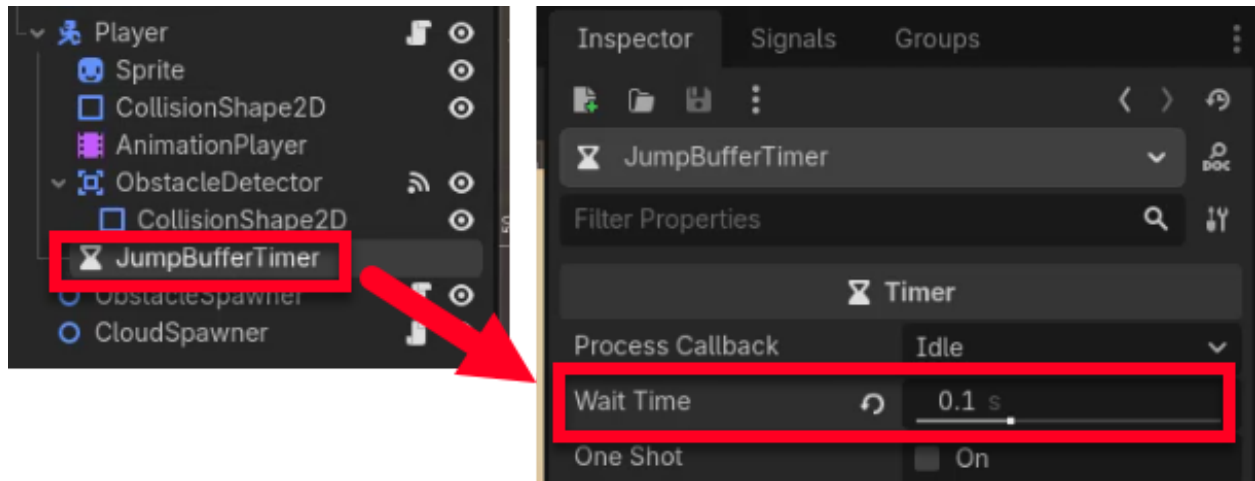
To solve this, we will implement jump buffering. This technique involves “remembering” the player’s jump input for a very short period. If the player presses the jump button while airborne but close to landing, the game will store this input. As soon as the player touches the ground, the game will check for this “buffered” input and execute the jump automatically.

Adding the Buffer Timer

In order to implement the effect we’re after, we will use a `Timer` node to manage the duration of our jump buffer. The process involves starting the timer when a jump is attempted and clearing the jump input when the timer runs out.

To begin, open your `Player` scene, add a new `Timer` child node, and rename it to `JumpBufferTimer`.

In the Inspector, set its **Wait Time** to `0.1` seconds. This means our jump input will be buffered for one-tenth of a second, which is usually sufficient to catch early inputs without feeling unintentional.



Scripting the Buffer Logic

Now we need to integrate this timer into our player's logic. Let's open the `player.gd` script to make the necessary changes.

First, we need to get a reference to our new timer node. Add the following line at the top of your script with your other `@onready` variables.

```
@onready var jump_buffer_timer : Timer = $JumpBufferTimer
```

Next, we need to modify the `_try_jump` function. Previously, this function checked if the player was on the floor before registering a jump. We will remove that check and instead start our buffer timer. This allows the jump input to be registered even when the player is not on the ground.

Update your `_try_jump` function to match the following:

```
# Buffer the jump input so near-miss taps still register
func _try_jump ():
    if game_manager.is_game_over:
        return

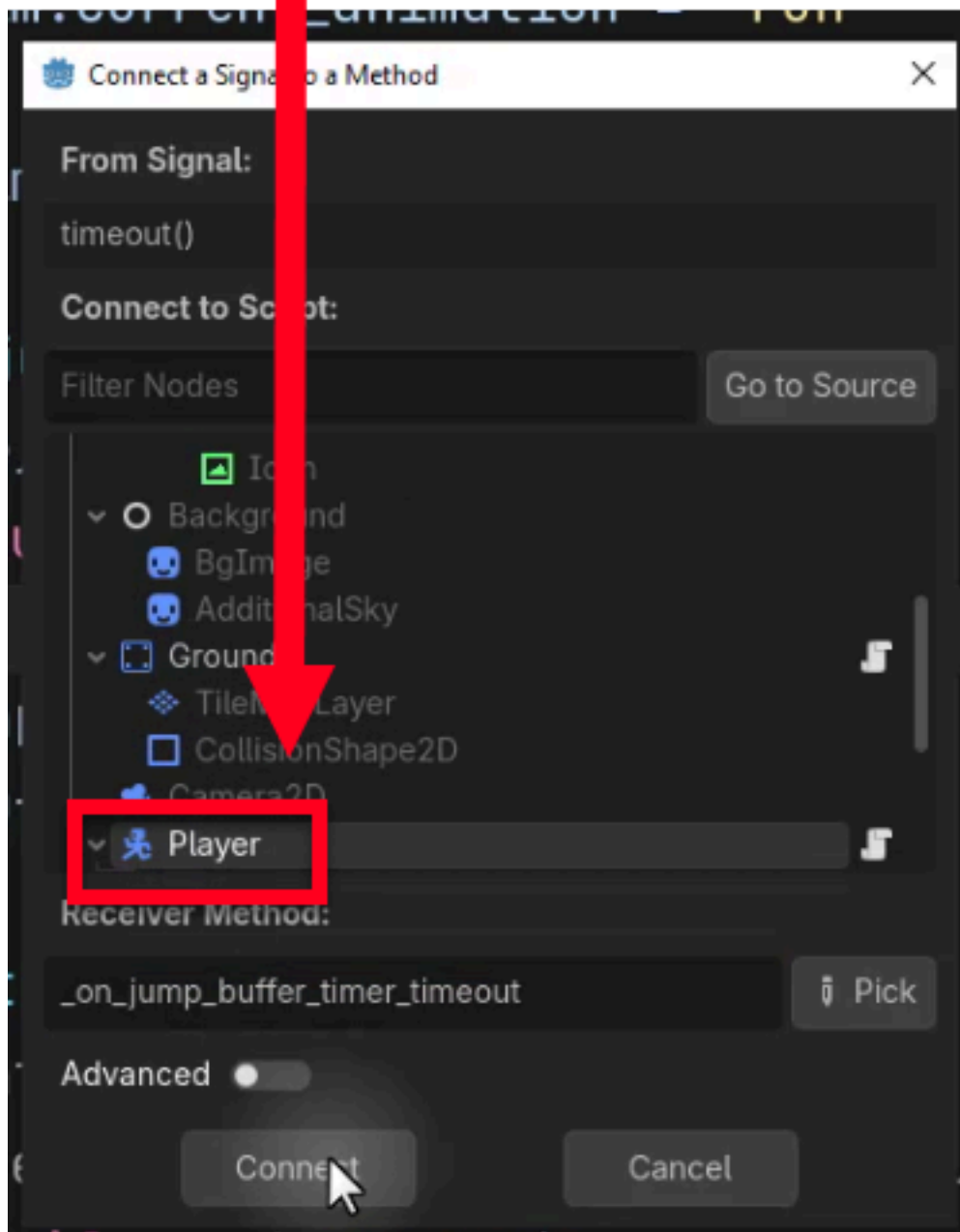
    jump_input = true
    jump_buffer_timer.start()
```

Now, whenever the player attempts to jump, the `jump_input` variable is set to `true`, and our 0.1-second timer begins.

Resetting the Input

If the player doesn't land within the 0.1-second buffer window, we need to reset `jump_input` to `false`. We can do this by connecting the `JumpBufferTimer`'s timeout signal.

1. Select the `JumpBufferTimer` node and open the **Signals** tab in the **Node** panel next to the Inspector.
2. Double-click the `timeout()` signal.
3. Ensure your `Player` node is selected and click **Connect**.

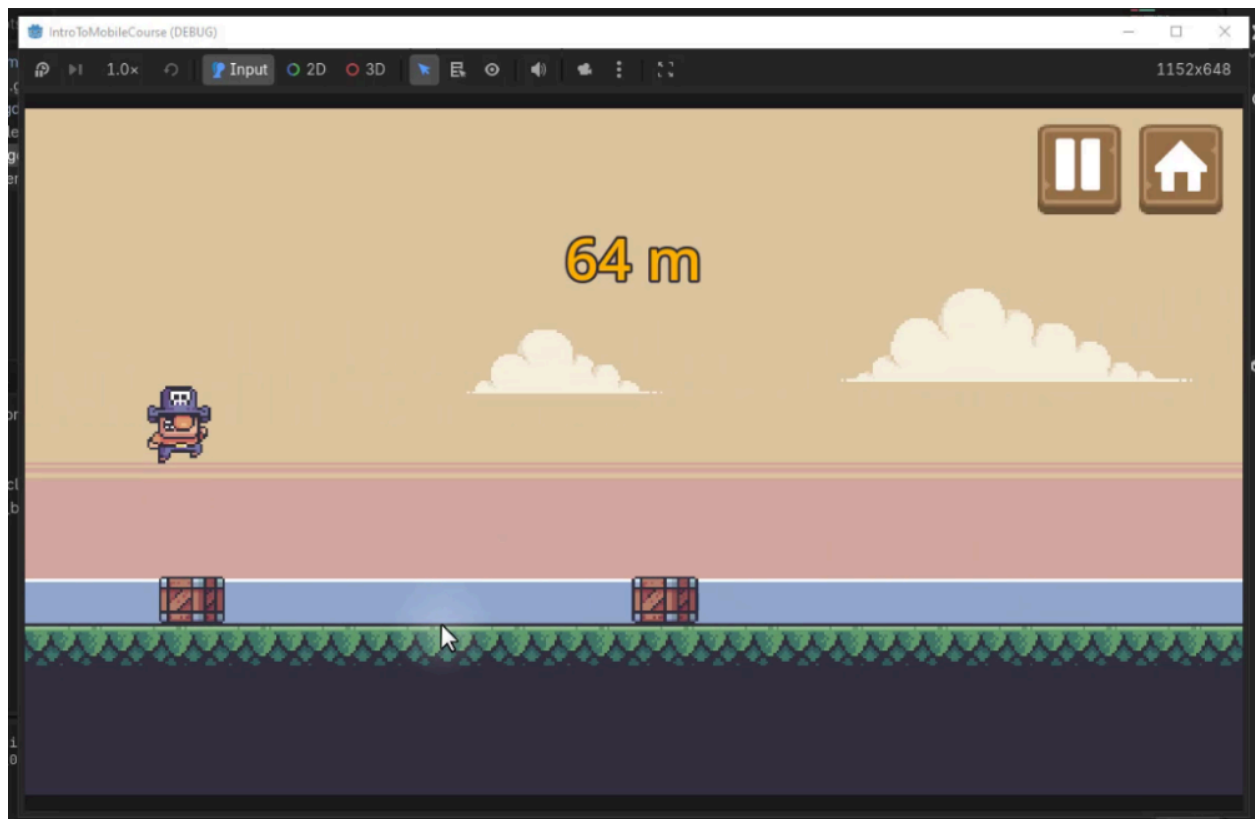


Godot will create a new function in your `player.gd` script called `_on_jump_buffer_timer_timeout`. Add the following code inside this function to clear the input:

```
# Clear the buffered input when the timer expires  
func _on_jump_buffer_timer_timeout():  
    jump_input = false
```

With these changes, you can now test the game. While it may not look different on the surface, you should feel the improvement. Try pressing the jump button just before your character lands. You will notice that the character now jumps immediately upon touching the ground, creating a much smoother and more intuitive gameplay experience.

Of course, if we press jump too early, the character doesn't automatically jump again, making the controls feel properly responsive.



Audio and Atmosphere

Even with great visuals and responsive controls, a game can feel surprisingly flat without sound. Audio reinforces every action the player takes, from the satisfying pop of a jump to the tension of background music shifting on a game over screen. In this section, we will implement the audio for our game, including a jump sound effect and background music that changes when the game ends.

Implementing the Jump Sound Effect

First, let's add a sound effect that plays whenever the player jumps. We will use an `AudioStreamPlayer` node, and not `AudioStreamPlayer2D`, because this sound does not need to be positional (i.e. it does not need to get quieter as the player moves away from it).

Godot Insight: `AudioStreamPlayer` types

Godot has three audio player nodes. `AudioStreamPlayer` plays sound globally with no spatial effects, making it ideal for UI sounds, background music, and full-screen effects like our jump sound.

`AudioStreamPlayer2D` attenuates volume based on the listener's distance in 2D space, so sounds get quieter as the camera moves away.

`AudioStreamPlayer3D` does the same in 3D. Choose the version that matches your game's needs: global for non-positional audio, 2D/3D for sounds tied to a location in the world.

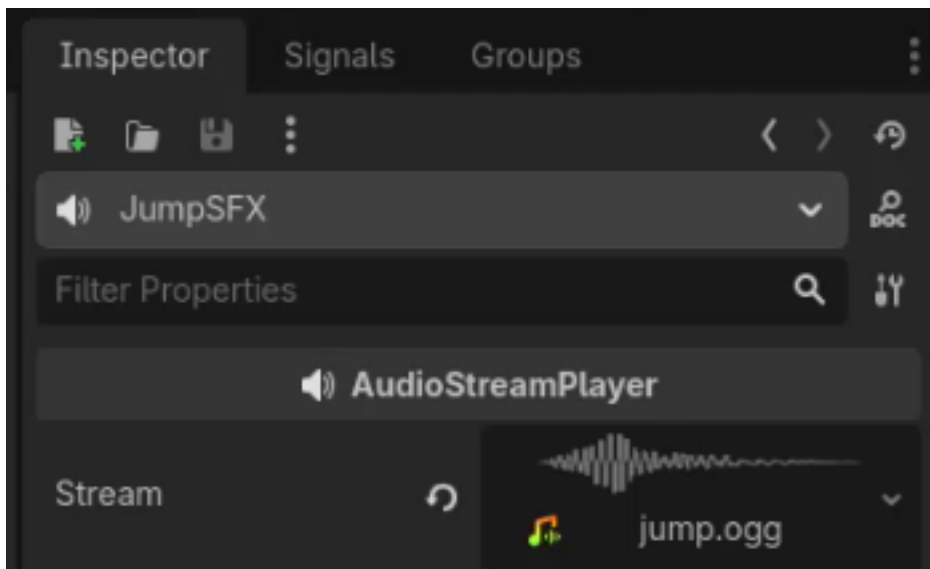
Learn more: [AudioStreamPlayer](https://docs.godotengine.org/en/4.6/classes/class_audioplayer.html)

(https://docs.godotengine.org/en/4.6/classes/class_audioplayer.html)

To start, select the `Player` node in the Scene dock. As mentioned, add an `AudioStreamPlayer` child node and rename it to `JumpSFX`.



With the JumpSFX node selected, locate the Stream property in the Inspector. Drag your jump sound file (jump.ogg) from the FileSystem dock into this property.



Now, we need to trigger this sound from our script. Let's open the player.gd script and create a reference to the JumpSFX node at the top.

```
@onready var jump_sfx : AudioStreamPlayer = $JumpSFX
```

Next, we'll locate the logic where the player character jumps. In our `_physics_process` function, this is where we set the vertical velocity. Add a line to play the sound effect.

```

func _physics_process (delta):
    # Apply gravity only while airborne
    if not is_on_floor():
        velocity.y += gravity * delta

    # Execute a buffered jump if the player is on the ground
    if jump_input and is_on_floor():
        jump_input = false
        velocity.y = -jump_force
        jump_particle.emitting = true
        jump_sfx.play()

    move_and_slide()

    # ... rest of the function

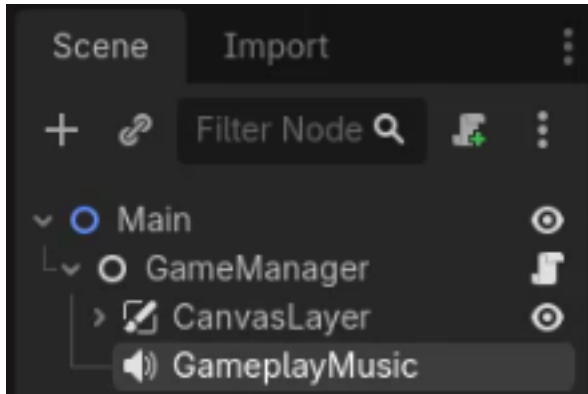
```

When you run the game, the jump sound should now play each time you jump.

Adding Background Music

Next, we will add background music. Since the music persists regardless of the player's state, we will manage it in our GameManager.

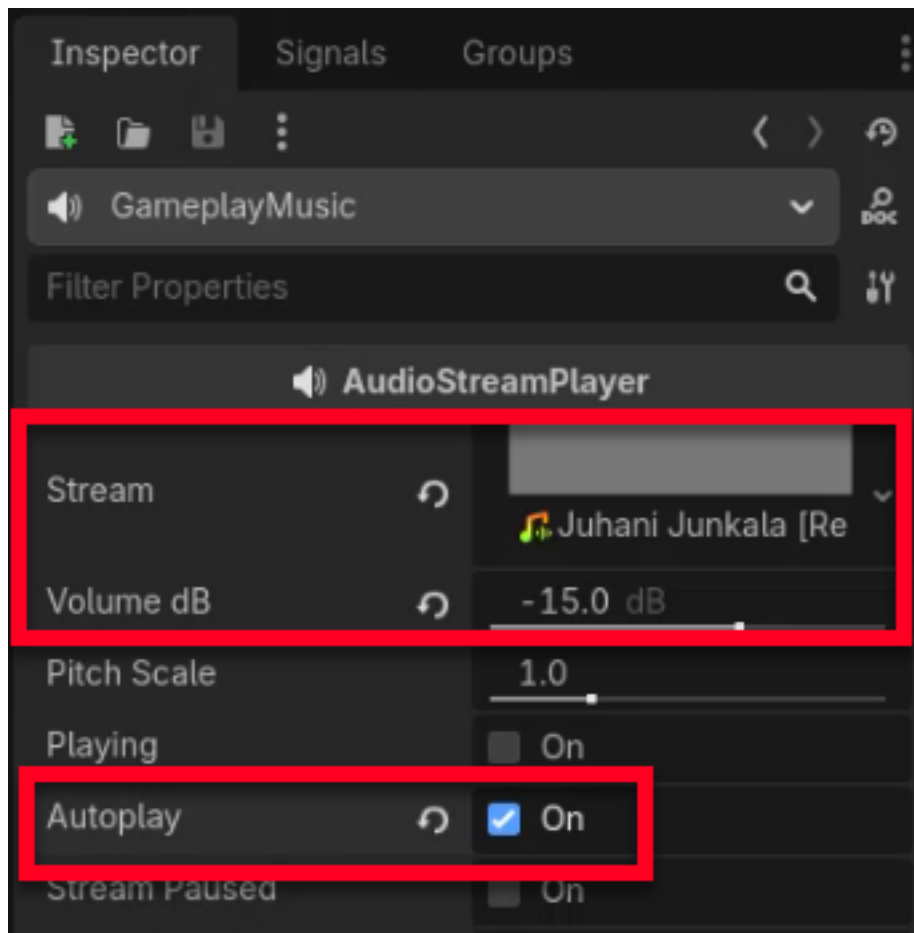
Open your main scene and select the GameManager node. We'll add an AudioStreamPlayer node as a child of GameManager. Let's rename this node to GameplayMusic.



In the Inspector, drag your main music track into the Stream property.

To ensure the music is not too loud compared to the sound effects, adjust the Volume Db property. A value of around -15 is a good starting point.

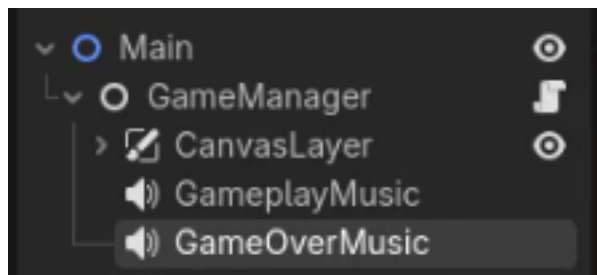
Finally, enable the Autoplay property. This will make the music start playing automatically as soon as the game begins.



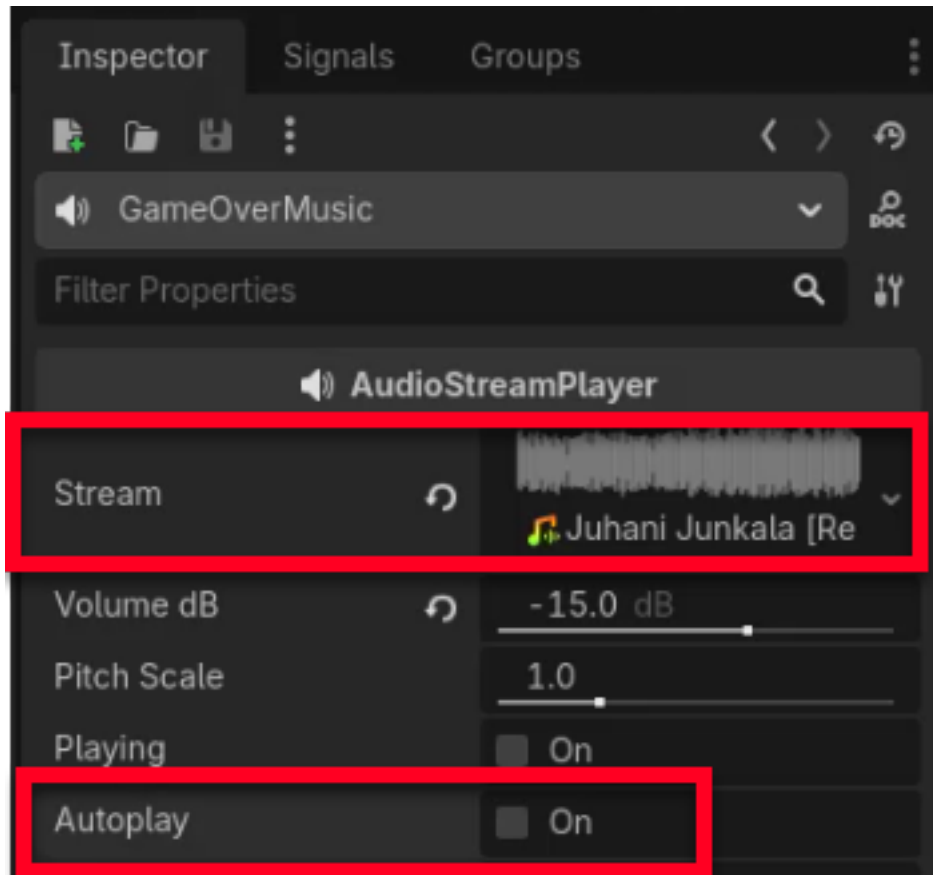
Switching to Game Over Music

When the player hits an obstacle, we want to stop the main music and play a different “game over” track to shift the mood.

In the Scene dock, duplicate the GameplayMusic node and rename the copy to GameOverMusic.



With GameOverMusic selected, drag your game over music track into the Stream property. Ensure the Autoplay property for this node is **disabled**, as we will control it via script.



Now, let's modify the `game_manager.gd` script to handle the music transition. First, create references to both music player nodes.

```
@onready var gameplay_music : AudioStreamPlayer = $GameplayMusic
@onready var game_over_music : AudioStreamPlayer = $GameOverMusic
```

Next, navigate to the `game_over` function. Here, we will stop the gameplay music and play the game over music.

```
# Called by the player script when an obstacle is hit
func game_over ():
    current_speed = 0.0
    is_game_over = true
```

```
game_over_screen.visible = true

# Swap background music to the game over track
gameplay_music.stop()
game_over_music.play()

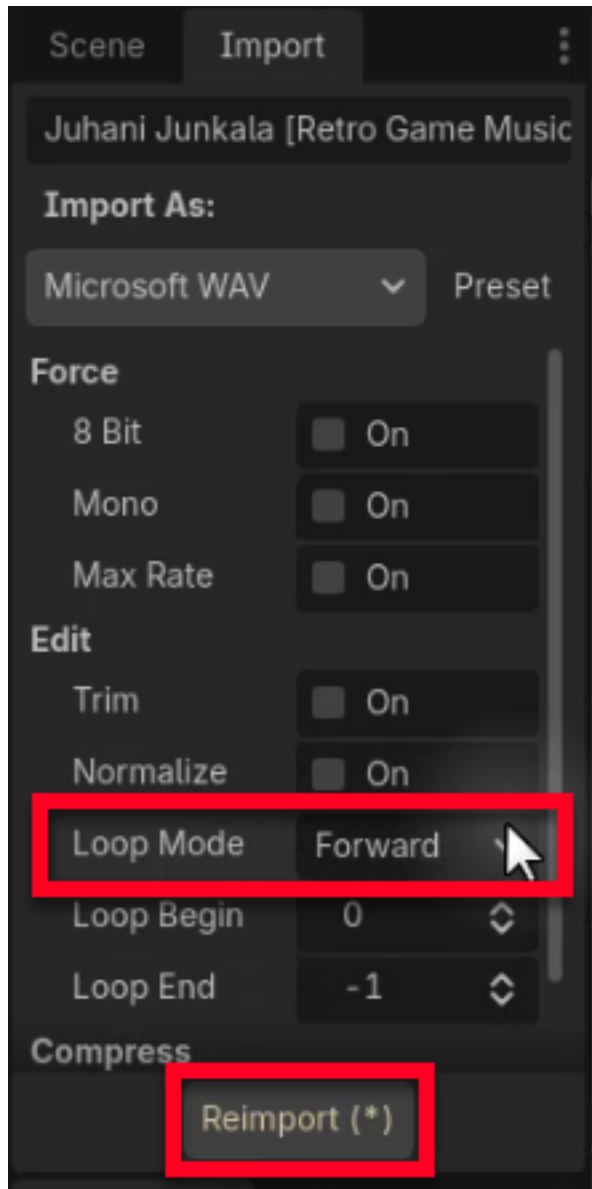
$CanvasLayer/GameOverScreen/GameOverText.text = "Game Over\nYou
survived " + str(int(distance)) + " m"
```

Looping Music Tracks

By default, audio streams in Godot do not loop. If a music track finishes, it will simply stop playing. To ensure our music loops continuously, we need to adjust its import settings.

In the FileSystem dock, select one of your music files, then switch to the **Import** dock (usually next to the Scene dock, top left). Next, find the Loop Mode setting and change its value from Detect from WAV (or Disabled) to Forward. Click the **Reimport** button at the bottom of the Import dock to finish the process.

Finally, repeat this process for your other music track.



The jump sound effect does not need to be looped, as it is triggered manually by our script each time the player jumps.

With these changes, your game's audio is now fully implemented. The jump feels responsive thanks to buffering, and the audio cues provide essential feedback and atmosphere.

In the next chapter, we will make some minor adjustments to improve our UI for mobile devices and also discuss how we can export our project.

Finalization and Mobile Export

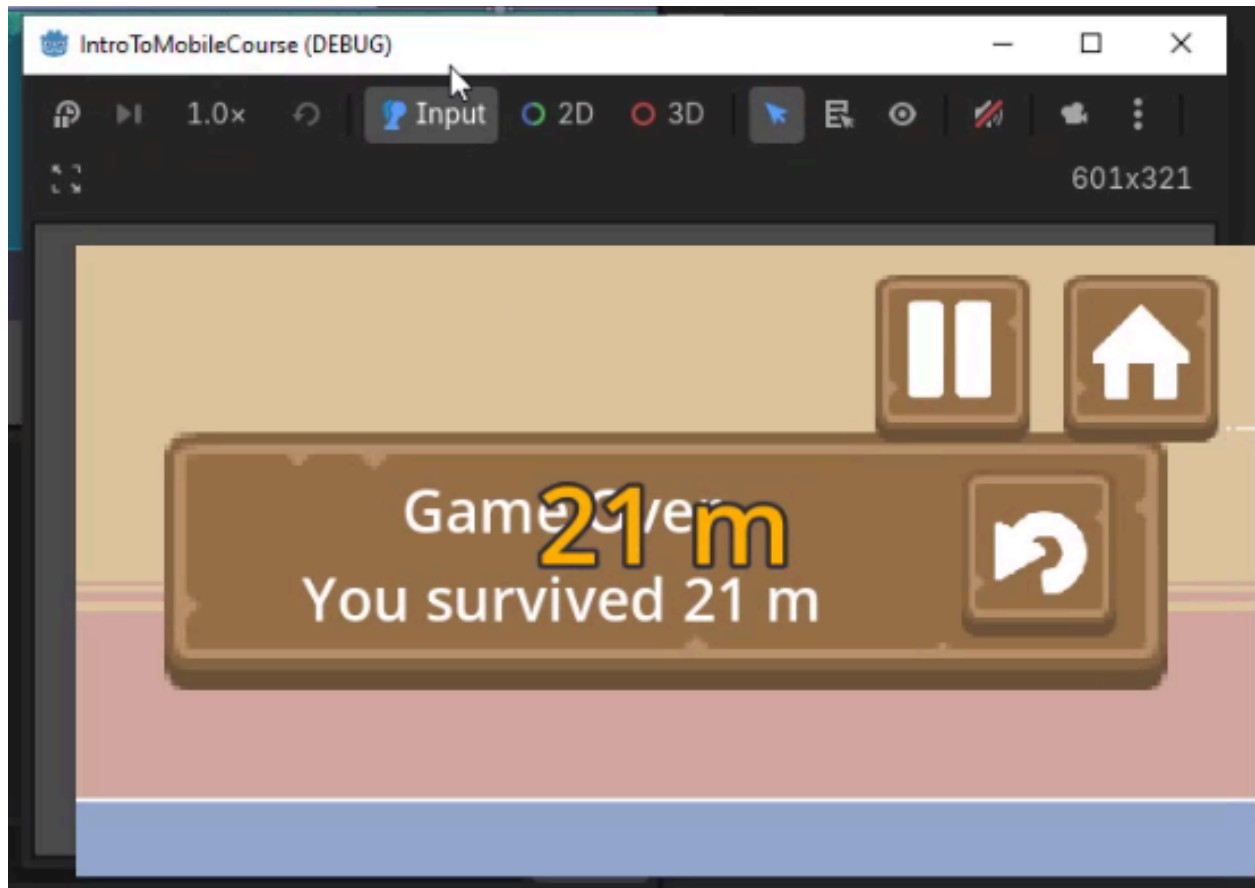
With our game loop complete, the user interface functional, and the audio atmosphere established, we have a fully playable game. However, to turn this project into a polished mobile product, we need to address a few critical technical aspects.

In this chapter, we will finalize the project by ensuring it scales correctly across the wide variety of mobile screen sizes and aspect ratios. We will also implement a simple optimization to keep performance high during long play sessions. Finally, we will walk through the specific steps required to export your game for both Android and iOS devices.

Screen Scaling

When developing games, especially for mobile platforms, it is crucial to consider the vast array of screen sizes, resolutions, and aspect ratios available in the market.

By default, a Godot project maps one game pixel to one screen pixel. While this behavior is straightforward, it often leads to a poor user experience on modern high-resolution mobile screens, where the game content and UI elements appear tiny. Conversely, on screens with different aspect ratios, elements might overlap or be cut off.

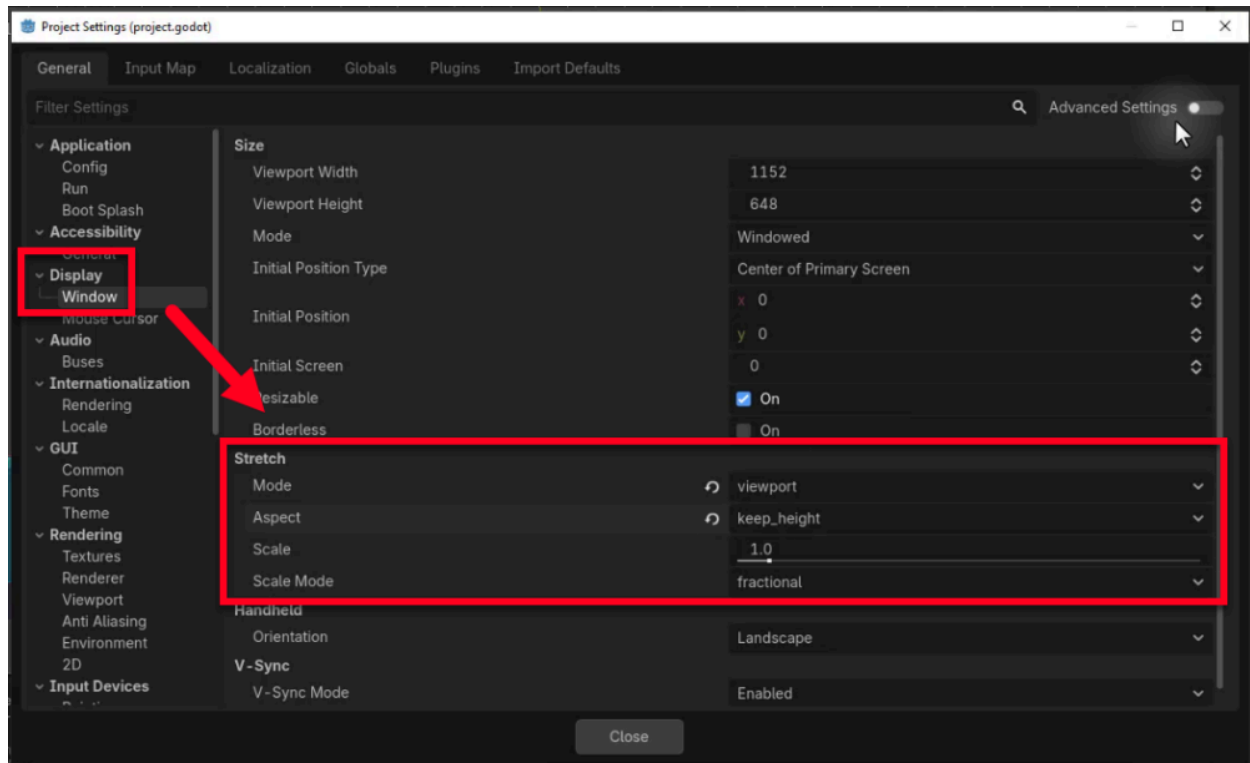


Our goal is to configure the game so that the visuals and UI scale uniformly, preserving the intended layout and readability regardless of the device.

Configuring Stretch Settings

Godot provides built-in settings to handle screen scaling automatically. We will adjust the project's stretch settings to ensure the viewport scales consistently to fit the device screen.

1. Navigate to the main menu and open **Project > Project Settings....**
2. In the Project Settings window, scroll down the left-hand menu and select the **Display > Window** category.
3. Locate the **Stretch** section.
4. Change the **Mode** from *Disabled* to **viewport**, and set **Aspect** from *Keep* to **keep_height**.
5. Close the Project Settings window.

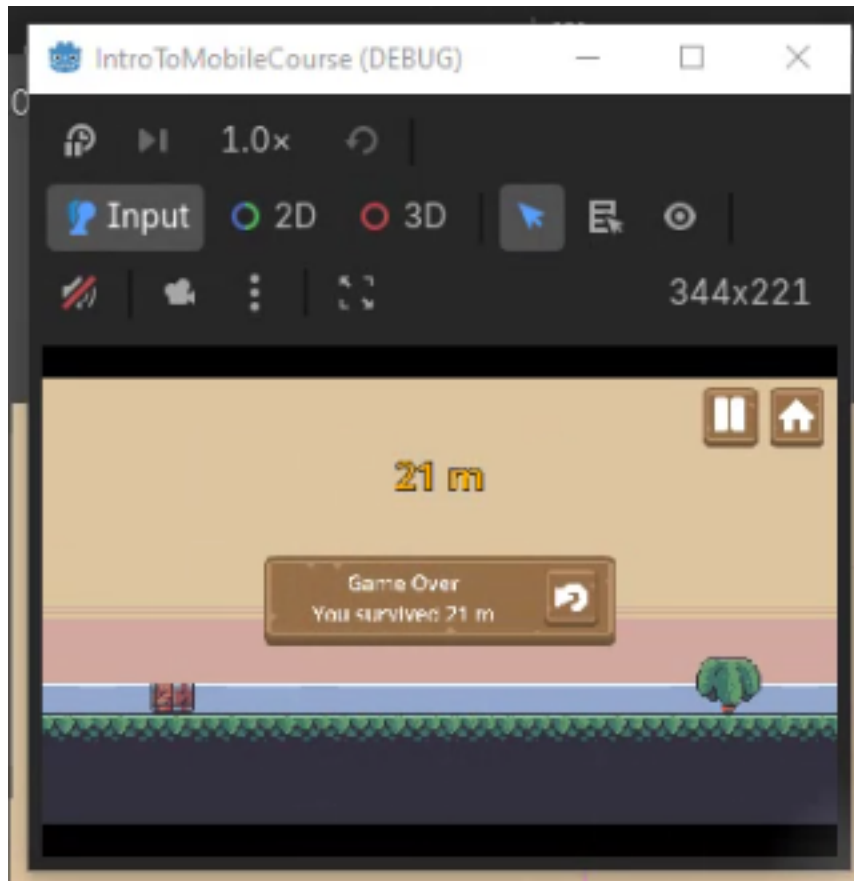


Understanding the Settings

These two settings we just changed work together to solve our scaling issues:

- **Mode: Viewport:** This tells Godot to render the game at the base resolution defined in your project settings and then scale the resulting image to fit the device's screen. This treats your game like a canvas that stretches to fill the display, ensuring your pixel art remains visible and your UI elements retain their relative size.
- **Aspect: Keep Height:** This setting determines how the engine handles aspect ratios that differ from your base resolution. By selecting *Keep Height*, the vertical field of view remains constant. If the device screen is wider than your base resolution, the player will simply see more of the level to the left and right. This is an excellent strategy for side-scrolling runners, as it prevents vertical cropping that could hide the ground or sky.

With these adjustments, your game will now scale correctly across various mobile devices, providing a consistent experience whether played on a phone or a tablet.



Performance Optimization

Before we export, we should address a subtle issue related to performance. Currently, when obstacles are spawned, they move across the screen to the left and continue moving indefinitely.

Although they eventually leave the visible area, these objects remain in memory. Over a long play session, the accumulation of thousands of off-screen nodes could lead to performance degradation or even crash the game.

To solve this, we will implement a cleanup mechanism to destroy obstacles once they are far enough off-screen.

Cleaning up Off-screen Nodes

To fix this issue, we will modify the obstacle script to check its position every frame. If the obstacle moves beyond a specific threshold to the left, we will remove it from the scene tree.

Open the `obstacle.gd` script. We will update the `_process()` function to include this check.

1. We will use an X-coordinate of `-1200` as our threshold. This ensures the object is well off-screen before it is removed.
2. Inside the `_process()` function, check if `global_position.x` is less than `-1200`.
3. If the condition is met, call `queue_free()`. This function safely queues the node for deletion at the end of the current frame.

Update your `obstacle.gd` script as follows:

```
func _process(delta):  
    # Multiply by parallax so background elements drift slower  
    var speed : float = game_manager.current_speed * delta * parallax  
    global_position.x -= speed  
  
    # Remove off-screen objects to prevent memory buildup  
    if global_position.x < -1200:  
        queue_free()
```

While advanced games often use “object pooling” (reusing objects instead of destroying them) to maximize performance, our spawn rate is low enough that simply destroying and instantiating objects is perfectly sufficient.

Godot Insight: `queue_free()` and node lifecycle

When you call `queue_free()` on a node, Godot marks it for deletion at the end of the current frame. This is safer than calling `free()` directly, which removes the node immediately and can cause crashes if other code still references it during the same frame (for example, inside a physics callback).

Once freed, a node is removed from the scene tree and its memory is released. Any child nodes are freed along with it. This makes `queue_free()` the standard way to remove spawned objects, projectiles, and effects once they are no longer needed.

Learn more: [Node](#)

(https://docs.godotengine.org/en/4.6/classes/class_node.html) This change

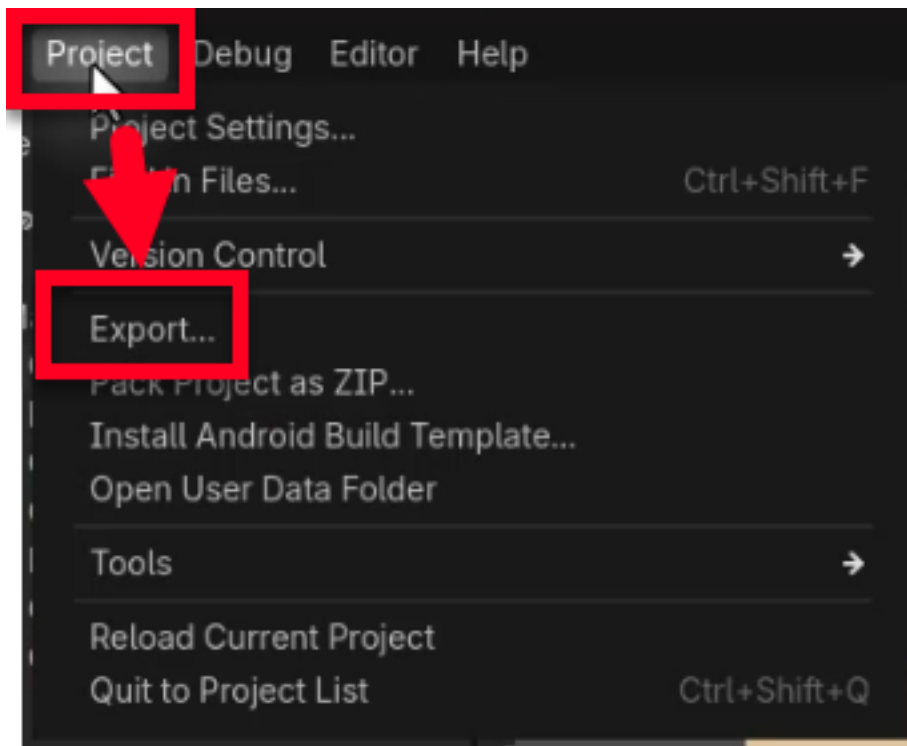
ensures our game remains efficient regardless of how long the player survives.

Exporting to Android

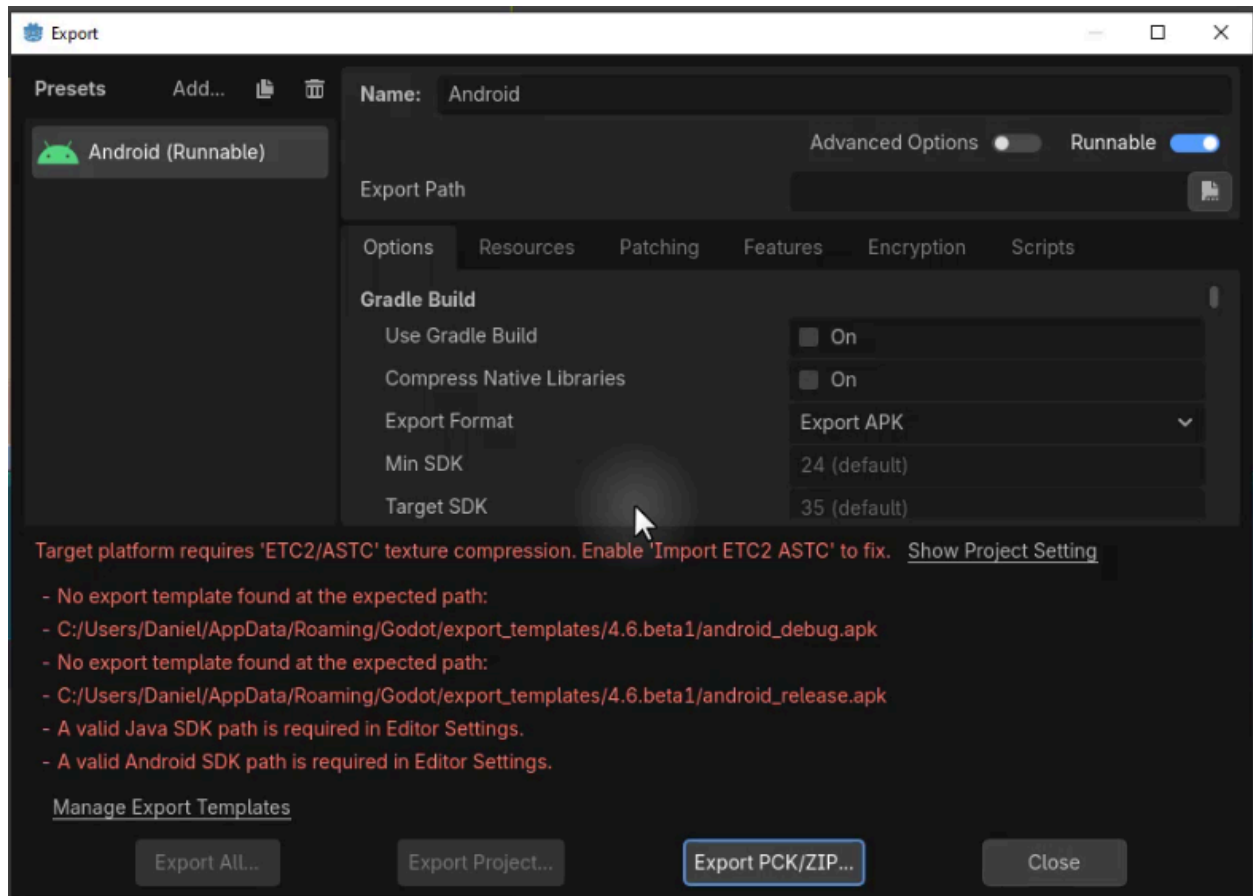
With our project configured and optimized, we can now look at exporting it to a mobile device. We will start with Android. This process involves setting up export presets, installing build templates, and configuring external SDKs.

Setting up the Android Export Preset

First, we need to tell Godot that we intend to export for Android. First, navigate to the top menu and select **Project > Export...**



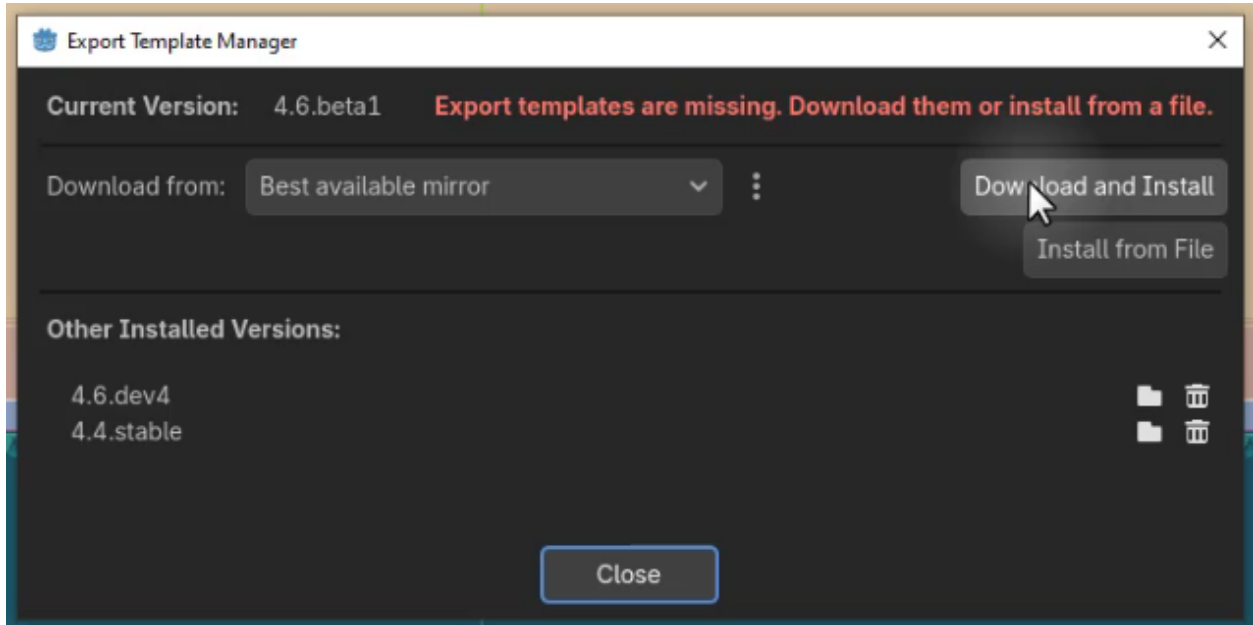
In the Export window, click **Add...** and select **Android** from the dropdown list. You will likely see several error messages at the bottom of the window. We will resolve these step-by-step.



Installing Android Build Templates

Godot requires specific build templates to package games for Android. These are not included in the standard editor download to keep the file size small.

In the top menu, go to **Project > Install Android Build Template....** Next, in the popup window, click **Manage Templates**, then click **Download and Install** in the Export Template Manager.

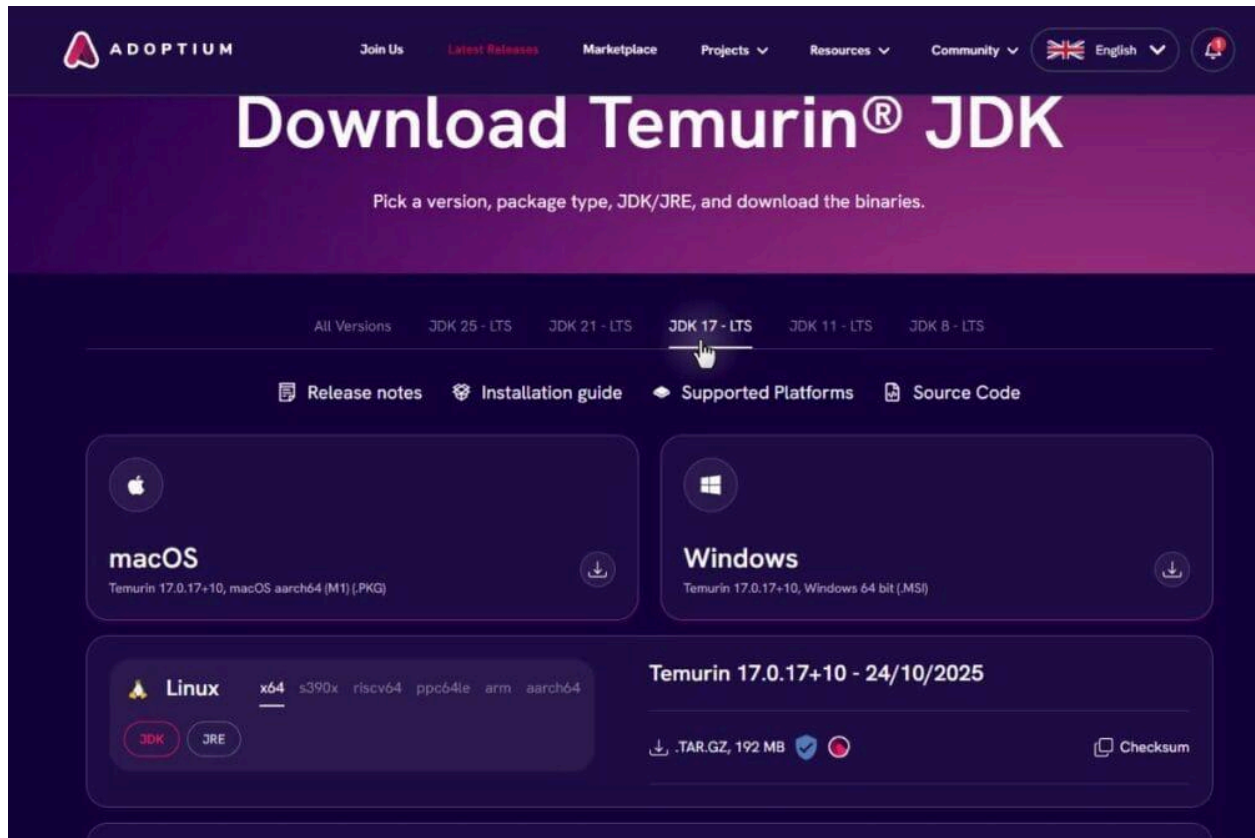


This download may take some time depending on your internet connection.

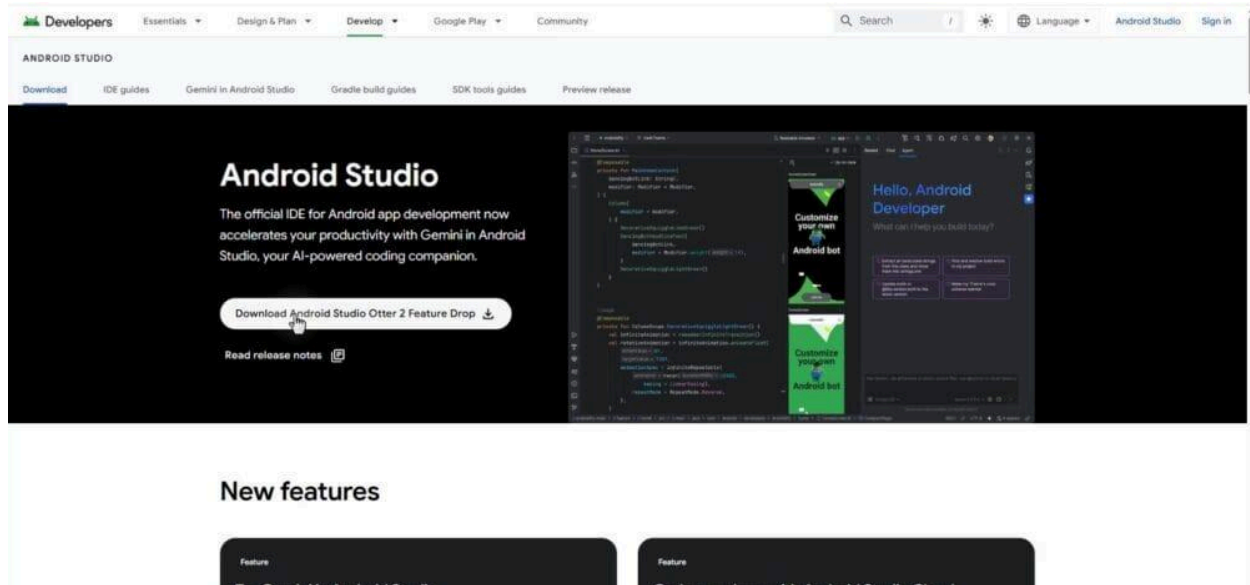
Installing Required SDKs

To build the final APK file, Godot relies on two external tools: the Android SDK and OpenJDK. The Android SDK provides the platform libraries and build tools that package your game into an installable APK, while OpenJDK supplies the Java compiler that the Android build system requires under the hood.

1. **OpenJDK:** Godot requires a specific version of the Java Development Kit (which you can find listed in the official documentation: https://docs.godotengine.org/en/stable/engine_details/development/compiling/compiling_for_android.html). Download and install the recommended JDK LTS (Long-Term Support) version, typically JDK 17, from the Adoptium website (<https://adoptium.net/>). Note the installation directory.



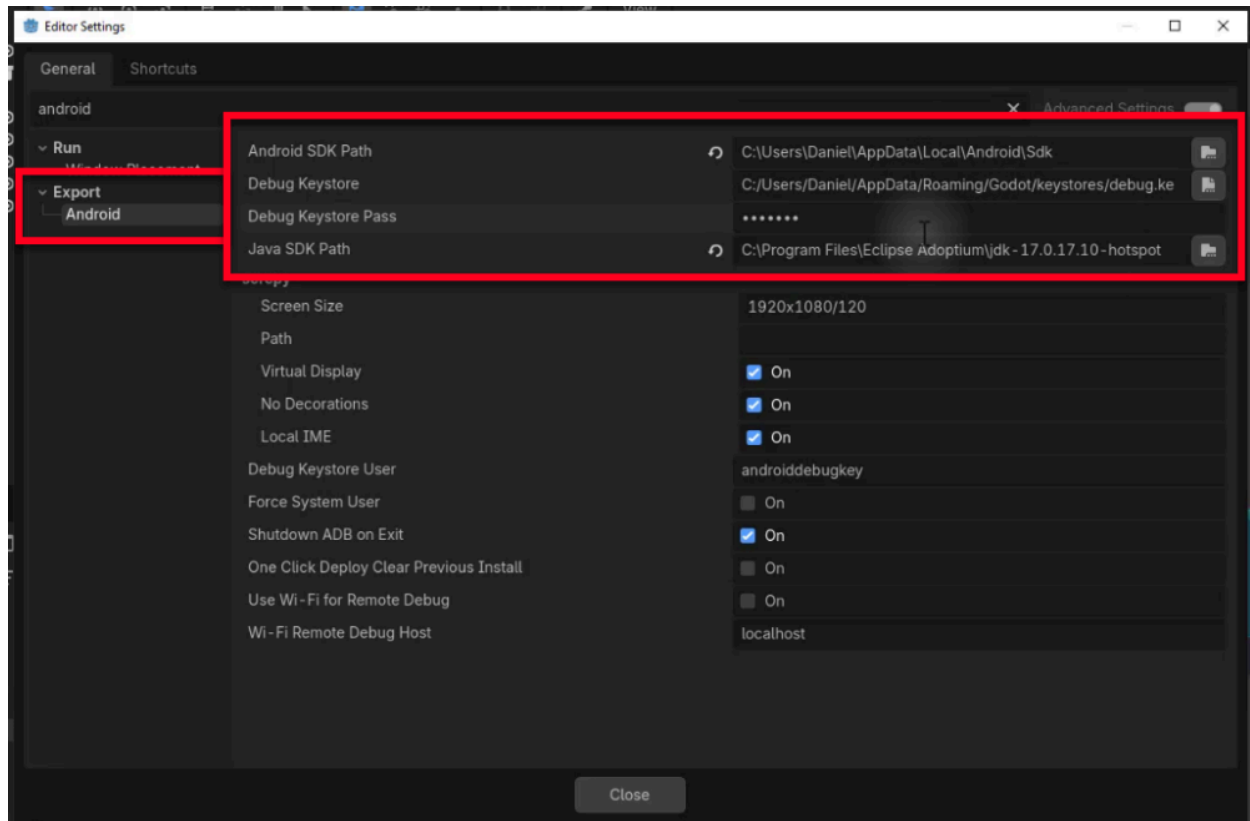
2. **Android SDK:** The easiest way to acquire the Android SDK is by installing Android Studio (<https://developer.android.com/studio>). During installation, note the location of the SDK folder (usually in AppData/Local/Android/sdk on Windows or Library/Android/sdk on macOS).



After installing these tools, restart your computer to ensure all system environment variables are updated.

Configuring SDK Paths in Godot

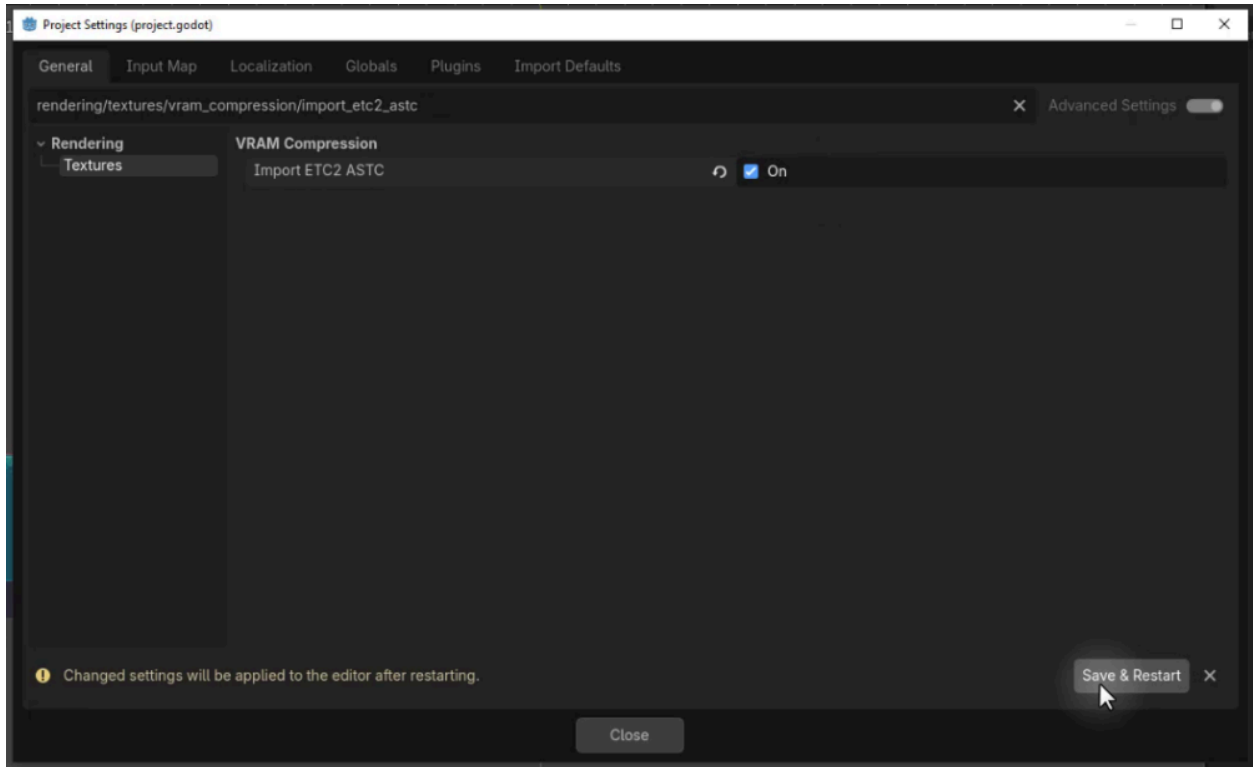
Once the external tools are installed, we must link them to Godot. In Godot, go to **Editor > Editor Settings** and scroll down to the **Export > Android** section. Set the **Android Sdk Path** and the **Java Sdk Path** to the folders where the Android SDK and OpenJDK were installed, respectively.



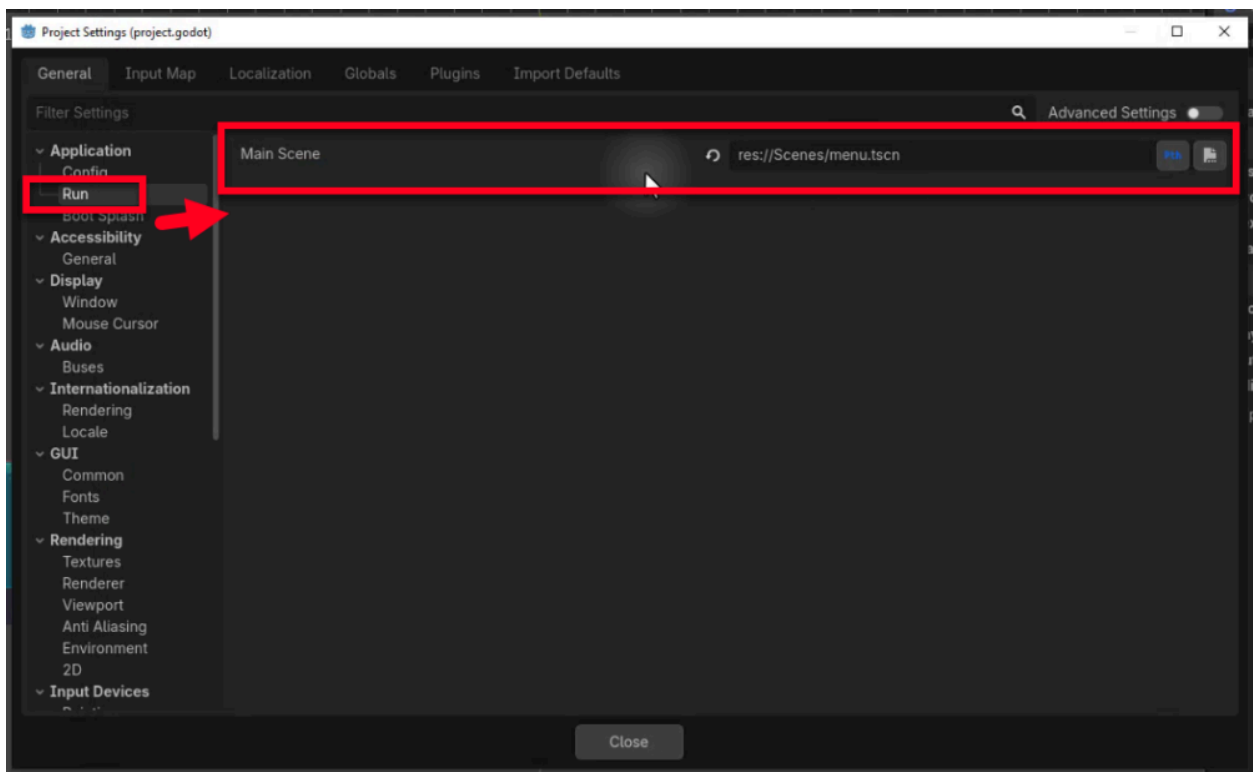
Final Project Configuration

We need to adjust a few project settings to ensure the app runs correctly on a phone. Open the Project Settings window and make the following changes:

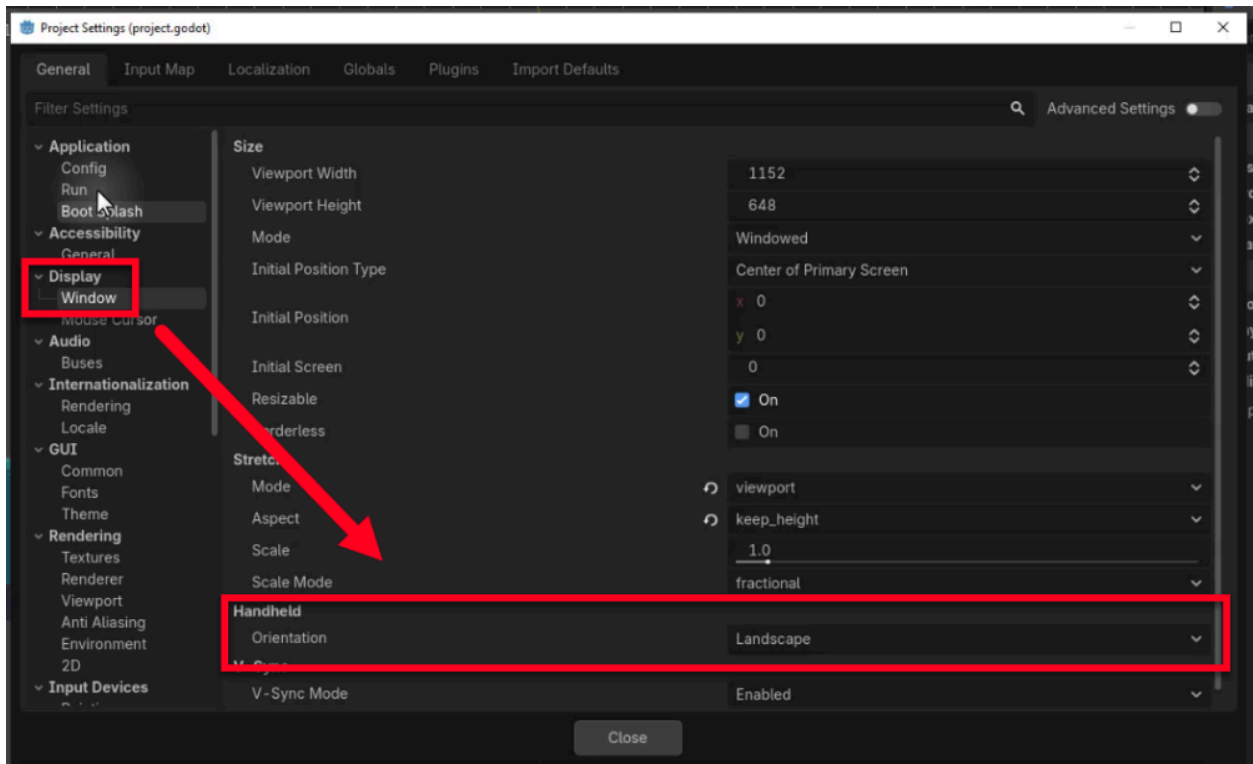
- **Texture Compression:** Mobile GPUs cannot decode desktop texture formats directly. VRAM compression (ETC2/ASTC) re-encodes your textures into a format that mobile hardware can read efficiently, reducing memory usage and improving rendering speed. In **Project > Project Settings > Rendering > Textures**, enable **Vram Compression** (specifically for ETC2/ASTC). You will need to save and restart the editor after changing this.



- **Main Scene:** In Project Settings > Application > Run, ensure Main Scene is set to your menu.tscn so the game starts at the menu.



- **Orientation:** In **Project Settings > Display > Window > Handheld**, set **Orientation** to **landscape**. This locks the screen rotation to match our game design.



- **Icons:** In **Project Settings > Application > Config**, you can set the **Name** of your app and assign a custom **Icon**.

Exporting the APK

After this, all our issues should be resolved. As such, we should be ready to create the APK - the file that Android uses for mobile apps. Return to **Project > Export...** and select the Android preset. As mentioned, the errors should now be resolved. As such, click **Export Project**, choose a save location, and click **Save**.

Godot will then build the APK file. You can transfer this file to your Android device and install it to play your game!

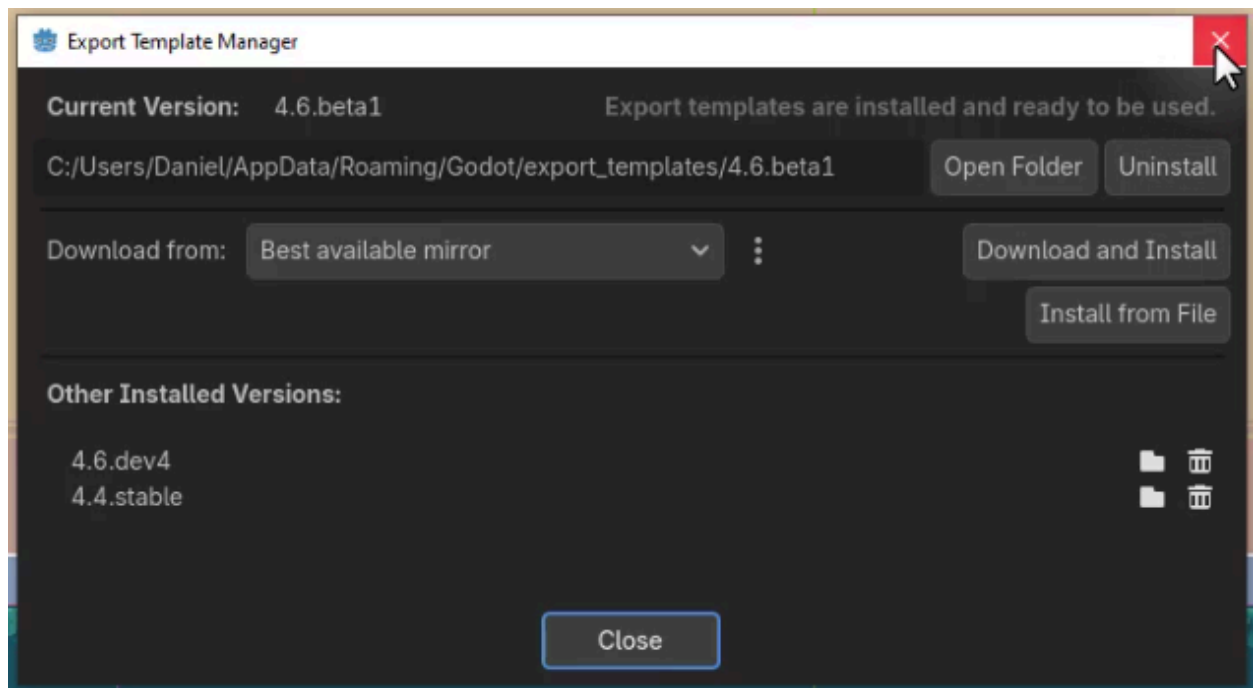
Exporting to iOS

Exporting to iOS requires a macOS computer and Xcode (from the Mac App Store or Apple Developer website). If you are working on Windows or Linux, you cannot directly build an iOS application.

Besides these requirements, you'll also require an Apple Developer Account. You'll need to sign the app and deploy it to a device in order to test it. Note a free account works for local testing, but a paid account is needed for the App Store.

Export Templates

If you haven't already installed the export templates during the Android setup, you will need to do so now via **Editor > Manage Export Templates**. Once again, this can take a while as these are large files.



Project Configuration

Similar to Android, ensure your project settings are correct:

- **Name and Icon:** Set in **Project Settings > Application > Config**.
- **Orientation:** Set to **Landscape** in **Project Settings > Display > Window > Handheld**.

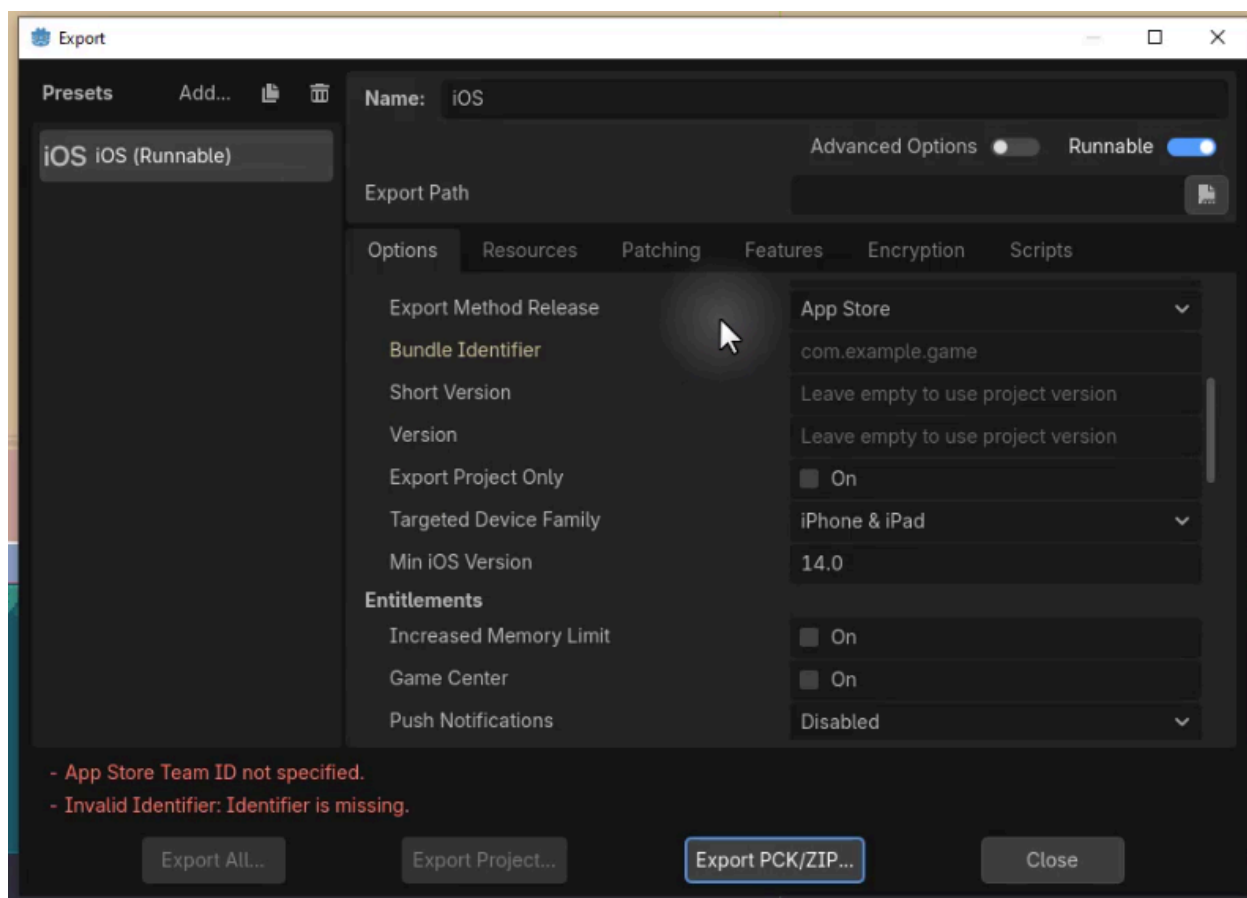
Creating the Xcode Project

With our Project Settings handled, we can now create the project files that need to be opened in Xcode.

To begin, go to **Project > Export...**, click **Add...**, and select **iOS**. In the iOS preset settings, you must fill in:

- **App Store Team ID:** Your 10-character Team ID from your Apple Developer account.
- **Bundle Identifier:** A unique ID like `com.yourname.gamename`.

Once these aspects are filled out, click **Export Project**. You just then need to select an empty folder to save the project files.



From here, Godot will generate an Xcode project folder. You can open the `.xcodeproj` file in Xcode, connect your iOS device, and build/run the app directly from there.

For detailed troubleshooting and advanced iOS configuration, always refer to the [official Godot documentation on exporting for iOS](https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_ios.html) (https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_ios.html).

In the final chapter, we will conclude our journey and discuss next steps for your game development path.

Closing Words

Congratulations! You have built an infinite runner from scratch, and it runs on a real mobile device.

That is no small thing. Game development is a complex discipline that requires you to wear many hats: programmer, designer, artist, and sound engineer. Over the course of these chapters, you have worn all of them. You started with an empty Godot project and finished with a polished, playable mobile game that you can proudly show to friends, family, or anyone who will hold still long enough to tap “Play.”

Take a moment to appreciate what you have built. It is easy to look at the finished product and only see the code, but the real value lies in the problem-solving skills you developed along the way.

What you Accomplished

You have done more than just follow a set of tutorials. You have internalized the “Godot way” of thinking about game architecture. Let’s review the core skills you now have in your toolkit:

The Node System and Scene Tree

You learned how to compose complex behaviors from simple building blocks. You understand that everything in Godot is a node, and that scenes can be instanced and nested to create modular, reusable game objects.

Scripting and Logic

You moved beyond basic syntax to handle real-time logic. You wrote scripts that drive physics calculations, track game state with simple boolean flags, and use group-based collision detection to decide what happens when objects meet. You learned how to use Signals to decouple your code, ensuring that your UI, player, and game manager can communicate cleanly without creating a tangled web of dependencies.

Game Feel and Polish

A working game is not always a *fun* game. You learned how to bridge that gap with “juice.” You added particle systems for visual impact, layered in audio for a satisfying feedback loop, and tweaked animations until they felt just right. You learned firsthand that small details often matter more than high-fidelity graphics.

User Interface and State Management

You built a UI system that adapts to different screen sizes using Godot’s container nodes and anchors. You learned how to pause the game, manage menus, and handle game-over flows so the player always has a clear path back to the action. You tackled the challenge of mobile resolution and aspect ratios as well, ensuring your game looks good on any device.

The Build Pipeline

Finally, you stepped out of the editor. You learned how to configure export templates, handle platform-specific settings for mobile, and generate a build that you can install and share with the world.

Where to go from Here

The end of this book is the beginning of your own independent practice. Here are a few concrete steps you can take to continue your journey:

Expand this Project

The best way to solidify your knowledge is to modify code you already understand. Try adding features to the game we built together:

- Create a new obstacle type with a different challenge or shape.
- Add a power-up system that temporarily changes the player’s abilities.
- Implement a high-score table that saves the top five runs.
- Add controller support for PC builds.

Participate in a Game Jam

Game jams are events where you build a game from scratch in a limited time (usually 48 to 72 hours). They are the single best way to learn as they

force you to scope your ideas realistically and finish a project. The Godot community is very active on [itch.io](https://itch.io/jams) (<https://itch.io/jams>), and there is almost always a jam happening.

Explore the Documentation

We covered a lot of ground, but Godot is vast. Make a habit of reading the official documentation. It is well-written and full of examples. Whenever you wonder, “Can Godot do this?”, the answer is usually in the class reference.

See the [Godot Documentation](https://docs.godotengine.org) (<https://docs.godotengine.org>).

Join the Community

Game development can be solitary, but you do not have to do it alone. The Godot community is known for being welcoming and helpful. Join the official [Discord](https://discord.com/invite/godotengine) (<https://discord.com/invite/godotengine>), browse the [Reddit community](https://www.reddit.com/r/godot/) (<https://www.reddit.com/r/godot/>), or check out the [Godot Forums](https://forum.godotengine.org/) (<https://forum.godotengine.org/>). Sharing your work and seeing what others are building is a great source of motivation.

Continue Your Learning

For hands-on, project-driven learning in Godot and programming, visit Zenva Academy (<https://academy.zenva.com>). Our platform features 300+ courses and 40+ Learning Pathways that guide you from foundational skills to advanced mastery.

Final Thoughts

Game development is a practice, not a destination. You will write bugs. You will design mechanics that do not work. You will stare at code you wrote yesterday and wonder what you were thinking. This is all part of the process, and every developer goes through it.

Every error message is an opportunity to learn how the engine works. Every failed mechanic teaches you something about game design. The most important skill you can cultivate is persistence.

You have the tools. You have the knowledge. And you have proof that you can finish a project, because you just did. Now, go build something amazing!

Reference: Full Source Code

In this appendix, we provide the complete source code for the Infinite Runner project. This reference is intended to help you debug your own code, verify your implementation, or serve as a base for future expansions.

You can also download the complete project files from the course resources at the beginning of this book to compare your work directly.

game_manager.gd

Found in folder: /Scripts

The GameManager class serves as the central controller for the game loop. It manages global variables such as the current game speed, the maximum speed limit, and the distance traveled. Additionally, it handles the game over state, music transitions, and UI button callbacks for restarting, pausing, and returning to the home menu.

```
class_name GameManager
extends Node

var current_speed : float = 200
var max_speed : float = 1500
var speed_increase_rate : float = 5
var distance : float = 0

var is_game_over : bool = false

@onready var game_over_screen : TextureRect =
    $CanvasLayer/GameOverScreen
@onready var distance_text : Label = $CanvasLayer/DistanceText

@onready var gameplay_music : AudioStreamPlayer = $GameplayMusic
@onready var game_over_music : AudioStreamPlayer = $GameOverMusic

func _ready():
    # Hide the game over panel until the player actually loses
    game_over_screen.visible = false

func _process(delta):
    # Stop updating once the game ends
```

```

if is_game_over:
    return

if current_speed < max_speed:
    # Gradually ramp up difficulty over time
    current_speed += speed_increase_rate * delta
    # Convert speed into a human-readable "meters" value
    distance += current_speed / 32 * delta
    distance_text.text = str(int(distance)) + " m"

# Called by the player script when an obstacle is hit
func game_over ():
    current_speed = 0.0
    is_game_over = true

    game_over_screen.visible = true

    # Swap background music to the game over track
    gameplay_music.stop()
    game_over_music.play()

    $CanvasLayer/GameOverScreen/GameOverText.text = "Game Over\nYou
survived " + str(int(distance)) + " m"

func _on_restart_button_pressed():
    # Reload the entire scene to reset all state
    get_tree().reload_current_scene()

func _on_home_button_pressed():
    get_tree().change_scene_to_file("res://Scenes/menu.tscn")

# Toggle pause by freezing/unfreezing the engine clock
func _on_pause_button_pressed():
    if Engine.time_scale > 0:
        Engine.time_scale = 0.0
    else:
        Engine.time_scale = 1.0

```

ground.gd

Found in folder: /Scripts

Attached to the ground object, this script creates the illusion of infinite movement. It moves the StaticBody2D node to the left based on the global

game speed. When the node moves past a specific X-coordinate (end), it teleports back to the start position, creating a seamless loop.

```
extends StaticBody2D
```

```
@onready var game_manager : GameManager = $"./GameManager"
```

```
# Loop boundaries set in the Inspector to match the tile pattern
```

```
@export var start : float
```

```
@export var end : float
```

```
func _process(delta):
```

```
# Move the ground leftward at the current game speed
```

```
var speed : float = game_manager.current_speed * delta
```

```
global_position += Vector2.LEFT * speed
```

```
# Teleport back to create a seamless infinite loop
```

```
if global_position.x < end:
```

```
    global_position.x = start
```

menu.gd

Found in folder: /Scripts

This script manages the main menu scene. It listens for the “Play” button press event and transitions the game to the main gameplay scene (main.tscn).

```
extends Control
```

```
func _on_play_button_pressed():
```

```
# Transition from the menu to the main gameplay scene
```

```
get_tree().change_scene_to_file("res://Scenes/main.tscn")
```

obstacle.gd

Found in folder: /Scripts

This script controls the movement of environmental objects, including obstacles and clouds. It applies a leftward velocity based on the game speed and a parallax factor, allowing background elements to move slower than

foreground elements. It also handles memory management by deleting objects once they move off-screen.

extends Node2D

```
# Absolute path because spawned objects are children of the spawner, not Main
```

```
@onready var game_manager : GameManager =  
get_node("/root/Main/GameManager")
```

```
# Controls depth-based speed: 1.0 = foreground, lower = further back
```

```
@export_range(0.0, 1.0) var parallax : float = 1.0
```

```
func _process(delta):
```

```
# Multiply by parallax so background elements drift slower
```

```
var speed : float = game_manager.current_speed * delta * parallax  
global_position.x -= speed
```

```
# Remove off-screen objects to prevent memory buildup
```

```
if global_position.x < -1200:  
    queue_free()
```

player.gd

Found in folder: /Scripts

The player controller handles the character's physics and input. It applies gravity, processes jump inputs (including buffering), and manages visual feedback such as animations and particle effects. It also listens for collisions with obstacles to trigger the game over state.

extends CharacterBody2D

```
@export var jump_force : float = 500
```

```
@export var gravity : float = 2200
```

```
var jump_input : bool = false
```

```
@onready var anim : AnimationPlayer = $AnimationPlayer
```

```
@onready var game_manager : GameManager = $"./GameManager"
```

```
@onready var jump_buffer_timer : Timer = $JumpBufferTimer
```

```
@onready var run_particle : CPUParticles2D = $RunParticle
```

```

@onready var jump_particle : CPUParticles2D = $JumpParticle
@onready var jump_sfx : AudioStreamPlayer = $JumpSFX

func _physics_process (delta):
    # Apply gravity only while airborne
    if not is_on_floor():
        velocity.y += gravity * delta

    # Execute a buffered jump if the player is on the ground
    if jump_input and is_on_floor():
        jump_input = false
        velocity.y = -jump_force
        jump_particle.emitting = true
        jump_sfx.play()

    move_and_slide()

    # Switch animation and particles based on ground contact
    if is_on_floor():
        anim.current_animation = "run"
        run_particle.emitting = true
    else:
        anim.current_animation = "jump"
        run_particle.emitting = false

# Buffer the jump input so near-miss taps still register
func _try_jump ():
    if game_manager.is_game_over:
        return

    jump_input = true
    jump_buffer_timer.start()

# Handle both mouse clicks (desktop) and screen taps (mobile)
func _input (event):
    if event is InputEventMouseButton:
        if event.button_index == MOUSE_BUTTON_LEFT:
            if event.pressed:
                _try_jump()

    if event is InputEventScreenTouch:
        if event.pressed:
            _try_jump()

# Triggered by the Area2D detector in front of the player

```

```

func _on_obstacle_detector_body_entered(body):
    if game_manager.is_game_over:
        return

    if body.is_in_group("Obstacle"):
        game_manager.game_over()

        # Fling the player backward for visual feedback
        velocity.x = -500
        velocity.y = -jump_force

# Clear the buffered input when the timer expires
func _on_jump_buffer_timer_timeout():
    jump_input = false

```

spawner.gd

Found in folder: /Scripts

The spawner script automates the creation of obstacles and clouds. It instantiates scenes from a provided array at random intervals and vertical positions. This ensures an endless and varied stream of content for the player to navigate.

```

extends Node2D

@export var scenes_to_spawn : Array[PackedScene]

@export var min_spawn_range : int
@export var max_spawn_range : int
var next_spawn_range : float

# Maximum vertical offset for spawn variety (clouds at different heights)
@export var max_height : int

var previous_object : Node2D

func _process(delta):
    # Spawn the first object immediately, then wait for distance
    if not previous_object:
        _spawn()
        return

```

```

    # Spawn again once the last object has moved far enough away
    if global_position.x - previous_object.global_position.x >
next_spawn_range:
    _spawn()

# Instantiate a random scene from the array and place it at the spawner
func _spawn ():
    var scene_to_spawn : PackedScene = scenes_to_spawn[randi() %
len(scenes_to_spawn)]
    var obj : Node2D = scene_to_spawn.instantiate()
    add_child(obj)

    obj.global_position = global_position

    # Add a random vertical offset for natural placement
    var y_pos : float = global_position.y + randi_range(0, max_height)
    obj.global_position.y = y_pos

    # Randomize the gap before the next spawn
    next_spawn_range = randi_range(min_spawn_range, max_spawn_range)

    previous_object = obj

```